



# Cloud Systems

## Chapter 1: Introduction

Dr Lauritz Thamsen  
[lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)  
School of Computing Science  
University of Glasgow



# Acknowledgements

- This part of the course draws on materials developed with colleagues at TU Berlin and Hasso Plattner Institute
  - Prof. Dr. Odej Kao
  - Dr. Daniel Warneke
  - Jossekin Beilharz
  - Dr. Anton Gulenko
  - Lukas Pirl
  - Morgan Geldenhuys
  - Philipp Wiesner



# A Bit More on Me

Dr Lauritz Thamsen (he/him)

- Lecturer in Computer Systems
- Systems Research Section
- Low-Carbon and Sustainable Computing theme
- [lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)
- <https://lauritzthamsen.org>
- Office: S152
- Office hours: Tuesdays 11 am



Research interests:

- Distributed Systems
- Edge/Cloud Computing
- Resource Management & Scheduling
- Carbon-Aware Computing

# First Half: Lecture Chapters

- Focus of this part: **Cloud Resource Management**
  - Management of virtual resources on host systems
  - Larger systems running on sets of virtual resources
  - Carbon impact assessment and optimization
- 1. Cloud Computing Intro
- 2. Virtual Machines
- 3. Containers
- 4. Cloud Infrastructure Management
- 5. Cloud Sustainability



# First Half: Labs and Exercise

- Weekly lab sessions:
  1. Lab 1: (Linux Systems) Benchmarking Basics
  2. Lab 2: Virtual Machines (Google Cloud and QEMU)
  3. Lab 3: Containers (Docker)
  4. AE1 formative feedback session
  5. General AE1 support
- Assessed Exercise 1 (AE1)
  - Virtual resources benchmarking project
  - Written report with plots showing own measurements
  - Release: 20.1. 10 am (Monday Week 2); Deadline: 14.2. 4:30 pm (Friday Week 5)
  - Weight 25%

# The Audience

---

- Virtualization?
- Docker?
- Kubernetes?
- Infrastructure as Code?
- Edge computing?
- FaaS?
- Carbon-Aware Computing?

# Outline of Chapter 1: Cloud Computing Intro

---

1.1 Clusters, Grids, Clouds, and Edge/Fog

1.2 The NIST Definition's Dimensions

— short break (10-15 minutes) —

1.3 Intro to Virtualization

1.4 Intro to VMs and Containers

# Outline of Chapter 1: Cloud Computing Intro

---

## **1.1** Clusters, Grids, Clouds, and Edge/Fog

## **1.2** The NIST Definition's Dimensions

— short break (10-15 minutes) —

## **1.3** Intro to Virtualization

## **1.4** Intro to VMs and Containers

# Background

## Clusters

- Many locally connected computers

## Grids

- Loosely coupled and widely distributed computers or clusters

## Clouds

- IT resources as a utility

## Edge/Fog

- Cloud services in closer proximity to users and edge devices

# Clusters

- Mostly homogeneous (commodity) compute resources and software stacks viewed as one system
- Interconnected by a low-latency and high-bandwidth (local area) network
- Goal: improving availability, resource utilization, price/performance
- Examples
  - Analytics clusters at Google, Microsoft, Meta, Alibaba, Amazon etc.!

# Grids

- Widely distributed heterogeneous compute resources
- Connected via a slow network (i.e. the internet)
- Heterogeneous software stacks unified by a middleware
- Used for a common goal
- Examples
  - Worldwide LHC Computing Grid (WLCG)
  - Berkeley Open Infrastructure for Network Computing (BOINC)

# Clouds

- Definition of Cloud Computing according to NIST [1]:

“Cloud computing is a model for enabling **ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

This cloud model is composed of five essential characteristics, three service models, and four deployment models.”

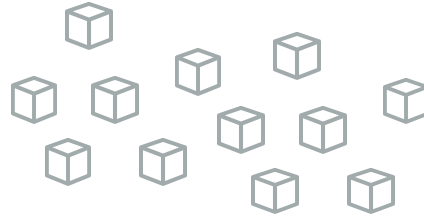
Standard 800-145

- NIST: National Institute of Standards and Technology



# Cloud and the Internet of Things

Cloud

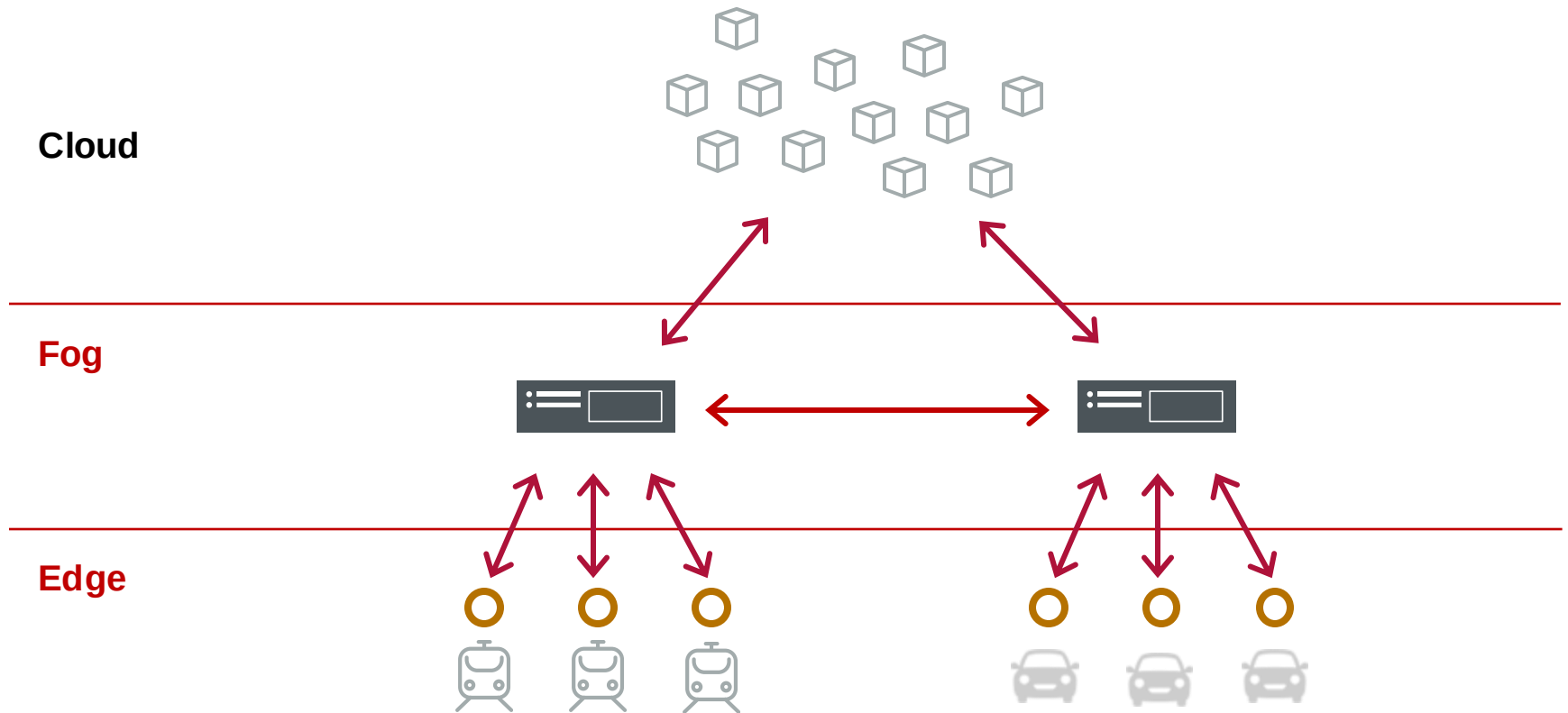


Devices and  
Sensors



Cloud Systems

# Fog Computing



# Outline of Chapter 1: Cloud Computing Intro

---

**1.1** Clusters, Grids, Clouds, and Edge/Fog

**1.2 The NIST Definition's Dimensions**

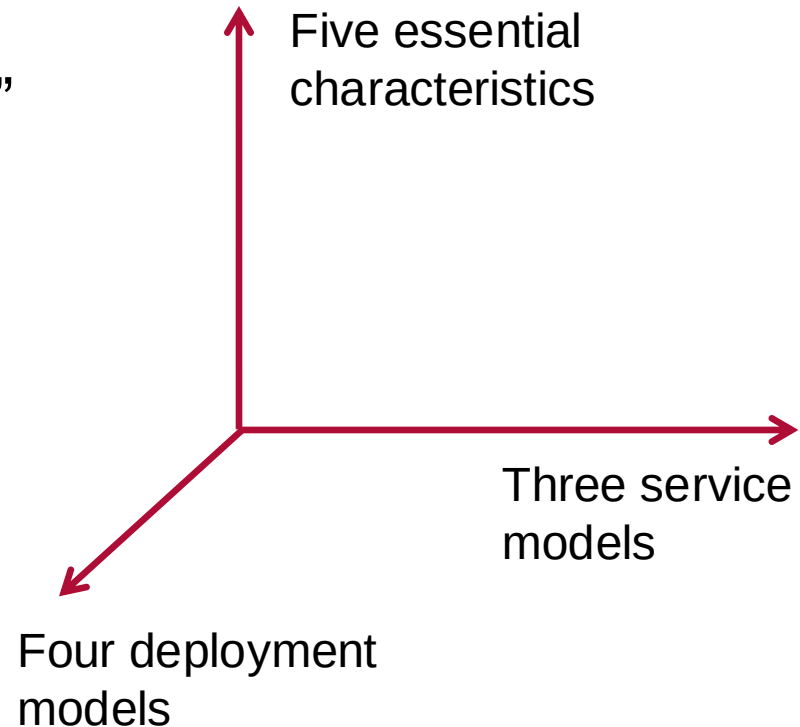
– short break (10-15 minutes) –

**1.3** Intro to Virtualization

**1.4** Intro to VMs and Containers

# Dimensions of Cloud Computing (NIST)

- NIST: “Cloud model is composed of
  - Five essential characteristics
  - Three service models
  - Four deployment models”



# Five Characteristics of Cloud Computing (NIST)

- On-demand self-service
  - No human interaction required for resource provisioning
- Broad network access
  - Accessible over networks with standard mechanisms
- Resource pooling
  - Pooled resources dynamically shared among several consumers, with location independence
- Rapid elasticity
  - Capabilities can be provisioned/released on demand
- Measured service
  - Resource usage is monitored, controlled, and reported

# Three Service Models of Cloud Computing (NIST)

- Software as a Service (SaaS)
  - Provider's application runs on cloud infrastructure and can be accessed over the network
  - Consumer does not control/manage underlying infrastructure
- Platform as a Service (PaaS)
  - Consumer can deploy custom application onto cloud infrastructure using programming languages, libraries, services, and tools supported by the provider
  - Consumer does not control/manage underlying infrastructure
- Infrastructure as a Service (IaaS)
  - Provider provides processing, storage, network resources to consumer
  - Consumer does not control/manage underlying infrastructure but has control over operating systems, storage, and deployed applications

# Four Deployment Models of Cloud Computing (NIST)

|                                       | Private Cloud                                  | Community Cloud                                 | Public Cloud                                                    | Hybrid Cloud                                                                                                                                                                                                     |
|---------------------------------------|------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User of the cloud infrastructure?     | Single organization                            | Organizations with shared concerns              | Open for the general public                                     | Composition of private / community / public cloud: <ul style="list-style-type: none"> <li>Clouds remain distinct, but bound together by standard mechanisms</li> <li>General goal: Enable portability</li> </ul> |
| Owner of the cloud infrastructure?    | Organization, third party, combination thereof | Organizations, third party, combination thereof | Business academic, government organization, combination thereof |                                                                                                                                                                                                                  |
| Location of the cloud infrastructure? | On premise, off premise                        | On premise, off premise                         | On premise of cloud provider                                    |                                                                                                                                                                                                                  |

# Public Cloud Examples

---

- AWS EC2
- DigitalOcean Droplets
- Google Kubernetes Engine
- AWS S3
- Amazon EMR
- AWS Lambda
- Azure Machine Learning Studio
- AWS SageMaker
- IBM Watson
- Google Mail, Google Docs, MS Office 365...



# Outline of Chapter 1: Cloud Computing Intro

---

1.1 Clusters, Grids, Clouds, and Edge/Fog

1.2 The NIST Definition's Dimensions

**– short break (10-15 minutes) –**

1.3 Intro to Virtualization

1.4 Intro to VMs and Containers

# Outline of Chapter 1: Cloud Computing Intro

---

**1.1** Clusters, Grids, Clouds, and Edge/Fog

**1.2** The NIST Definition's Dimensions

— short break (10-15 minutes) —

**1.3** Intro to Virtualization

**1.4** Intro to VMs and Containers

# Challenges for Cloud Providers

---

- Rapid provisioning
  - Resources must be available to the consumer quickly
  - No human interaction during provisioning
- Elasticity
  - Create illusion of infinite resources
  - Yet, manage data center in a cost-efficient manner
- Isolation of different consumers
  - Users must not interfere with each other
- Performance
  - Maintain good performance despite other challenges

# Approach: Virtualization

“Cloud computing is a model for enabling ubiquitous, convenient, on-demand **network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services)** that can be rapidly provisioned and released with minimal management effort or service provider interaction.

Standard 800-145

- Main idea on IaaS-level: provide *virtual resources*
  - Provide compute resources in the form of **virtual machines** and **containers**
  - Also other computing resources such as virtual networks, virtual network functions, virtual storage

# What is Virtualization?

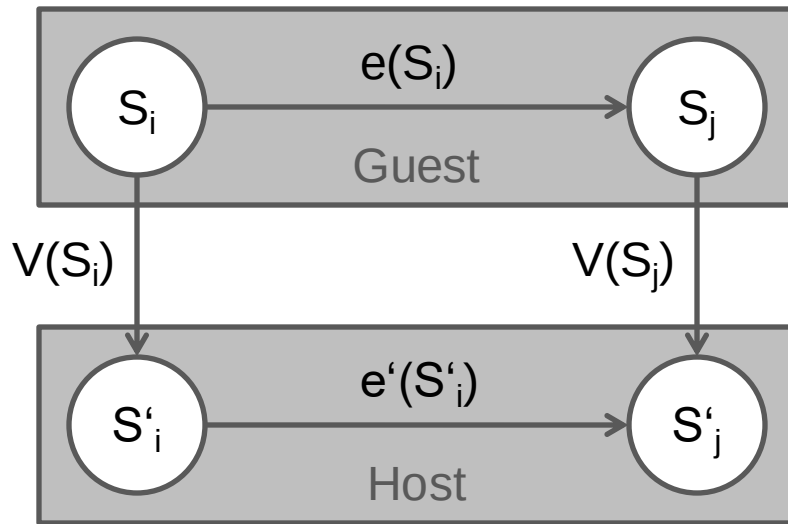
- Definition of virtualization according to NIST [2]:

“Virtualization is the **simulation of the software and/or hardware upon which other software runs**. This simulated environment is called a virtual machine (VM).”

Standard 800-125

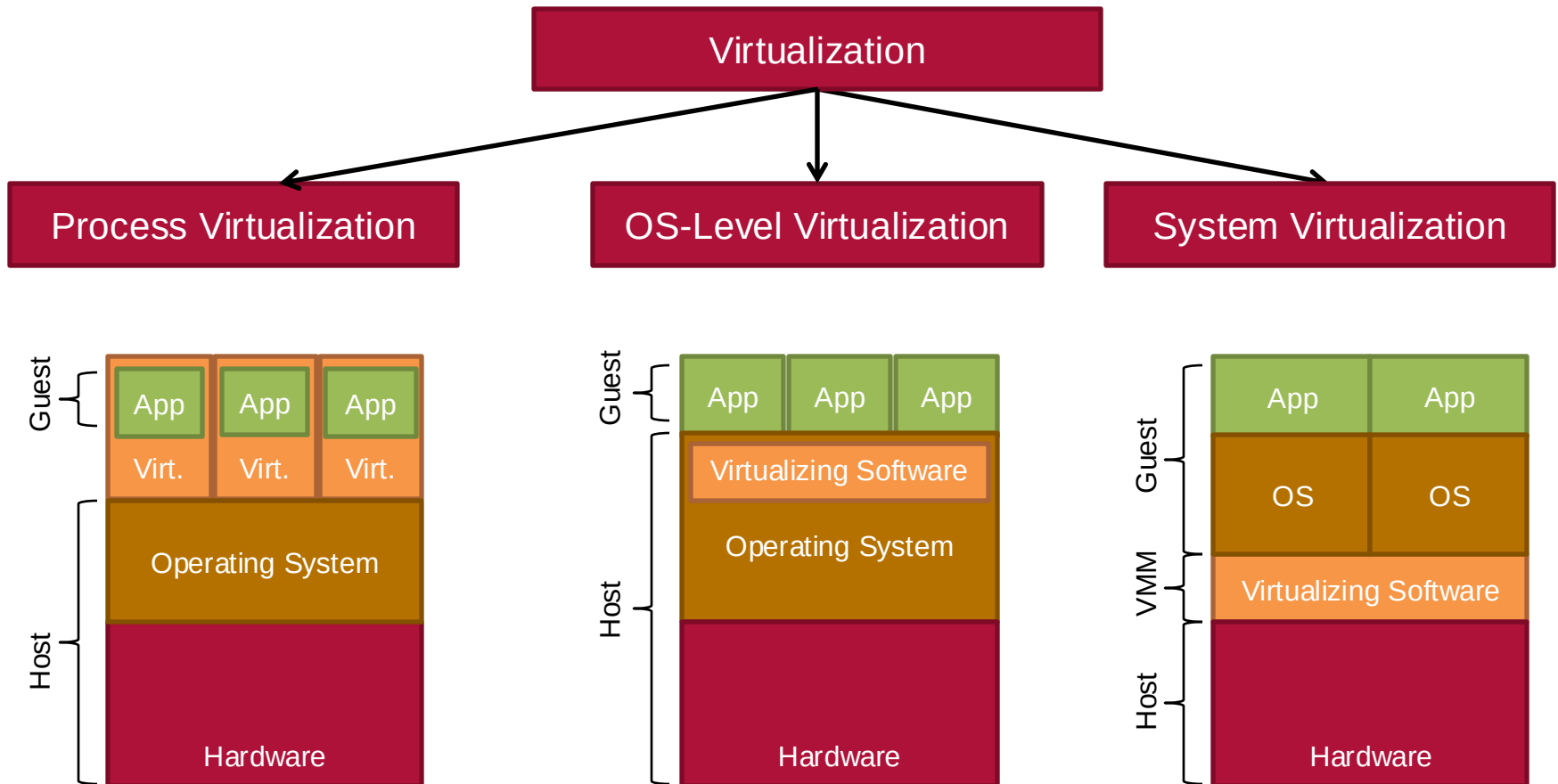
- Virtualization can transform a real system so
  - it looks like a different virtual system
  - multiple virtual systems
- **Real system** is often referred to as **host (system)**
- **Virtual system** is often referred to as **guest (system)**

# Formal Definition of Virtualization

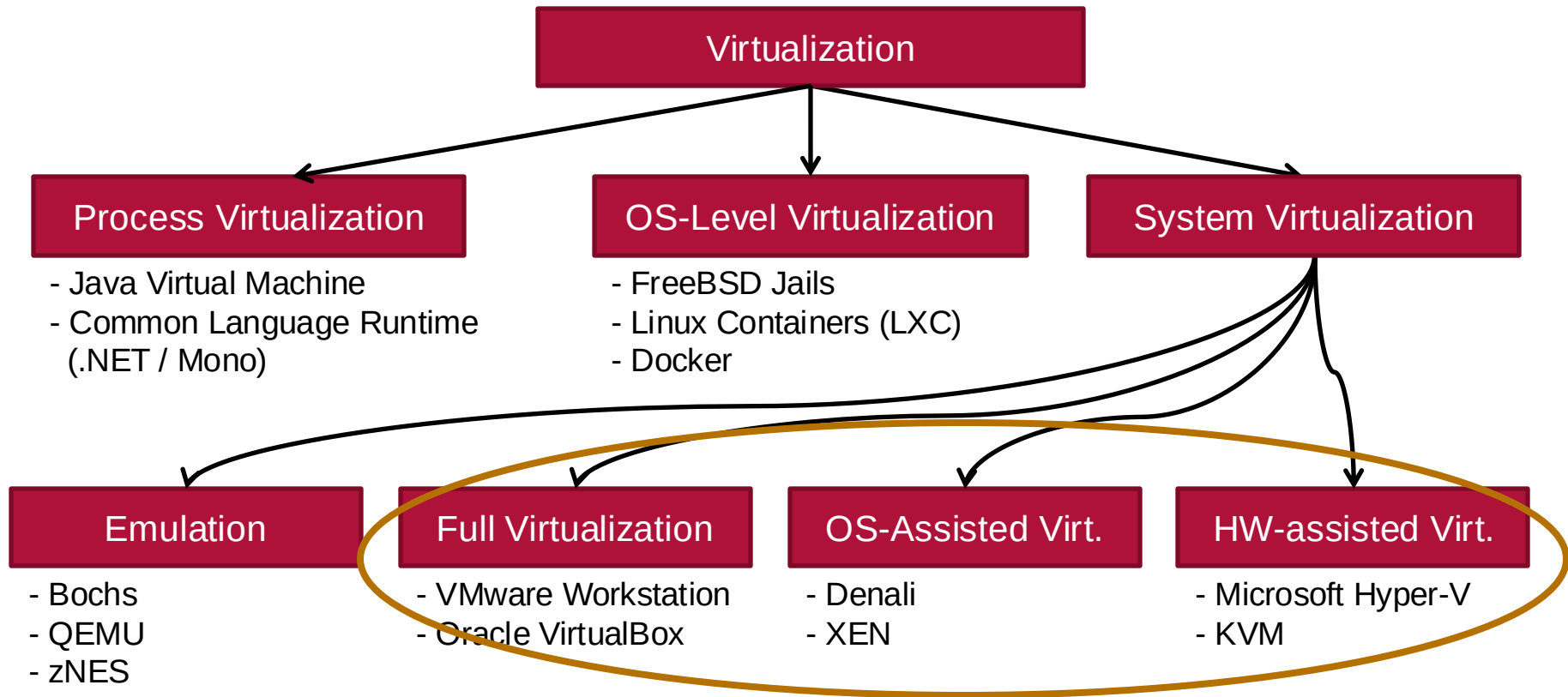


- Isomorphism  $V$ :
  - $S_i, S_j$ : States of machine
  - $e$ : Sequence of operations
- Isomorphism  $V$  maps guest state to host state such that
  - for  $e$  that modifies the guest's state from  $S_i$  to  $S_j$
  - there exists a corresponding sequence of operations  $e'$  that performs an equivalent modification between host's states ( $S'_i$  to  $S'_j$ )

# Categories of Virtualization (1/2)



# Categories of Virtualization (2/2)



Most relevant for Infrastructure as a Services



# Outline of Chapter 1: Cloud Computing Intro

---

1.1 Clusters, Grids, Clouds, and Edge/Fog

1.2 The NIST Definition's Dimensions

— short break (10-15 minutes) —

1.3 Intro to Virtualization

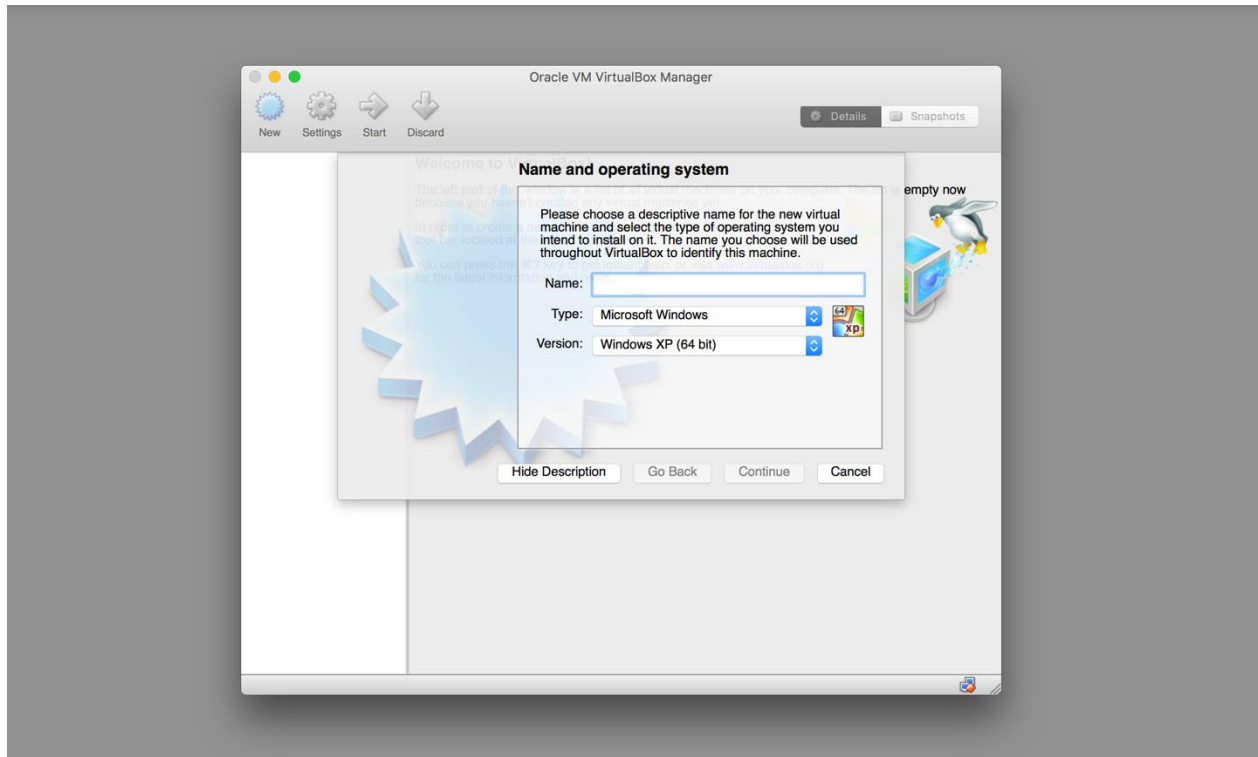
1.4 Intro to VMs and Containers

# Virtual Machines

- Virtual machine (VM): hardware that is used by operating system and running processes is not real, but virtual → OS and processes run on software emulating hardware
- VMs can have different hardware characteristics (including different from the physical host)
  - e.g. number of cores, assigned memory, disk space
- VMs are often started from available images (with an operating system and some applications installed)
  - e.g. Windows with SQL Enterprise

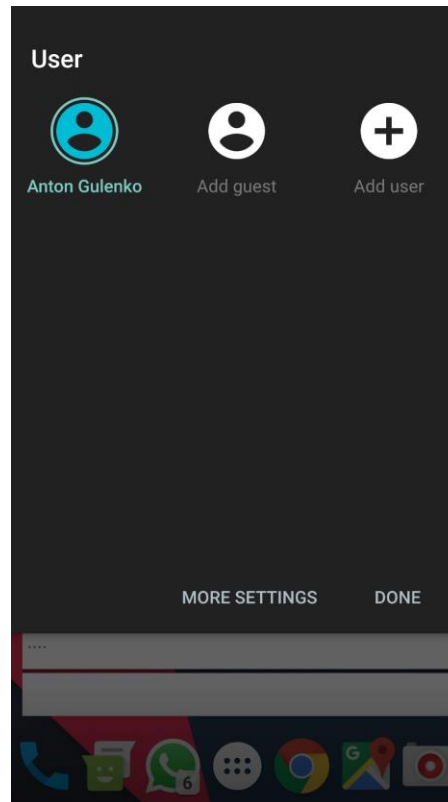
# Use Cases for Virtual Machines (1/4)

- Run a different operating system than installed on the host system



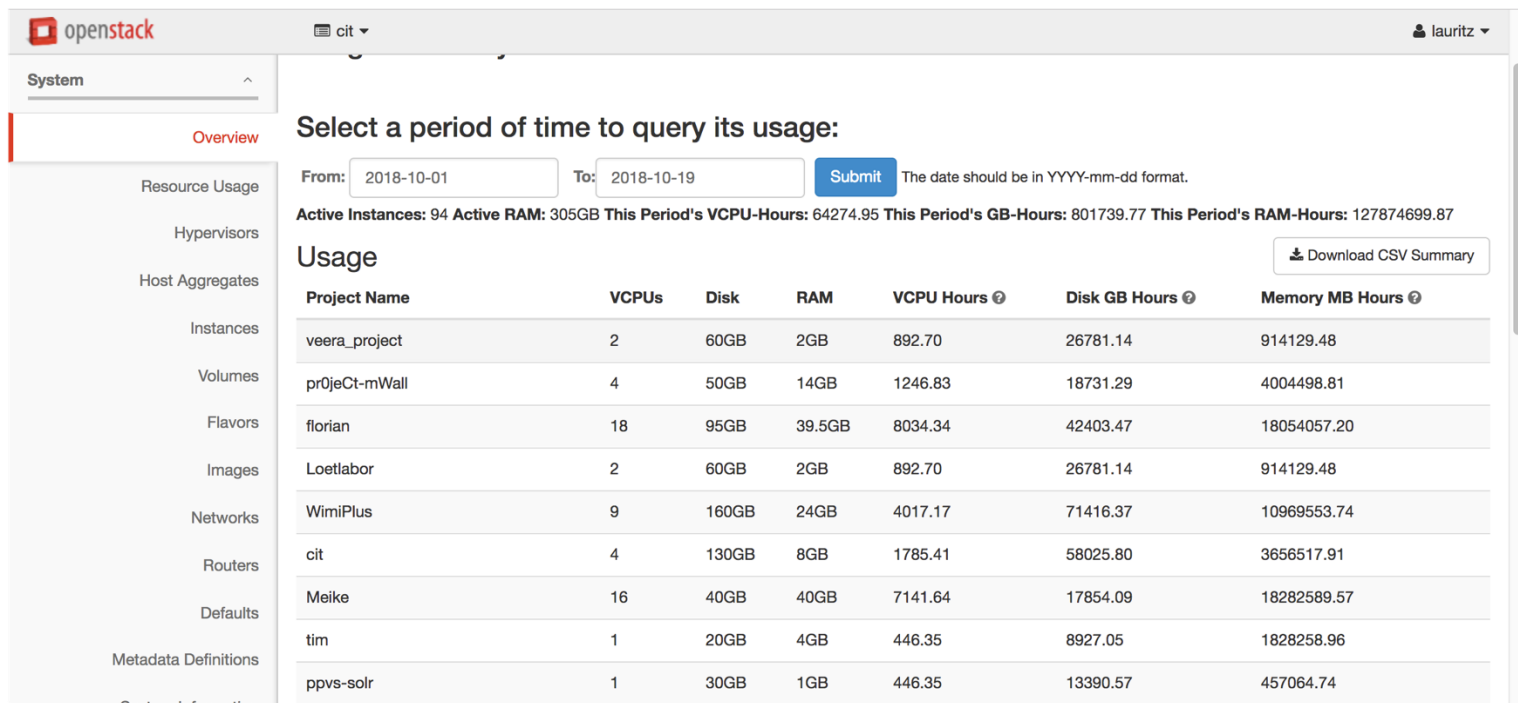
# Use Cases for Virtual Machines (2/4)

- Operating multiple isolated environments on a host (e.g. different user accounts on a phone)



# Use Cases for Virtual Machines (3/4)

- Pool of resources shared by multiple users and applications in a private cloud



The screenshot displays the OpenStack dashboard interface. On the left, a sidebar menu lists various system components: System, Overview, Resource Usage, Hypervisors, Host Aggregates, Instances, Volumes, Flavors, Images, Networks, Routers, Defaults, Metadata Definitions, and System Information. The 'Overview' tab is selected. The main content area shows a summary of resource usage for the 'cit' project. It includes a date range selector (From: 2018-10-01, To: 2018-10-19) and a 'Submit' button. Below this, a summary line states: 'Active Instances: 94 Active RAM: 305GB This Period's VCPU-Hours: 64274.95 This Period's GB-Hours: 801739.77 This Period's RAM-Hours: 127874699.87'. A 'Download CSV Summary' button is also present. The 'Usage' section features a table with the following data:

| Project Name  | VCPUs | Disk  | RAM    | VCPU Hours ? | Disk GB Hours ? | Memory MB Hours ? |
|---------------|-------|-------|--------|--------------|-----------------|-------------------|
| veera_project | 2     | 60GB  | 2GB    | 892.70       | 26781.14        | 914129.48         |
| pr0jeCt-mWall | 4     | 50GB  | 14GB   | 1246.83      | 18731.29        | 4004498.81        |
| florian       | 18    | 95GB  | 39.5GB | 8034.34      | 42403.47        | 18054057.20       |
| Loetlabor     | 2     | 60GB  | 2GB    | 892.70       | 26781.14        | 914129.48         |
| WimiPlus      | 9     | 160GB | 24GB   | 4017.17      | 71416.37        | 10969553.74       |
| cit           | 4     | 130GB | 8GB    | 1785.41      | 58025.80        | 3656517.91        |
| Meike         | 16    | 40GB  | 40GB   | 7141.64      | 17854.09        | 18282589.57       |
| tim           | 1     | 20GB  | 4GB    | 446.35       | 8927.05         | 1828258.96        |
| ppvs-solr     | 1     | 30GB  | 1GB    | 446.35       | 13390.57        | 457064.74         |

# Use Cases for Virtual Machines (4/4)

- Virtual machines available in a public cloud (AWS)

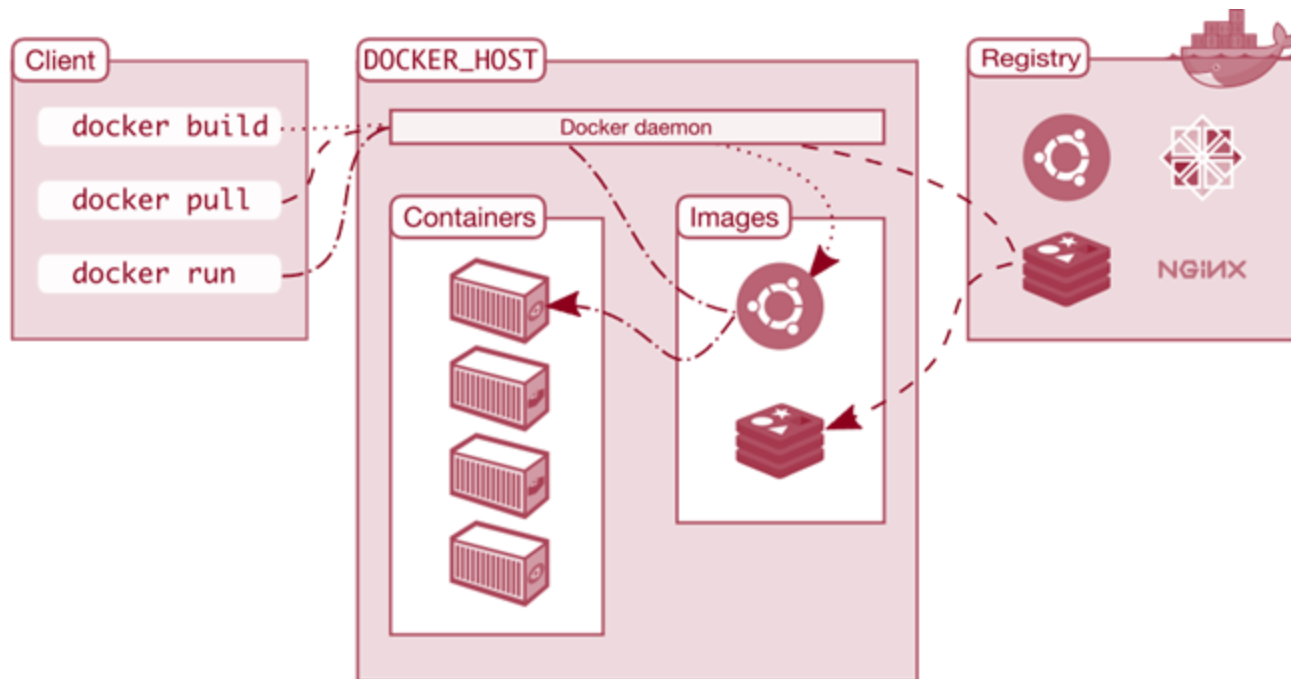
| Linux                                | RHEL                    | SLES               | Windows                   | Windows with SQL Standard | Windows with SQL Web |
|--------------------------------------|-------------------------|--------------------|---------------------------|---------------------------|----------------------|
| Windows with SQL Enterprise          | Linux with SQL Standard | Linux with SQL Web | Linux with SQL Enterprise |                           |                      |
| Region: EU (Frankfurt) ▾             |                         |                    |                           |                           |                      |
|                                      | vCPU                    | ECU                | Memory (GiB)              | Instance Storage (GB)     | Linux/UNIX Usage     |
| General Purpose - Current Generation |                         |                    |                           |                           |                      |
| t3.nano                              | 2                       | Variable           | 0.5 GiB                   | EBS Only                  | \$0.006 per Hour     |
| t3.micro                             | 2                       | Variable           | 1 GiB                     | EBS Only                  | \$0.012 per Hour     |
| t3.small                             | 2                       | Variable           | 2 GiB                     | EBS Only                  | \$0.024 per Hour     |
| t3.medium                            | 2                       | Variable           | 4 GiB                     | EBS Only                  | \$0.048 per Hour     |
| t3.large                             | 2                       | Variable           | 8 GiB                     | EBS Only                  | \$0.096 per Hour     |
| t3.xlarge                            | 4                       | Variable           | 16 GiB                    | EBS Only                  | \$0.192 per Hour     |
| t3.2xlarge                           | 8                       | Variable           | 32 GiB                    | EBS Only                  | \$0.384 per Hour     |
| t2.nano                              | 1                       | Variable           | 0.5 GiB                   | EBS Only                  | \$0.0067 per Hour    |

# Containers

- Lightweight “OS-level virtualization”: virtual environments for single applications → processes run in isolation within an OS
  - Containers rely on containment features of the OS kernel (e.g. the *chroot* and *cgroups* features of Linux)
- Use cases:
  - Isolating applications on the same host from each other (→ resource sharing/pooling in a cluster)
  - Bundling dependencies and the environment of applications (libs and config) to ship software
- Containers can have different sizes, are usually started from images, and can be run on cloud services

# Use Cases for Containers (1/2)

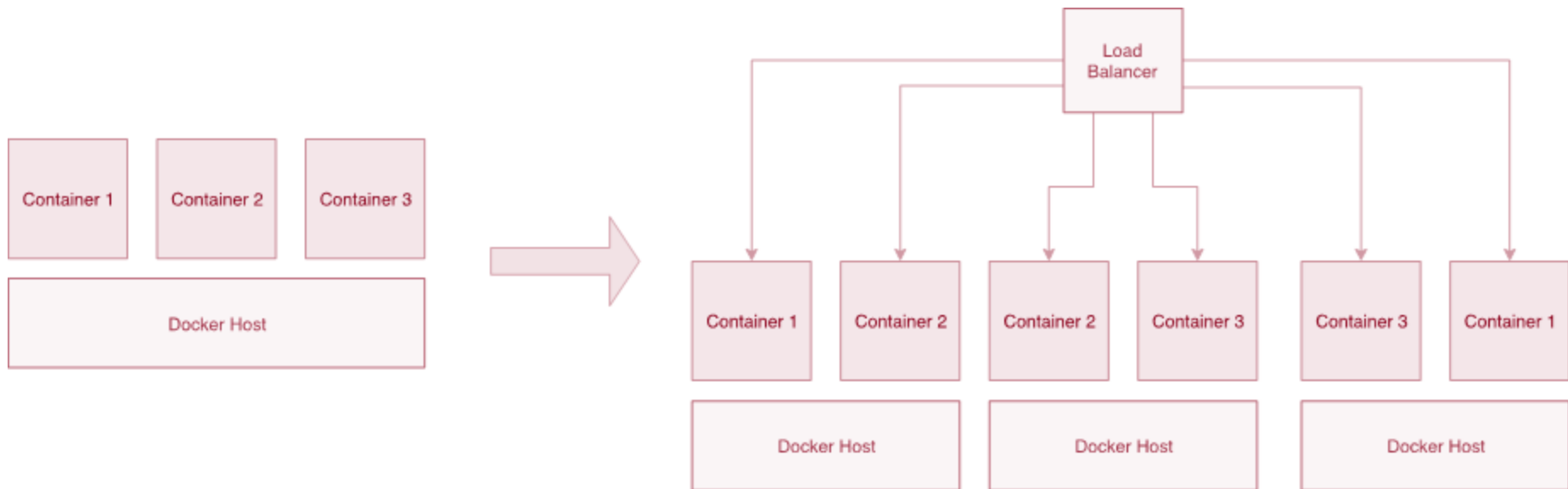
- Build and ship your application in a container to be able to run it everywhere, e.g. development on your host, but then deployment to staging / production





# Use Cases for Containers (2/2)

- Having many applications share the resources of a cluster



# Containers vs. Virtual Machines

- Containers
  - No virtual hardware: cannot run an application build for a different architecture or operating system kernel
  - Reduced scope: single application in pre-build environments
  - Reduced isolation: containerized applications share the same kernel, but are isolated on process-level
  - Live migration of containers is less mature
- VMs
  - Larger images, slower VM startup, and more overhead per VM on a physical node

# (Literature and) References

- References:

- [1] P. Mell, T. Grance: “The NIST Definition of Cloud Computing”, Technical Report, National Institute of Standards and Technology, 2011,  
<https://csrc.nist.gov/pubs/sp/800/145/final>
- [2] K. Scarfone, M. Souppaya, P. Hoffman: “Guide to Security for Full Virtualization Technologies”, 2010,  
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-125.pdf>



# Cloud Systems

## Chapter 2: Virtual Machines

Dr Lauritz Thamsen

[lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)

School of Computing Science

University of Glasgow



# First Half: Lecture Chapters

---

Cloud Resource Management:

1. Cloud Computing Intro
- 2. Virtual Machines**
3. Containers
4. Cloud Infrastructure Management
5. Cloud Sustainability

# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

2.3 OS- and Hardware-Assisted Virtualization

2.4 Resource Isolation and Performance Implications

2.5 Case Study: Amazon EC2 / AWS

# Outline of Chapter 2: Virtual Machines

---

## 2.1 Virtualizability

## 2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

## 2.3 OS- and Hardware-Assisted Virtualization

## 2.4 Resource Isolation and Performance Implications

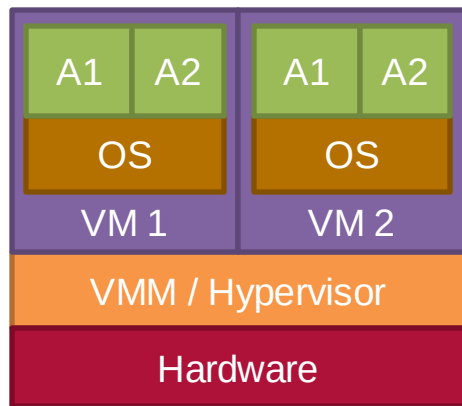
## 2.5 Case Study: Amazon EC2 / AWS

# Basic Designs for System Virtualization

- VMM Type I

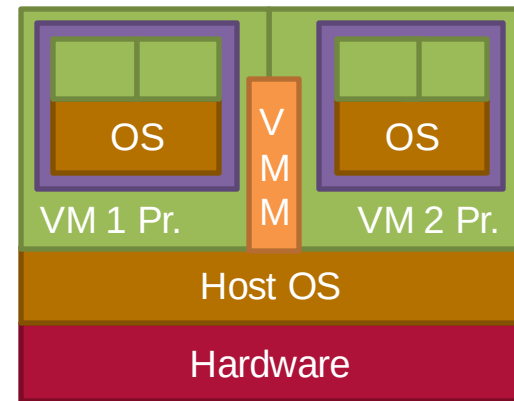
- Directly on hardware
- “Basic OS” to run VMs
- Pro: More efficient
- Con: Requires special device drivers

Also known as  
bare-metal /  
native  
hypervisor  
virtualization



- VMM Type II

- VMM as host OS proc.
- VMs run as processes, supported by VMM
- Pro: No special drivers
- Con: More overhead



Also known as  
hosted  
virtualization



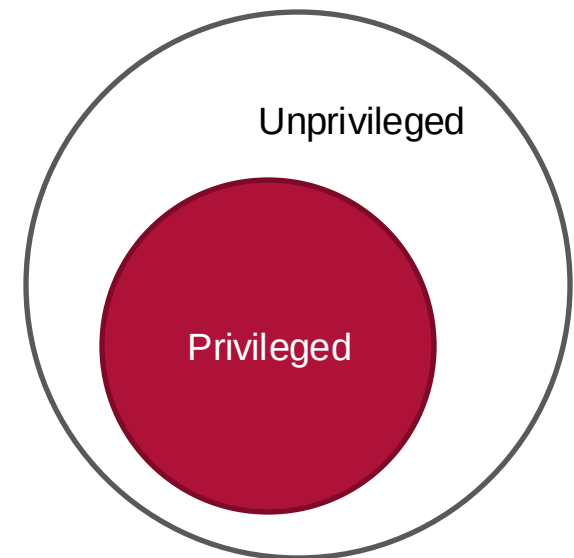
# Virtualizability (VMM Type I)

- Fundamental problem for system virtualization:
  - VMM must have ultimate control over hardware
  - Guest operating system must be disempowered without noticing
- Four assumptions in analysis of Popek and Goldberg [3]
  1. One processor and uniformly addressable memory
  2. Two processor modes: system and user mode
  3. Subset of instruction set only available in system mode
  4. Memory addressing is relative to relocation register

# Recap: Categories of Processor Instructions (1/2)

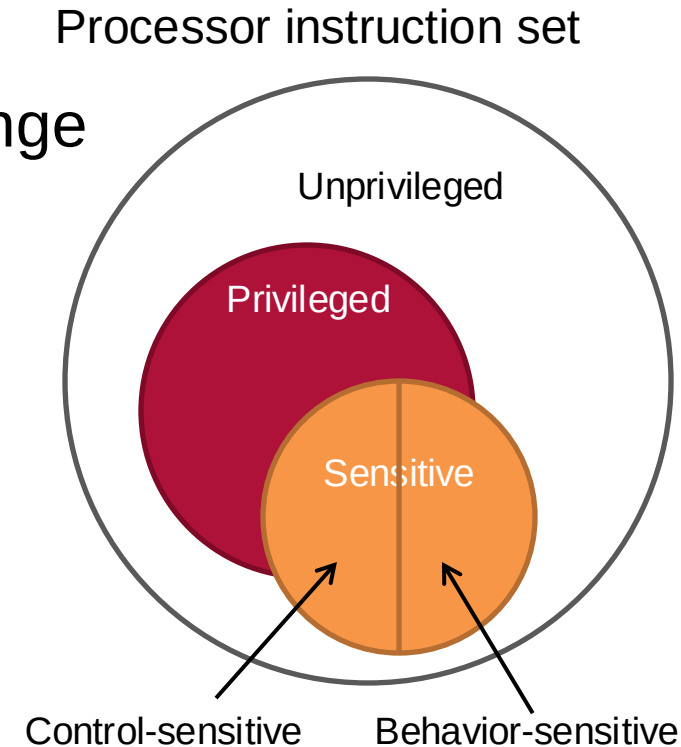
- Privileged instructions
  - Can only be executed in system mode
  - Trap when processor is in user mode
- Examples of privileged instruct.
  - Load PSW (S/370)
    - ◆ One bit to indicate system mode
    - ◆ Malicious program could modify bit
  - Set CPU Timer (S/370)
    - ◆ Defines when user code loses CPU

Processor instruction set



# Recap: Categories of Processor Instructions (2/2)

- Sensitive instructions
  - Control-sensitive instructions: Change configuration of resources
  - Behavior-sensitive instructions: Behave different depending on configuration of resource
- Examples
  - Load Real Address (S/370)
  - Pop Stack into Flags Register (IA-32)

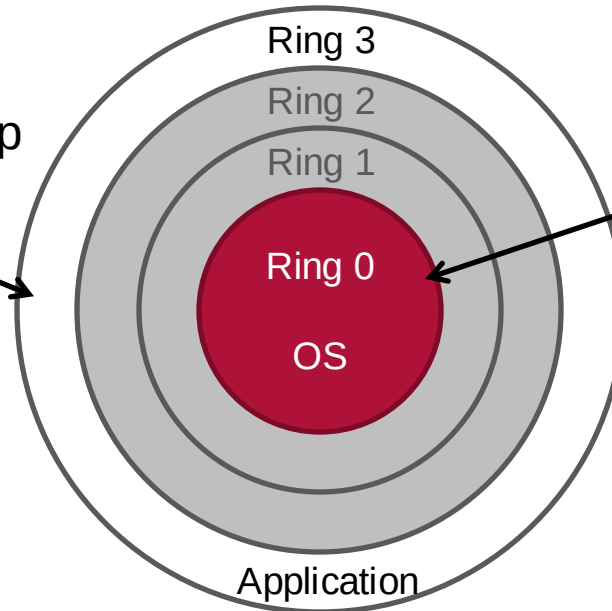


# Recap: IA-32 Architectures

- IA-32 uses rings to manage privileges
  - Four different code privileges possible
  - Designed as generalization of two processor modes
  - For portability, only Ring 0 and 3 used in practice

User space

- Runs applications
- Privileged instructions trap
- Execution of privileged instruction requires syscall



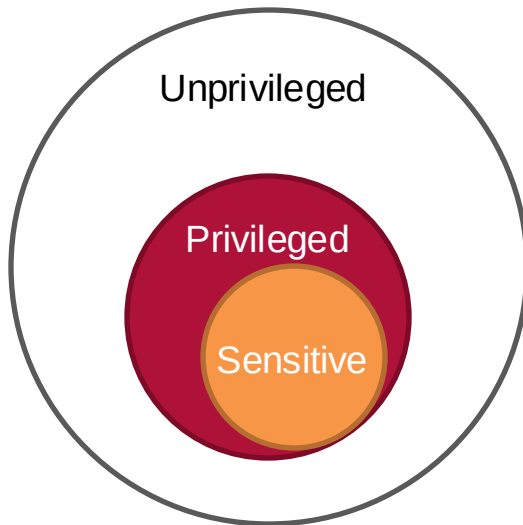
Supervisor mode

- Full control over CPU
- OS runs in this ring
- Equiv. to system mode

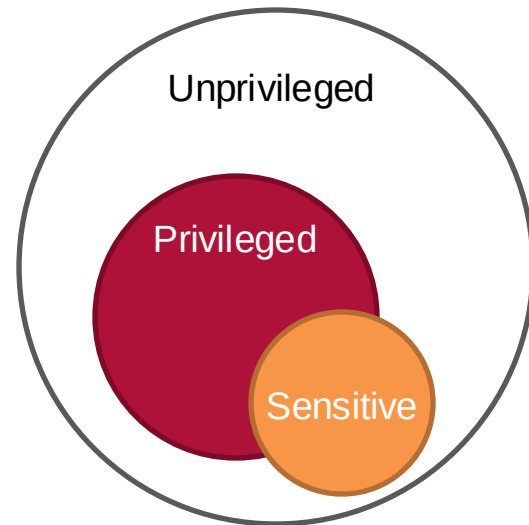
# Popek and Goldberg's Theorem

- Basic condition for the construction of *efficient* VMMs

“... a virtual machine monitor may be constructed if the set of **sensitive instructions** for that computer is a **subset** of the set of **privileged instructions**.”



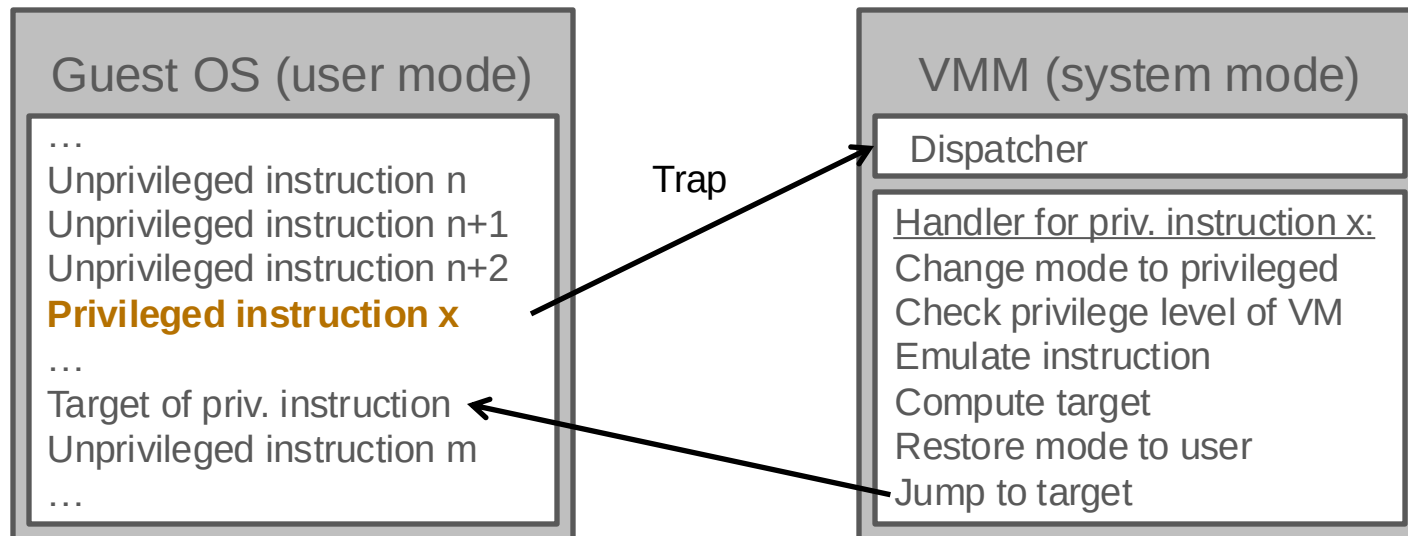
Condition satisfied



Condition unsatisfied

# Implications of Popek and Goldberg's Theorem

- Efficient VMM: All non-sensitive instructions run natively on processor
- *Trap and emulate*: Guest OS calls sensitive instruction
  - Instruction traps
  - VMM emulates instruction operation



# Popek and Goldberg's Requirements in Practice

- Which ISAs satisfy Popek and Goldberg's requirement?
  - IBM Power ✓
  - Sun Sparc ✓
  - Intel IA-32 ✗
    - ◆ ~17 critical instructions (= sensitive but not privileged) [4]
    - ◆ Critical instructions do not trap, but have different semantics if not executed in system mode
  - Arm ✗
- Apparently, virtualization on IA-32 and ARM is possible. So, how?

# Virtualization of IA-32 Architectures

1. Full Virtualization using Binary Translation

2. OS-Assisted Virtualization

3. Hardware-Assisted Virtualization



# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

**2.2 Full Virtualization (with Binary Translation)**

– short break (10-15 minutes) –

2.3 OS- and Hardware-Assisted Virtualization

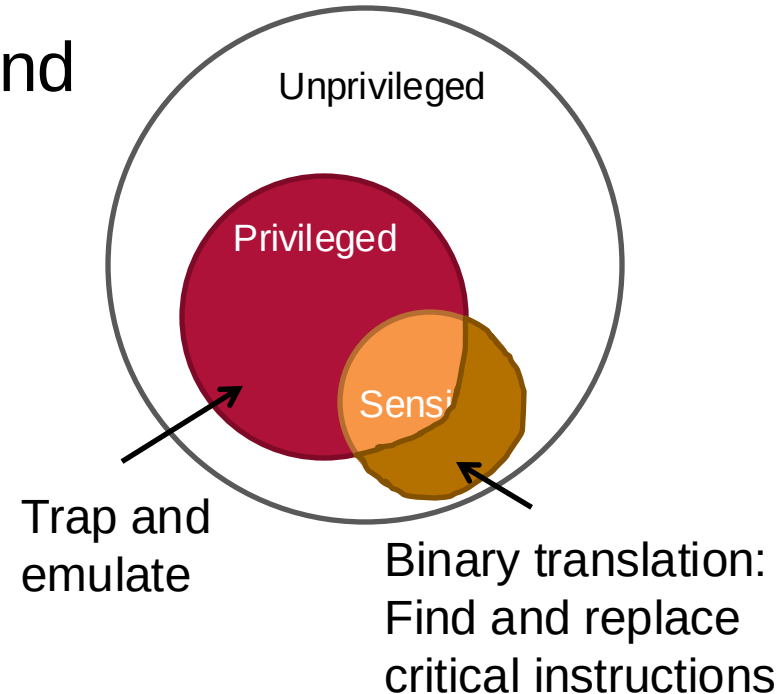
2.4 Resource Isolation and Performance Implications

2.5 Case Study: Amazon EC2 / AWS

# Full Virtualization using Binary Translation

- Idea: Find critical instructions and replace them

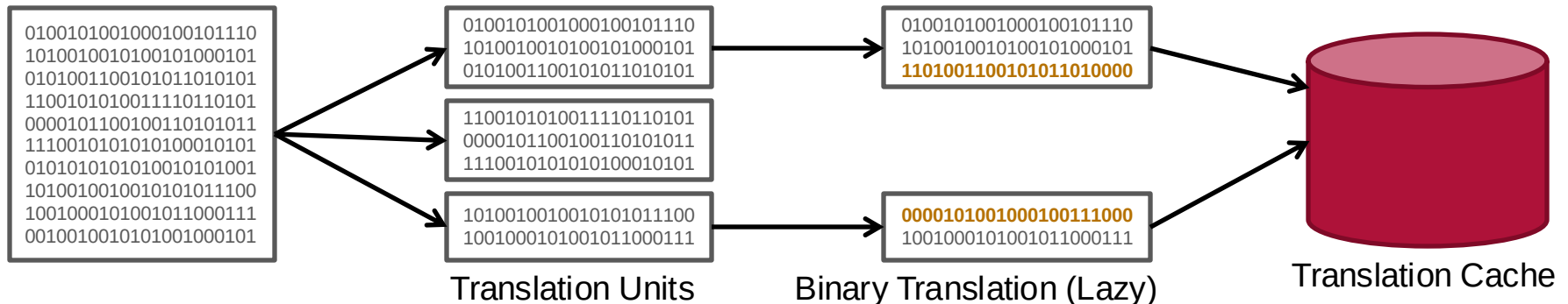
1. Run *unprivileged instructions* directly on CPU
2. Trap and emulate *privileged instructions*
3. Find *critical instructions* and replace with exception



- Whether an instruction is critical or not can depend on parameters used (e.g. LOAD instruction)
  - Replacement must be done at runtime
- Translating a book word for word is inefficient

# Basic Approach for Binary Translation

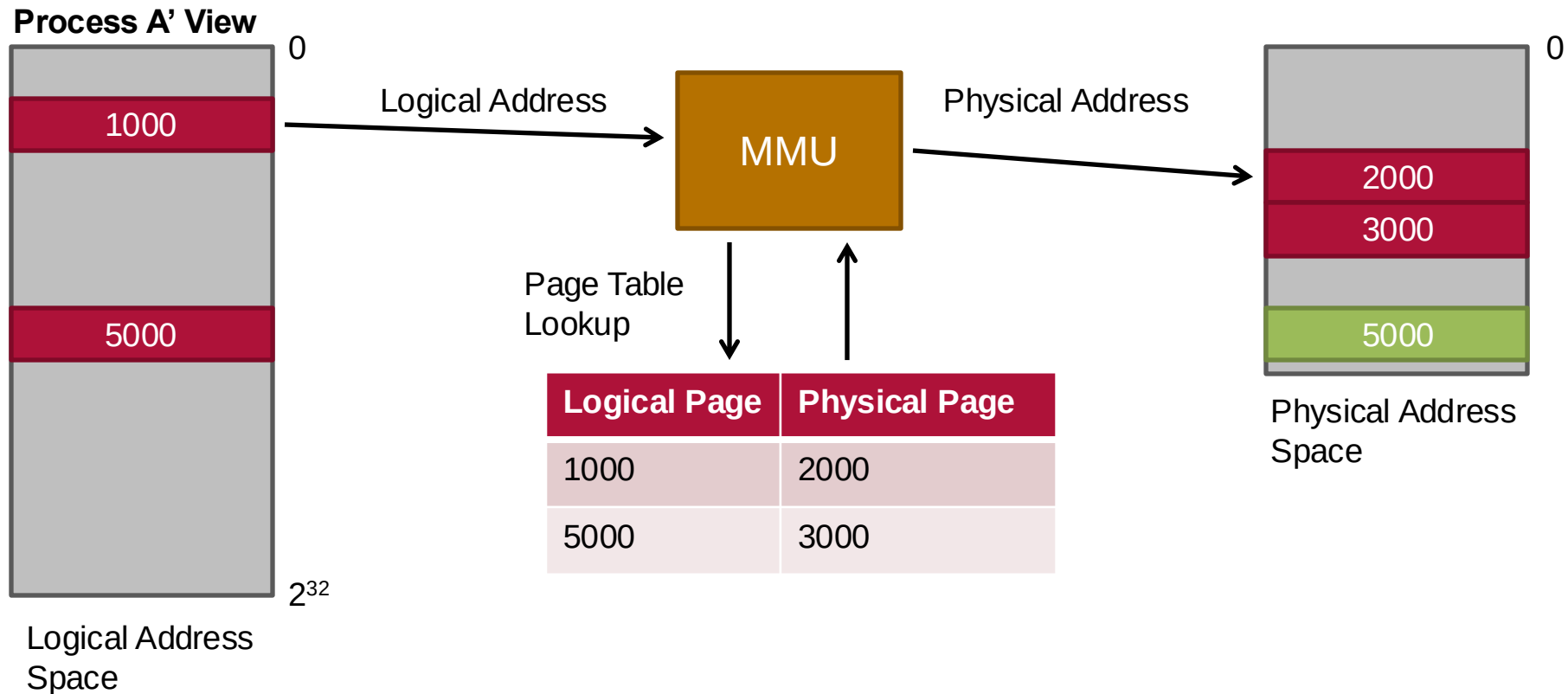
1. Separate instruction sequence into translation units
2. Check units for critical instructions and modify code
3. Modified code is stored in translation cache



- Translation is done lazily
  - Some units may be never translated (e.g. except. handl.)
  - Frequently used units benefit from translation cache

# Recap: Memory Management on IA-32 Architectures (1/2)

- MMU translates logical to physical memory addresses

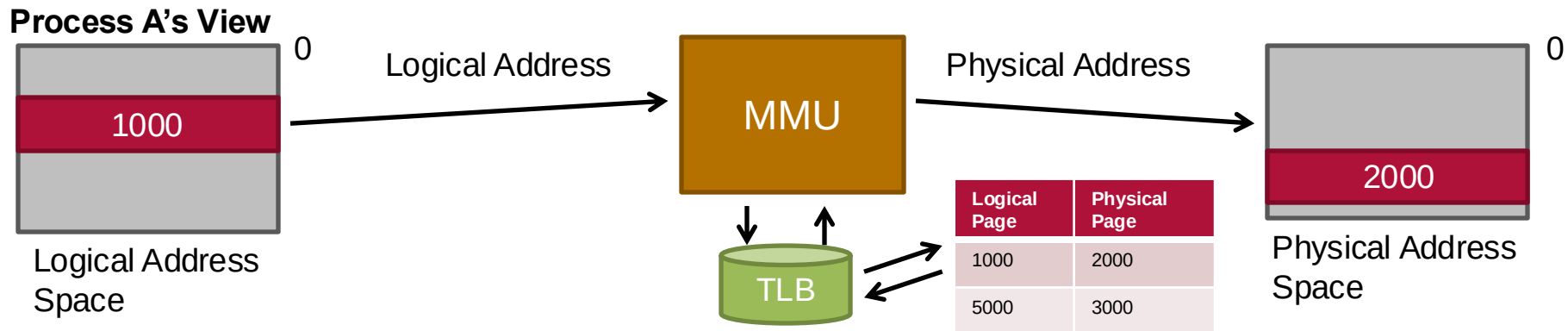


# Recap: Memory Management on IA-32 Architectures (2/2)

- Page tables are part of IA-32 architecture
  - Hardware knows the layout of page tables
  - OS modifies the page table, lookup happens transparently
- Page tables reside in main memory themselves → Overhead of memory access essentially doubles
- Idea: Introduce special hardware-accelerated cache to remember recent address translations  
→ Translation Lookaside Buffer (TLB)

# Recap: Translation Lookaside Buffer (TLB)

- TLB acts as cache of the MMU
  - Typically, really fast (~1 cycle hit time)
  - Typically, really good hit rate (> 99%)

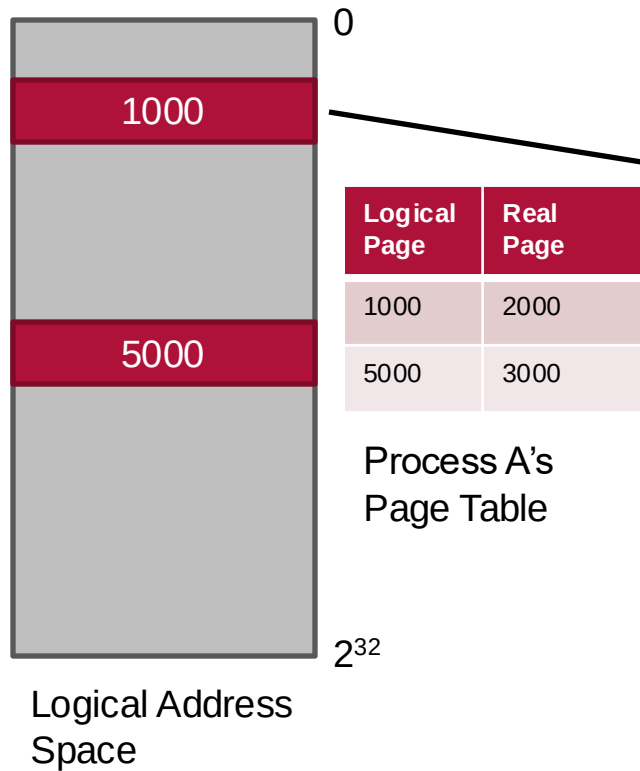


- On IA-32, the TLB is invisible to the operating system
  - Is updated by hardware on every page table lookup
  - It is flushed on every context switch

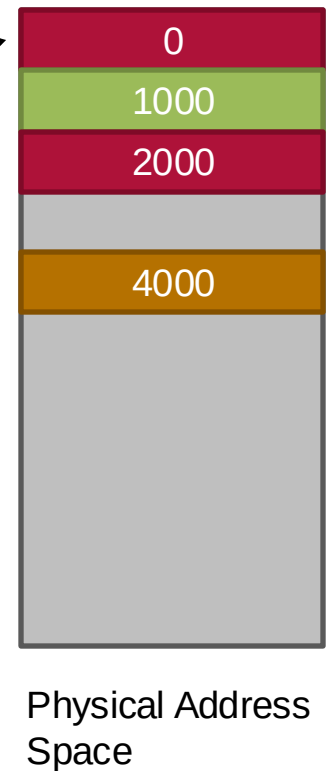
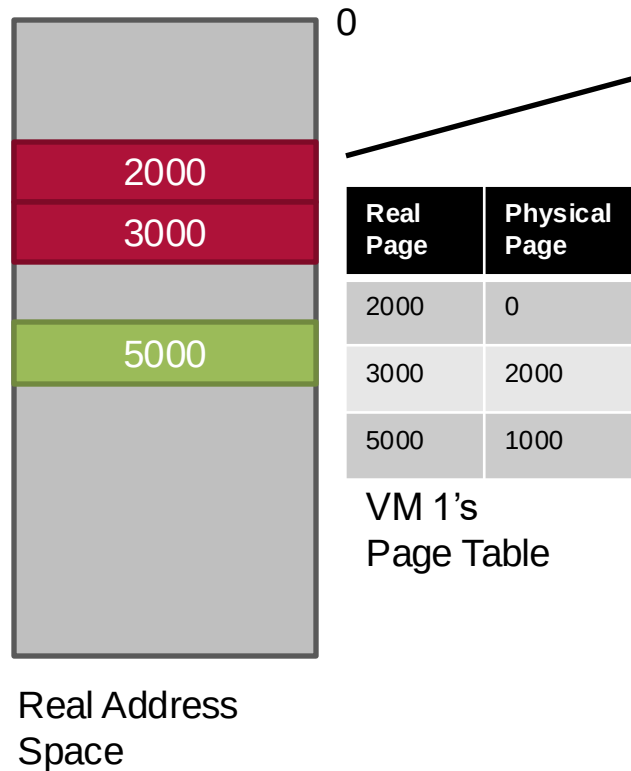
# Memory Management and Full Virtualization (1/2)

- General idea: Add another level of indirection

Process A's View in VM 1



Guest OS's View in VM 1

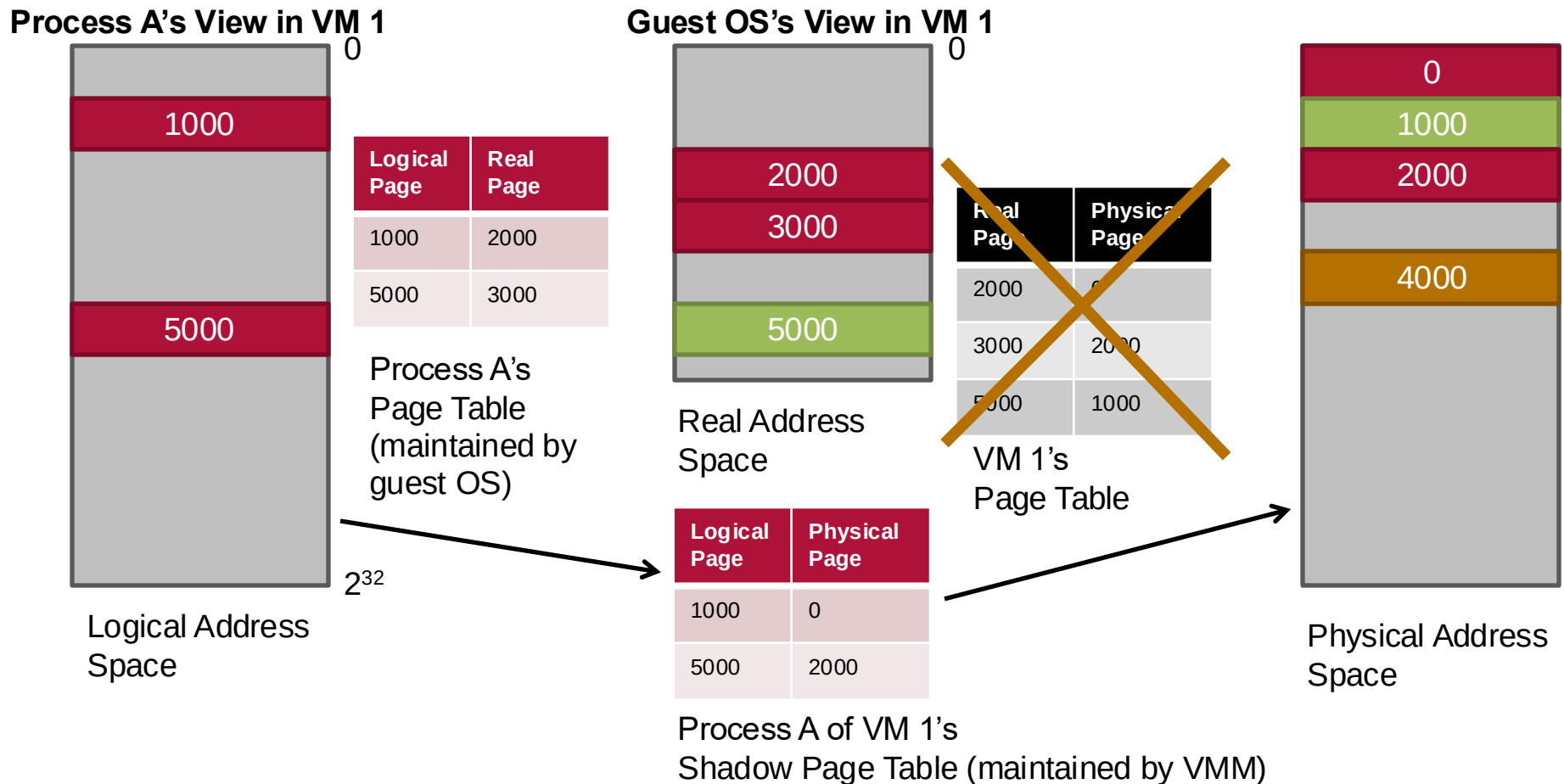


# Memory Management and Full Virtualization (2/2)

- Problem: Additional memory access required to resolve address → significant performance decrease
- Practical implementation: *Shadow page tables*
  - Guest OSs maintain own page tables (for compatibility)
  - But modifications to guest's page table trap and entries are copied to the VMM's shadow page table
  - Shadow page table is then used by hardware
    - ◆ Keeps TLB up-to-date
    - ◆ Works through virtualization of page table pointer

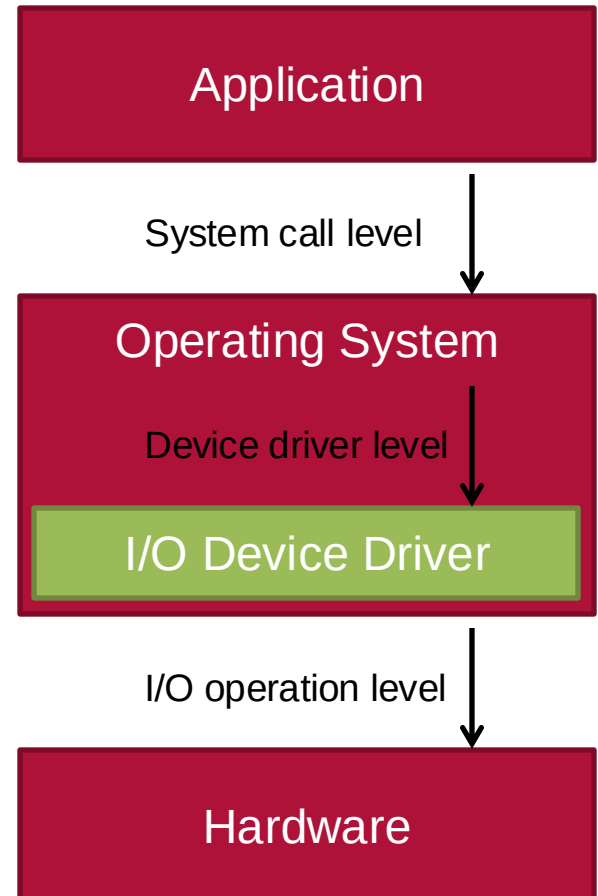


# Memory Virtualization with Shadow Page Tables



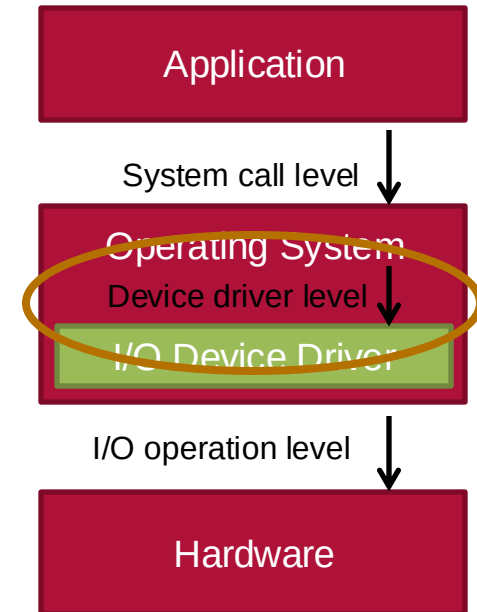
# Full Virtualization and I/O

- Different levels of I/O virtualization possible
  1. At system call level
  2. At device driver level
  3. At I/O operation level



# I/O Virtualization at Device Driver Level

- VMM intercepts calls to virt. device driver
  - Converts virtual device information to corresponding physical device
  - Redirects calls to physical device's driver program
- Pro: Natural point for virtualization
  - No “reverse engineering” required
- Con: Requires knowledge of guest's device driver interface
- ➔ Works in practice (e.g. for Linux and Windows guests)



# Summary Full Virtualization with Binary Translation

- Requires modified guest OS? **NO**
- Requires hardware support? **NO**
- Performance
  - Fine for compute-intensive applications
    - ◆ Unprivileged instructions run directly on CPU
  - Decreased performance for data-intensive applications
    - ◆ I/O requires syscalls → privileged instructions & traps
    - ◆ “trap and emulate” often requires context switches
    - ◆ Context switches lead to flush of TLB etc.

# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

**– short break (10-15 minutes) –**

2.3 OS- and Hardware-Assisted Virtualization

2.4 Resource Isolation and Performance Implications

2.5 Case Study: Amazon EC2 / AWS

# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

**2.3 OS- and Hardware-Assisted Virtualization**

2.4 Resource Isolation and Performance Implications

2.5 Case Study: Amazon EC2 / AWS

# OS-Assisted Virtualization

- Idea of OS-assisted virtualization
  - Make guest OS aware that it is running in a VM
  - Modify the guest source code so that it avoids assistance of the VMM as far as possible
- Denali project also coined term *paravirtualization* [7]
- Today, most virtualization platforms are aware of using virtual devices (→ OS-assisted virtualization for I/O)
- Requirements for (pure) OS-assisted approach
  - Source code of guest operating system is available
  - Modified guest OS maintains application binary interface

# OS-Assisted Virtualization in Practice: The Example of Xen

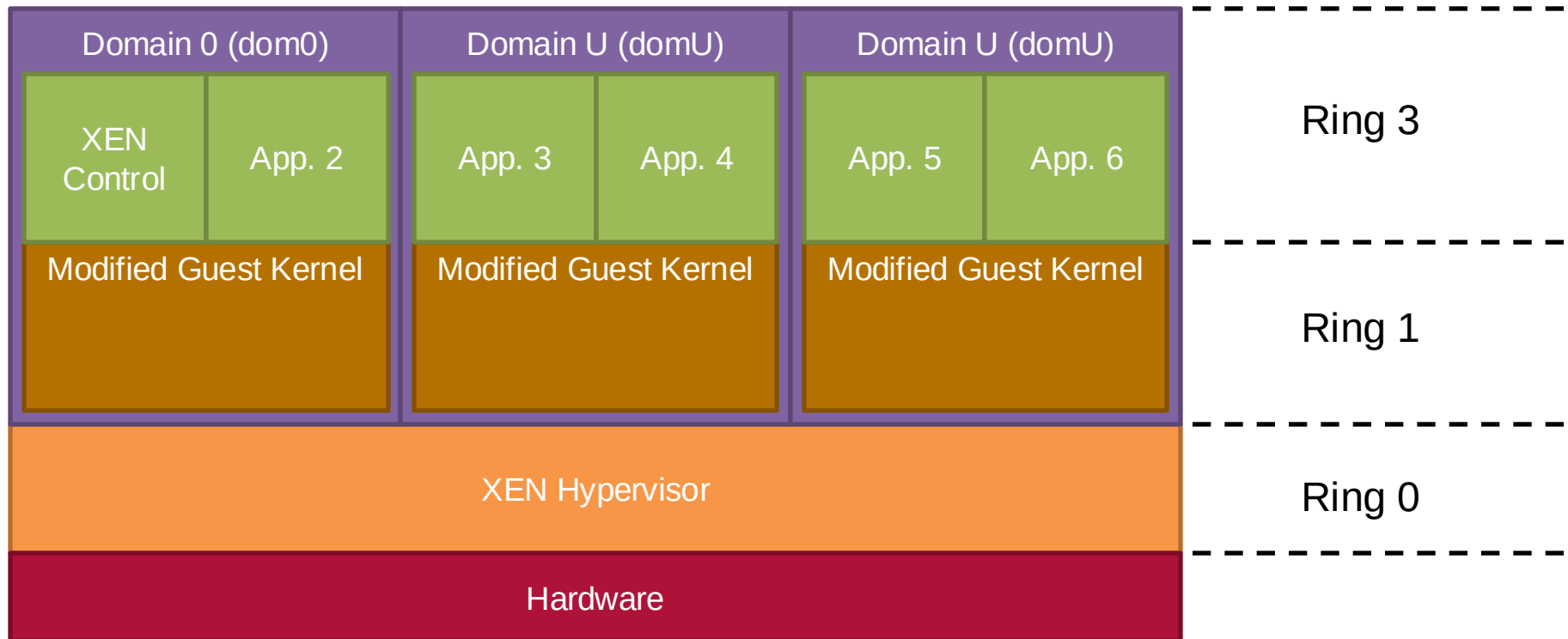
- Classic representative for OS-assisted virtualization: XEN [8]
  - Type I Hypervisor
  - Available as open-source software
  - Originally developed at the University of Cambridge, UK, in collaboration with Microsoft Research Cambridge
- Prime example for paravirtualization, but can also be used to run unmodified guest kernels using HW-assisted virtualization





# XEN Architecture and Domains

- Domain 0: Privileged guest for control/management
- Domain U: Guest with XEN-enabled OS

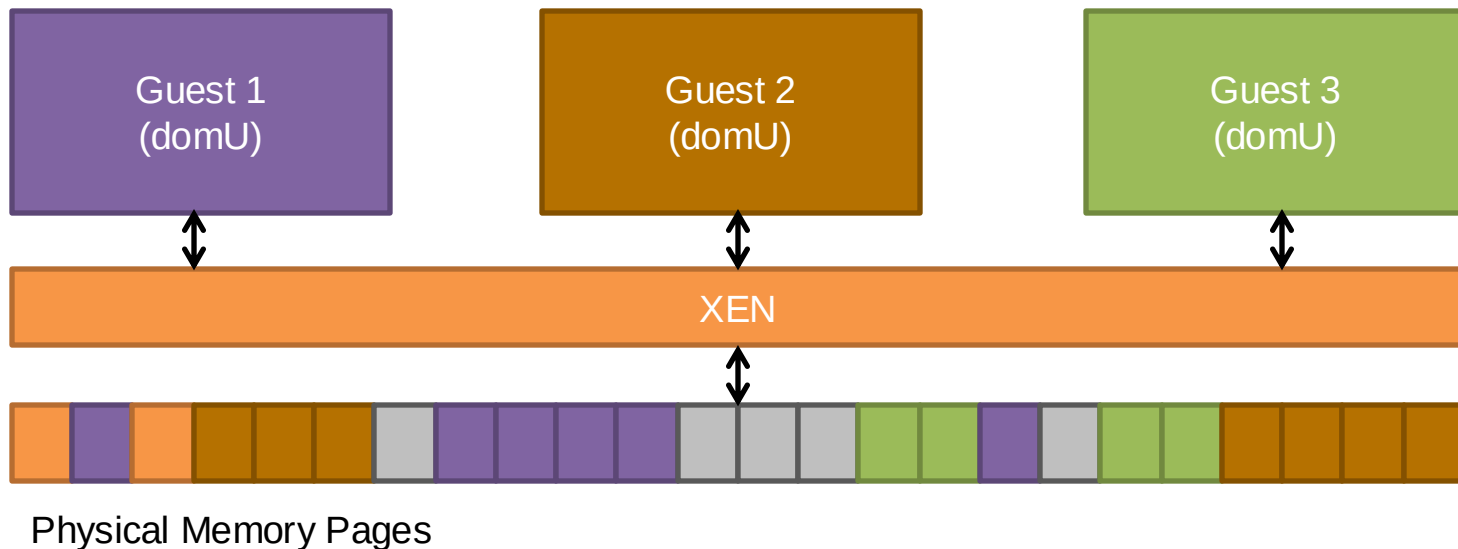


# XEN and CPU Virtualization

- Critical instructions do not trap on IA-32
  - Guest OS is aware of virtualization → Critical instructions can be avoided
- Still, frequent VMM intervention required, e.g. for system calls and page table updates
- However, XEN minimizes frequencies and costs
  - Some system calls to guest OS (Ring 1) without VMM intervention (registering guest OS handlers with Xen)
  - Mapping hypervisor code into process address spaces (so hypercalls do not require context switches)
  - Command batching (e.g. subsequent page table updates)

# XEN and Physical Memory

- Domain gets fraction of phys. memory at creation time
  - Static partitioning among domains
  - No guarantee that partition is contiguous
  - Hypervisor knows which domain „owns“ which pages

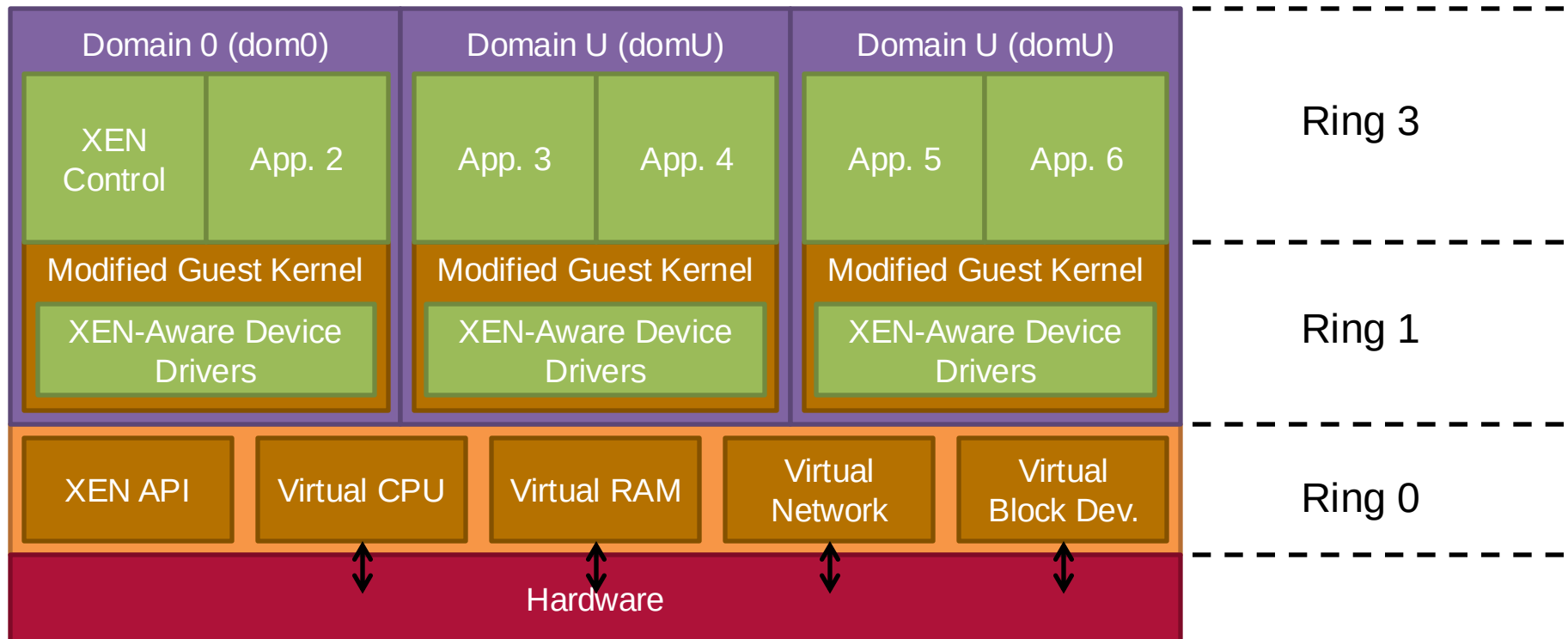


# XEN and Memory Virtualization

- XEN lets guests maintain their own page tables
  - Prerequisite: Guest OS knows its fraction of physical memory
  - Guest page tables are used by the guest's MMU
  - XEN validates page table updates to ensure isolation
  - No need for hypervisor intervention on reads
- Procedure for writes
  1. Guest requests page table update via hypercall
  2. XEN checks if mapping address belongs to domain
  3. If okay, allows update to page table

# XEN and I/O Virtualization

- XEN presents virtual devices, domains use lightweight virtual device drivers, and physical device drivers reside in dom0
- Communication between XEN and domains:
  - Hypercall: Synchronous call from domain to XEN
  - Event: Asynchronous notification from XEN to domain



# Summary OS-Assisted Virtualization

- Requires modified guest OS? **YES**
- Requires hardware support? **NO**
- Pros:
  - Better performance through cooperation between hypervisor and guest OS
- Cons:
  - Limited compatibility, not generally applicable
  - Increased management overhead for data center operator as different version of OS must be maintained

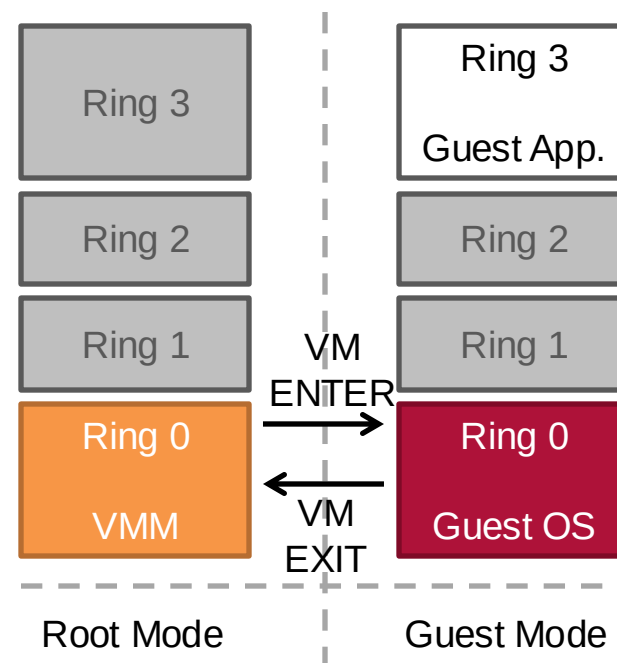
# Hardware-Assisted Virtualization

- Most virtualization difficulties caused by IA-32 design
  - Sensitive instructions do not always trap in rings  $> 0$
  - Guests can observe they are not running in Ring 0
- Success of VMware has demonstrated demand for virtualization
- Idea: Extend IA-32 architecture to circumvent virtualization obstacles on the hardware level
  - Independent developments by Intel and AMD
  - Yet, developments share same basic ideas



# CPU Virtualization Extensions

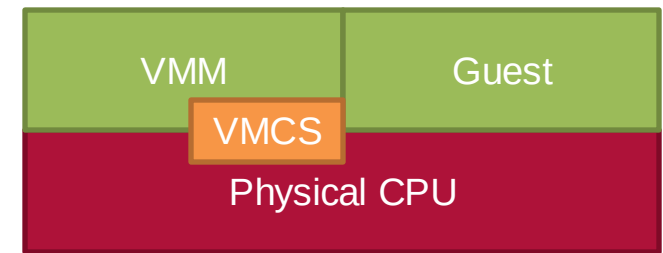
- Two new CPU modes: root mode vs. guest mode
  - VMM runs in root mode
  - Guest OS in guest mode
- VMM and guest run as “co-routines”
  - VMM can give CPU to guest OS (VM ENTER)
  - VMM can define conditions when to regain CPU (VM EXIT)





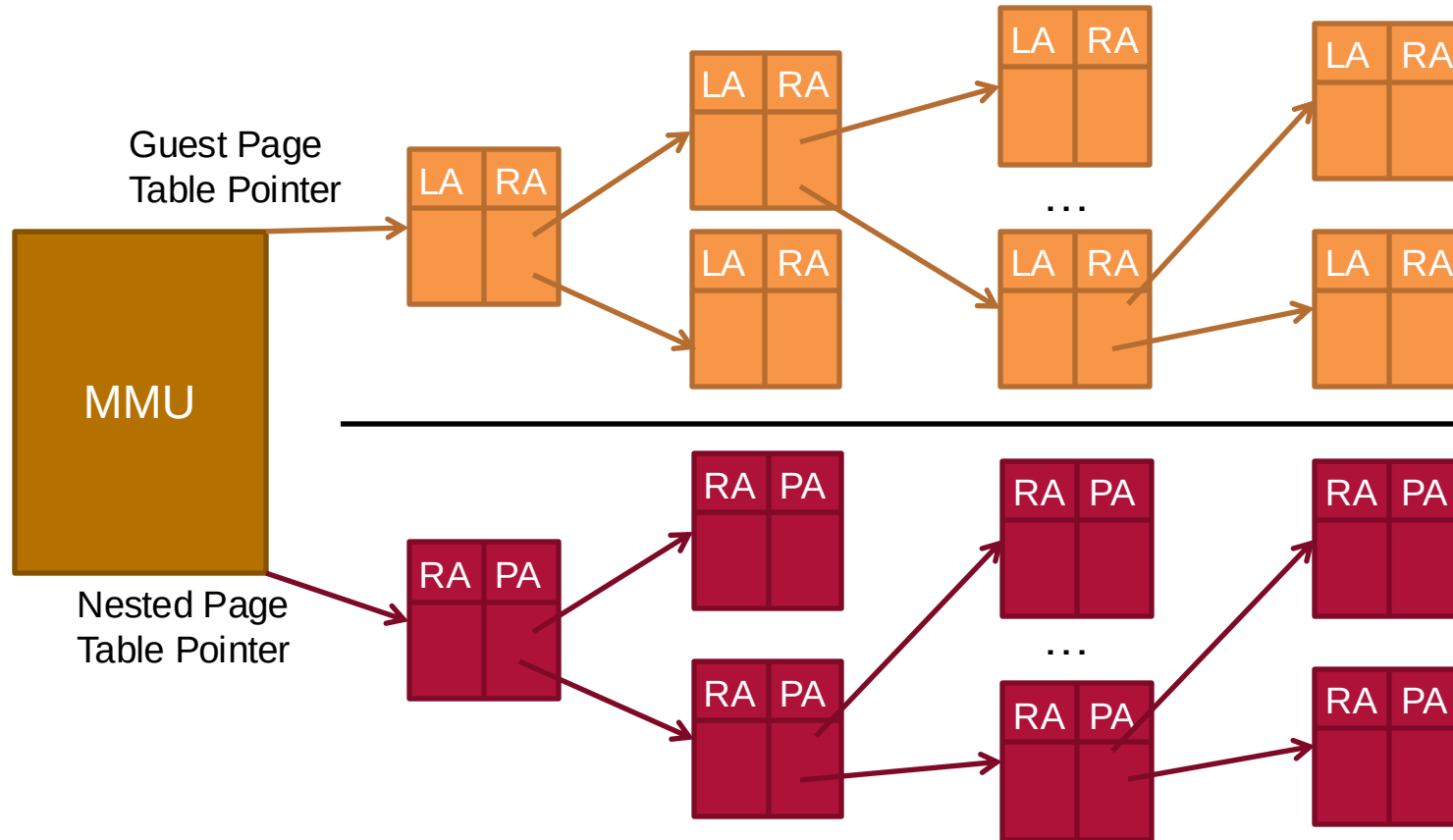
# VMM Control Structures [13]

- VMM controls guest through HW-defined structure
  - Intel: VMCS (virtual machine control structure)
  - AMD: VMCB (virtual machine control block)
- VMCS/VMCB contains
  - Guest state
  - Control bits defining criteria for VM EXIT
    - ◆ Exit on IN, OUT, CPUID, ...
    - ◆ Exit on write to page table register, ...
    - ◆ Exit on page fault, interrupt, ...
  - VMM uses control bits to “confine” and observe guest



# Memory Virtualization Extensions

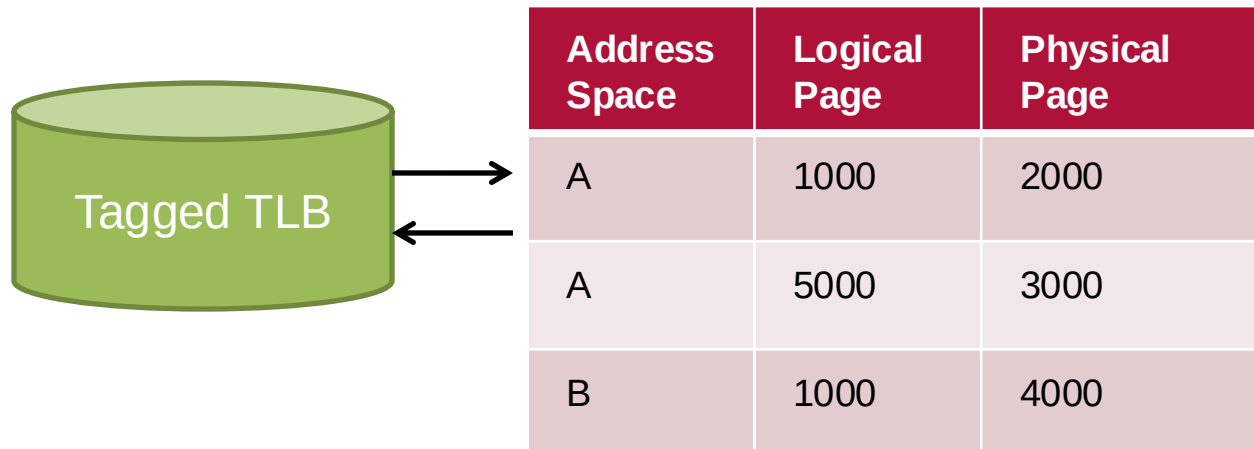
- Extended Page Tables/Nested Page Tables



LA: Logical Address, RA: Real Address, PA: Physical Address

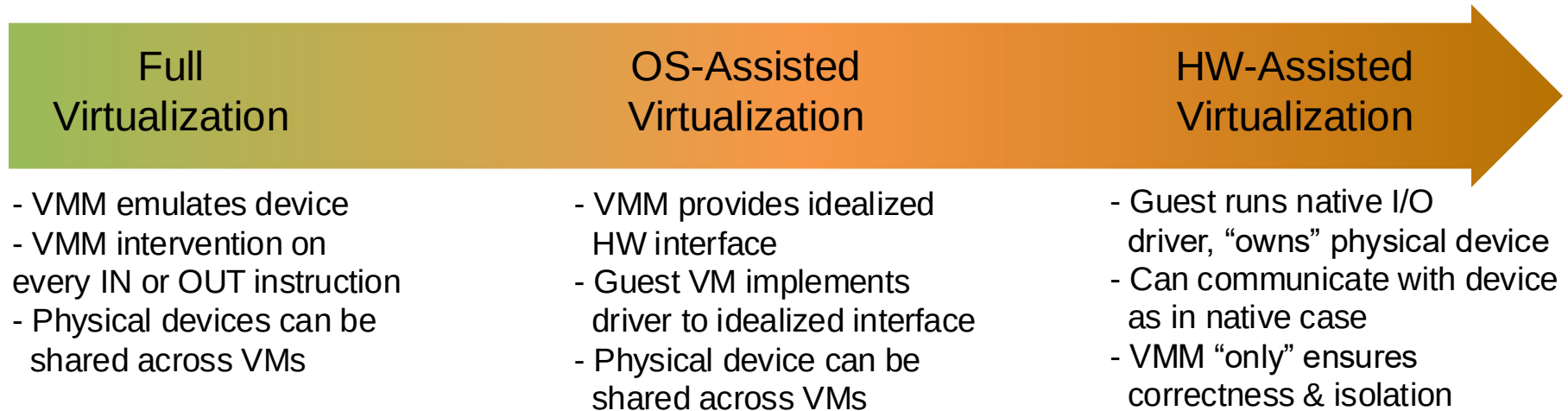
# Tagged Translation Lookaside Buffer

- Translation lookaside buffer continues to cache LA → PA address translation
- Both Intel and AMD introduced tagged TLBs
  - Every TLB entry associated with address space tag
  - Only some entries are invalid on context switch!



# Focus of HW-Support for I/O: Direct Assignment

- Direct assignment: Guest VM owns a physical device
  - No sharing of device between several VMs
  - Guest VMs run the unmodified device drivers
  - Goal: Efficient I/O without VMM intervention
  - Challenge: VMM must still ensure correctness & isolation



# Summary HW-Assisted Virtualization

- Requires modified guest OS? **NO**
- Requires hardware support? **YES**
- Pros:
  - Improved performance even for unmodified guest OSs
- Cons:
  - Specialized hardware required and reduced flexibility due to hardware constraints

# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

2.3 OS- and Hardware-Assisted Virtualization

**2.4 Resource Isolation and Performance Implications**

2.5 Case Study: Amazon EC2 / AWS

# Resource Sharing & Performance Implications

---

- In commercial IaaS clouds, many VMs often run on the same physical hardware
- Main questions:
  - How are resources of a physical node shared?
  - How is fairness enforced?
  - What are the implications of resource sharing?

# Resource Distribution Among VMs

- Storage space: statically partitioned
  - Each VM typically receives predefined fraction of disk
- Main memory: statically partitioned
  - Each VM typically receives predefined fraction of RAM
- CPU: Different methods possible
  - Pinning: Each VM is statically assigned CPU cores
  - Scheduling: VMM dynamically assigns time slots to VMs
- I/O Access: Typically, FCFS
  - More sophisticated methods subject to research



# CPU Scheduling

- Goals of the schedulers
  - Each VM supposed to receive “fair” share of the CPU
  - High CPU utilization
  - Low response times
- Different algorithms available for e.g. XEN, including
  - a general-purpose scheduler
  - a scheduler for latency-sensitive jobs
  - several experimental real-time schedulers
  - ...

# CPU Scheduling and Shared I/O

- Currently, VM scheduling focuses on CPU
- Results in good fairness/response times for compute-intensive applications
- However, processing of I/O requests by VMM also consumes CPU time
  - In particular when I/O is bursty
  - Guest OSs unaware of that source of processing delay
  - Perceived as high delay variations by the guests and negatively impacts performance

# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

2.3 OS- and Hardware-Assisted Virtualization

2.4 Resource Isolation and Performance Implications

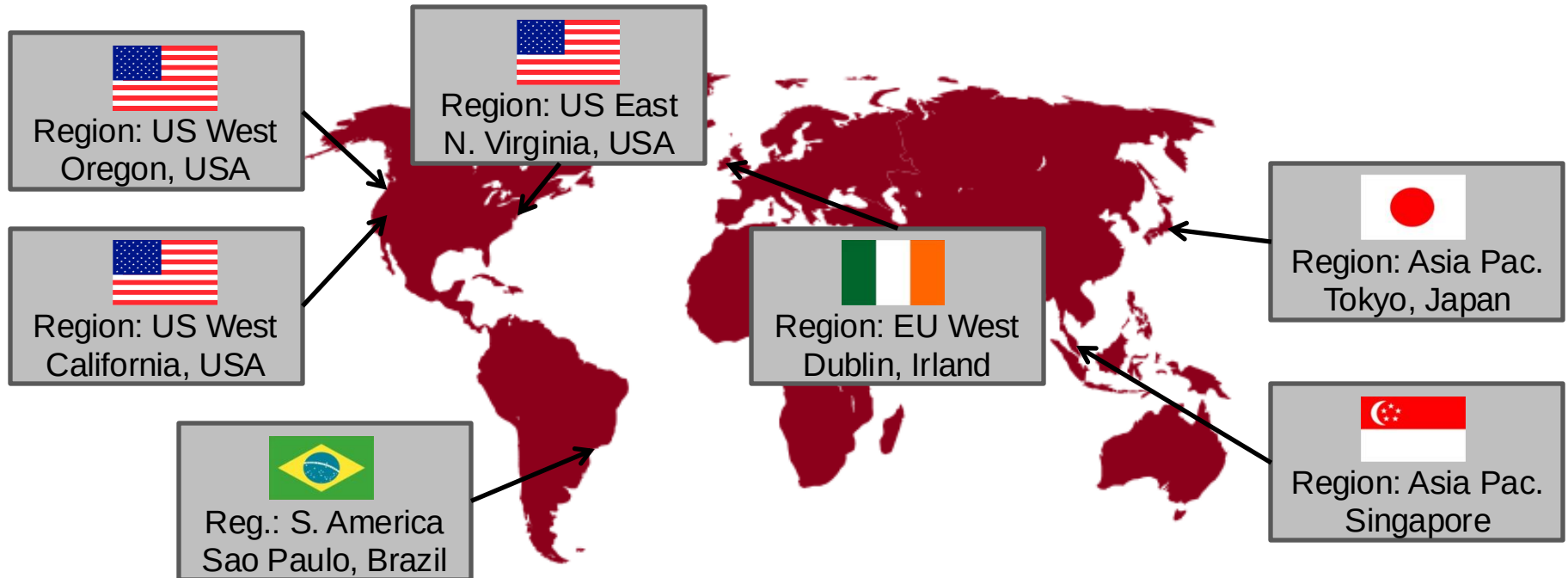
**2.5 Case Study: Amazon EC2 / AWS**

# Amazon Elastic Compute Cloud (EC2)

- Public IaaS cloud by Amazon Web Services (AWS)
  - Subsidiary of Amazon.com, Inc.
  - Launched in 2006
  - Most used cloud platform today
- AWS encompasses large number of service, e.g.
  - Computing: EC2, Lambda
  - Storage: S3, ECB
  - Databases: RDS, DynamoDB
  - Analytics: EMR, SageMaker



# Geographic Distribution of EC2 Data Centers



- Amazon calls each geographic location a *region*
  - Regions are again divided into *availability zones*
    - ◆ Intra-availability zone traffic is free of charge
    - ◆ Per-GB fee for inter-availability zone/inter-region traffic

# EC2 Per-Hour Pricing Model

- Per-time pricing model (hence the term “Elastic”)
  - Amazon charges fee for each started hour of VM usage
  - Customer can shutdown VMs at anytime
  - No long-term obligations, reduced risk of over-/under-provisioning
- Concrete per-hour cost depends on several factors
  - Region
  - Virtual machine type (EC2 calls those instance types)
  - Operating system, image (possible license costs)
  - Usage of external services (EBS, Internet traffic, ...)

# EC2 Instance Types (1/2)

- Instance types define VM classes with particular hardware characteristics

| Instance      | t2 Small                     | t2 Medium                     | t2 Large                       | t2 xLarge                      |
|---------------|------------------------------|-------------------------------|--------------------------------|--------------------------------|
| Compute power | 1 virtual core, 1 comp. unit | 1 virtual core, 2 comp. units | 2 virtual cores, 4 comp. units | 4 virtual cores, 8 comp. Units |
| Main memory   | 2 GiB                        | 4 GiB                         | 8 GiB                          | 16 GiB                         |
| Storage       | via EBS                      |                               |                                |                                |
| Platform      | 32/64-Bit                    | 32/64-Bit                     | 64-Bit                         | 64-Bit                         |
| Price         | USD 0.02                     | USD 0.05                      | USD 0.09                       | USD 0.19                       |

Example from region US East (North Virginia), Linux/UNIX usage, general purpose

# EC2 Instance Types (2/2)

- “EC2 Compute Unit”: Abstract unit for compute power
  - One compute unit corresponded to a 1.0-1.2 GHz AMD Opteron or Intel Xeon of 2007
  - Introduced as a reference measure, allowing predictability, with different generations of HW inside data centers
- Schad et al. examined EC2 performance variance [18]
  - Instances of the same type may be hosted on different generations of hardware
  - ➔ Significant performance variations across different instances of the same type are possible!



# Outline of Chapter 2: Virtual Machines

---

2.1 Virtualizability

2.2 Full Virtualization (with Binary Translation)

– short break (10-15 minutes) –

2.3 OS- and Hardware-Assisted Virtualization

2.4 Resource Isolation and Performance Implications

2.5 Case Study: Amazon EC2 / AWS

# Summary: Virtual Machines

---

- IaaS clouds let consumers rent basic IT resources in form of virtual machine
  - Customers have full control over OS and deployed applications in VMs
- System virtualization as the enabling method
  - Several customers can share physical infrastructure
  - Different techniques to realize system virtualization
  - Performance overhead depends on specific virtualization technique, application, and hardware

# References

- Andrew S. Tanenbaum, Herbert Bos: Modern Operating Systems, Pearson, 2015
  - William Stallings: Operating Systems – Internals and Design Principles, 2015
- 
- [3] G.J. Popek and R.P. Goldberg: “Formal Requirements for Virtualizable Third Generation Architectures”, Communications of the ACM, 17 (7), 1974
  - [4] J.S. Robin, C.E. Irvine: “Analysis of the Intel Pentium’s Ability to Support a Secure Virtual Machine Monitor”, Proc. of the 9th Conference on USENIX Security Symposium, 2000
  - [6] K. Adams, O. Agesen: “A Comparison of Software and Hardware Techniques for x86 Virtualization”, Proc. of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems, 2006
  - [7] A. Whitaker , M. Shaw , S.D. Gribble: “Denali: Lightweight Virtual Machines for Distributed and Networked Applications”, Proc. of the 2002 USENIX Annual Technical Conference, 2002
  - [8] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, A. Warfield: “Xen and the Art of Virtualization”, Proc. of the 19th ACM Symposium on Operating Systems principles, 2003
  - [11] O. Agesen: “Performance Aspects of x86 Virtualization”, VMWORLD 2007
  - [13] G. Neiger, A. Santoni, F. Leung, D. Rodgers, R. Uhlig: “Intel Virtualization Technology: Hardware Support for Efficient Processor Virtualization”, Intel Technology Journal, 10 (3), 2006
  - [18] J. Schad, J. Dittrich, J.-A. Quiané-Ruiz: “Runtime Measurements in the Cloud: Observing, Analyzing, and Reducing Variance”, Proc. of the VLDB Endowment, 3 (1-2), 2010



# Cloud Systems

## Chapter 3: Containers

Dr Lauritz Thamsen

[lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)

School of Computing Science

University of Glasgow



# First Half: Lecture Chapters

---

Cloud Resource Management:

1. Cloud Computing Intro
2. Virtual Machines
- 3. Containers**
4. Cloud Infrastructure Management
5. Cloud Sustainability

# **Outline of Chapter 3: Containers & Container Mngt.**

---

**3.1** Intro to Containerization

**3.2** Linux Containment Features

**3.3** Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

**3.4** Containers vs. VMs

**3.5** Container Orchestration

**3.6** Orchestration Systems (like Kubernetes)

# **Outline of Chapter 3: Containers & Container Mngt.**

---

## **3.1 Intro to Containerization**

## **3.2 Linux Containment Features**

## **3.3 Container Technologies (like LXC and Docker)**

— short break (10-15 minutes) —

## **3.4 Containers vs. VMs**

## **3.5 Container Orchestration**

## **3.6 Orchestration Systems (like Kubernetes)**

# Containerization

- Also known as OS-Level Virtualization
- Motivation: System virtualization comes with too much overhead, large images, and long boot times
- Idea: Do not create *virtual machines*, instead...
  - Reuse operating system kernel but *isolate applications*
  - Virtualize access to resources used by processes
    - ◆ File system
    - ◆ Devices
    - ◆ Network
    - ◆ Other processes
    - ◆ ...

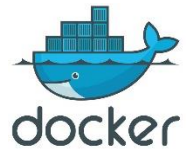


# History of OS-Level Virtualization

- Unix v7 **chroot** system call (1979)
  - Allows setting the file system root for processes
  - Unix philosophy: (almost) everything is accessed through the file system → (almost) everything can be virtualized though **chroot**
- Wave of container technologies in early 2000s
  - FreeBSD Jails, Linux VServer, Solaris Zones, ...
  - Different degrees of isolation, different tool chains
  - Mostly specific to Linux distributions
  - Not widely popular, mainly used by sys admins in large companies

# History of OS-Level Virtualization

- LXC (2008)
  - User space tools for accessing Linux kernel process isolation features (e.g. *cgroups* and *namespaces*)
- Docker (2013)
  - Most widely used container technology and ecosystem
- Rocket (CoreOS, 2014)
  - Alternative to Docker, more focused on security and standardization
- LXD (Canonical, 2014)
  - Image-based, focused on entire Linux distributions
- Ubuntu Snap (Canonical, 2018)
  - Universal packaging format for Linux distros



# **Outline of Chapter 3: Containers & Container Mngt.**

---

**3.1** Intro to Containerization

**3.2** Linux Containment Features

**3.3** Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

**3.4** Containers vs. VMs

**3.5** Container Orchestration

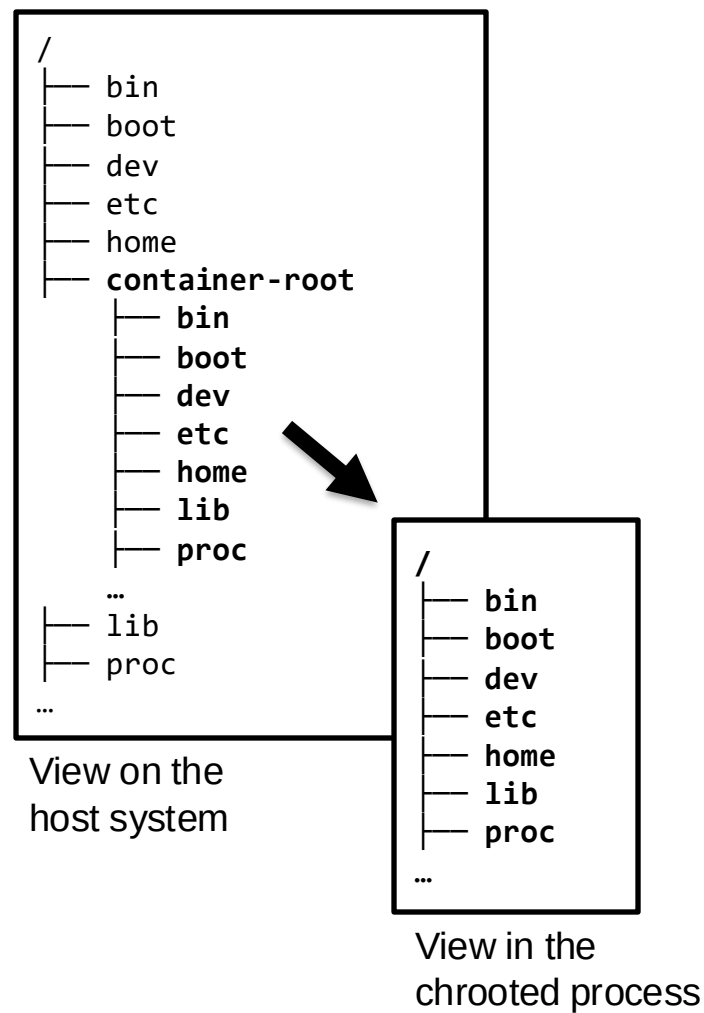
**3.6** Orchestration Systems (like Kubernetes)

# Linux Kernel Features for OS-Level Virtualization

- Linux kernel contains numerous mechanisms for process isolation:
  - chroot system call
  - Namespaces
  - Capabilities
  - cgroups
  - SELinux
  - seccomp
  - ...
- Partially overlapping functionality, yet different configuration approaches

# chroot

- Oldest mechanism for process isolation
- System call that changes the root directory (“/”) of the calling process
- Since “everything is a file” in Linux, many aspects of the underlying system can be virtualized
- System calls, networking, etc. remain unchanged



# Linux Namespaces

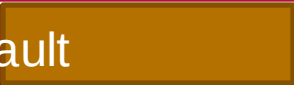
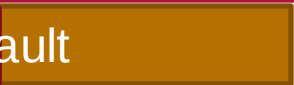
- Provide separate views on kernel resources for processes in different namespaces
- Resources that can be isolated:
  - PID (processes)
  - IPC (inter-process communication)
  - Network (devices, protocol stacks, firewalls, ports, ...)
  - Mount (mount points, file system structure)
  - User (users and groups)
  - UTS (hostname and domain name)

# Linux Namespaces Example

- Container 1: Share users and hostname, but isolate processes, network, and mount points

 Namespaces used by processes running in Container 1

 Namespaces used by other processes

|         |                                                                                                                                                                                  |                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| PID     |  Default                                                                                       |  Container 1 |
| Network |  Default                                                                                       |  Container 1 |
| Mount   |  Default                                                                                       |  Container 1 |
| IPC     |  Default  |                                                                                                 |
| User    |  Default  |                                                                                                 |
| UTS     |  Default  |                                                                                                 |

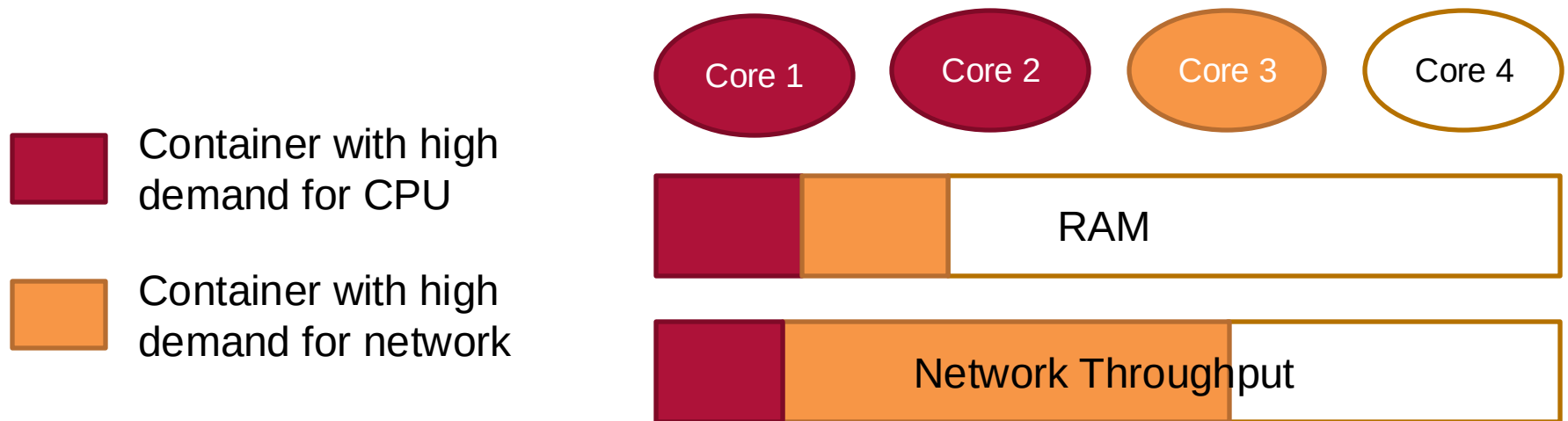
# Linux Capabilities

- Traditionally, super user (root) is the only one to perform administrative tasks on a Linux system
- Linux capabilities allow to assign selected rights to new processes
- Long list of possible capabilities, including
  - many system calls,
  - device access,
  - file system modifications,
  - ...



# Linux cgroups (Control Groups)

- Allow definition of resource usage constraints for parts of the process tree
- Used for limiting CPU, RAM, network, and disk usage for containers



# Linux Security Policies

- SELinux, AppArmor
  - Optional kernel modules for security
  - Allow rule-based confinement of processes to limit their access to certain files and devices only
  - SELinux: complex, but powerful rule definitions
  - AppArmor: simpler security profiles
- Seccomp policies
  - seccomp system call puts the process into a secure computation mode, where only selected system calls are allowed

# Outline of Chapter 3: Containers & Container Mngt.

---

3.1 Intro to Containerization

3.2 Linux Containment Features

**3.3 Container Technologies (like LXC and Docker)**

— short break (10-15 minutes) —

3.4 Containers vs. VMs

3.5 Container Orchestration

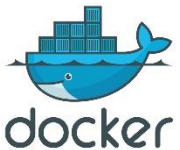
3.6 Orchestration Systems (like Kubernetes)

# Examples for Container Technologies



**LXC**

- LXC: First container technology widely adopted



- Docker: Currently most widely used container platform and eco system

# LXC Containers

- LXC: Library and userspace tools (scripts) to access kernel process isolation features
- No daemon, instead containers and their file systems are accessible via `/var/lib/lxc`

## Language Bindings

(Python, Lua, Go, Python, Ruby, ...)

## Userspace tools

`lxc-create`, `lxc-start`, `lxc-stop`, ...

**liblxc**

## Kernel features:

Namespaces, capabilities, chroot, cgroups,  
AppArmor, SELinux, seccomp

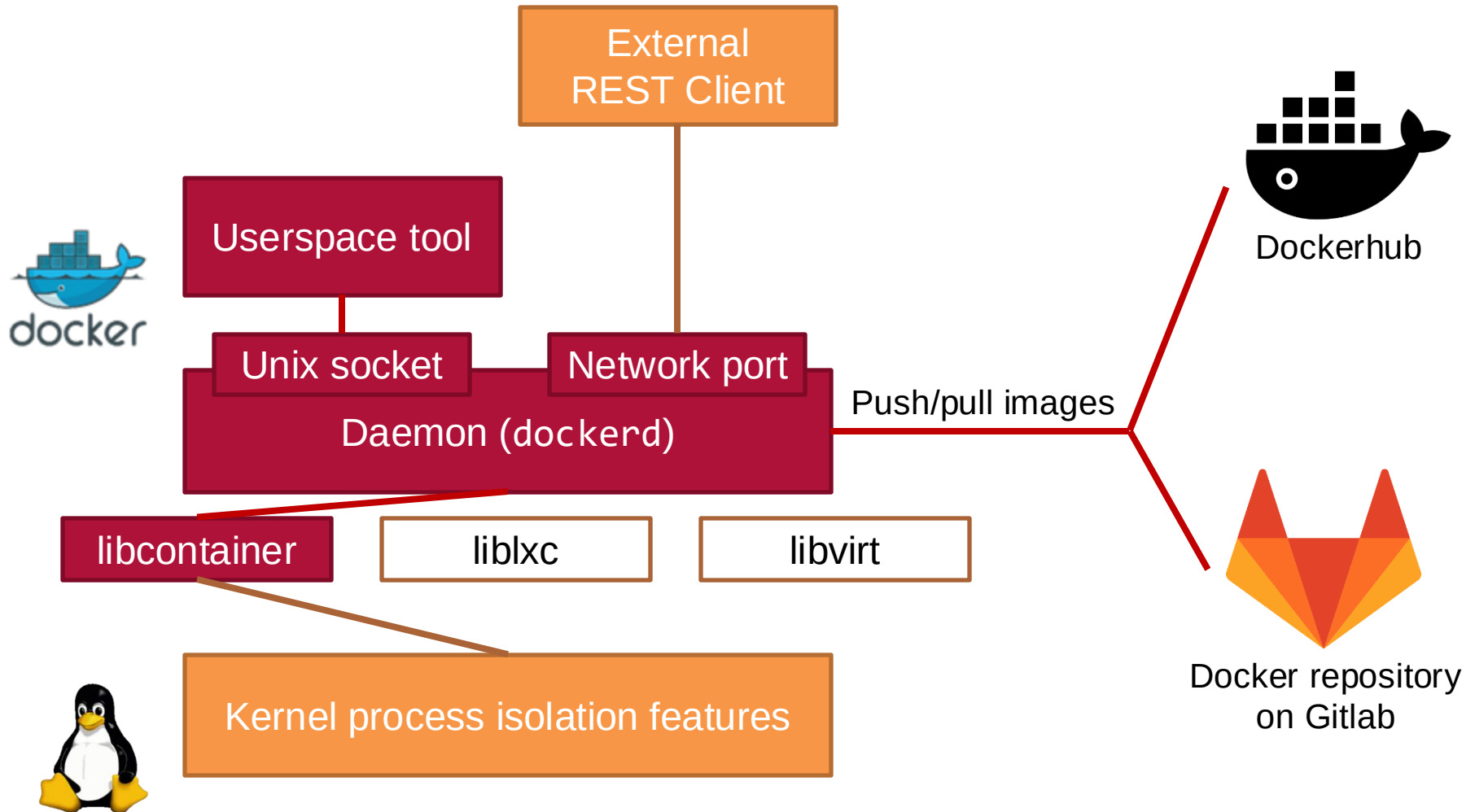
# LXC Container Creation

- Container Creation
  1. Container directory allocated under `/var/lib/lxc`
  2. “Template script” executed to populate the file system of the container
  3. Process is spawned and chrooted into its file system
  4. Process isolation is configured
  5. Main application process is spawned, all child processes will inherit isolation context
- LXC does not rely on an image format
- Not much infrastructure for sharing images

# Docker

- Most popular container technology at the moment
- Docker includes:
  - Daemon and user space tools for managing local containers and images
  - Hierarchical image format
  - Build tool for images, called *Dockerfiles*
  - Private and public repositories for sharing images (e.g. *Dockerhub*)
  - Internal and external tools for automatic orchestration of container infrastructures

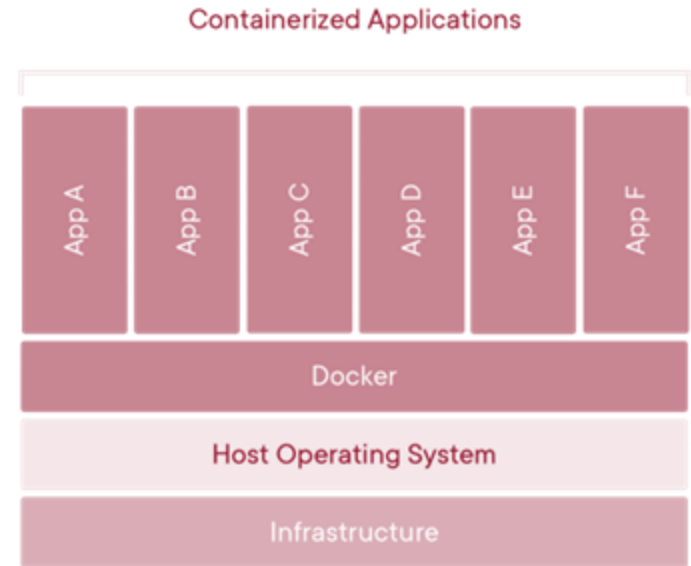
# Docker Components





# Docker Instances, Images, and Files

- Multiple containers share a single OS kernel, isolated by kernel containment features
- Container instance: running an isolated process, started from an image
- Docker image: snapshot of a container, packaging the application, its dependencies, and its configuration
- Dockerfile: instructions for creating images

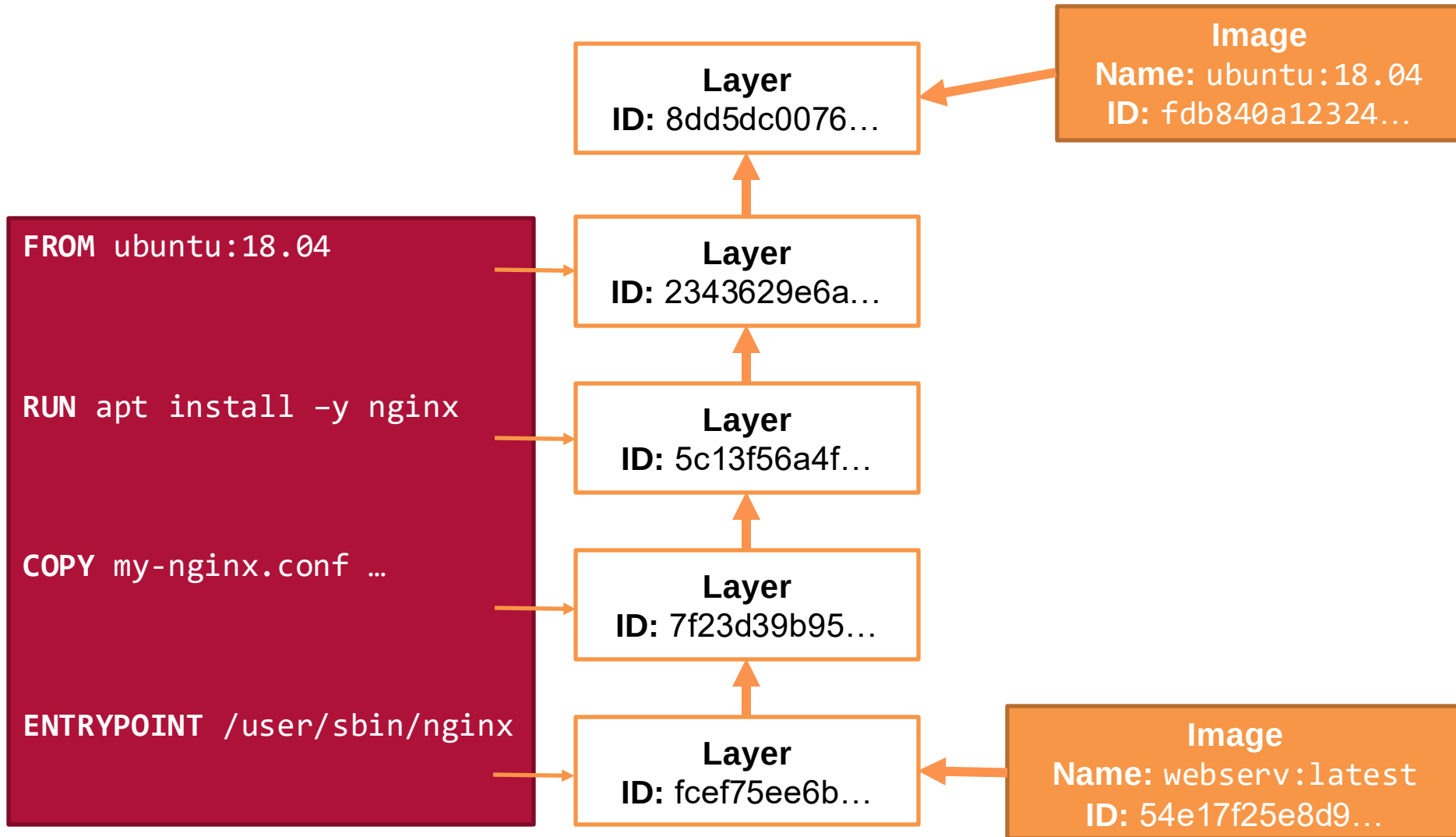


# Dockerfiles

- Automatic creation of container images
- Text file with commands that modify files or change configuration
- Intermediate steps can be cached, resulting in reduced image build times

```
FROM ubuntu:18.04
RUN apt install -y nginx
COPY my-nginx.conf /etc/nginx/nginx.conf
ENTRYPOINT /user/sbin/nginx
```

# Docker Image Format



# Docker Images

- Docker images contain:
  - List of layers
  - ID (Hash of layer IDs and other config data)
  - Configuration data: ports, mounts, env. variables, ...
  - Meta data (creation time, author, history, ...)
- Each image layer contains:
  - ID of the layer (hash of all files) and ID of parent layer
  - Layer files (tarball): Files that were added/changed on this layer, relative to parent layer
  - Command that was used to create the layer

# Another Dockerfiles Example

- Creating docker images from configuration files
- Example: Dockerfile for an Express web server image using NodeJS

```
$ docker build --tag express .
```

```
$ docker run --rm express
```

```
# Create image based on the official Node 6 image from dockerhub
FROM node:9

# Create a dir where our app will be placed
RUN mkdir -p /usr/src/app

# Change dir so that our commands run inside this new dir
WORKDIR /usr/src/app

# Copy dependency definitions
COPY package.json /usr/src/app

# Install dependencies
RUN npm install

# Get all the code needed to run the app
COPY . /usr/src/app

# Expose the port the app runs in
EXPOSE 3000

# Serve the app
CMD ["npm", "start"]
```

# Outline of Chapter 3: Containers & Container Mngt.

---

3.1 Intro to Containerization

3.2 Linux Containment Features

3.3 Container Technologies (like LXC and Docker)

**– short break (10-15 minutes) –**

3.4 Containers vs. VMs

3.5 Container Orchestration

3.6 Orchestration Systems (like Kubernetes)

# **Outline of Chapter 3: Containers & Container Mngt.**

---

**3.1** Intro to Containerization

**3.2** Linux Containment Features

**3.3** Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

**3.4** Containers vs. VMs

**3.5** Container Orchestration

**3.6** Orchestration Systems (like Kubernetes)

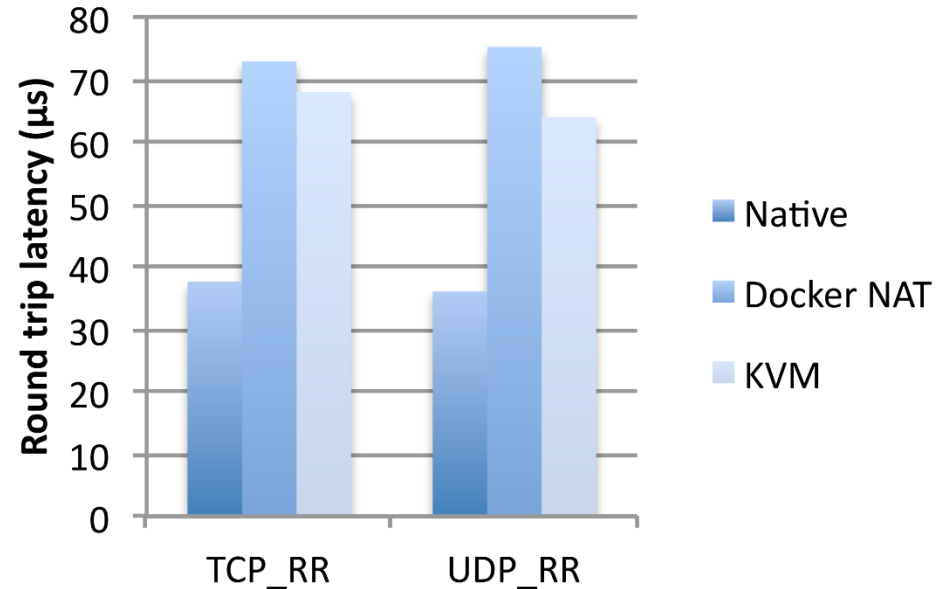
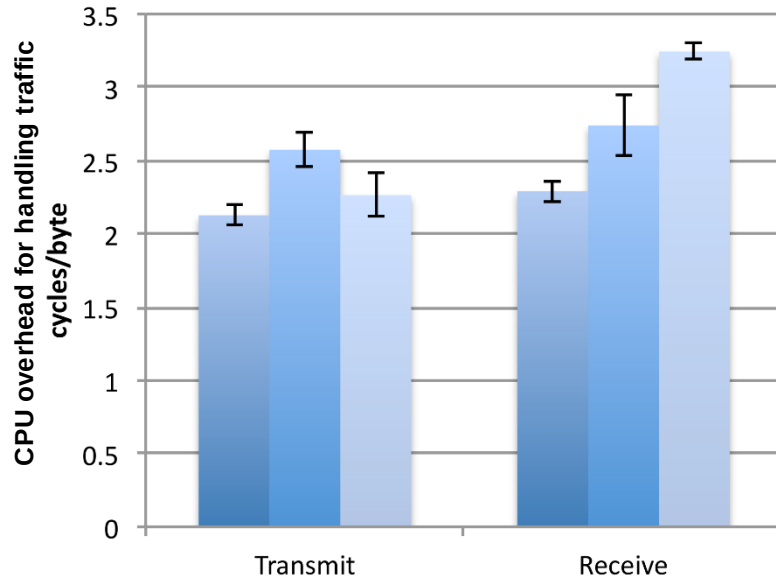
# Containers vs VMs [19]: Performance (CPU & RAM)

| Workload                         | Native                   | Docker                   | KVM                      |
|----------------------------------|--------------------------|--------------------------|--------------------------|
| Linpack (GFLOPS)                 | 290.8 [ $\pm 1.13$ ]     | 290.9 [ $\pm 0.98$ ]     | 284.2 [ $\pm 1.45$ ]     |
| Memory (Random Access, GIOps/s)  | 0.0126 [ $\pm 0.00029$ ] | 0.0124 [ $\pm 0.00044$ ] | 0.0125 [ $\pm 0.00032$ ] |
| Memory (Sequential Access, GB/s) | 45.8 [ $\pm 0.21$ ]      | 45.6 [ $\pm 0.55$ ]      | 45.0 [ $\pm 0.19$ ]      |

- VMs and container introduce low CPU and memory access overhead
  - Prerequisite: exposing cache topology and CPU acceleration features (e.g. NUMA, FPU, SSE)

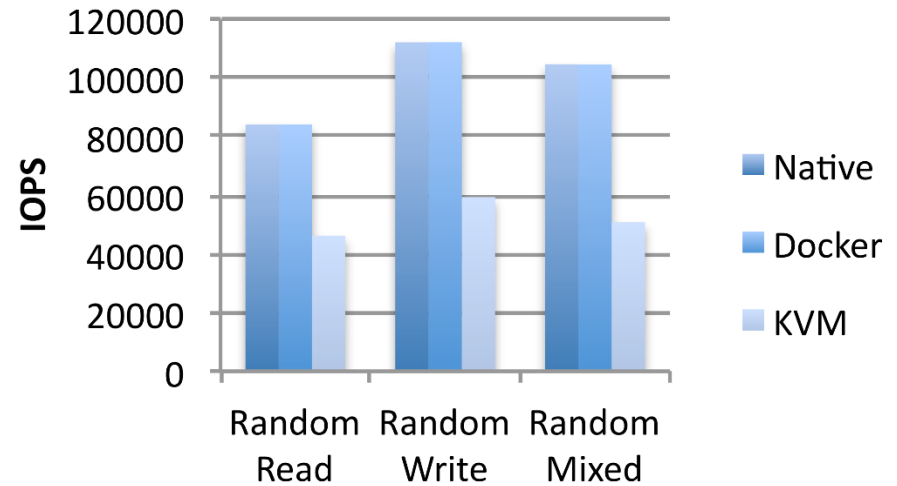
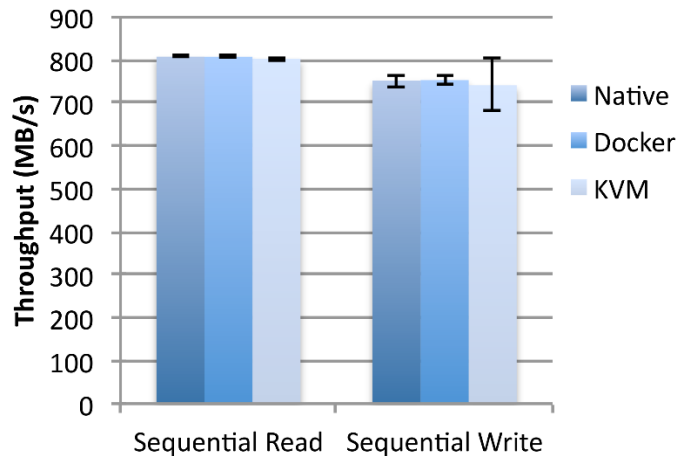


# Containers vs VMs [19]: Performance (Network)

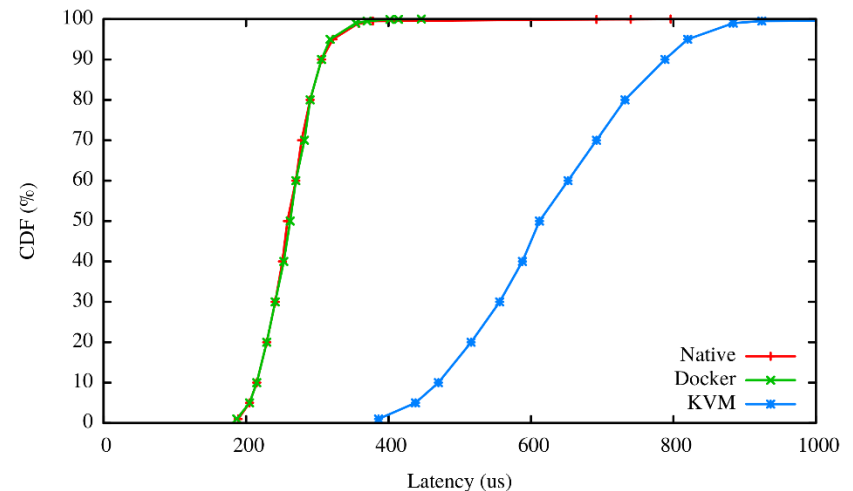


- Low CPU overhead per packet
- Reasons for latency increase:
  - NAT in Docker networking
  - Virtual network device in KVM (NAT might further increase latency)

# Containers vs VMs [19]: Performance (Disk IO)



- Similar throughput
- Penalty for latency and IOPs due to virtual IO device



# Containers vs VMs: Image Size & Boot Time

- Virtual machine images usually larger than container images (contain entire OS)
- Image caching on execution hosts for both VMs and containers
- Boot time of VMs can be orders of magnitude longer than container startup

# Containers vs VMs: Isolation & Security

- Containers share the host OS kernel
  - Vulnerable to Linux kernel bugs
  - Denial-of-Service attacks: Resource usage, system calls, and context switches of one container can starve others
  - Kernel parameters cannot be tuned to workload
- VMM orders of magnitude less code than OS kernels  
→ more exploits discovered and fixed?

# Outline of Chapter 3: Containers & Container Mngt.

---

3.1 Intro to Containerization

3.2 Linux Containment Features

3.3 Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

3.4 Containers vs. VMs

**3.5 Container Orchestration**

3.6 Orchestration Systems (like Kubernetes)

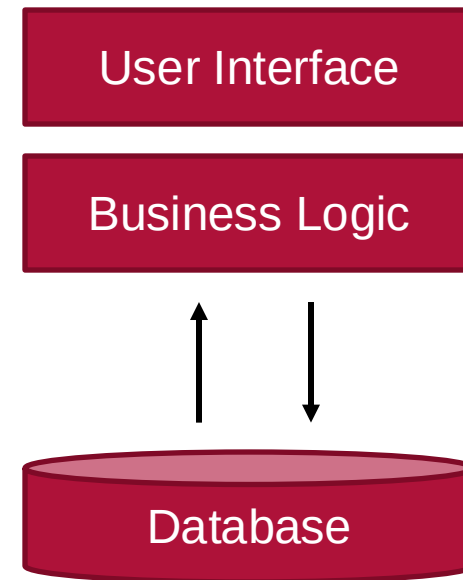
# Container Analogy

- Containers
  - Standardized
  - Easy to move
  - Isolated
  - Many containers fit hosts
- Dependency management: libs and config bundled with the application to run it everywhere → build your app in a container, test the container locally or your staging environment, then ship the container



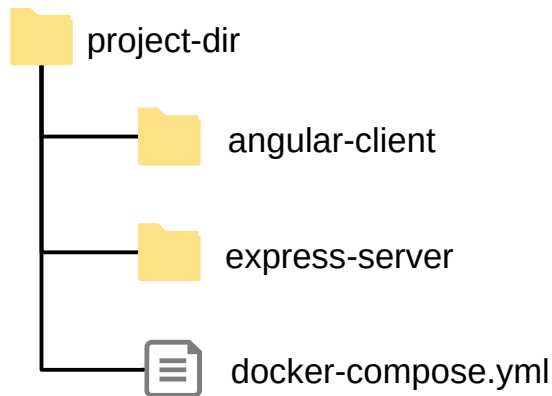
# Container Composition

- Applications typically consist of multiple connected components intended to run as a single service
- e.g. Web Application
  - Front-end framework
  - Back-end framework
  - Persistent data store



# Docker Compose

- Tool for defining and running multi-container Docker applications on one node



```
version: '2' # specify docker-compose version

# Define the services/containers to be run
services:
  angular:
    build: angular-client # Dockerfile directory
    ports:
      - "4200:4200" # port forwarding

  express:
    build: express-server # Dockerfile directory
    ports:
      - "3000:3000" # port forwarding

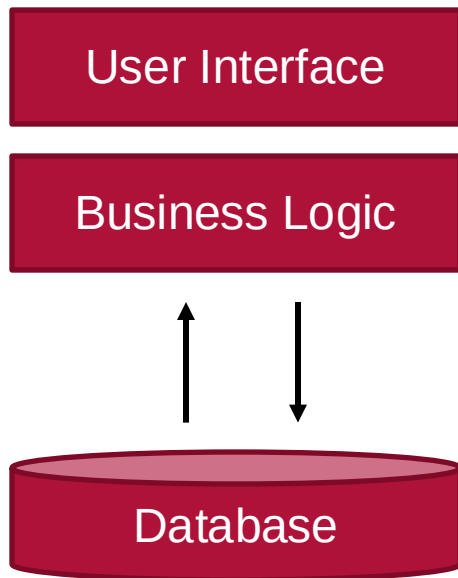
  database:
    image: mongo # image to build container from
    ports:
      - "27017:27017" # port forwarding
```

```
$ docker compose up
```

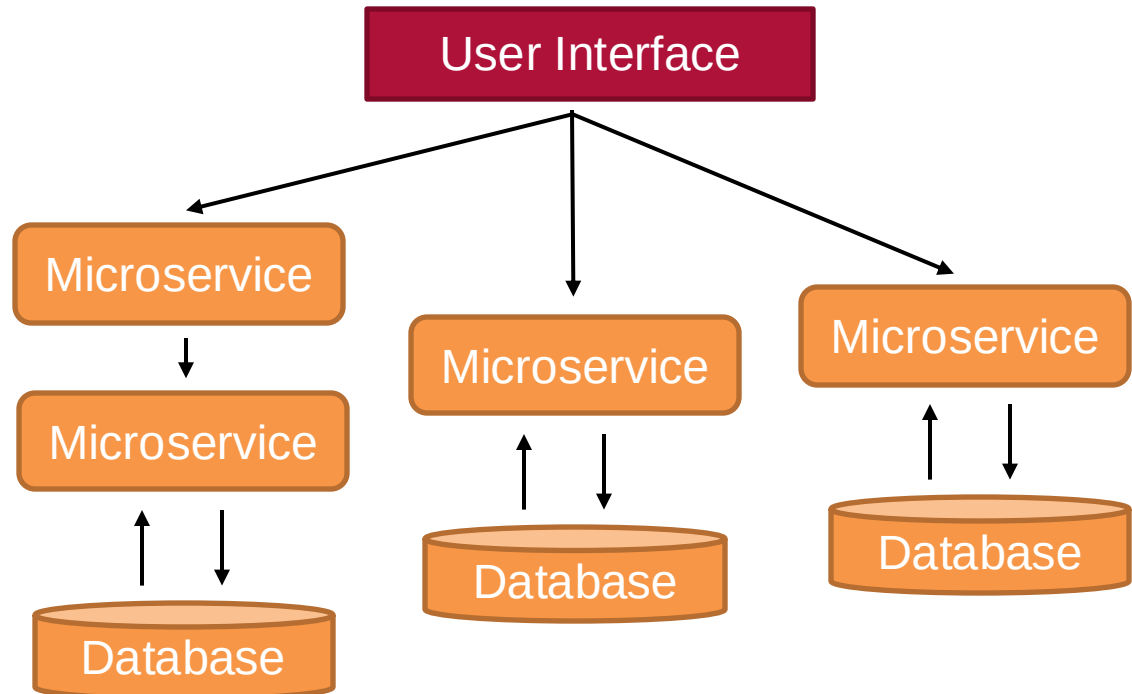


# Microservices (1/2)

## Monolith



## Microservices



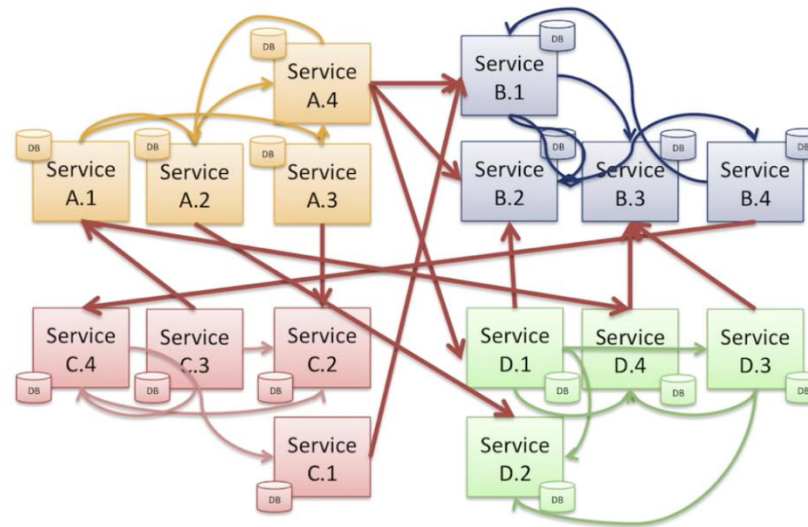
# Microservices (2/2)

- Advantages

- Independent development
- Small teams
- Fault isolation
- Scalable

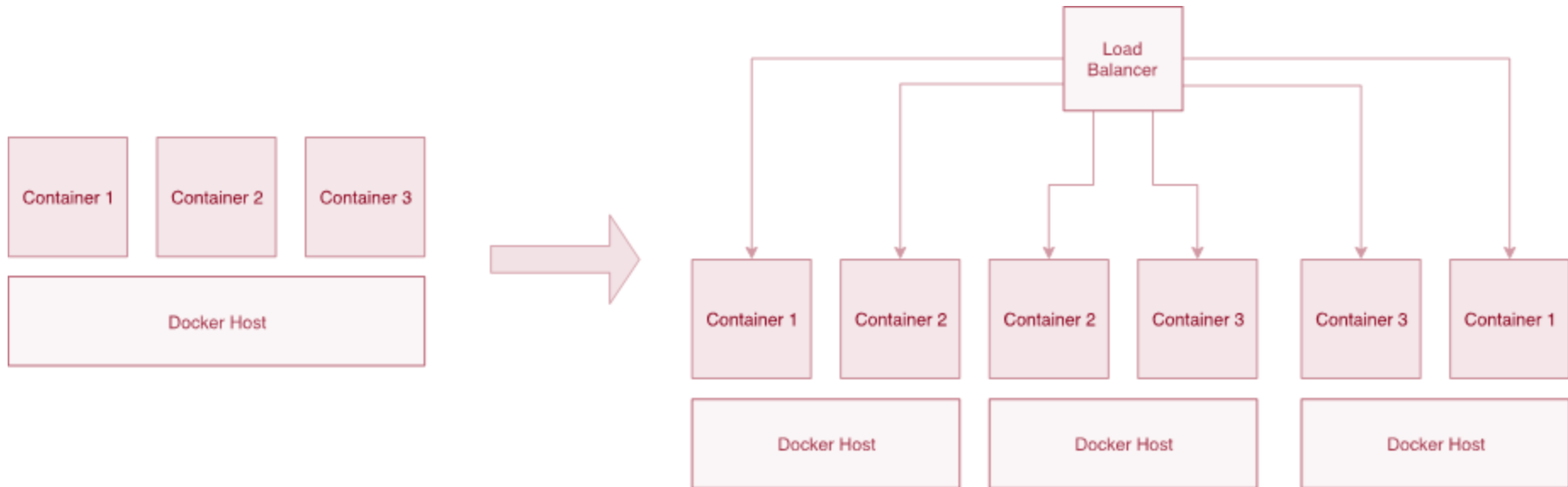
- Disadvantages

- Overhead (duplicated tech, e.g. databases)
- Complexity of services and networking



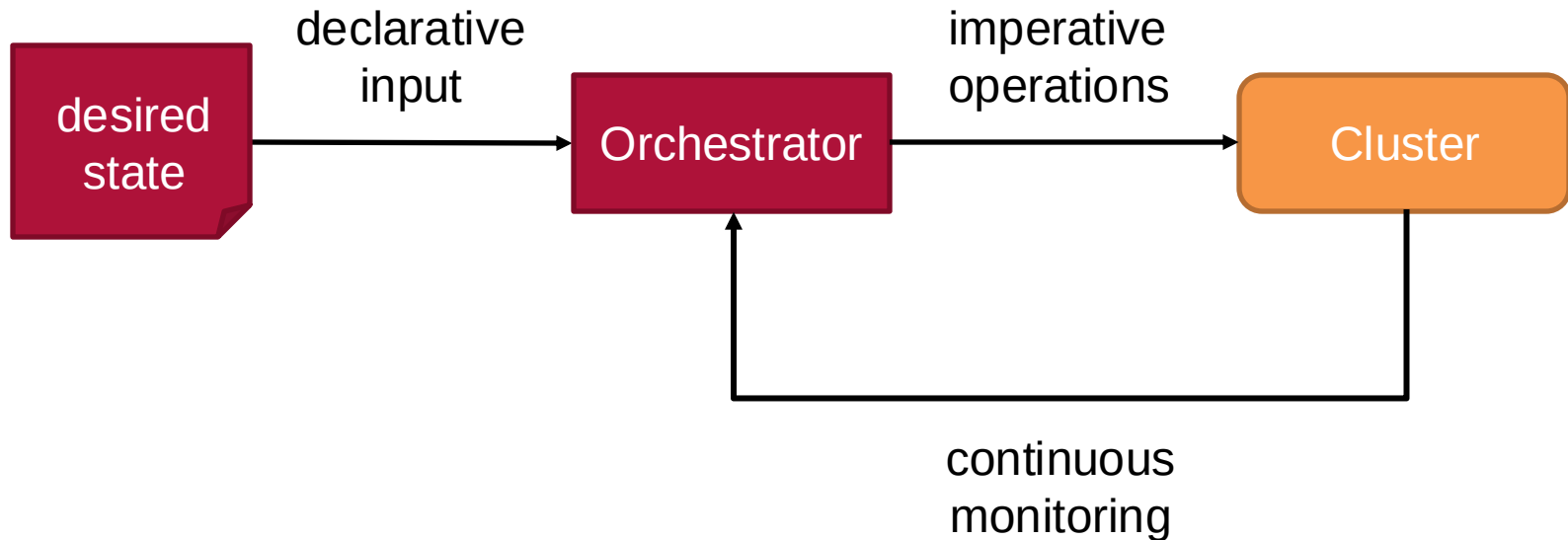
# Docker Cluster

- Running an application on a cluster of docker hosts for fault tolerance and scalability



# Resource Orchestration

- Orchestration tools: Control systems for clusters



# Container Orchestration

Container Orchestration

Distributed container management

Container Runtime

Local container management

Infrastructure

Container-agnostic infrastructure

- Provisioning and deployment of containers
- Configuration and networking of containers
- Replication and availability of containers
- Monitoring of replicated containers
- Load balancing across replicated containers

# Outline of Chapter 3: Containers & Container Mngt.

---

3.1 Intro to Containerization

3.2 Linux Containment Features

3.3 Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

3.4 Containers vs. VMs

3.5 Container Orchestration

**3.6 Orchestration Systems (like Kubernetes)**

# Kubernetes

- Greek for “helmsman” or “pilot”, abbreviated “k8s”
- Open-sourced by Google in 2014
- Version 1.0 released in July 2015
- Google and the Linux Foundation created the Cloud Native Computing Foundation (CNCF) to make Kubernetes an open standard
- Google Kubernetes Engine (GKE), Azure Container Service (AKS), AWS Elastic Container Service (EKS)

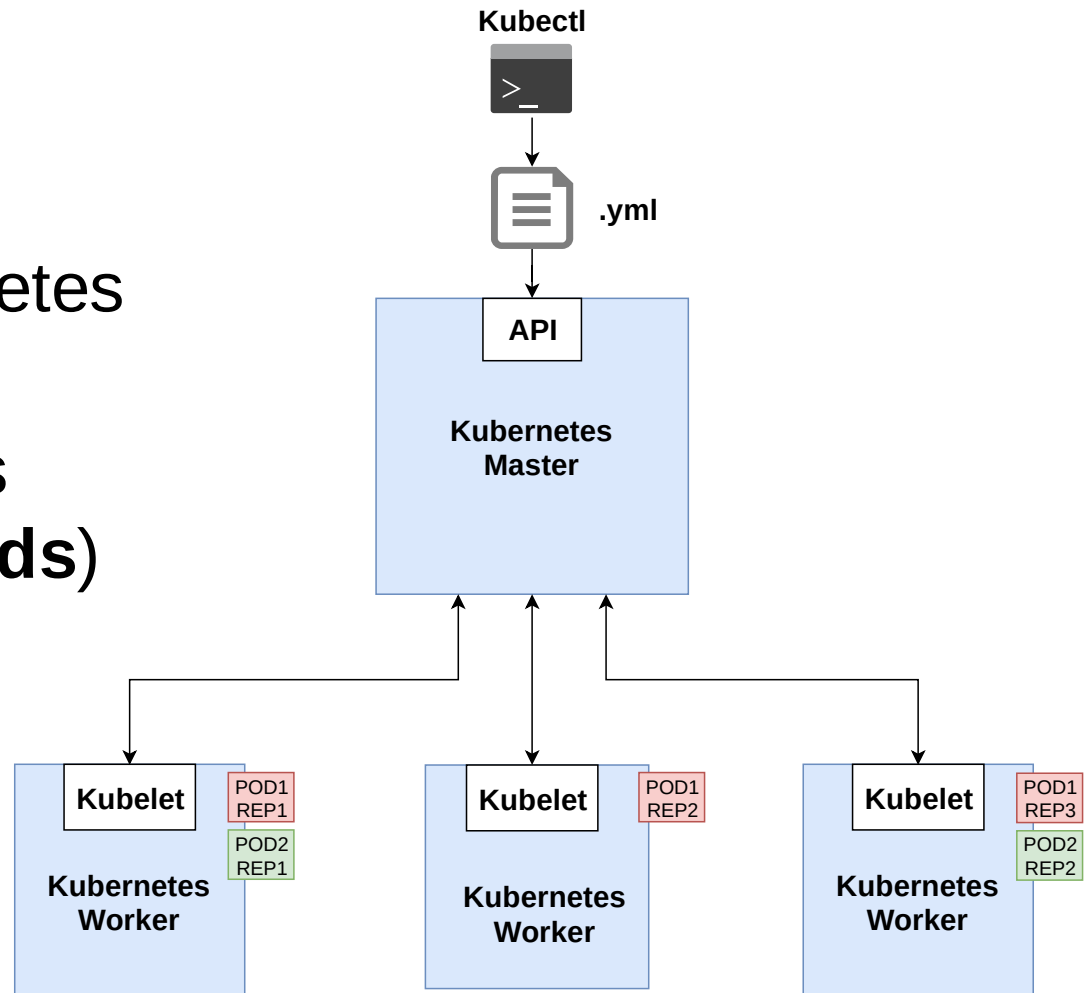
# What Does Kubernetes Do for You?

- Automation engine for cluster management, based on a declarative description of the desired cluster state
- Deployment, monitoring, recovery, and scaling
  - Instantiating sets of containers
  - Connecting containers through agreed interfaces
  - Exposing services to machines outside of the cluster
  - Monitoring, logging, and re-starting/scheduling
  - Dynamically scaling the cluster



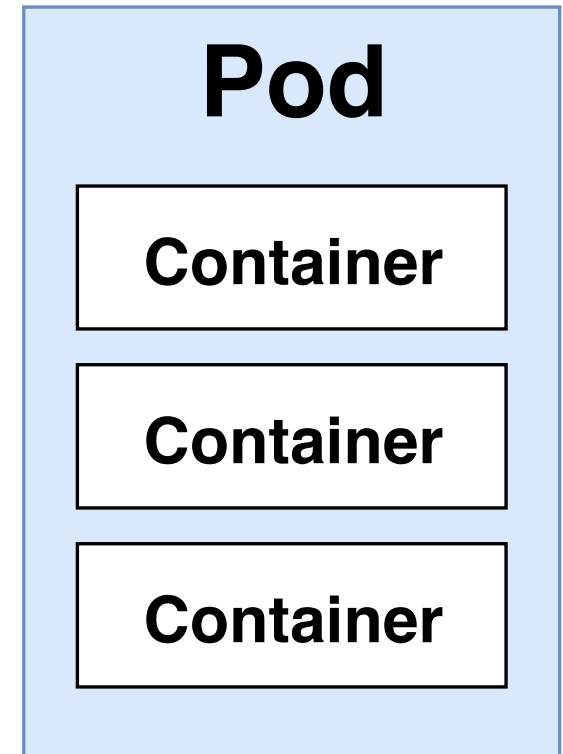
# Kubernetes Cluster

- Collection of nodes (either bare-metal or virtual machines), managed by Kubernetes
- Runs groups of replicated containers (which are called **Pods**)



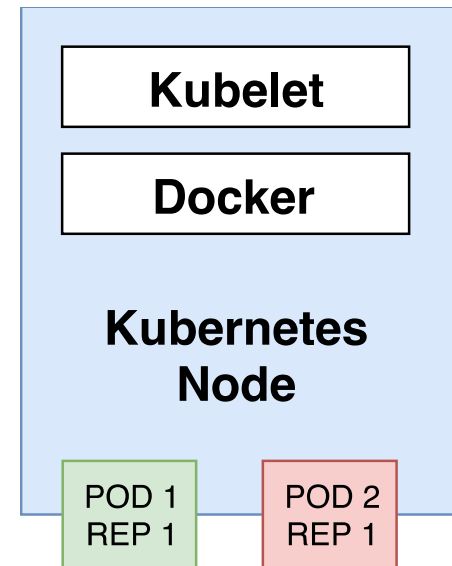
# Kubernetes Pods

- **Pods** consist of
  - Group of containers
  - Container configurations
  - Shared storage
- Containers in a pod
  - are scheduled together
  - are guaranteed to be on the same node
- **Replicas** of pods



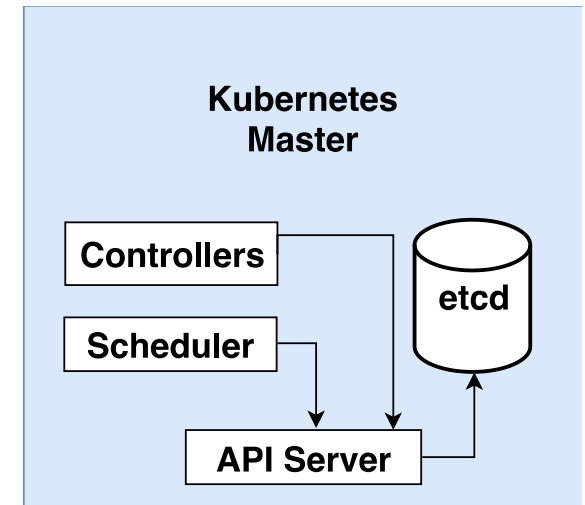
# Kubernetes Nodes

- Each worker node in the cluster runs two processes:
  - **kubelet**: agent that communicates with the Kubernetes master (master → cluster nodes)
  - **kube-proxy**: makes defined services available on each node (cluster nodes → master)



# Kubernetes Master

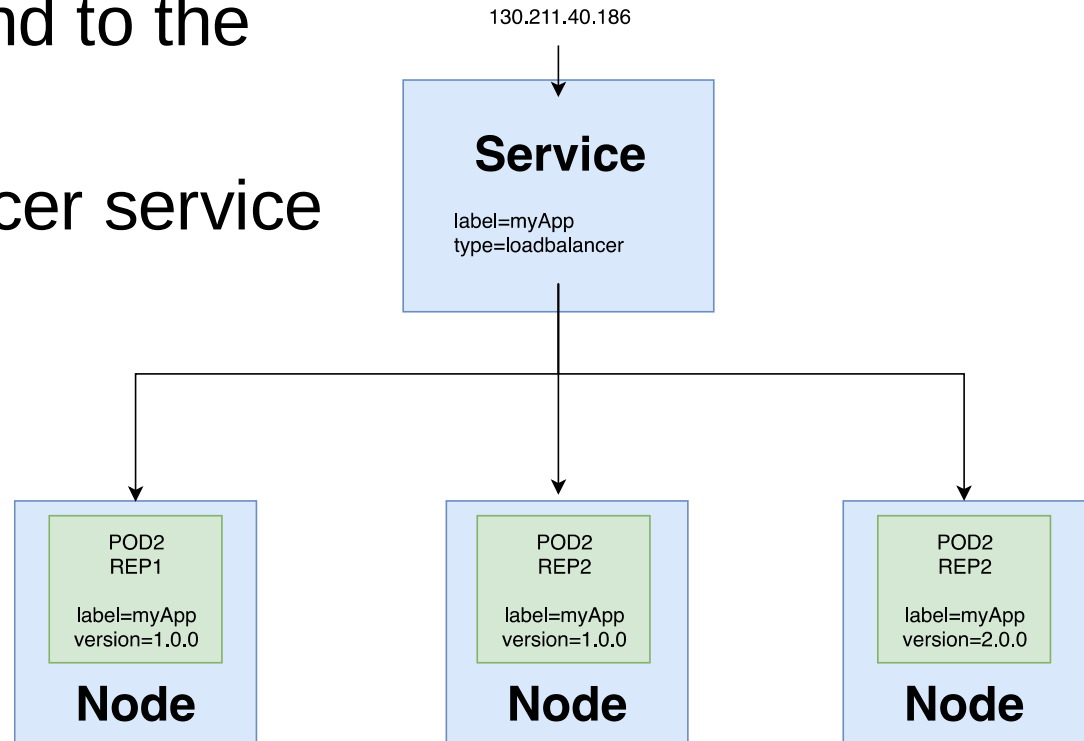
- Collection of processes managing the cluster state on a single node of the cluster
- **Controllers**, e.g. replication and scaling controllers
- **Scheduler**: places pods based on resource requirements, hardware and software constraints, data locality, deadlines...
- **etcd**: reliable distributed key-value store, used for the cluster state



# Kubernetes Services

- **Services** expose functionality of pods in the cluster and to the outside
- Example: load balancer service

```
apiVersion: v1
kind: Service
metadata:
  name: myApp-service
spec:
  type: LoadBalancer
  ports:
    - protocol: TCP
      # accessible externally
      nodePort: 80
      # port in Pod
      targetPort: 8080
  selector:
    app: myApp
```



# Kubernetes Cluster State

- Kubernetes allows to specify the state of pods in a cluster and then manages the cluster accordingly
  - **Deployments** for stateless pods:
    - ◆ Do not preserve pod state and do not need permanent storage and IDs for networking
    - ◆ Used e.g. for web server pods (like Nginx and Apache)
  - **StatefulSets** for stateful pods:
    - ◆ Keep track of pod state by saving it to storage and pods communicate using persistent unique IDs
    - ◆ Used e.g. for database pods (like MongoDB)

# Kubernetes Deployments

- Specify the **deployment** of a stateless pod, which k8s then establishes (and re-establishes)

```
$ kubectl create -f nginx-deployment.yaml
```

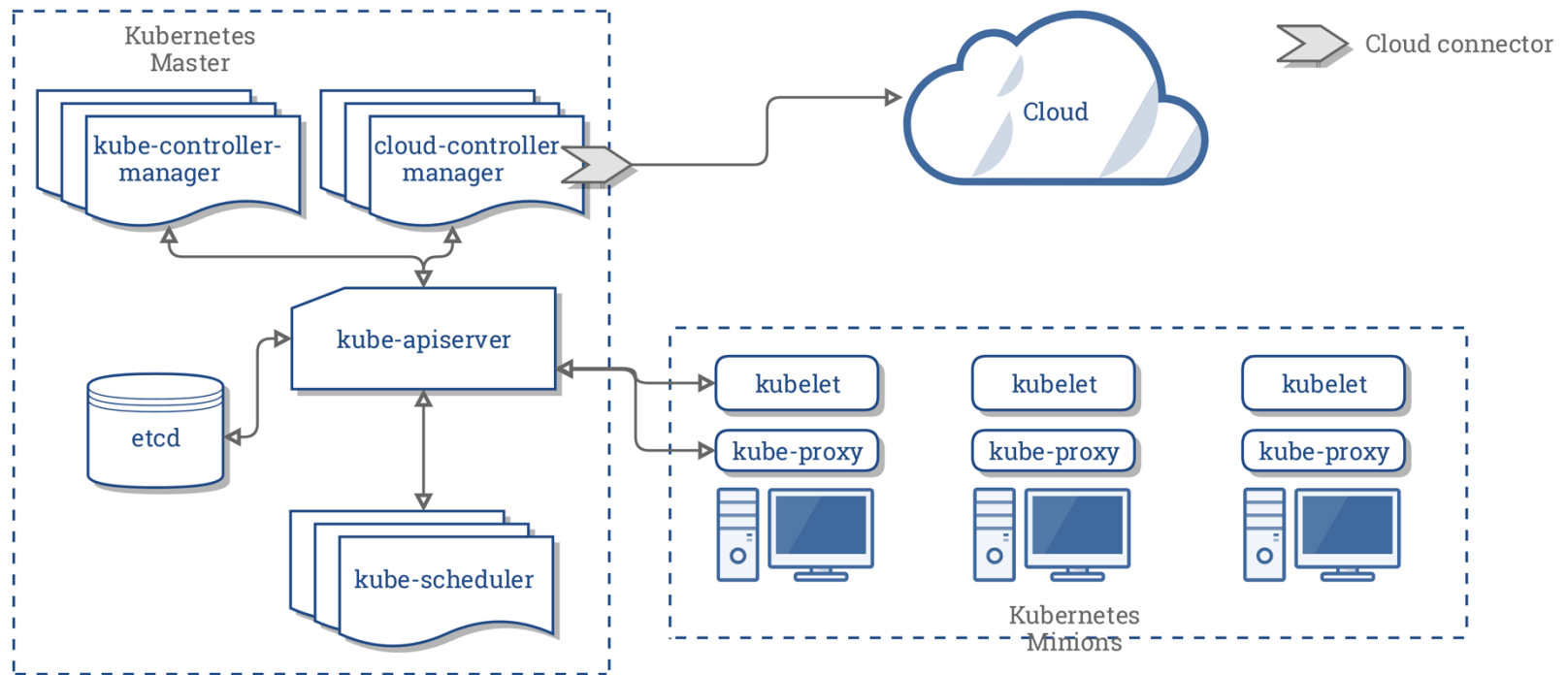
```
$ kubectl get deployments
```

| NAME             | DESIRED | CURRENT | UP-TO-DATE | AVAILABLE | AGE |
|------------------|---------|---------|------------|-----------|-----|
| nginx-deployment | 3       | 3       | 3          | 3         | 18s |

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.15.4
        ports:
        - containerPort: 80
```

# Kubernetes

## Horizontal Pod Autoscaler





# Kubernetes

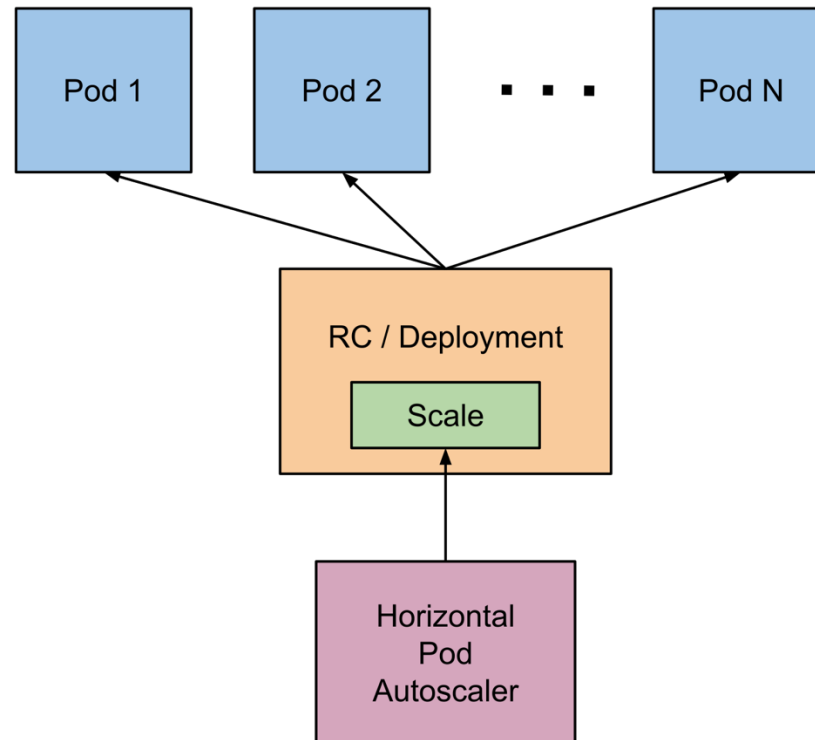
## Horizontal Pod Autoscaler

---

- Automatically scales the number of pods using the **replication controller**
- Based on
  - CPU utilization or custom metrics
  - Target value for the metric
- Autoscaler checks metrics regularly (e.g. every 30 sec)
- Scales the number of replicas to optimize the metric towards the target value

# Kubernetes

## Horizontal Pod Autoscaler



# Kubernetes

## Horizontal Pod Autoscaler

- Number of replicas is scaled by comparing an averaged current metric value to target value
- No scaling if metrics are within a tolerance (e.g.  $\leq 0.1$ )

```
desiredReplicas =  
    [ currentReplicas * (currentMetricValue/desiredMetricValue) ]
```

⑦ This assumes linear scaling!

- Autoscaler scales to the highest number of desired replicas in a sliding window of 5 minutes
  - Quick responses to more load, but reduces *thrashing*

# Sometimes Metrics are not Available

- Ignore pods that are currently being shut down
- Normally running pods with missing metrics
  - If result without these would be to scale out: assume metric to be 0
  - If result without these would be to scale in: assume metric to be 1
- Assume metric to be 0 for pods that are not yet ready

“Dampens”  
the scaling



# **Outline of Chapter 3: Containers & Container Mngt.**

---

**3.1** Intro to Containerization

**3.2** Linux Containment Features

**3.3** Container Technologies (like LXC and Docker)

— short break (10-15 minutes) —

**3.4** Containers vs. VMs

**3.5** Container Orchestration

**3.6** Orchestration Systems (like Kubernetes)

# Summary: Containers

---

- Containers provide an alternative or supplement to VMs for isolating individual applications
- Containerization using OS kernel containment features (e.g. chroot, namespaces, cgroups)
- Docker adds hierarchical images, image registries, and Dockerfiles
- Container orchestration to manage distributed, replicated multi-container applications

# References

[19] W. Felter, A. Ferreira, R. Rajamony, J. Rubio: “An updated performance comparison of virtual machines and Linux containers”, Proc. of the IEEE International Symposium on Performance Analysis of Systems and Software, 2015.

- Real-world systems discussed:
  - <https://linuxcontainers.org/>
  - <https://canonical.com/lxd>
  - <https://www.docker.com/>
  - <https://docs.docker.com/compose/>
  - <https://kubernetes.io/>



# Cloud Systems

## Chapter 4: Cloud Infrastructure Management

Dr Lauritz Thamsen

[lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)

School of Computing Science

University of Glasgow





# First Half: Lecture Chapters

---

Cloud Resource Management:

1. Cloud Computing Intro
2. Virtual Machines
3. Containers
- 4. Cloud Infrastructure Management**
5. Cloud Sustainability

# Outline of Chapter 4: Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

4.2 Cloud Operating Systems (like OpenStack)

4.3 VM Management (including Migration)

— short break (10-15 minutes) —

4.4 Infrastructure as Code and IaC Tools (like Ansible)

4.5 DevOps and CI/CD on Clouds

# **Outline of Chapter 4: Cloud Infrastructure Management**

---

**4.1 Intro to Cloud Infrastructure Management**

**4.2 Cloud Operating Systems (like OpenStack)**

**4.3 VM Management (including Migration)**

— short break (10-15 minutes) —

**4.4 Infrastructure as Code and IaC Tools (like Ansible)**

**4.5 DevOps and CI/CD on Clouds**

# “Iron Age”: Bare-Metal Servers



- Running components of an application on one or multiple physical servers:
  - Costs and agility: Purchase, housing, maintenance
  - Fault tolerance: Servers are single points of failure
  - Resource utilization: Servers are often underutilized, but sometimes also bottlenecks (with dynamic load)

# Bare-Metal Servers in One Organization

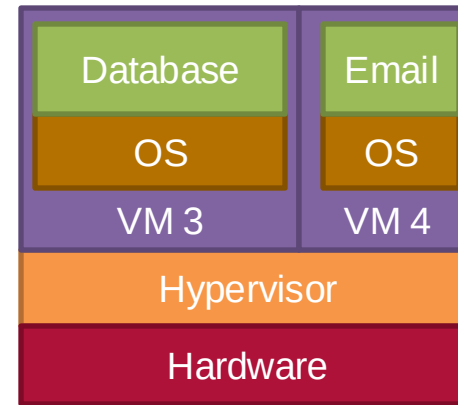
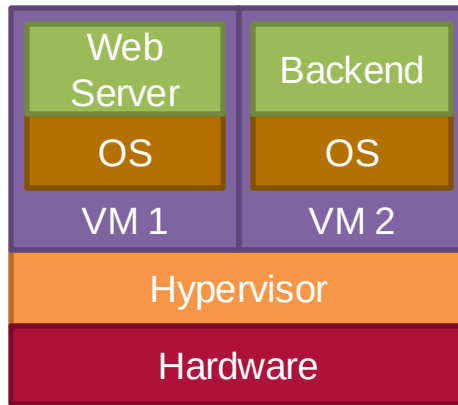
- Server consolidation: (Manually) assign components from different applications to shared physical nodes



- Only possible within one organization without isolation
- Static assignment (while loads will vary dynamically)

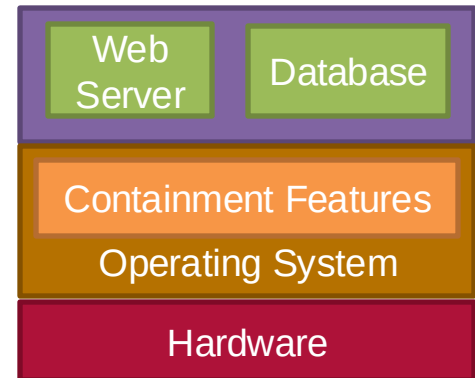
# Recap: Virtual Machines

- System virtualization: Same ISA, but possibly different OS and resource configuration (#cores, RAM, storage) and isolation of applications
- Same physical hosts run multiple virtual machines
- Live migration and snapshots of VMs allow for load balancing and fault tolerance



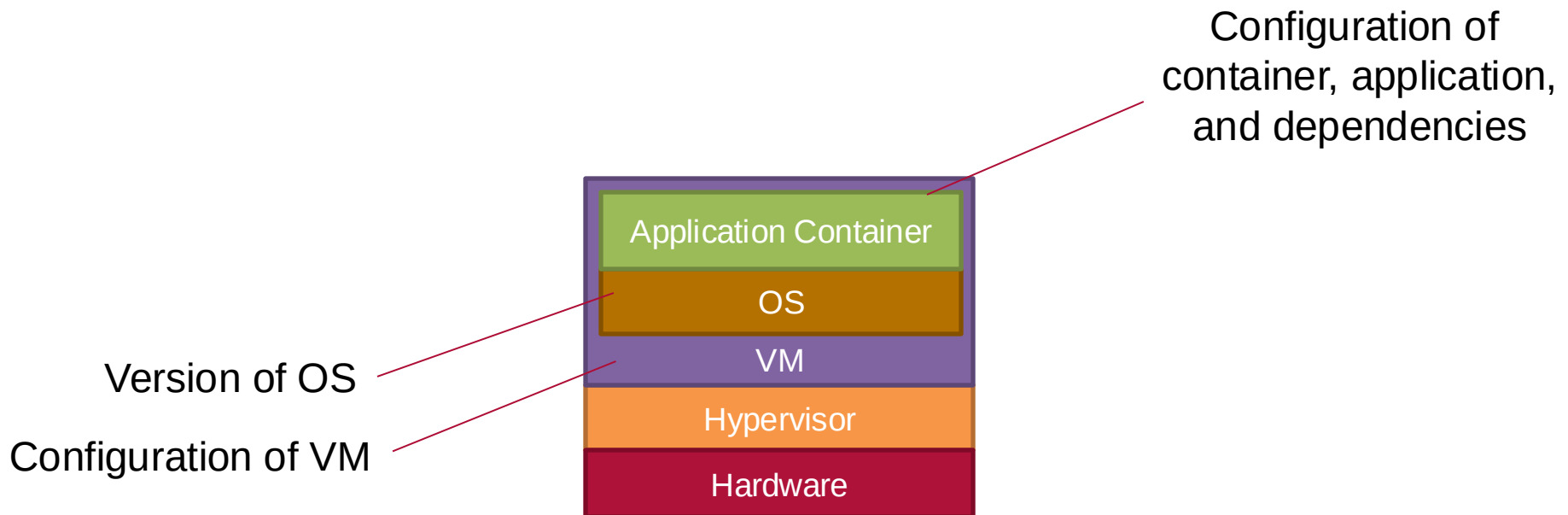
# Recap: Containers

- Motivation: VM images are large (contain OS) and starting VMs takes longer than processes (~10x)
- Containers: Same OS kernel, but different resource configuration and isolation of container processes
- Containment using Linux kernel features such as chroot, cgroups, and namespaces



# Management of Cloud Resources

- Major remaining problems in the “Cloud Age”: server sprawl, configuration drift, snowflake servers... → technical debt!





# Outline of Chapter 4:

## Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

**4.2 Cloud Operating Systems (like OpenStack)**

4.3 VM Management (including Migration)

— short break (10-15 minutes) —

4.4 Infrastructure as Code and IaC Tools (like Ansible)

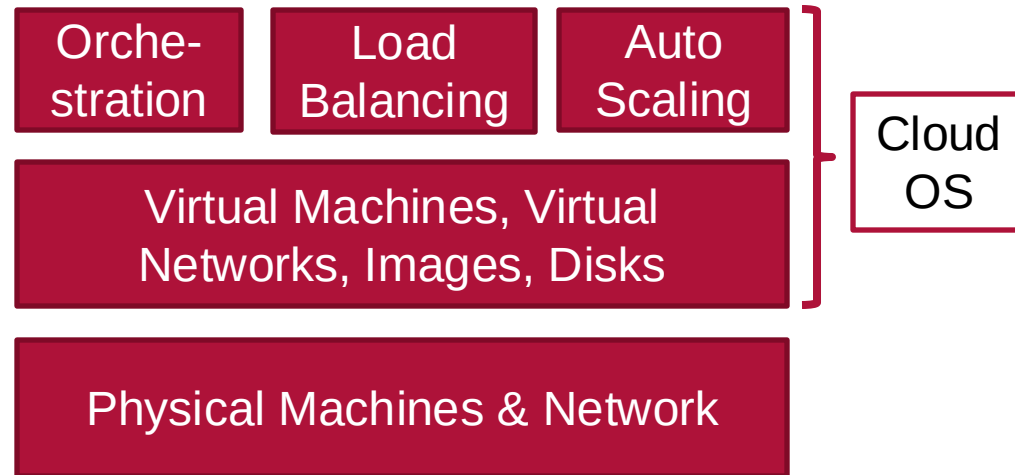
4.5 DevOps and CI/CD on Clouds

# Cloud Operating Systems (1/3)

- Virtualization is immensely useful, but managing many virtual machines manually is impractical
- Therefore: Cloud Operating Systems
  - Manage large pools of compute, storage, and networking resources
  - Provides dashboards and APIs for datacenter operators (administration) and users (provisioning) that abstracts infrastructures (e.g. the different hypervisors)
- Open-source systems: OpenStack, OpenNebula
- Commercial public clouds: e.g. EC2, GCE, Azure, Alibaba Cloud, Digital Ocean

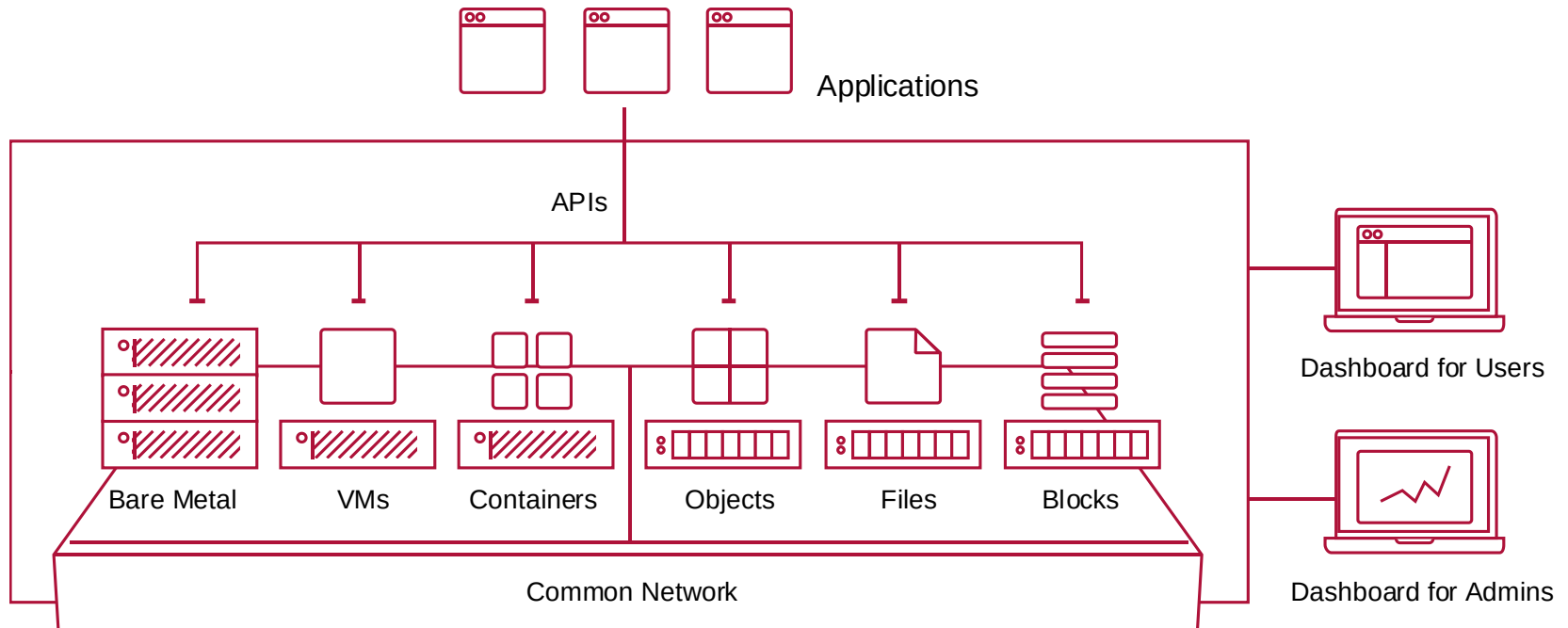
# Cloud Operating Systems (2/3)

- Basic functions: User Interface and APIs for
  - spawning and maintaining VMs,
  - virtual networking,
  - managing images and virtual disks
- Advanced functions:
  - orchestration,
  - load balancing,
  - and auto scaling



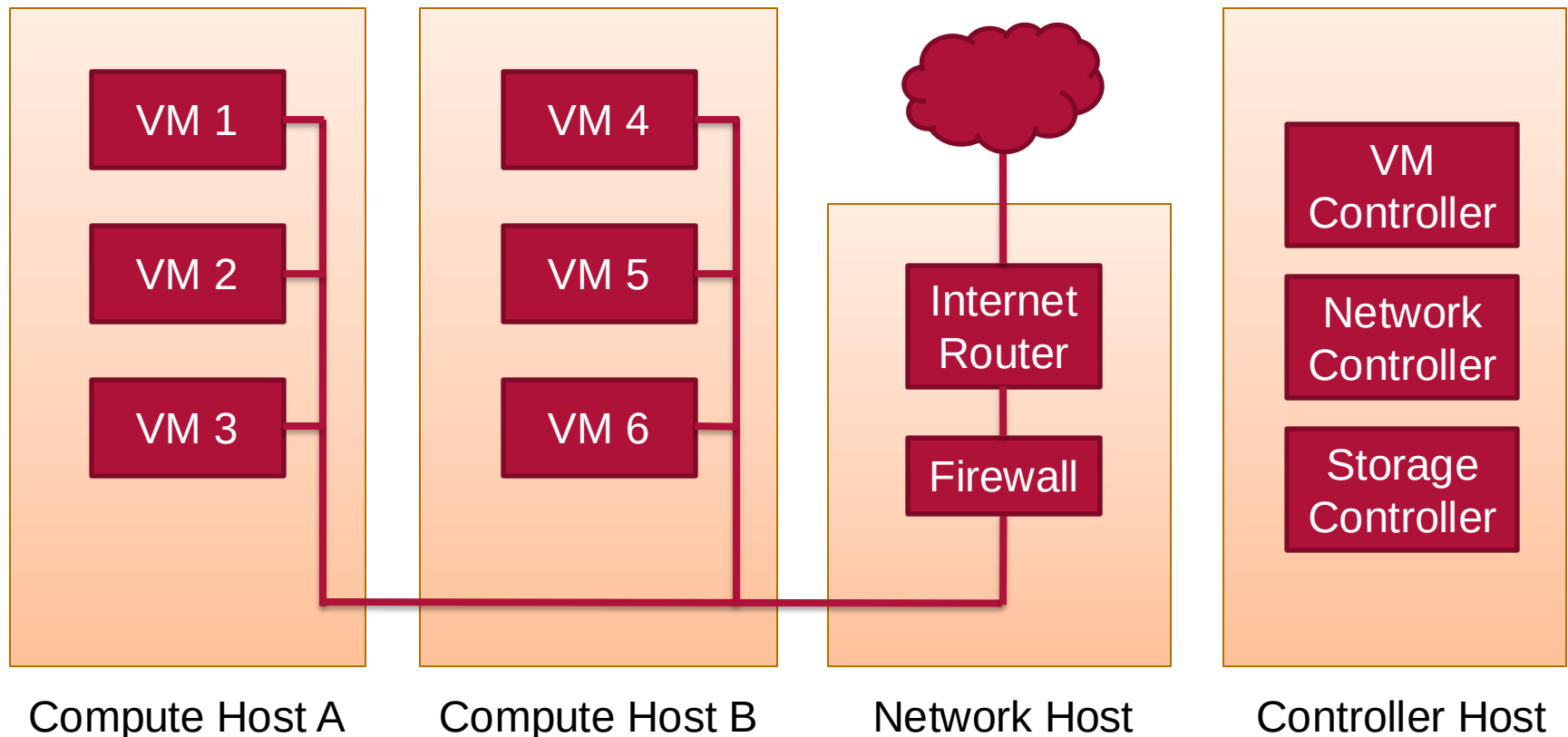
# Cloud Operating Systems (3/3)

- Interfaces to manage and provision virtual resources



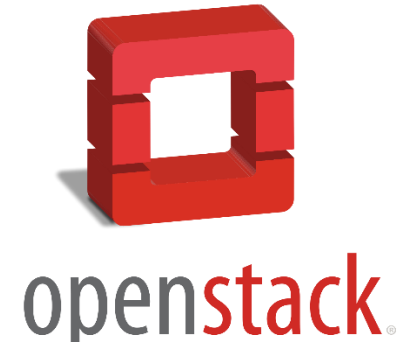
# Managed Cloud Environment

- Different roles for physical hosts: compute hosts, storage hosts, network hosts, and controller host

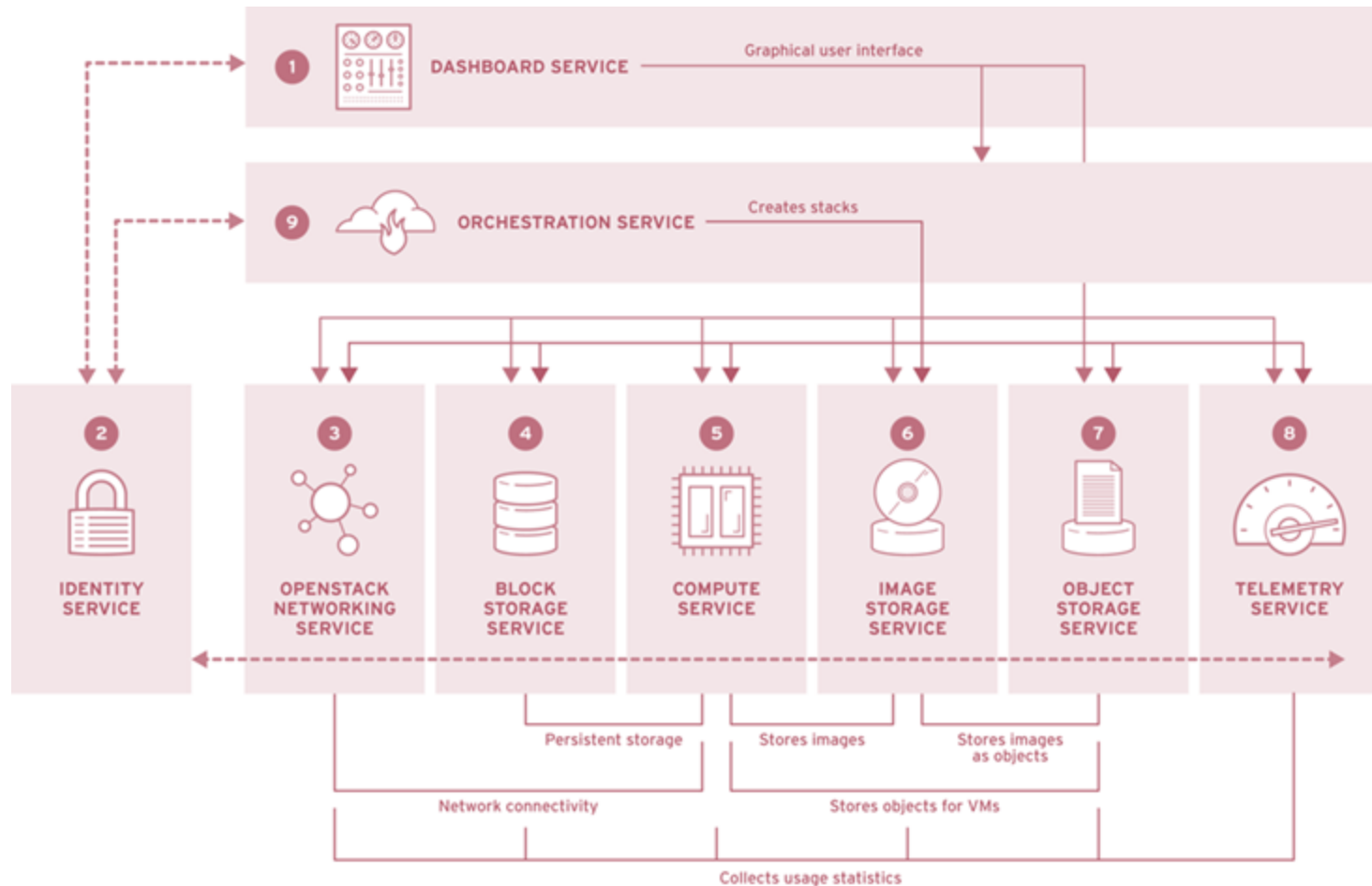


# OpenStack

- Open source cloud operating system
- Mostly deployed as IaaS, providing virtual machines and other resources to users
- Started in 2010 by Rackspace Hosting and NASA
- Contributions from IBM, Red Hat, HP, Cisco, Google, Oracle, EMC, VMware
- Most popular open cloud platform in both industry and academia

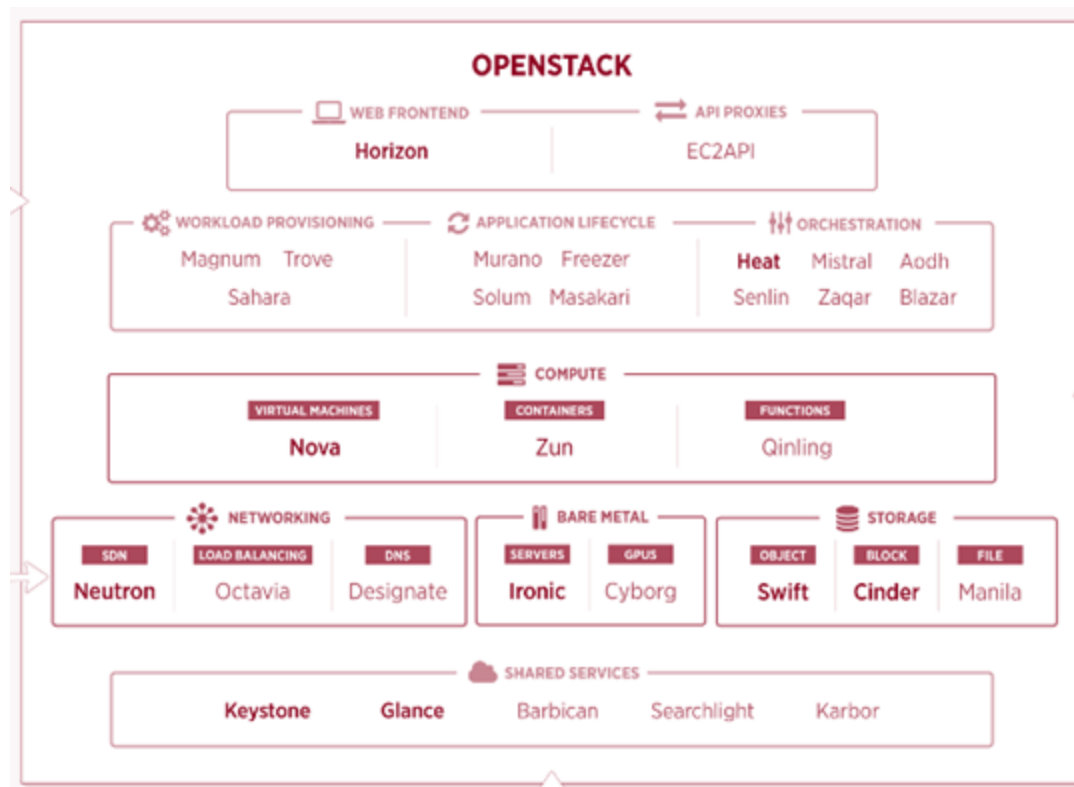


# Core Components of OpenStack (1/2)



# Core Components of OpenStack

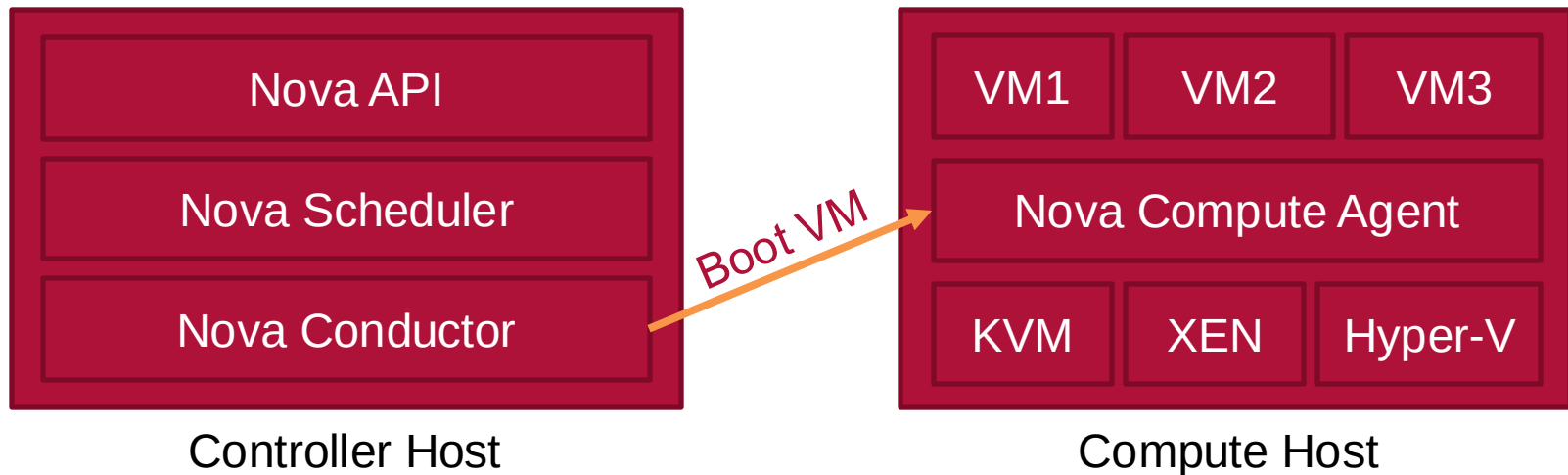
## (2/2)





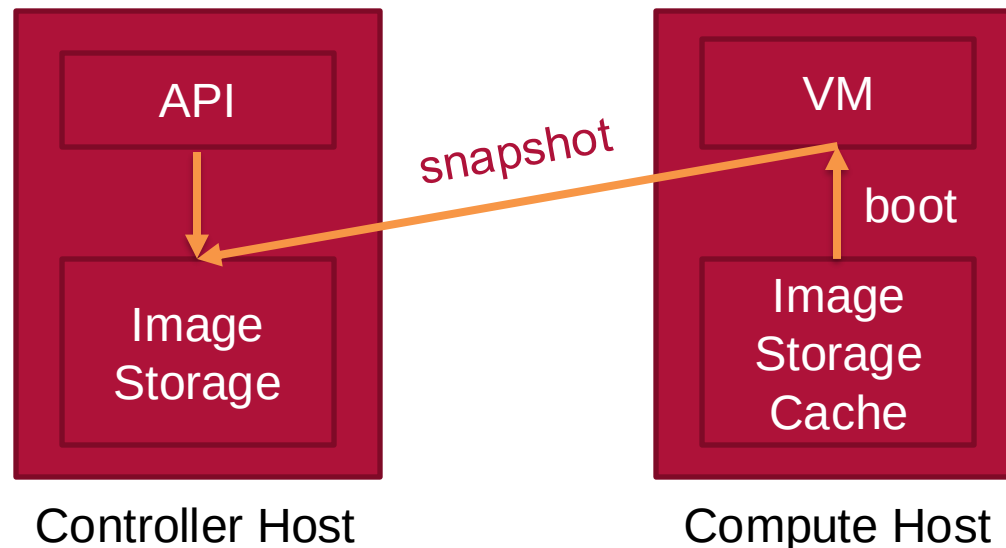
# Compute Service: *Nova*

- Allows to create and manage virtual machines on demand from images
- Schedules virtual machines to run on physical nodes
- Defines drivers that interact with underlying virtualization mechanisms



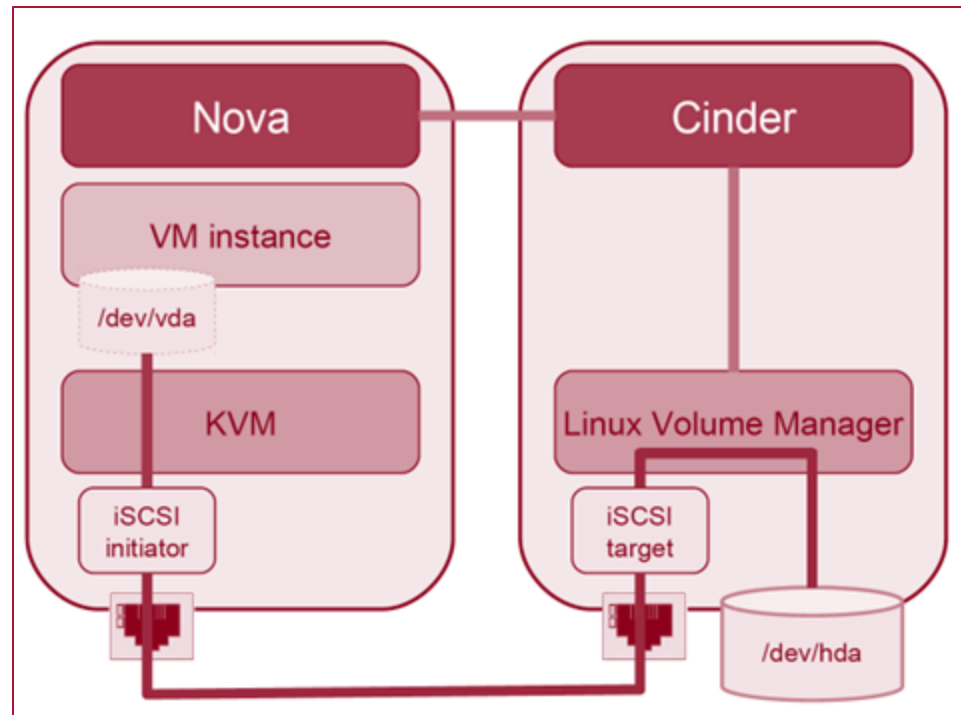
# Image Service: *Glance*

- Registry for virtual disk images
- Users can add new images or take snapshots of existing VMs
- Snapshots can be used as templates for new servers



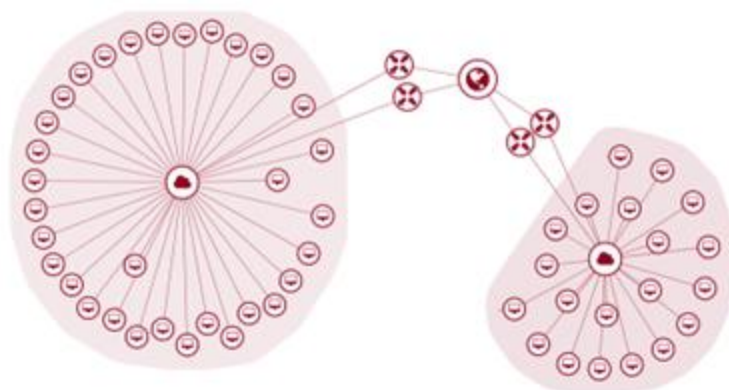
# Block Storage Service: *Cinder*

- Provides network-backed virtual block storage volumes
- Enables live migration of VMs and supports replication



# Virtual Networking: *Neutron*

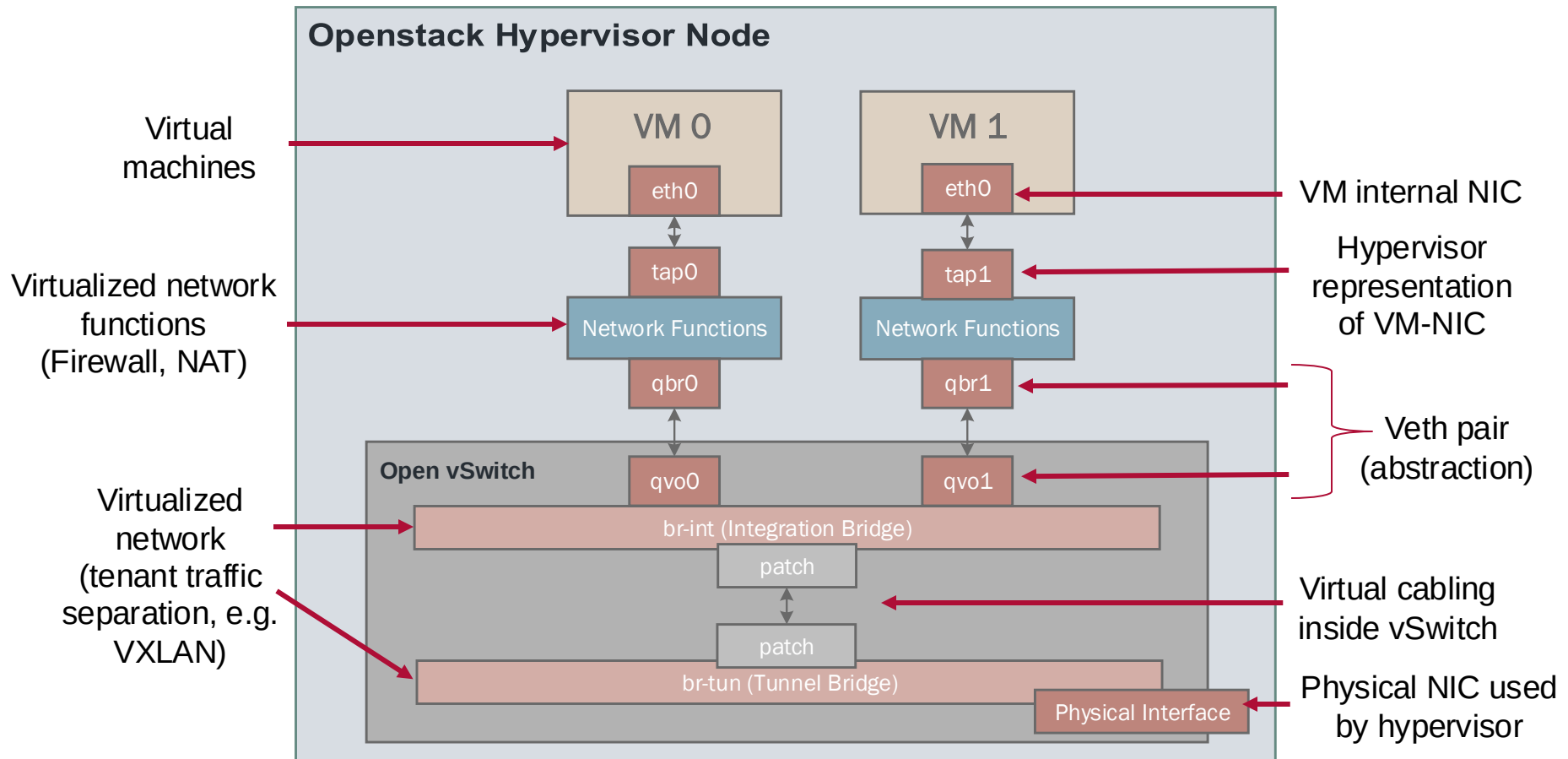
- Provides API for networking between virtual machines and physical network
- Interconnects virtual machines
  - within one hypervisor
  - between several hypervisors
  - with the Internet
- Provides additional network functions
  - Like firewalls, VPN, public IPs ...



# Virtual Switches & SDN

- Virtual switches: Virtual switching stack that enables network automation through programmatic extensions and standard protocols (e.g. OpenFlow)
- Software-Defined Networking (SDN)
  - Goal: Simplify network administration
  - Separation of control plane from data plane
  - Allows on the fly configuration of switches (controller-based decision making)

# Virtual Networking in OpenStack



# Outline of Chapter 4:

## Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

4.2 Cloud Operating Systems (like OpenStack)

**4.3 VM Management (including Migration)**

— short break (10-15 minutes) —

4.4 Infrastructure as Code and IaC Tools (like Ansible)

4.5 DevOps and CI/CD on Clouds

# VM Snapshots

- VM snapshots
  - Point-in-time copy that preserves memory and disk state in a VM image
  - Useful for roll-back or migration (and potentially for backups)
- Virtual snapshots
  - Reduce overhead for taking snapshots
  - *Copy-on-write* when changing the original, allowing to store just deltas (for one VM)
  - Not ideal for backups (as stored on the same host)



# VM Suspending vs. Pausing

- Suspending:
  - Full VM state is written to disk, so only disk resources and networking resources remain required
  - Resuming takes little time (way less than booting)
- Pausing:
  - Only the CPU activity is halted, so the VM does not run but does require main memory (and other resources)
  - Resuming takes very little time (less than resuming a suspended VM)

# VM Migration

- Migration: Move VM from one physical host to another
- Motivation:
  - Fault mgmt.: Host reports HW errors, must be shut down
  - Maintenance: Update of BIOS, hypervisor, ...
  - Load balancing: Move workload to another physical host
- **Cold migration:** Start a VM based on image elsewhere
- **Warm migration:** Suspend VM to disk; copy image and suspended state; resume execution elsewhere
- **Live migration:** Copy full state while VM executes

# Live VM Migration

- Desired property: Live migration
  - No shutdown of the virtual machine
  - No disruption of the service, no impact for the user
  - Minimize *downtime* and *total migration time*
- What needs to be migrated?
  - **Memory pages:** Ensure consistency between the memory state of the source and destination VMs
  - **Network resources:** Maintain open connections
  - **Storage resources:** Storage must be accessible both at the source and destination VM

# Strategies for Memory Migration

## 1. Push

- Source VM continues running, sends pages to destination
- Memory must potentially be sent multiple times
- Minimum downtime, potentially long migration time

## 2. Stop-and-copy

- Source VM stopped, pages are copied to destination VM
- Destination VM is started after having received all pages
- Short overall migration time, long downtime

## 3. Pull

- Execute new VM, pull accessed pages from source
- Performance depends on number of page faults

# Strategy for Memory Migration in XEN<sub>[15]</sub>

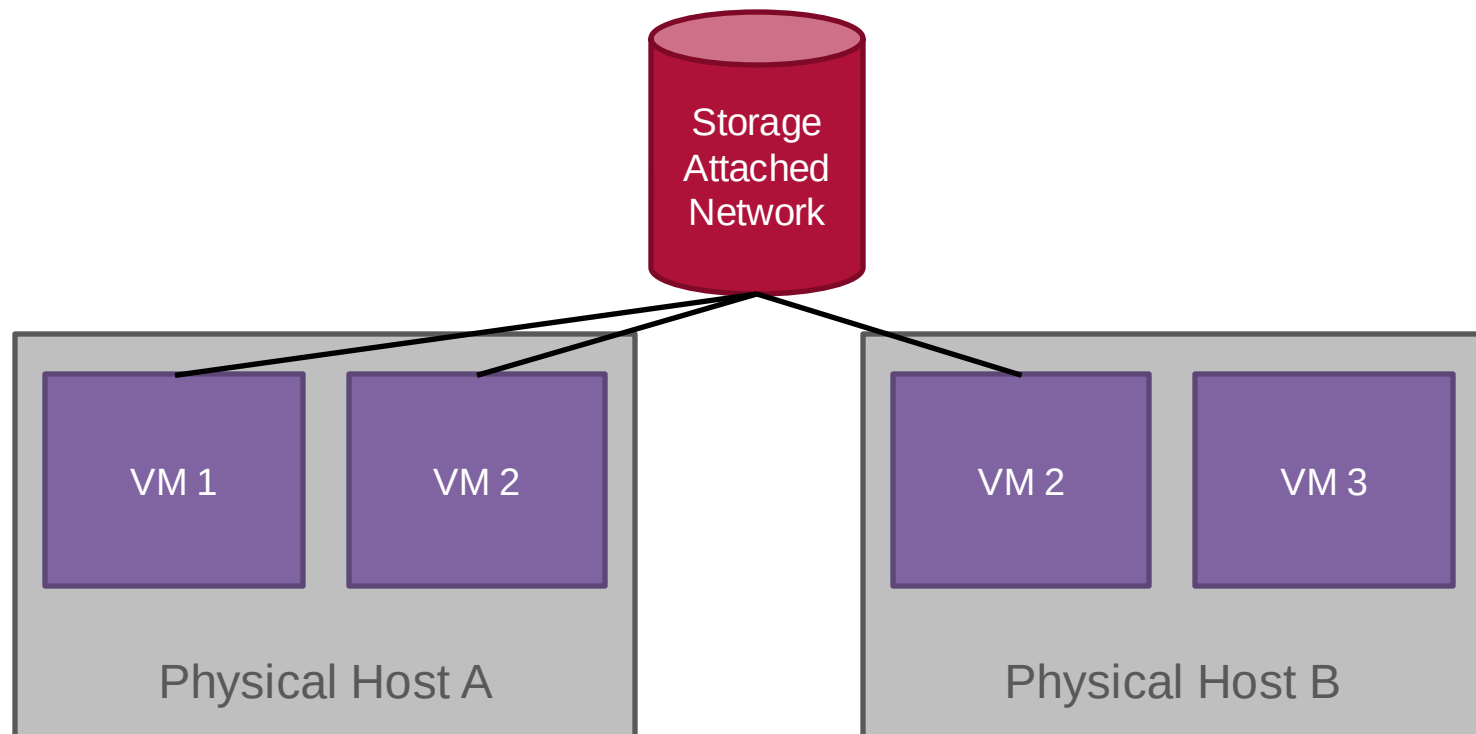
- XEN pursues *pre-copy* strategy for memory migration
  - Usage of a *push* and a *stop-and-copy* phase
  - Balances short downtime with short total migration time
- Iterative approach: Multiple rounds of push phase, then short stop-and-copy phase in the end
- Similar approach in VMware vMotion: vMotion also capable of slowing down source VM in case memory changes too fast for network transfer

# Strategy for Network Migration in XEN<sub>[15]</sub>

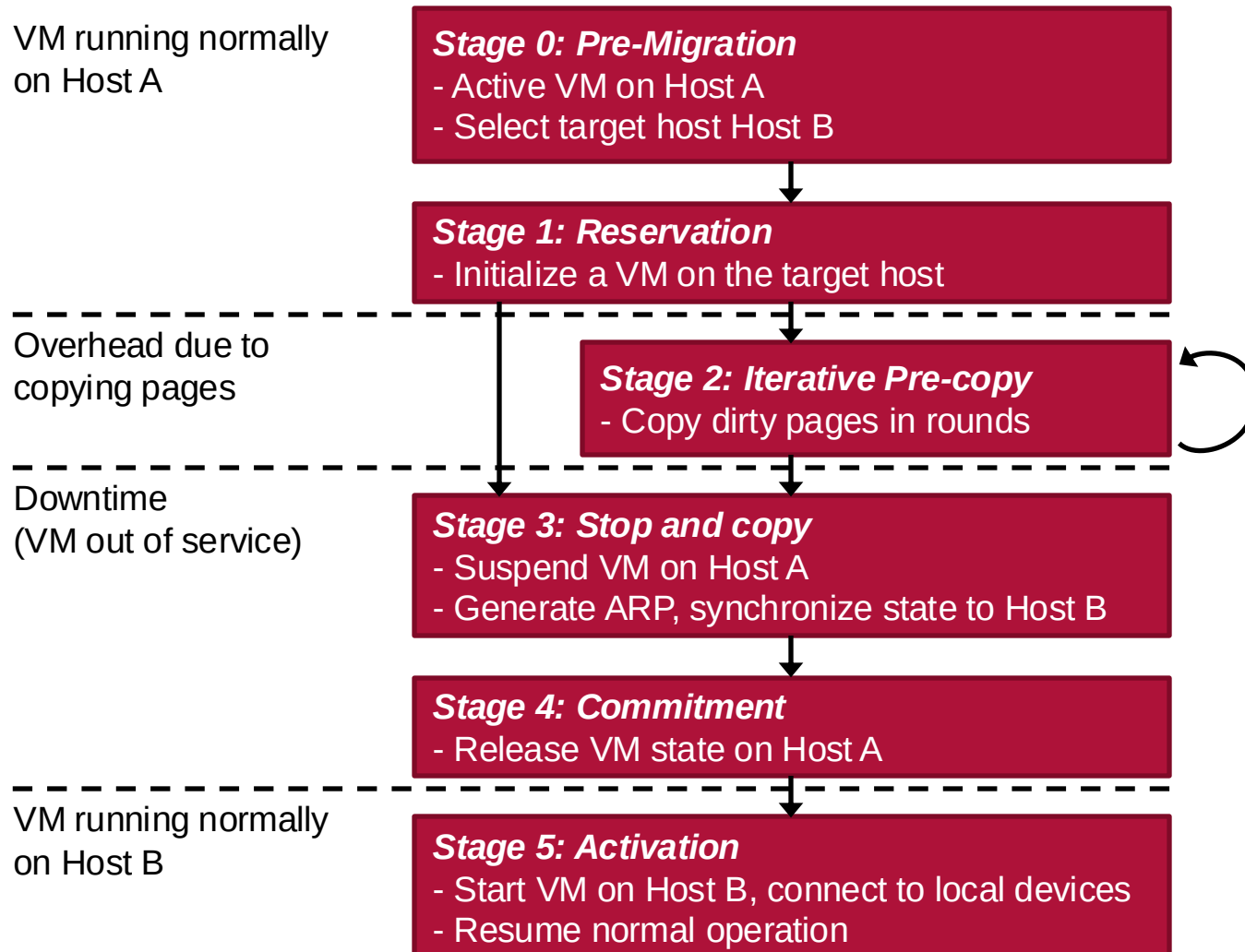
- Assumption: Source and destination VM are on same IP subnet (no IP-level routers involved)
- Approach for migration:
  - Destination VM will have new MAC but old IP address
  - After memory transfer, source host sends unsolicited ARP reply
    - ◆ Broadcast message to all hosts on the same network
    - ◆ Hosts will remove IP  $\leftrightarrow$  MAC mapping from caches
  - Upon new ARP request for IP, destination VM will return its MAC address

# Strategy for Storage Migration in XEN<sub>[15]</sub>

- XEN assumes VMs to reside on storage network (→ virtual / distributed storage)
  - Migration by rerouting network traffic
  - Solved with network migration

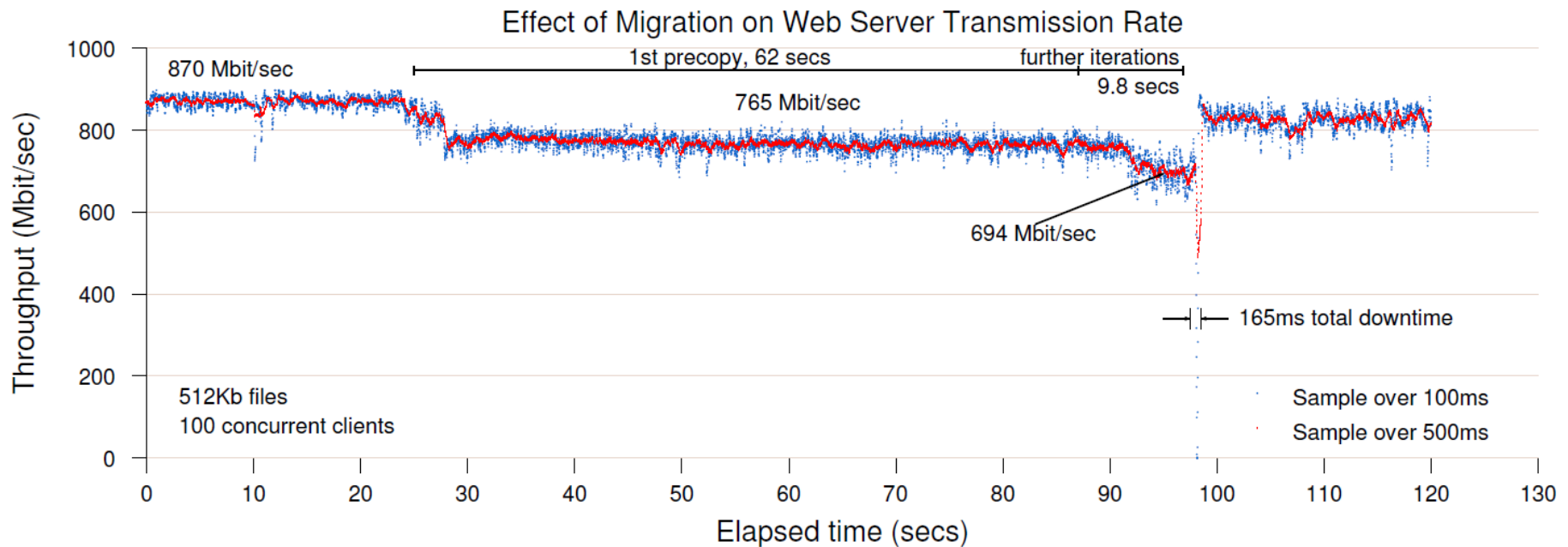


# XEN Migration Timeline<sup>[15]</sup>





# XEN Migration Performance Figures<sup>[15]</sup>



- Migration of a running web server VM (800 MB RAM)
  - Web server continuously serves 512 KB file to 100 clients

# Outline of Chapter 4: Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

4.2 Cloud Operating Systems (like OpenStack)

4.3 VM Management (including Migration)

**– short break (10-15 minutes) –**

4.4 Infrastructure as Code and IaC Tools (like Ansible)

4.5 DevOps and CI/CD on Clouds

# **Outline of Chapter 4:**

## **Cloud Infrastructure Management**

---

**4.1** Intro to Cloud Infrastructure Management

**4.2** Cloud Operating Systems (like OpenStack)

**4.3** VM Management (including Migration)

— short break (10-15 minutes) —

**4.4** Infrastructure as Code and IaC Tools (like Ansible)

**4.5** DevOps and CI/CD on Clouds

# Motivation

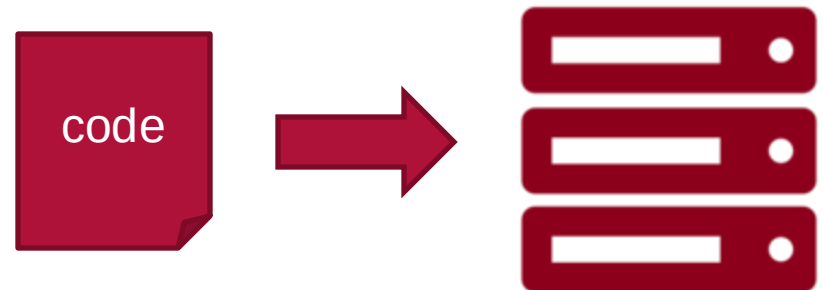
- Manual configuration of servers usually leads to
  - *Configuration Drift*: configuration of servers changed manually over time, without documentation
  - *and Snowflake Servers*:
    - ◆ Have a unique configuration
    - ◆ Are not reproducible when hardware dies
    - ◆ Are difficult to mirror for a test environment
    - ◆ Deliberate vs. default configuration
    - ◆ Host-specific vs. general configuration

# Open Questions

- What if a server fails?
- How to mirror a production environment for testing?
- How to keep a cluster of servers consistent?
- Disk snapshots allow to recover and mirror servers, but
  - still not straightforward to understand the state of servers
  - do not allow managing multiple servers that are mostly similar, but specific in a few configurations

# Infrastructure-as-Code (1/2)

- Goal: Make systems easily reproducible, support testing, and aid understanding configurations
- Solution: One single source of truth with a clear declaration of host-specific configuration → infrastructure definition files
  - that can be executed
  - that can be versioned
  - that can be shared
- Only use these definitions and automation tools, without manually updating single servers



# Infrastructure-as-Code (2/2)

- Define servers, networking, and other infrastructure elements in source code:
  - Configuration of environments
  - Dependencies with specific versions
- Automation tools built around infrastructure definitions
  - ➔ **immutable infrastructure**: (easily) rebuild servers instead of updating running servers

# Ansible

- Automation language and engine
- Resource orchestration
  - i.e. allocate and provision new servers
- Configuration management
  - i.e. configure servers
- Started in 2012, acquired by Red Hat in October 2015
- Well-documented conventions and best practices, but no artificial limits, so users can choose what to ignore

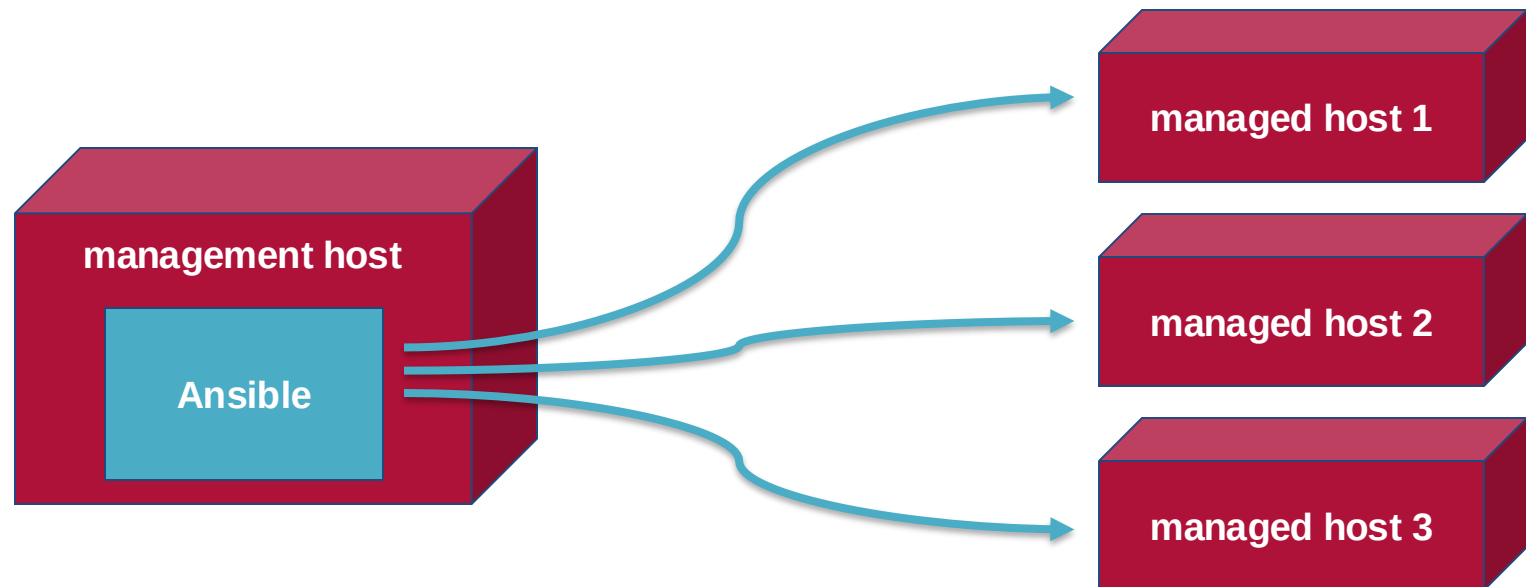




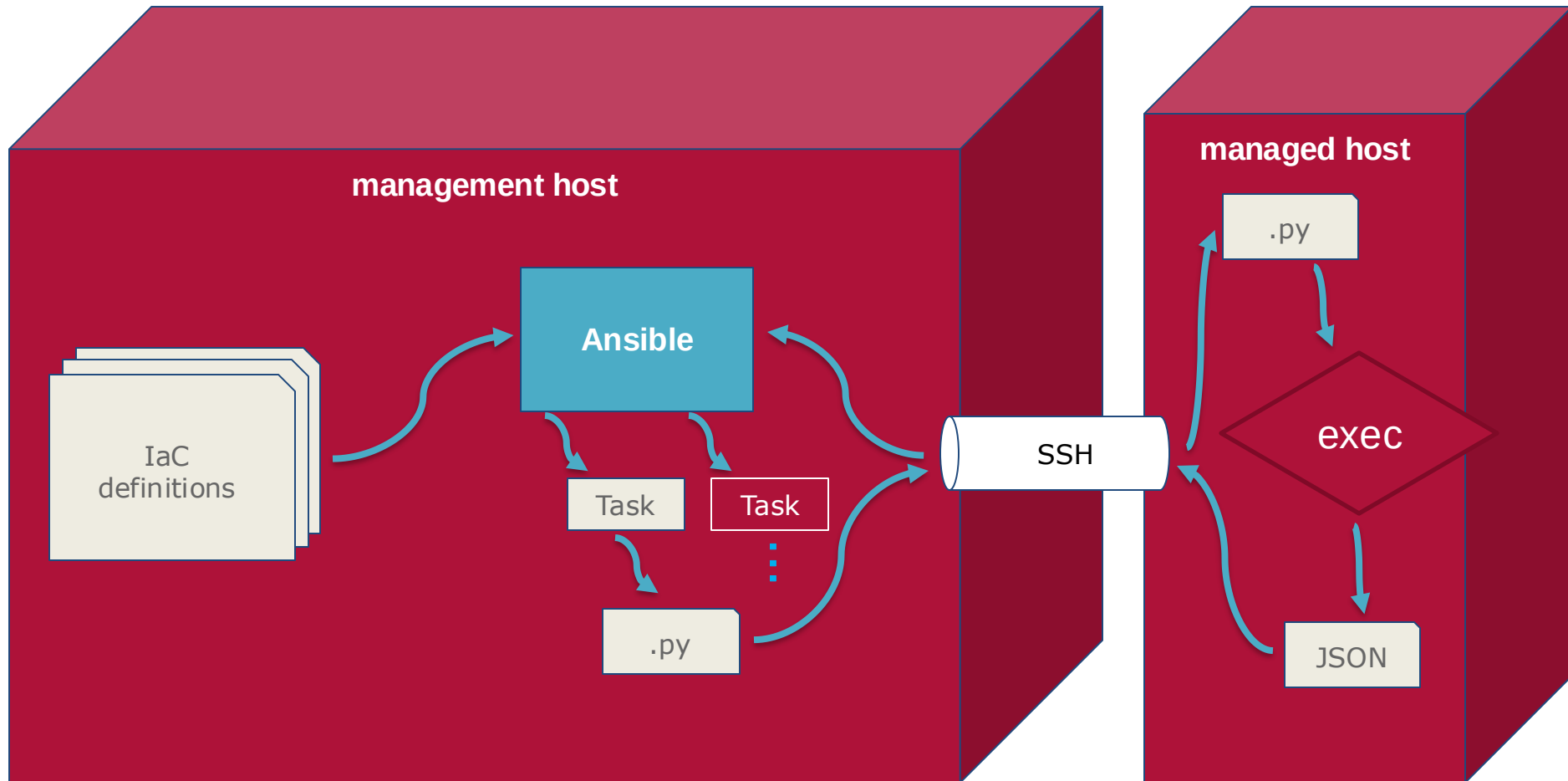
# Ansible

- Multi-platform
  - Operating systems: Linux, MacOS, Windows
  - Orchestration-related tools: LXC, libvirt, VMware, ...
  - IaaS services: AWS, Azure, OpenStack, ...
- Well-established technologies as the foundation
  - Python
  - SSH (PowerShell remoting, respectively)
  - YAML + Jinja templating
- Design: Declarative and agent-less

# Agent-Less Architecture (1/2)



# Agent-Less Architecture (2/2)



# Ansible Tasks (1/2)

- Declarative
  - i.e., define the desired state
- Opposed to imperative
  - i.e., define how to achieve the desired state
- Implies idempotency
  - i.e., applying the configuration again leads to same state
    - ◆ # that's the intention at least; you can, of course, create non-idempotent tasks

```
# an example "task":  
- name: "enforce permissions for .ssh directory of {{ ansible_user_id }}"  
  # ↑ "name" is optional but strongly encouraged  
  file: # ← the module name, ↓ followed by module parameters  
    dest: "{{ ansible_user_dir }}/.ssh"  
    state: directory  
    mode: u+rX,go-rwx  
    recurse: yes
```

# Ansible Tasks (2/2)

But to which hosts should this task be applied to?

```
# an example "task":  
- name: "enforce permissions for .ssh directory of {{ ansible_user_id }}"  
  # ↑ "name" is optional but strongly encouraged  
  file: # ← the module name, ↓ followed by module parameters  
    dest: "{{ ansible_user_dir }}/.ssh"  
    state: directory  
    mode: u+rX,go-rwx  
    recurse: yes
```

# Ansible Playbooks (1/2)

- Ansible's execution unit: Playbooks map everything to hosts
  - Tasks
  - Roles
  - Variables
- Further features include:
  - Rolling updates
    - ◆ Configure only n hosts at a time
    - ◆ Configure only x% of host at a time
  - Tolerate percentage of hosts where configuration fails
  - ...

```
# file: my_playbook.yml

# a "Play":
- name: configure internal Web servers
  hosts: webservers
  roles:
    - webserver
  tasks:
    - name: forbid external connections
      iptables:
        chain: INPUT
        source: !192.168.1.1/24
        jump: DROP
```

# Ansible Playbooks (2/2)

But how does Ansible  
know the hosts  
"webservers"?

```
# file: my_playbook.yml

# a "Play":
- name: configure internal Web servers
  hosts: webservers
  roles:
    - webserver
  tasks:
    - name: forbid external connections
      iptables:
        chain: INPUT
        source: !192.168.1.1/24
        jump: DROP
```

# Ansible Inventory

```
# no FQDNs required, as long as
# ``ssh <hostname>`` works (~/.ssh/config)
mail
```

```
[webservers]
w1.example.com
w2.example.com
```

```
[dbservers]
d1.example.com
d2.example.com
```

```
[appservers]
# we can override variables:
a01.example.com www_user=web
# we can use a pattern-like syntax:
a[02-12].example.com
```

```
# we can define variables for groups:
[appservers:vars]
www_user=www-data
```

```
# a group of groups
[rootservers:children]
appservers
dbservers
```

```
# alternatively, we can use YAML:
```

```
all:
  hosts:
    mail.example.com:
  children:
    webservers:
      hosts:
        w1.example.com:
        w2.example.com:
    dbservers:
      hosts:
        d1.example.com:
        d2.example.com:
  ...
```

- Inventories can also be obtained from external scripts / services (e.g. IaaS clouds)



# Ansible Variables (1/2)

- Variables can be defined in many places
  - Playbooks, Tasks, Inventory, CLI args, roles, facts, ...
  - But scattering variables all over the place is discouraged, of course
  - e.g., using role, group and host variables can suffice
- "facts"
  - A heap of auto-detected variables: e.g., Uptime, chroot?, user home, SSH host key, Python version, total and free memory, host name, FQDN, environment variables, block devices, mounts, interfaces, BIOS version, processor specs, OS name and version, ...
- Jinja templating also in variables files
  - e.g., app\_user: "{{ ansible\_env.SUDO\_USER|default(ansible\_user\_id) }}"

# Ansible Variables (2/2)

## ■ Host and group variables

◆ e.g., in project directory:

```
# e.g., set inventory to ./inventory.ini:
.ansible.cfg
inventory.ini
my_playbook.yml
group_vars/
    # applies to all hosts:
    all.yml
    # applies to webservers:
    webservers.yml
host_vars/
    mail.yml
```

```
# no FQDNs required, as long as
# ``ssh <hostname>`` works (~/mail
```

```
[webservers]
w1.example.com
w2.example.com
```

```
[dbservers]
d1.example.com
d2.example.com
```

```
[appservers]
# we can override variables:
a01.example.com www_user=web
# we can use a pattern-like syntax
a[02-12].example.com
```

```
# we can define variables for group
[appservers:vars]
www_user=www-data
```

```
# a group of groups
[rootservers:children]
appservers
dbservers
```

# Run a Playbook

```
$ cat my_playbook.yml
```

```
- name: test SSH connection
  hosts: pim:static
  tasks:
    - name: test if logging into host and running a module remotely works
      ping: # does not require any arguments
```

```
$ ansible-playbook my_playbook.yml
```

```
PLAY [test SSH connection] *****
```

```
TASK [Gathering Facts] *****
```

```
ok: [static]
```

```
ok: [pim]
```

```
TASK [test if logging in and running a module remotely works] *****
```

```
ok: [pim]
```

```
ok: [static]
```

```
PLAY RECAP *****
```

```
pim          : ok=2    changed=0    unreachable=0    failed=0
```

```
static       : ok=2    changed=0    unreachable=0    failed=0
```

# Outline of Chapter 4:

## Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

4.2 Cloud Operating Systems (like OpenStack)

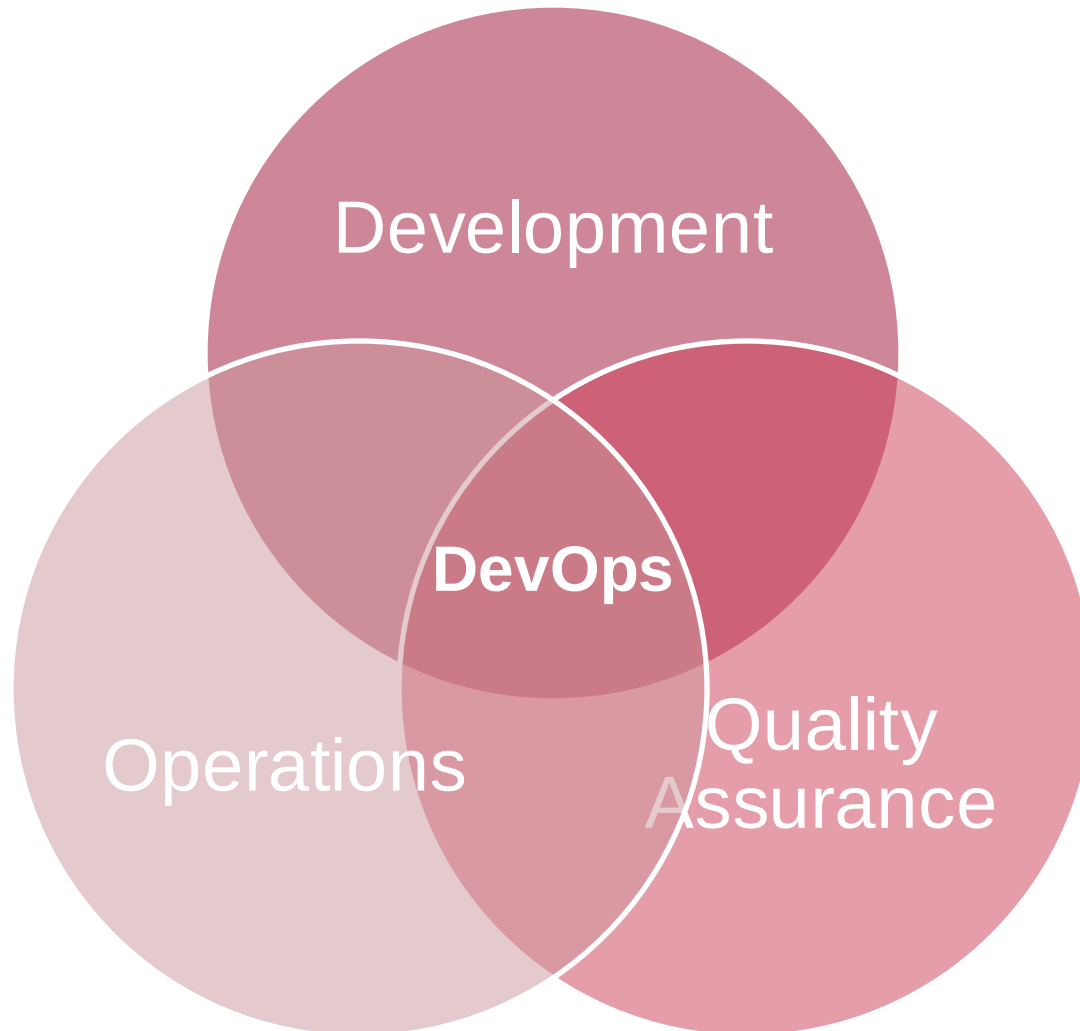
4.3 VM Management (including Migration)

— short break (10-15 minutes) —

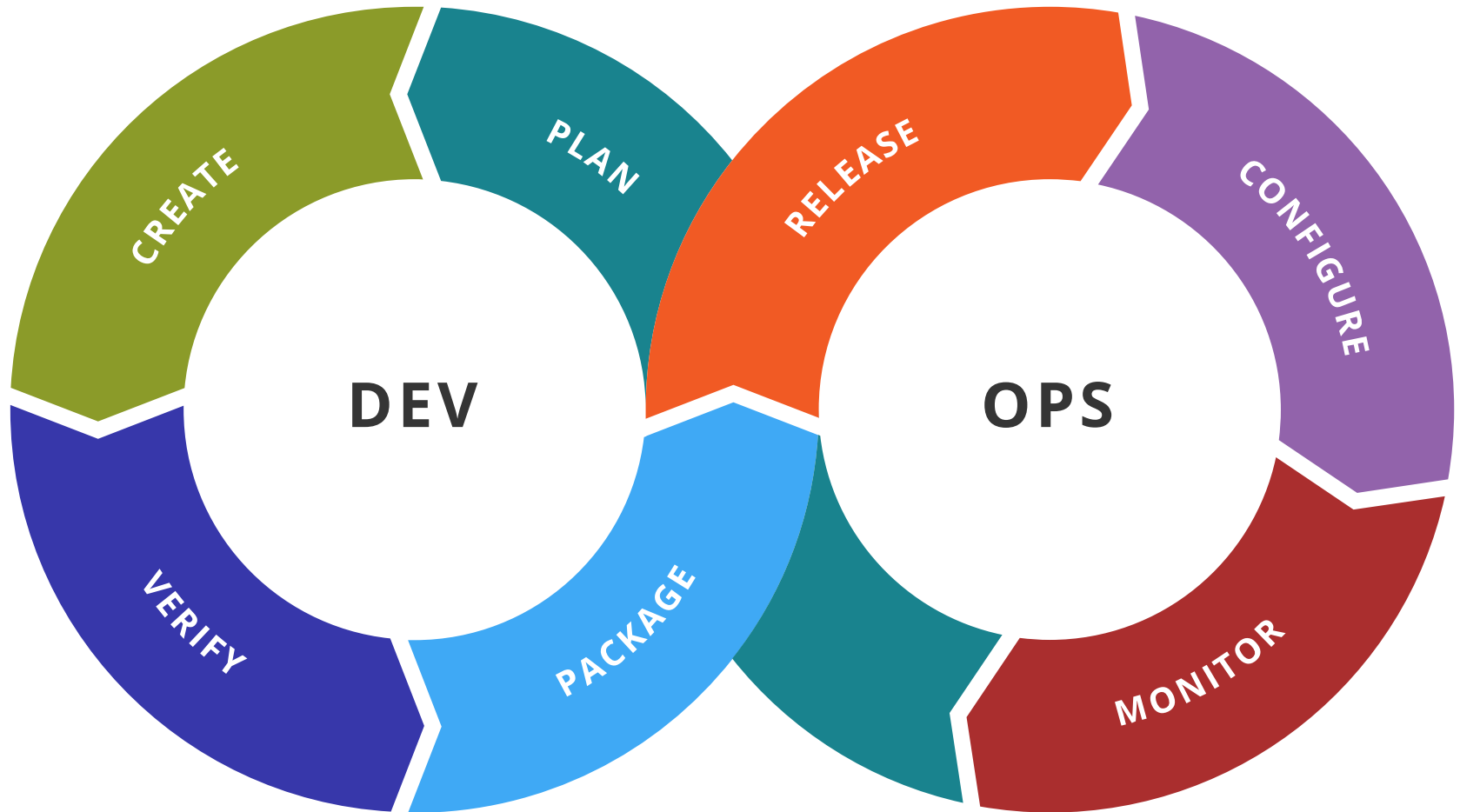
4.4 Infrastructure as Code and IaC Tools (like Ansible)

**4.5 DevOps and CI/CD on Clouds**

# DevOps



# DevOps

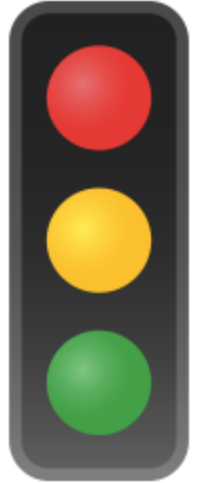


# Continuous Integration

- Automation of *integration and testing* of software
- First proposed by Grady Booch, 1991
- Adopted and popularized as part of XP (Extreme Programming)
- Motivation:
  - Evade “integration hell”: Find bugs while still easy to fix
  - Fast feedback on system-wide impact of local changes
  - Team has a common view of the software project state

# Continuous Integration: Practices

- One common (versioned) code repository
- Build automation (and short builds)
- Self-testing builds
- Regular commits
- Building every commit (usually on a CI server)
- Test & production environments as similar as possible
- Test results visible for everyone

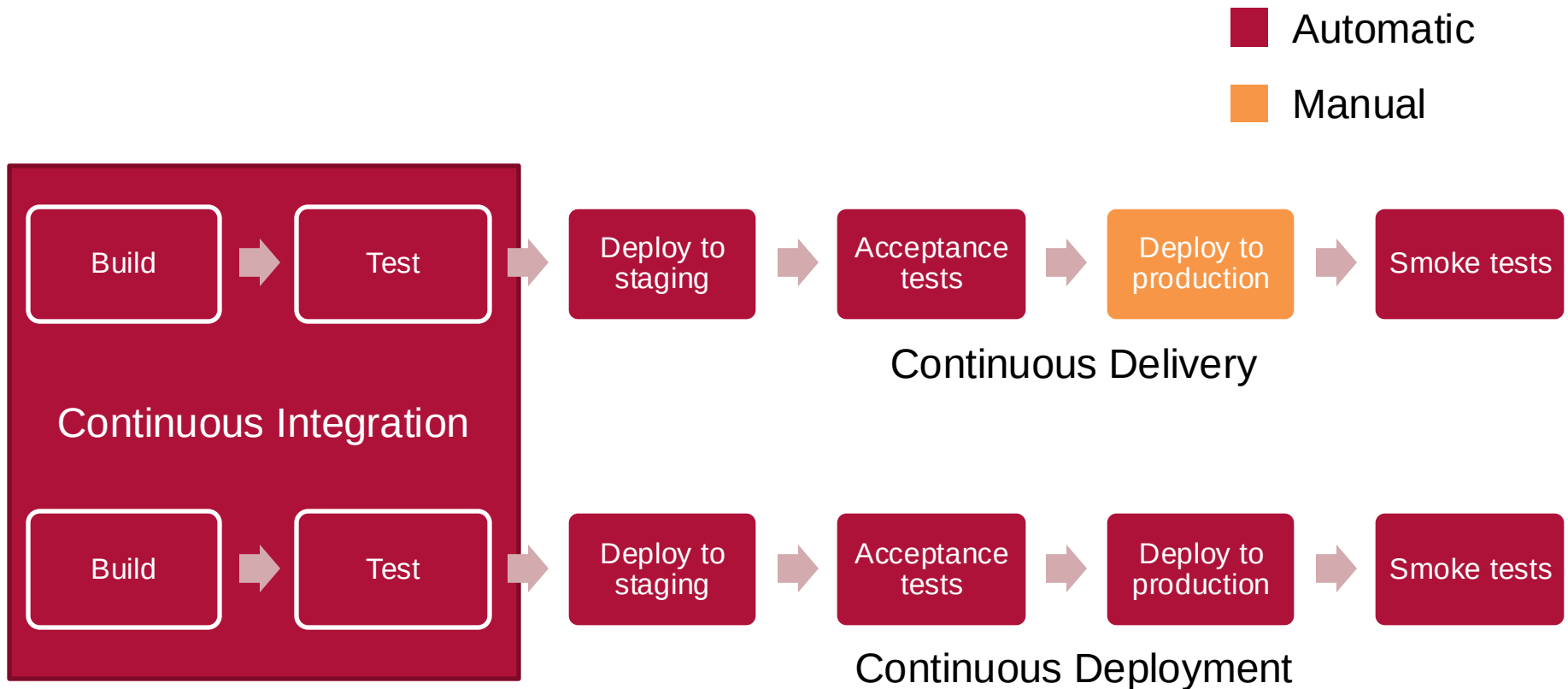




# Continuous Delivery

- Fast and reproducible software releases
- From the first principle of the agile manifesto:  
*“Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.”*
- CD Metric: How long does it take to release a single-line-code-change to production? (*cycle time*)
- Business case: Software development as investment  
→ optimizing ROI through small cycle time

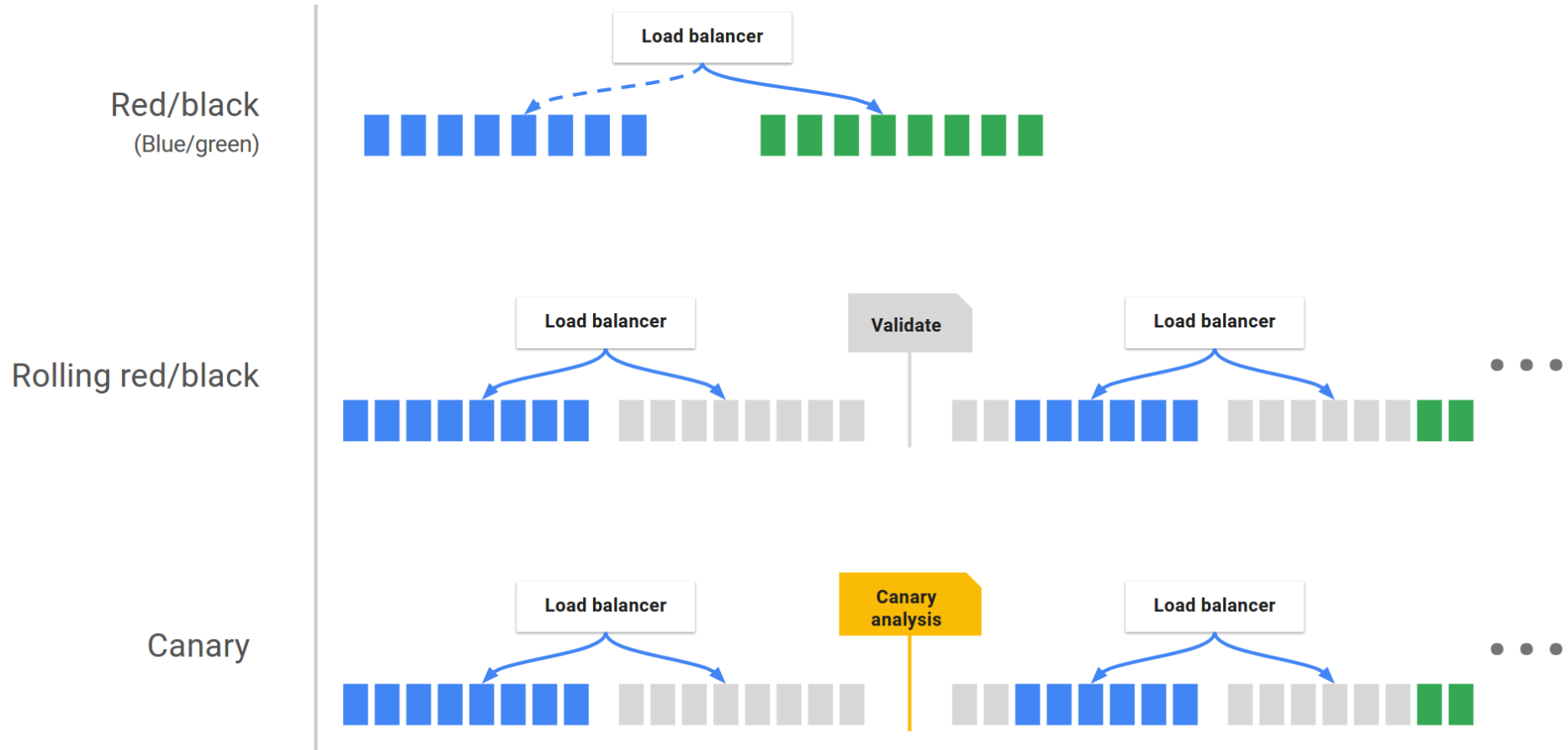
# Continuous Delivery vs. Continuous Deployment



# Deployment Strategies: Blue/Green

- Strategy to deploy an upgrade
- Two production environments: Blue and green
- Only one environment is active at a time
  1. Deploy to the non-active environment
  2. Switch the traffic coming through the load balancer over to the new environment
  3. Monitor new version, maybe switch back
- One environment always runs the last version, the other the newest version: easy rollbacks

# Deployment Strategies

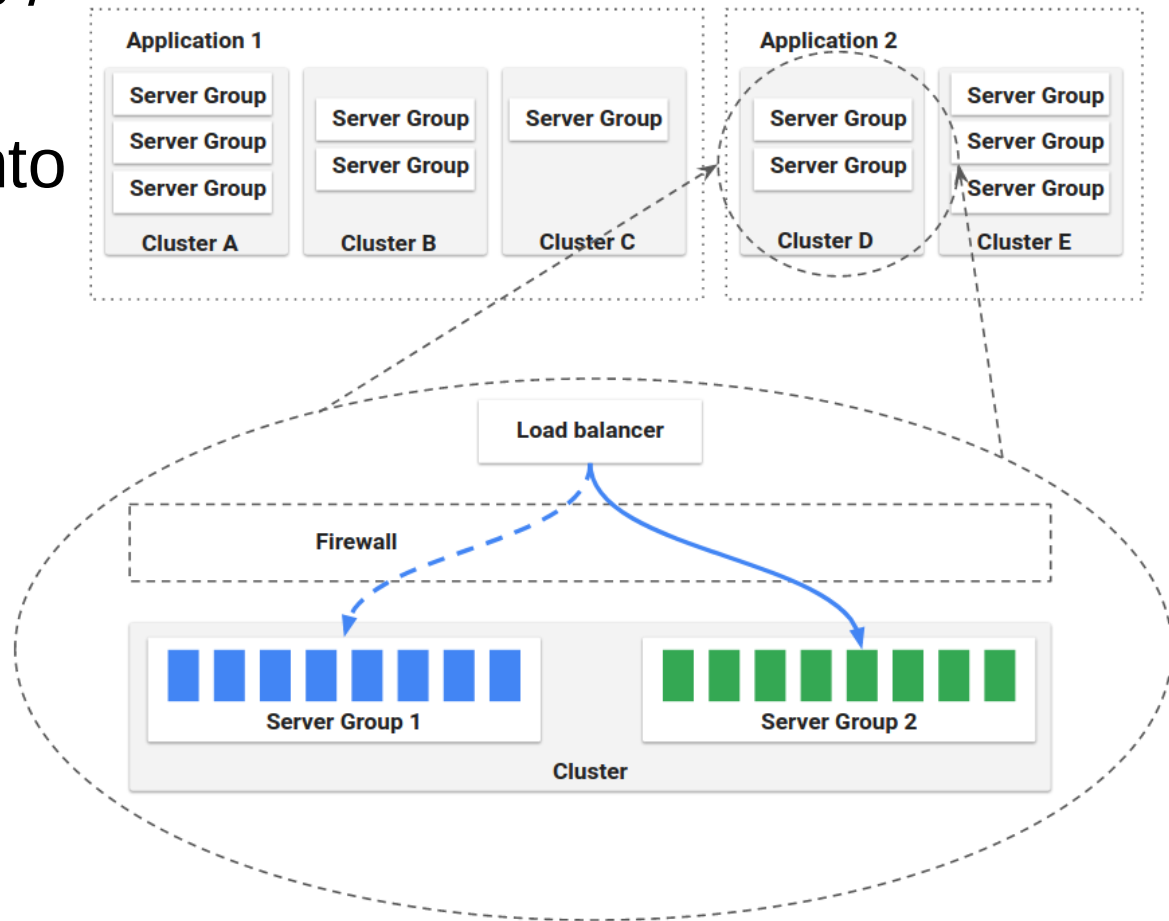


# Spinnaker

- Continuous delivery platform
- Started by Netflix, picked up and extended by Google
  - Initial release November 2015
  - Successor to Asgard, a web interface for automated AWS deployments
  - From baking to cloud deployment
- Support and common abstractions for EC2, GCE, Azure, Kubernetes...

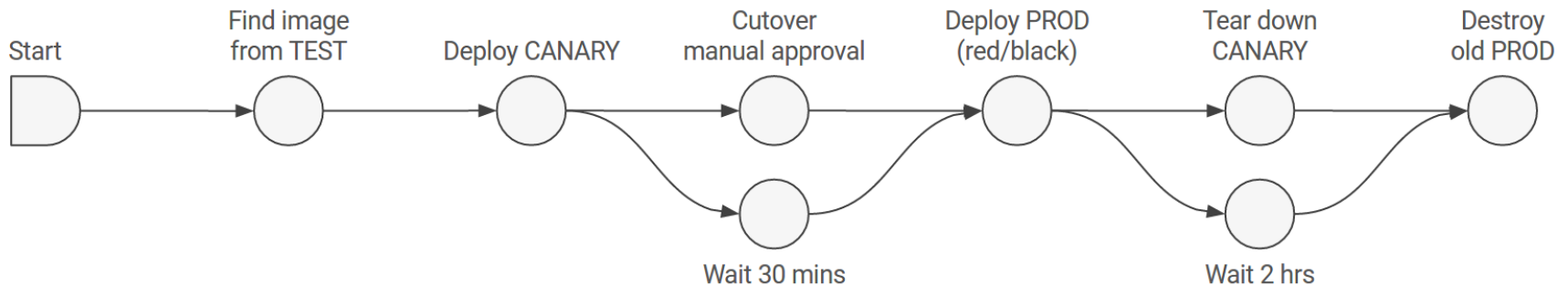
# Spinnaker

- Deploy applications / (micro)services
- Organize servers into
  - Clusters
  - Server groups

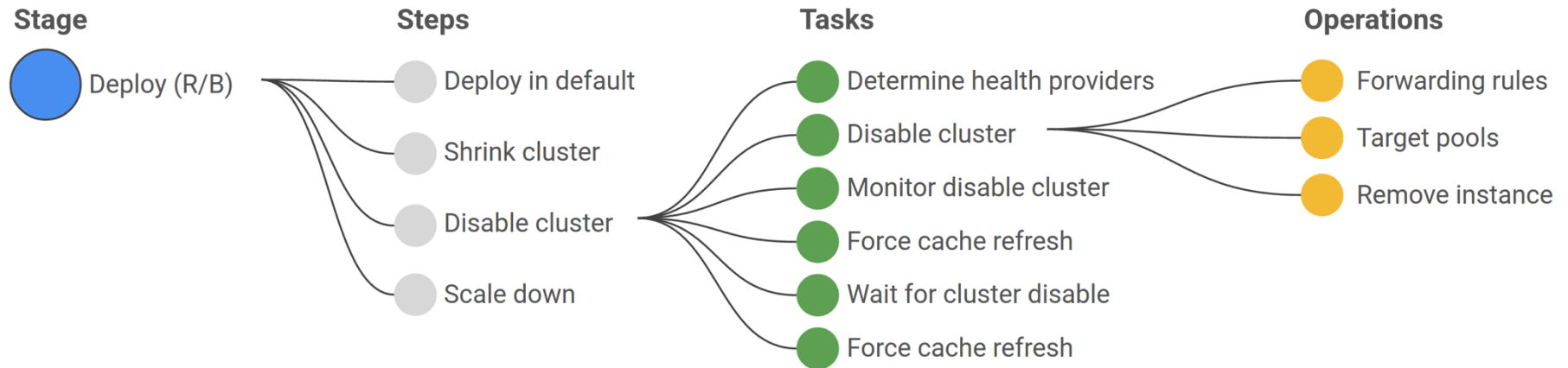


# Spinnaker Pipeline

- Deployment happens via pipelines
- Pipelines are built from stages
- Predefined stages exist for many things
  - Bake
  - Manual judgement
  - Clone server group
  - Deploy



# Spinnaker Stage Example





# Outline of Chapter 4:

## Cloud Infrastructure Management

---

4.1 Intro to Cloud Infrastructure Management

4.2 Cloud Operating Systems (like OpenStack)

4.3 VM Management: Pausing and Migration

– short break (10-15 minutes) –

4.4 Infrastructure as Code and IaC tools (like Ansible)

4.5 DevOps and CI/CD on Clouds

# Summary

---

- Server sprawl, configuration drift, and snowflake servers exemplify the complexity of managing cloud infrastructure
- Cloud operating and orchestration systems as well as automation engines help, but development and operating processes need to change as well:
  - IaC: Strictly automate and rebuild infrastructure
  - DevOps: Responsibility extends to operation
  - CI/CD: Integrate and deploy systems continuously

# References

- [15] C. Clark, K. Fraser, S. Hand, J.G. Hanseny, E. July, C. Limpach, I. Pratt, A. Wareld: “Live Migration of Virtual Machines”, Proc. of the 2nd Symposium on Networked Systems Design & Implementation, 2005
- Real-world systems discussed:
  - <https://www.openstack.org/>
  - <https://www.xenserver.com/>
  - <https://www.ansible.com/>
  - <https://spinnaker.io/>



# Cloud Systems

## Chapter 5: Cloud Sustainability

Dr Lauritz Thamsen

[lauritz.thamsen@glasgow.ac.uk](mailto:lauritz.thamsen@glasgow.ac.uk)

School of Computing Science

University of Glasgow



# First Half: Lecture Chapters

---

Cloud Resource Management:

1. Cloud Computing Intro
2. Virtual Machines
3. Containers
4. Cloud Infrastructure Management
- 5. Cloud Sustainability**

# Outline of Chapter 5: Cloud Sustainability

---

4.1 The Rising Footprint of ICT and Data Centers

4.2 Energy Efficiency, Energy Proportionality, and  
Power Usage Effectiveness

— short break (10-15 minutes) —

4.3 Cloud Carbon Footprint Assessment

4.4 Carbon-Aware Cloud Computing

# Outline of Chapter 5: Cloud Sustainability

---

**4.1 The Rising Footprint of ICT and Data Centers**

**4.2 Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness**

— short break (10-15 minutes) —

**4.3 Cloud Carbon Footprint Assessment**

**4.4 Carbon-Aware Cloud Computing**

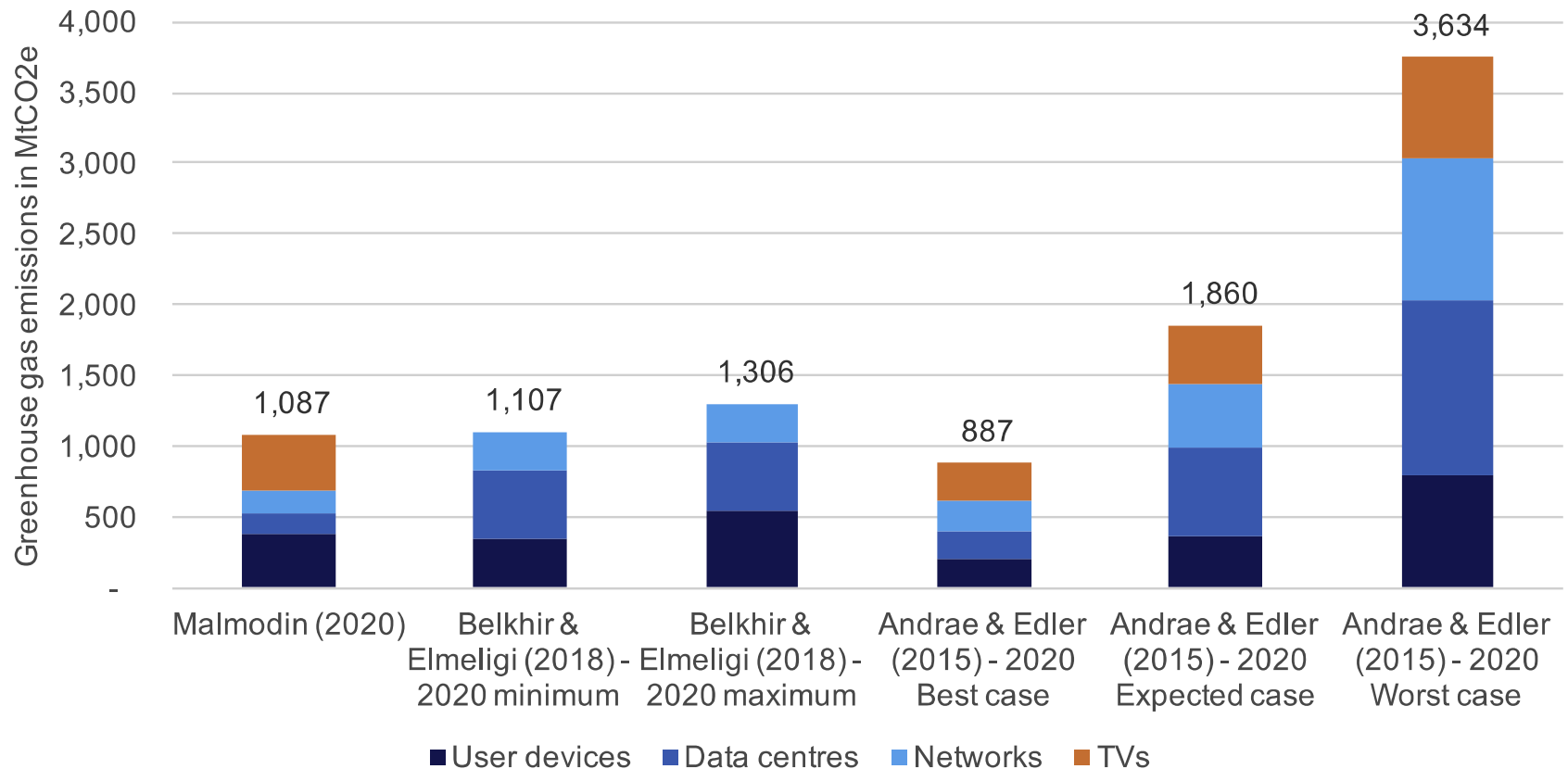
# Estimated Emissions from ICT

- ICT was estimated to produce between **1.0 and 1.7 gigatons of CO<sub>2</sub>e** in 2020 [1]
  - **Information and Communication Technologies (ICT):** mainly user devices, data centers, networking infrastructure
  - **CO<sub>2</sub>e: carbon dioxide equivalent** of all greenhouse gas (GHG) emissions
  - **Whole lifecycle emissions:** From material sourcing, manufacturing, and transport to the use of energy for operation and emissions for disposal/recycling



# Estimates of ICT's Footprint Differ

- Estimates for 2020 [1]

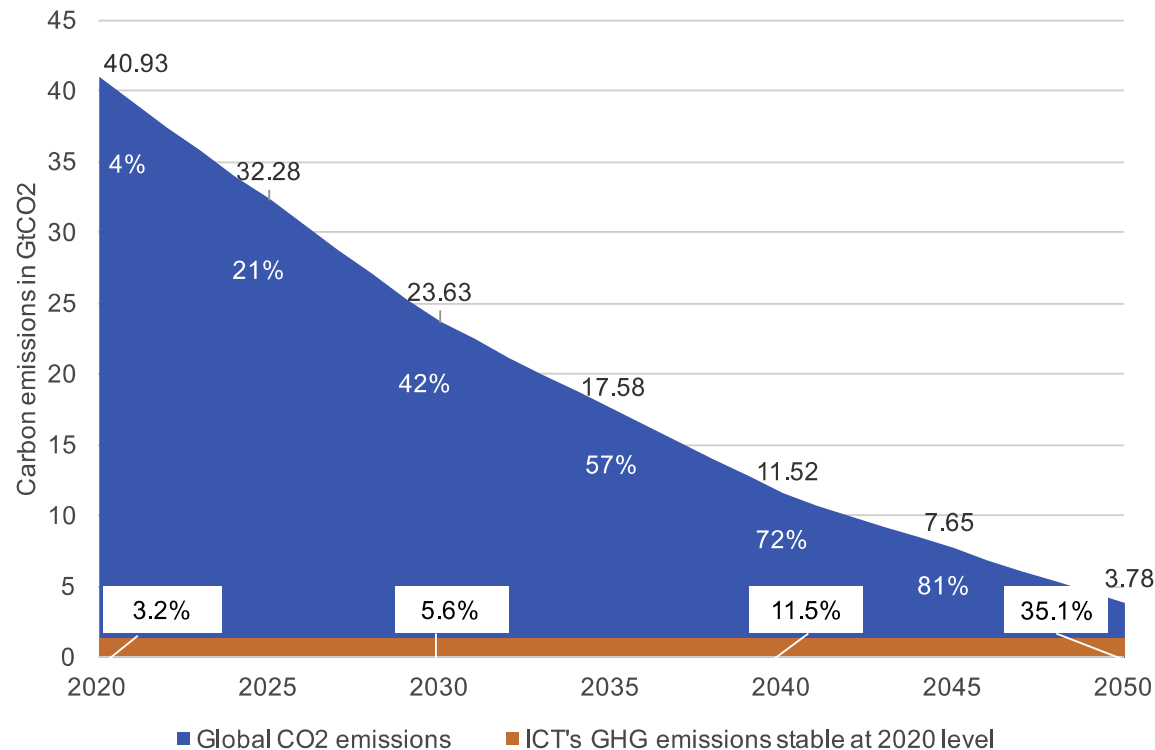


# How Does This Compare?

- 1.0 and 1.7 gigatons CO<sub>2</sub>e in 2020 equates to **1.8%–2.8% of global GHG emissions**
- World-wide commercial aviation? Estimated to be [around 2% in 2021](#)
- UK emissions in 2020? [Around 0.42 gigatons](#)

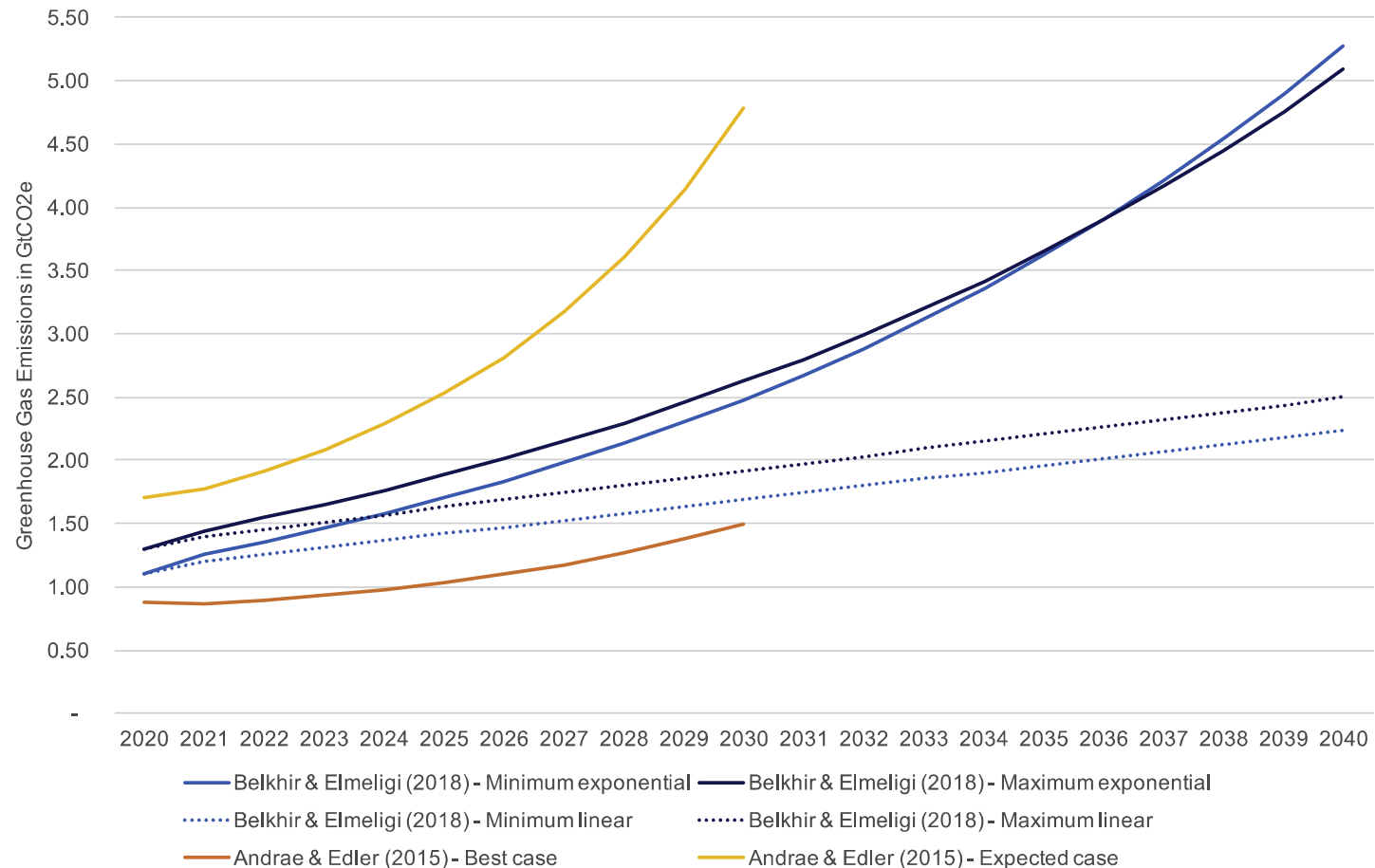
# ICT's Share Might be Rising

- What if ICT's carbon emissions stay the same, while other emissions go down as needed to likely limit global warming to 1.5 degrees (IPCC SSP2-19)? [1]



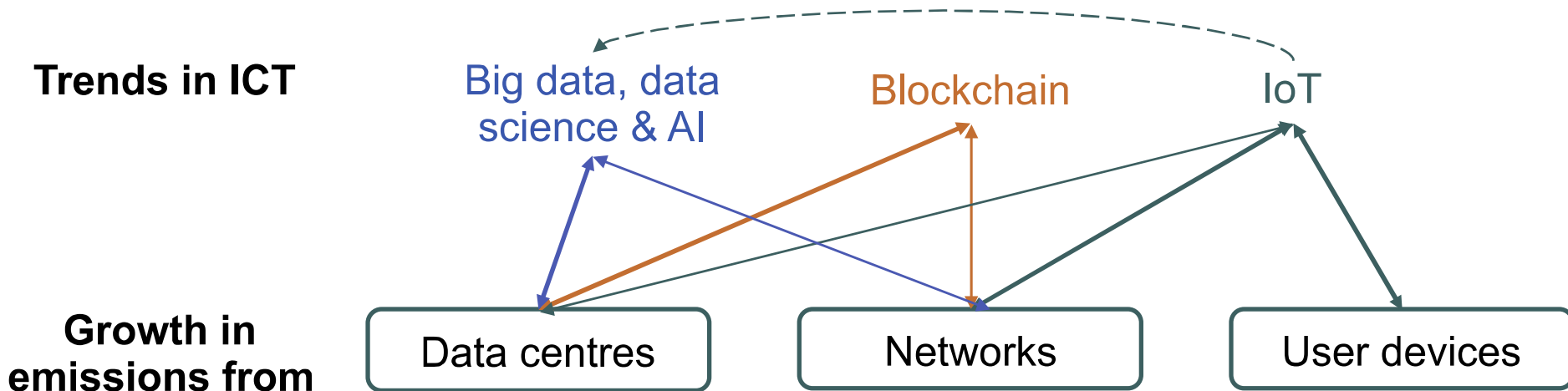
# Estimates Going Forward

- Projections beyond 2020 [1]



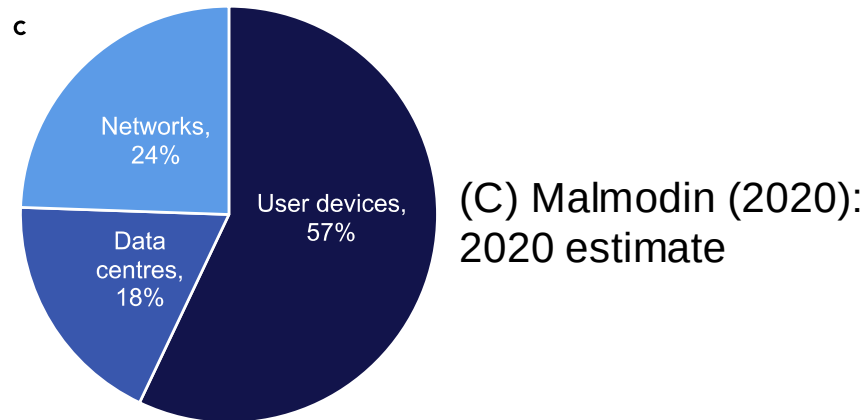
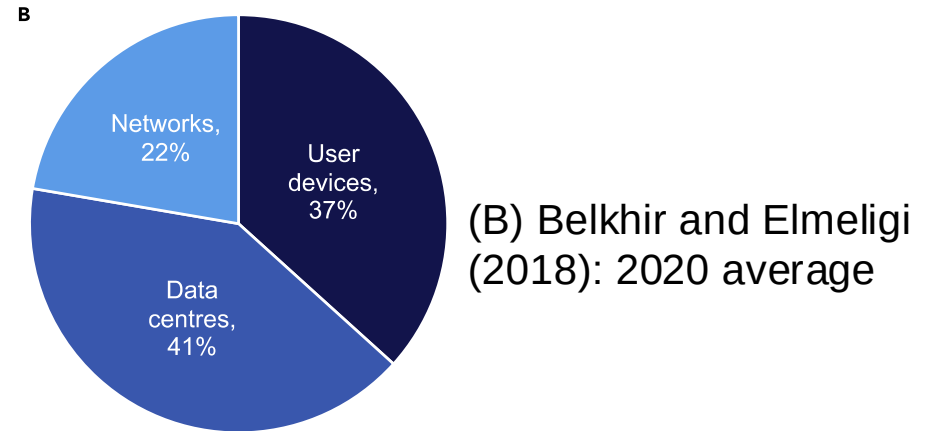
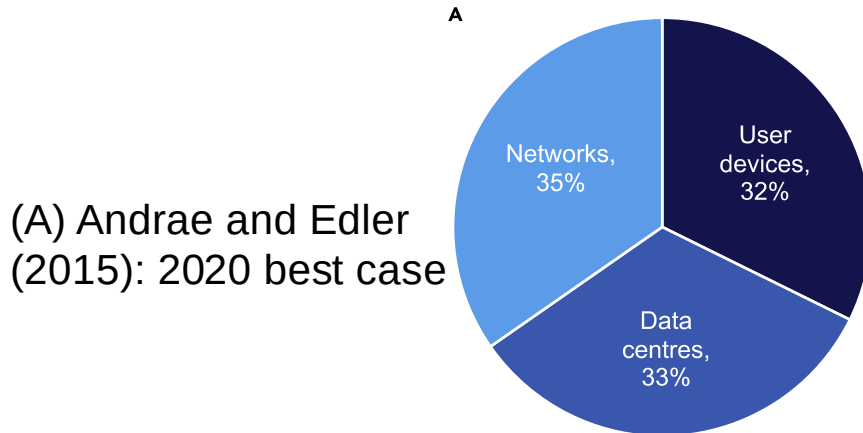
# What Drives This Growth?

- Several **major ICT trends** are routinely linked to growths in emissions [1]



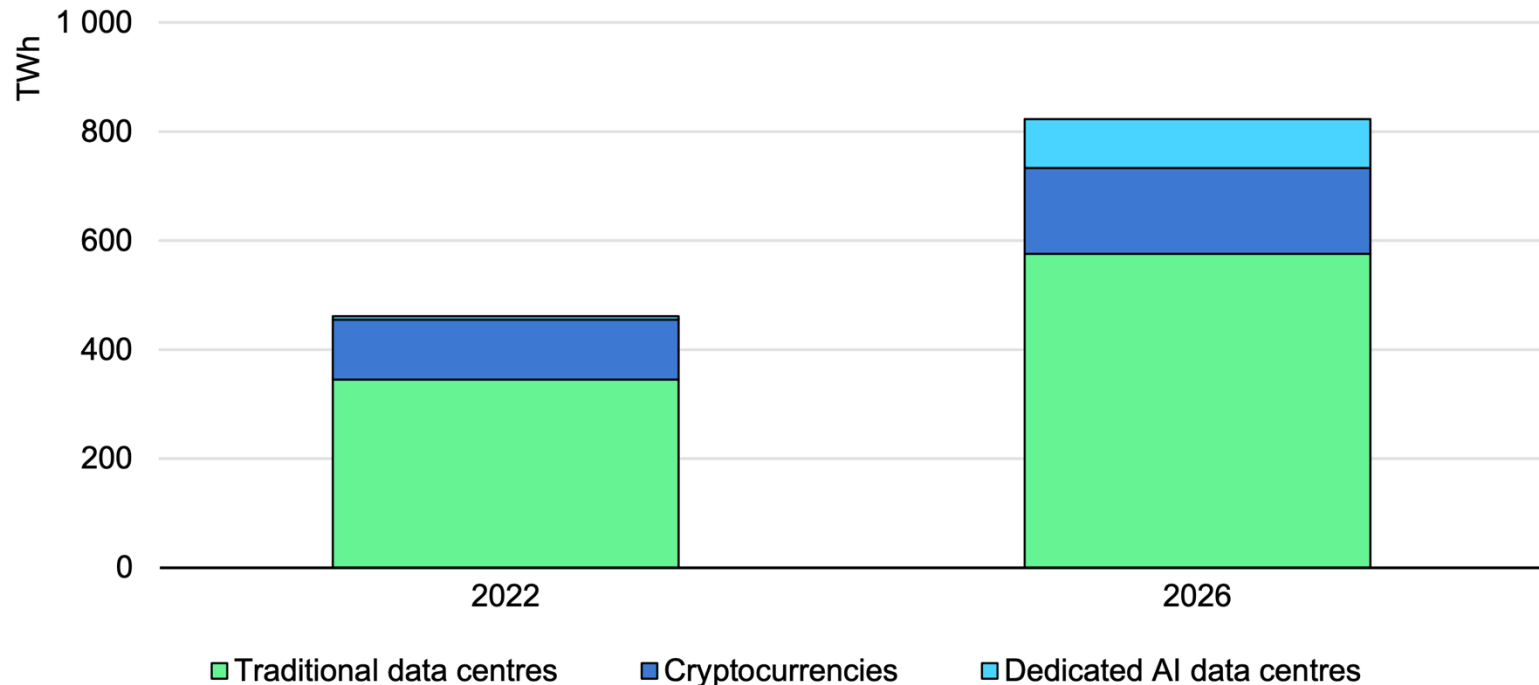
# What Drives This Growth?

- Networks, devices, and data centers [1]:



# Spotlight on Data Centers

- Estimated electricity demand from **traditional data centers**, **AI data centers**, and **cryptocurrencies** by the International Energy Agency (IEA) from January 2024 [8]:



IEA. CC BY 4.0.

# Lifecycle Emissions

- **Embodied emissions:** Emissions from raw material sourcing, manufacturing, transport
  - [1] suggests this is around 23% across various studies (but around 50% for consumer devices)
- **Operational emissions:** Emissions from using electricity for powering computing and networking hardware
  - [1] puts this to around 70% (but 50% for user devices)
- **End-of-life emissions:** Emissions from recycling and disposal of e-waste
  - [1] and [2] suggest this is around 5%



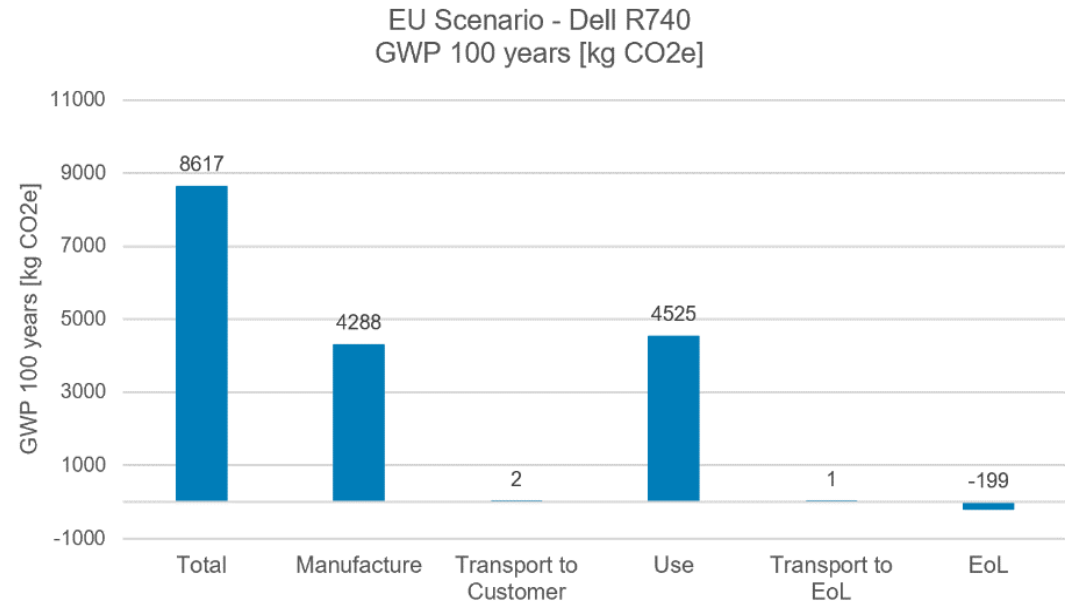
# Dell PowerEdge R740 LCA [3]

- One of the few detailed lifecycle assessments (LCA) published, for a server:



## Key assumptions:

- Lifetime of product: 4 years
- Use location: EU
- Processor: 2 x Intel Xeon 140W CPUs
- Memory: 12 x 32GB DIMMs
- Storage: 1 x 400GB SSD, 8 x 3.4TB SSDs



# What Can We Do?

- The Software Carbon Intensity (SCI) [5] specification of the Green Software Foundation's lists:
  - **Energy Efficiency**: Make software use less electricity to perform the same function
  - **Hardware Efficiency**: Make software use fewer physical resources to perform the same function
  - **Carbon Awareness**: Time- or region-shift computations to take advantage of lower carbon electricity

# Outline of Chapter 5: Cloud Sustainability

---

**4.1** The Rising Footprint of ICT and Data Centers

**4.2** Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness

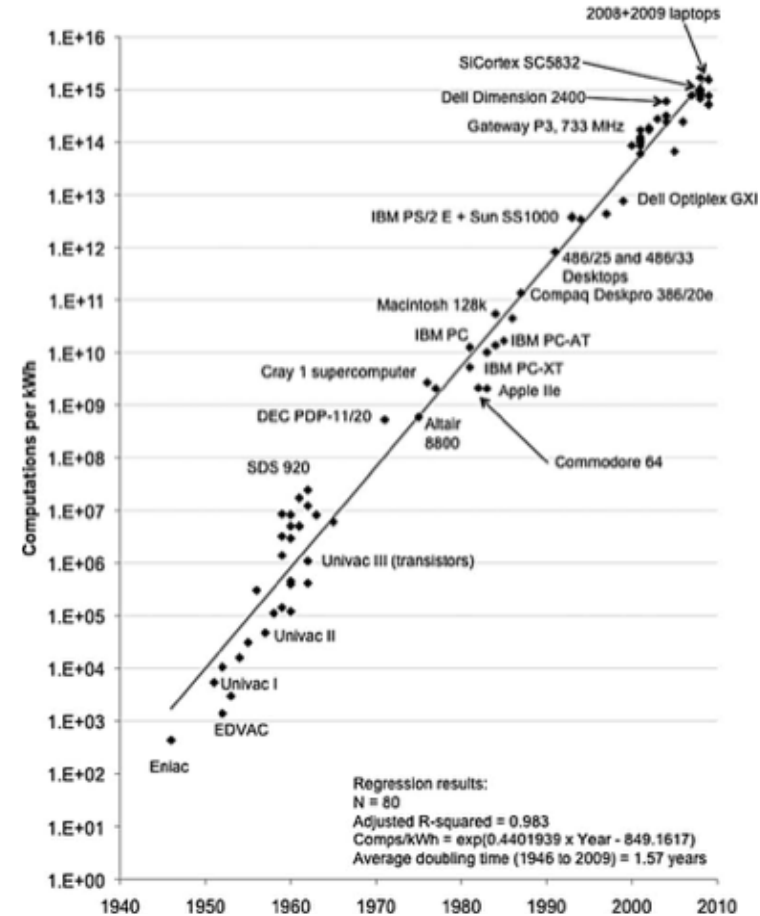
— short break (10-15 minutes) —

**4.3** Cloud Carbon Footprint Assessment

**4.4** Carbon-Aware Cloud Computing

# Energy Efficiency of Computers over Time [10]

- **Koomey's law:** number of computations per unit of energy doubled every few years
  - 1950s to 2000s: doubling every 1.57 years
  - 2000s to 2010s: doubling every 2.6 years
- **Slowing** in line with the slowing of Moore's law and the end of Dennard scaling

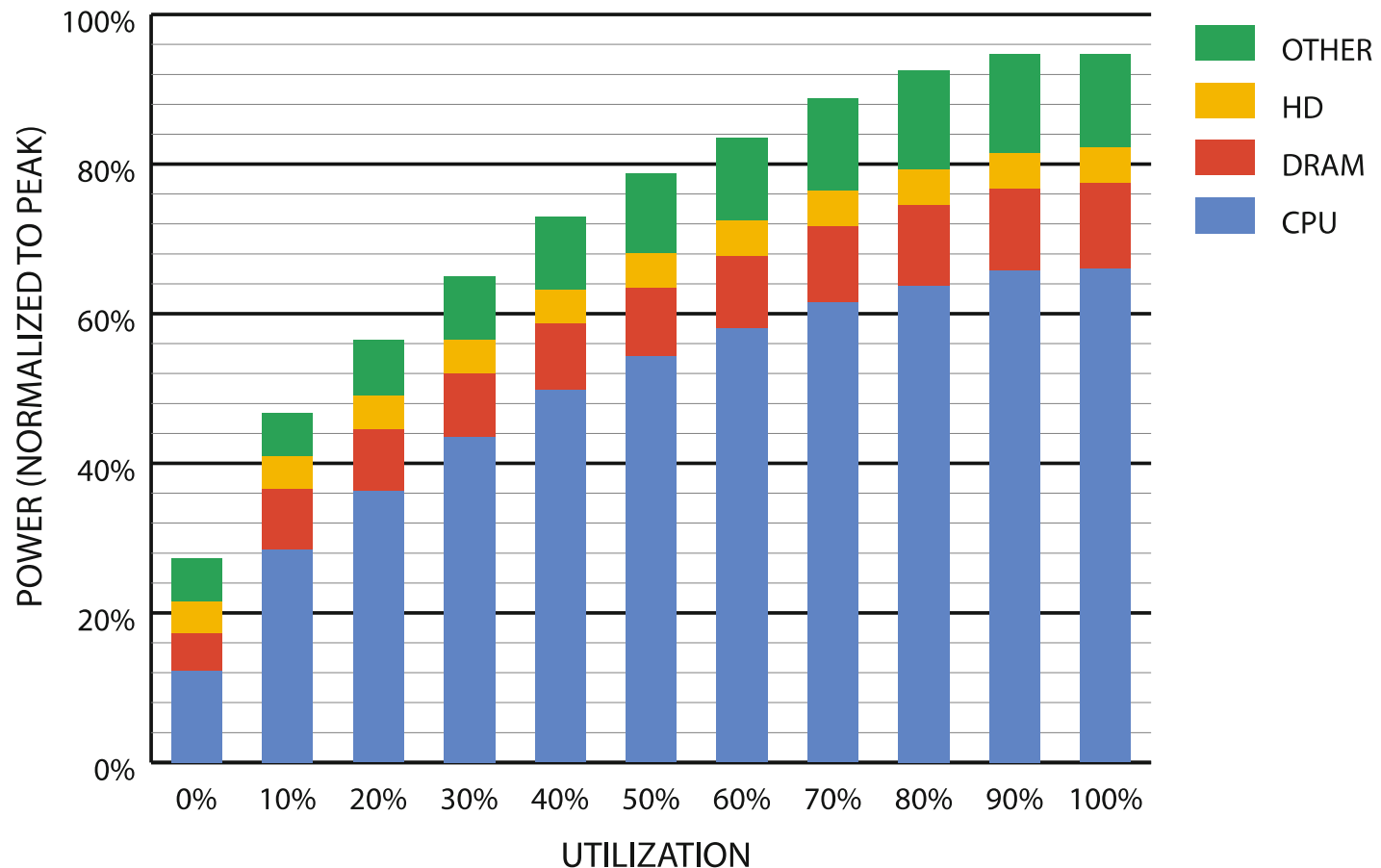


# Energy Demand of Hardware

- Recap: Energy = Power (W) x Time  
(1 Joule = 1 Ws; 3600 Joule = 1 Wh)
- We differentiate between
  - **Static power consumption:** This power is consumed as soon as the device is turned on (and idle)
  - **Dynamic power consumption:** This power demand scales with the load on the device
- Most devices follow a relatively linear power profile

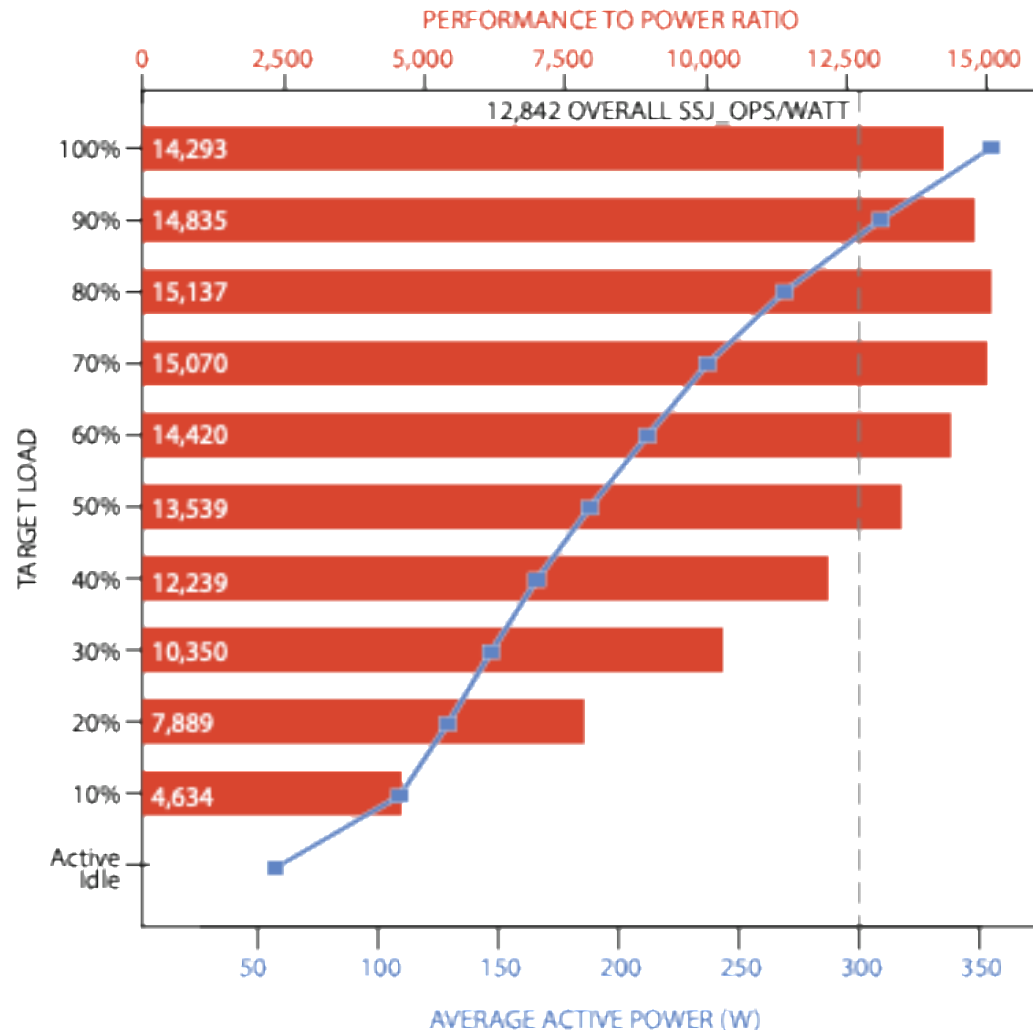
# Power Usage of an Intel Server at Google [4]

- Dynamic range of CPU, memory, and disk exemplified:



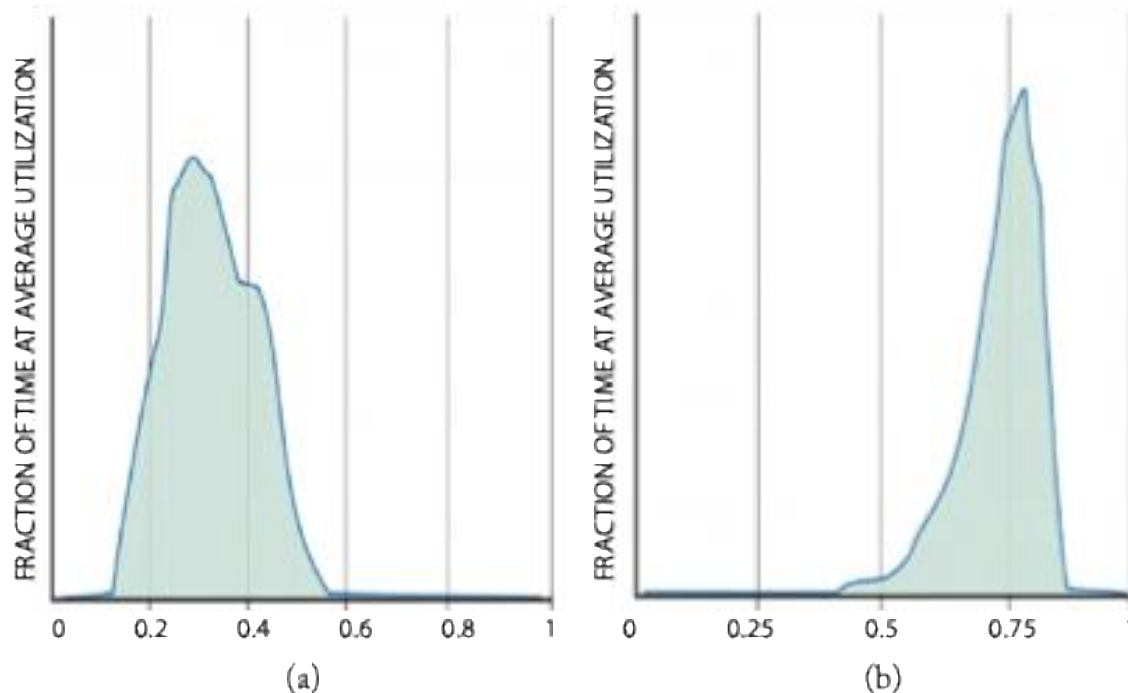
# Performance per Watt of an Intel Server [4]

- Computations performed per unit of energy
- Example from the *SPECpower\_ssj2008* benchmark [4]
- Performance per Watt depends on utilization



# Cloud Server Utilization [4]

- Data center servers are often not fully utilized
- Two Google clusters, each containing over 20,000 servers, sampled over 3 months in 2013:

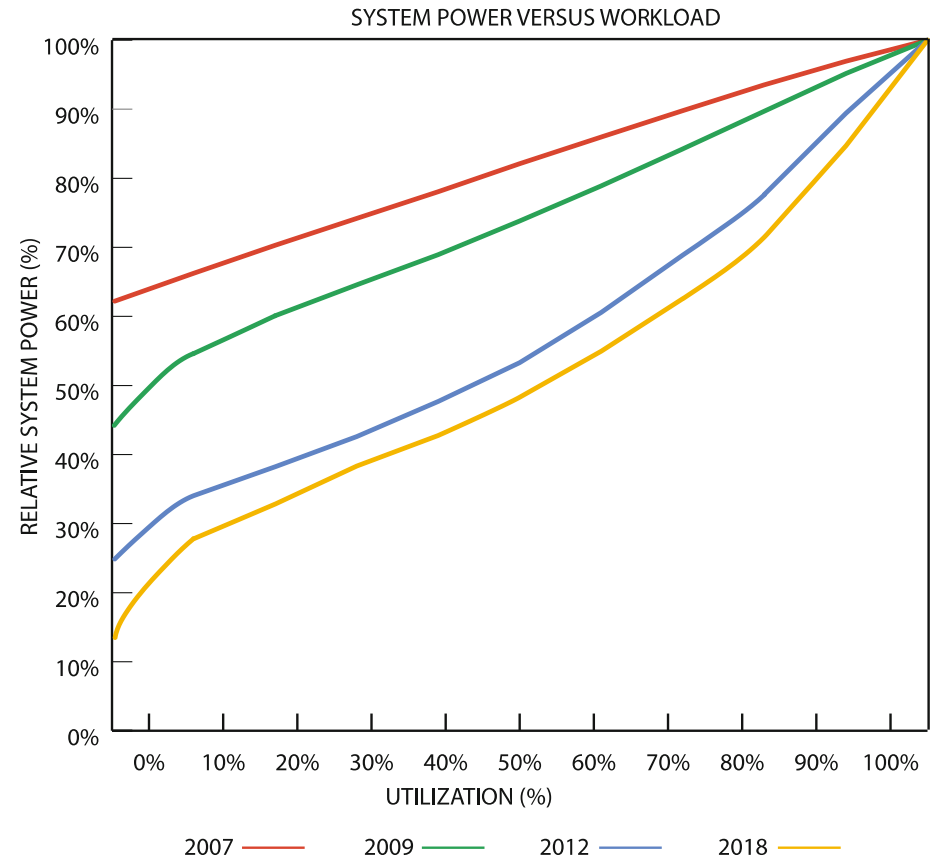




# Energy Proportionality [4]

- Goal of **energy-proportional computing**: Energy consumption per operation is independent of the current utilization
- Servers have become more energy-proportional

Intel servers between 2007-2018 used in Google's data centers:



# Energy-Proportional Computing

- **Energy-proportional data centers** require optimizations on many levels, e.g.
  - **Energy-proportional hardware:** CPUs and GPUs, memory and storage, and inter-data center network
  - **Energy-aware scheduling:** Consolidate workloads on as few servers as possible so others can be turned off
  - **Energy-aware data center design:** e.g. data centers should be designed so that parts of the cooling system and uninterruptible power supplies can be shut down too

# Power Usage Effectiveness (PUE)

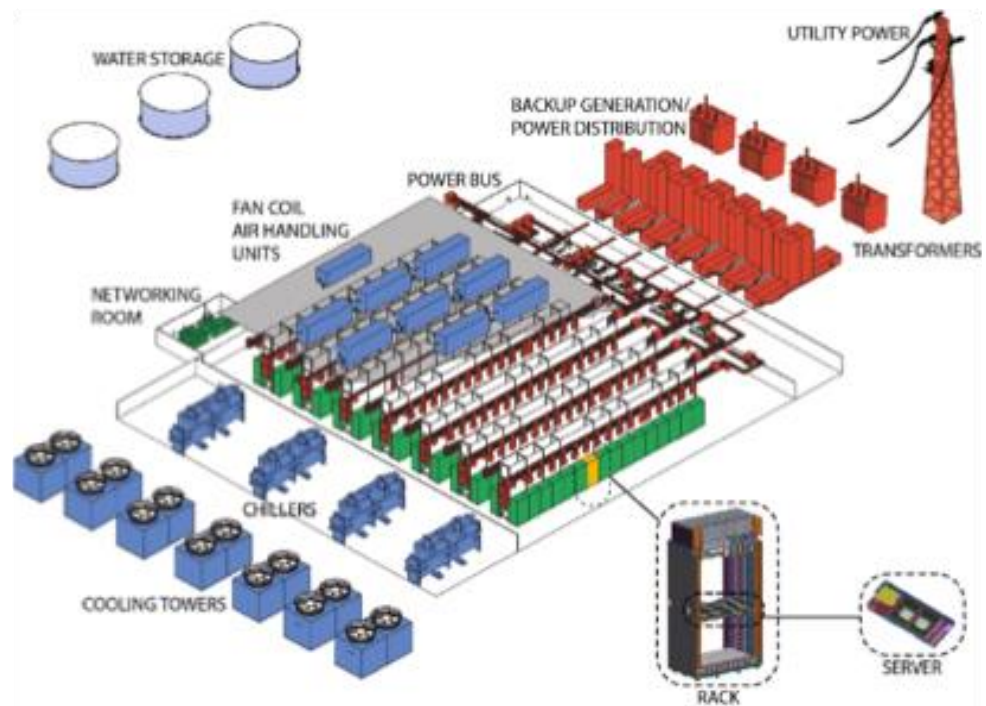
- **Power Usage Effectiveness (PUE)** describes how efficiently a data center uses energy:

$$PUE = \frac{\text{all electricity used by a data center}}{\text{electricity used by computing hardware}}$$

- Examples:
  - A PUE of 1.0 would be ideal: all the electricity entering a data center would be used only for computing
  - Modern hyperscale data centers can have PUEs of around 1.1, while the average for data centers globally in 2021 was estimated to be 1.6.

# Powering a Data Center [4]

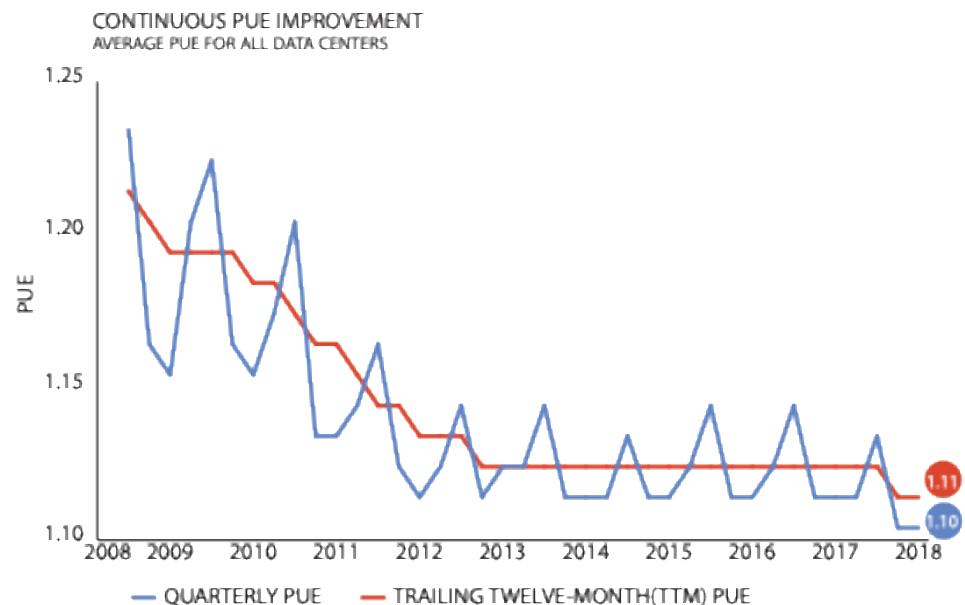
- **IT power usage:** Power consumed by computing, networking, and other IT equipment
- **Facility power usage:** building's mechanical and electrical systems
  - Cooling
  - Power distribution
  - Lighting
  - Access control
  - ...



# PUE over Time [4]

- PUE for all large-scale Google data centers over time:
- Yearly variations as more cooling needed in the summer months
- Scale enables very good PUE

PUE of large-scale Google data centers:

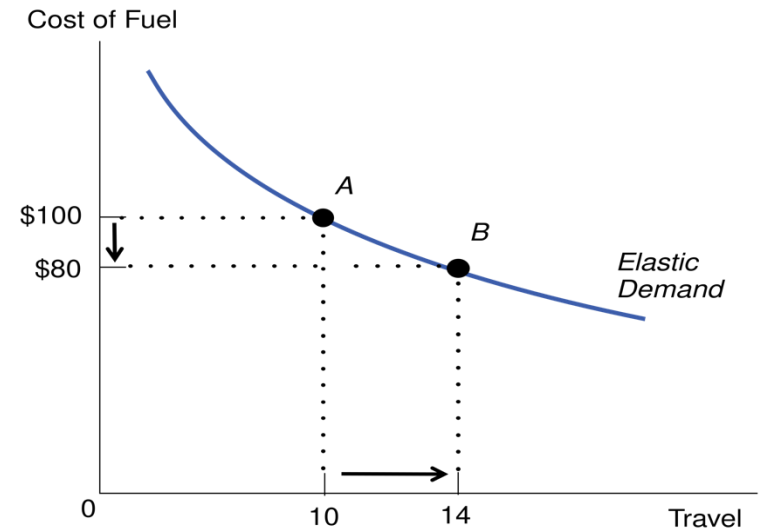


# Issues With PUE

- Differences in **measurement methodologies**
  - What overhead is included exactly?
  - How long, how often, and how exactly are measurements taken?
  - What is the workload while measurements are taken?
- Also, PUE does not necessarily reflect **whole systems tradeoffs**: e.g. heat from data centers can be re-used (so high PUE would not be as problematic)

# Beware of Rebound Effects

- **Jevons' Paradox:** As technology makes resource use more efficient, demand increases, so resource use overall often increases
  - Formulated by William Stanley Jevons in 1865
  - Observing this effect for coal consumption after the Watt Steam engine...
- We have seen the same with all kinds of energy efficiencies, including for computing



# Outline of Chapter 5: Cloud Sustainability

---

4.1 The Rising Footprint of ICT and Data Centers

4.2 Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness

**– short break (10-15 minutes) –**

4.3 Cloud Carbon Footprint Assessment

4.4 Carbon-Aware Cloud Computing



# Outline of Chapter 5: Cloud Sustainability

---

**4.1** The Rising Footprint of ICT and Data Centers

**4.2** Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness

— short break (10-15 minutes) —

**4.3** Cloud Carbon Footprint Assessment

**4.4** Carbon-Aware Cloud Computing

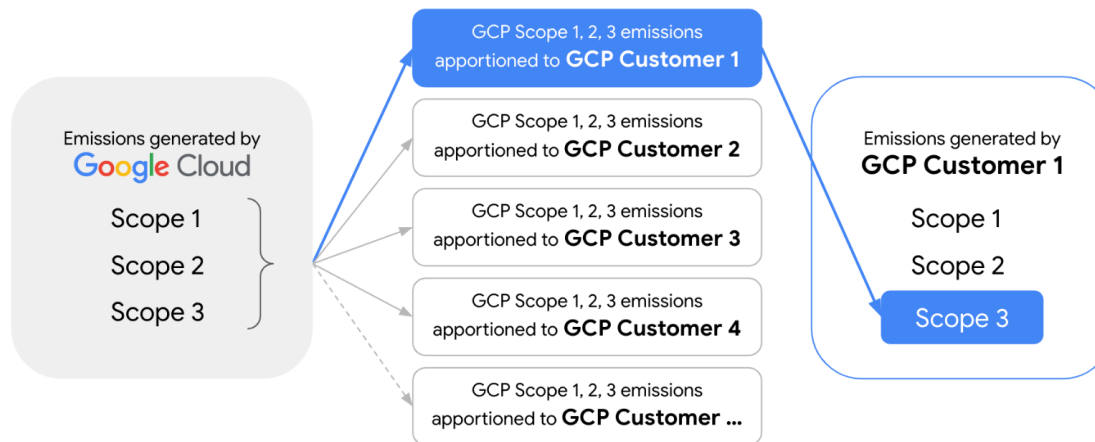
# Raising Awareness: Emissions Accounting

- How much emissions does my software application produce?
- The **Greenhouse Gas (GHG) Protocol** is the most widely used greenhouse gas accounting standard
  - **Scope 1**: directly controlled emissions— e.g. emissions from burning fossil fuels in furnaces, boilers, and vehicles
  - **Scope 2**: indirect emissions – mainly through electricity purchases
  - **Scope 3**: covers emissions occurring in an organization's value chain – e.g. manufacturing and transportation of servers, networking equipment, cooling systems, power supplies, etc.
- Most emissions of cloud operators are in **Scopes 2 and 3**
- Cloud customers perceive these emissions within their **Scope 3**

# What Do Cloud Providers Report?

- The GHG-Protocol is applied by big cloud vendors like AWS, GCP, and Azure
- They increasingly report carbon emissions of utilized services

Apportioned GCP's Scope 1, 2, 3 emissions are accounted as part of GCP Customers' Scope 3



# However...

- Methodologies are currently not transparent and not consistent between operators
- Provider reporting is also often
  - not fine-grained (e.g. break down only over different services such as EC2 and S3),
  - not timely (e.g. reported with a delay of multiple months)
  - not comprehensive (especially for Scope 3 emissions)
- Market-based measures (including unbundled RECs) are used to minimize reported emissions

# Estimating Operational Emissions

- Third-party estimation methodologies have been emerging
- Key challenges for public cloud emissions estimates:
  - No access to hardware/software power meters, only high-level VM resource usage
  - No knowledge of exact underlying server hardware, so hardware can only be estimated
- General procedure for cloud resources:
  - Translate **resource usage** into **energy consumption** and then **carbon emissions**
  - **Conversion factors** for the energy consumption (for vCores, RAM, disk use) and grid emissions

# How to Estimate Emissions for e.g. AWS VMs? [11]

- Pioneered by engineering teams at Etsy and Teads
- Estimating VM power usage and operational emissions:
  - Testing of “bare metal instances” of VM instance types: Checking /proc/cpuinfo to identify CPUs & using RAPL and microbenchmarks to create linear power models
  - Guessing underlying hardware for other types
  - Attributing physical resources to VMs in a linear manner, without overcommitment (vCPU/vCore  $\approx$  1 physical core)
- Estimating embodied emissions: Based on differences of published server lifecycle assessments (such as R740 [3])

# Cloud Carbon Footprint (CCF) Estimation Methodology

- Estimates for operational and embodied emissions

*Operational emissions = cloud resource usage  
× energy conversion factor  
× PUE  
× grid emissions factor*



cloudcarbonfootprint.org  
<https://github.com/cloud-carbon-footprint/>

CCF **energy conversion** factors for AWS resources:

- vCore: 0.74 - 3.5 W per core hour  
(0% - 100% use)
- Memory: 0.39 Watt-Hours per Gigabyte-Hour
- HDD: 0.65 Watt-Hours per Terabyte-Hour
- SSD: 1.2 Watt-Hours per Terabyte-Hour
- Networking: Can be neglected (within a data center)

AWS **PUE**: around 1.2

Average **carbon intensity** of the US in 2019: 433 g CO<sub>2</sub>e / kWh

# CCF Example: A Large Scientific Workflow on AWS

- The Galactic Plane project used AWS Cloud instances to produce 45 TBs of astronomy image data [9]
  - Using 318,000 vCore hours, assuming 50% utilization →  
 $318,000 \text{ core hours} \times 2.12 \text{ W per core hour} = 674 \text{ kWh}$
  - $674 \text{ kWh} \times 1.2 \text{ PUE} = 809 \text{ kWh}$
  - $809 \text{ kWh} \times 433 \text{ g CO}_2\text{e / kWh} = 350 \text{ kg CO}_2\text{e}^{**}$

\*\* For comparison:  
www.ecopassenger.org put a  
one-way direct flight from  
Glasgow to Berlin to 114.7 kg  
CO<sub>2</sub>e emissions  
(in December 2024)

CCF **energy conversion** factors for AWS resources:

- vCore: 0.74 - 3.5 W per core hour  
(0% - 100% use)
- Memory: 0.39 Watt-Hours per Gigabyte-Hour
- HDD: 0.65 Watt-Hours per Terabyte-Hour
- SSD: 1.2 Watt-Hours per Terabyte-Hour
- Networking: Can be neglected (within a data center)

AWS **PUE**: around 1.2

Average **carbon intensity** of the US in 2019: 433 g CO<sub>2</sub>e / kWh



# CCF: Virtual Resource Estimates

- CCF presents provider-wide estimates:
  - AWS: 0.74 – 3.5 W\*
  - GCP: 0.71 – 4.26 W\*      \* per core hour
  - Azure: 0.78 – 3.76 W\*
- These estimates average estimates for instance types
  - Building directly on the work done by Etsy and Teads
  - Using [SPECpower\\_ssj® 2008](#) power consumption benchmark results (not turbostress)
- Example (using CCF data and scripts):
  - GCP e2-standard-8 uses Skylake CPUs
  - Power consumption average for all the Skylake CPUs known to be used for GCP: 0.645 – 3.899 W\*

# Outline of Chapter 5: Cloud Sustainability

---

4.1 The Rising Footprint of ICT and Data Centers

4.2 Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness

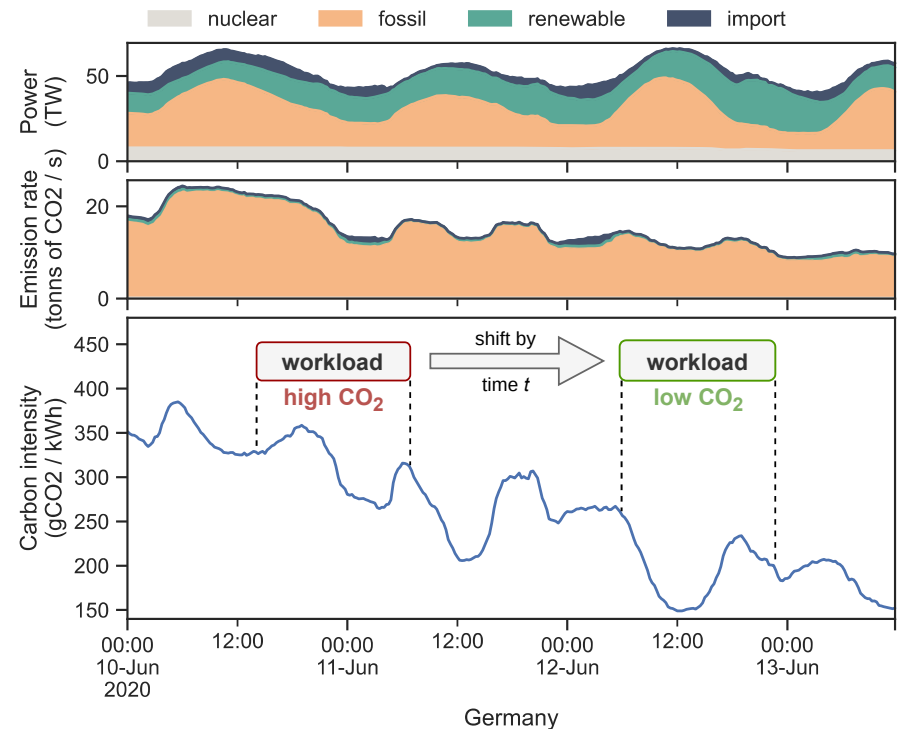
— short break (10-15 minutes) —

4.3 Cloud Carbon Footprint Assessment

**4.4 Carbon-Aware Cloud Computing**

# Basic Idea of Carbon-Aware Computing [6]

- Emissions of power grids are determined by the energy mix and demand
- Low-carbon objective: Compute when and where low-carbon energy is available



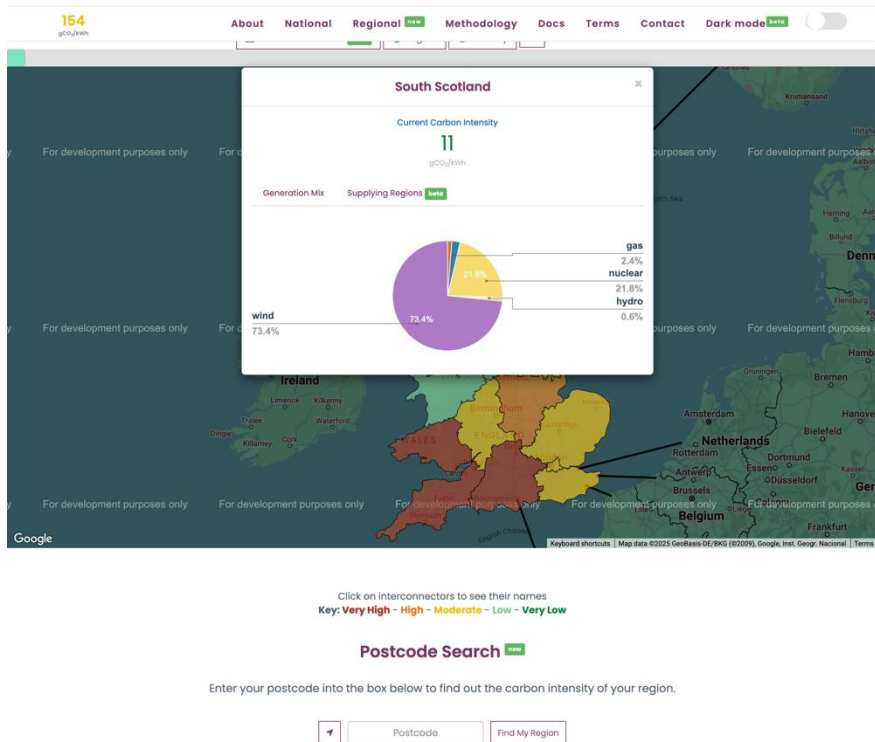
# Average Carbon Intensity [6]

- All produced power is weighted by the carbon intensity of their power source

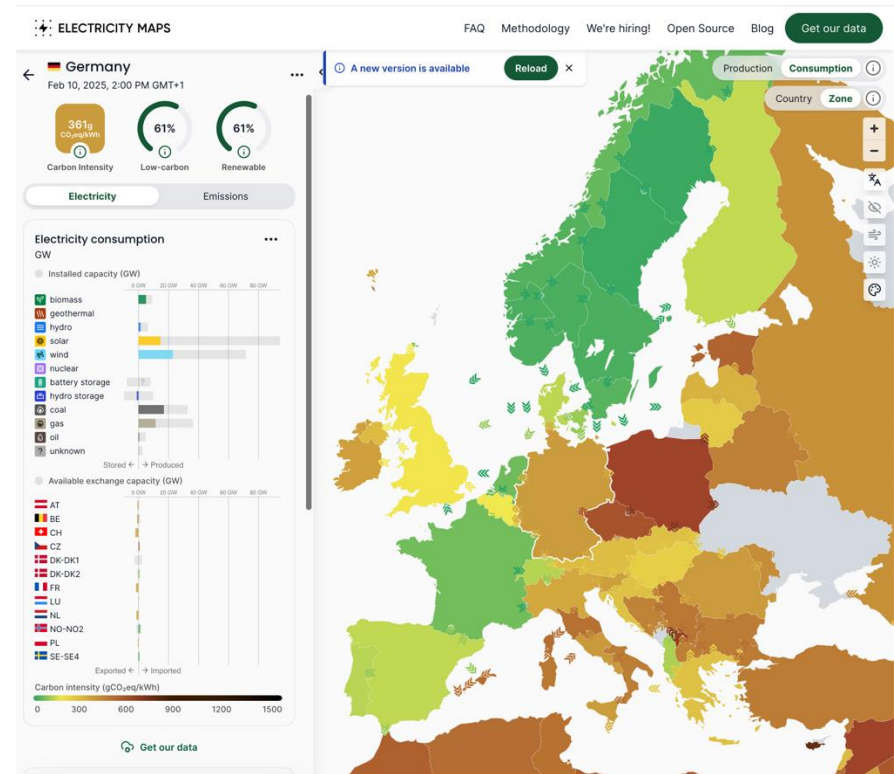
| Energy Source         | Biopower | Solar Energy | Geothermal Energy | Hydropower | Wind Energy | Nuclear Energy | Natural Gas | Oil | Coal |
|-----------------------|----------|--------------|-------------------|------------|-------------|----------------|-------------|-----|------|
| gCO <sub>2</sub> /kWh | 18       | 46           | 45                | 4          | 12          | 16             | 469         | 840 | 1001 |

- Good metric for carbon accounting
  - Metric is easy to understand and calculate
  - Power production per source and cross-border flows are publicly reported in the EU and similarly in other regions

# Average Carbon Intensity – 4:30 pm 10/02/2025



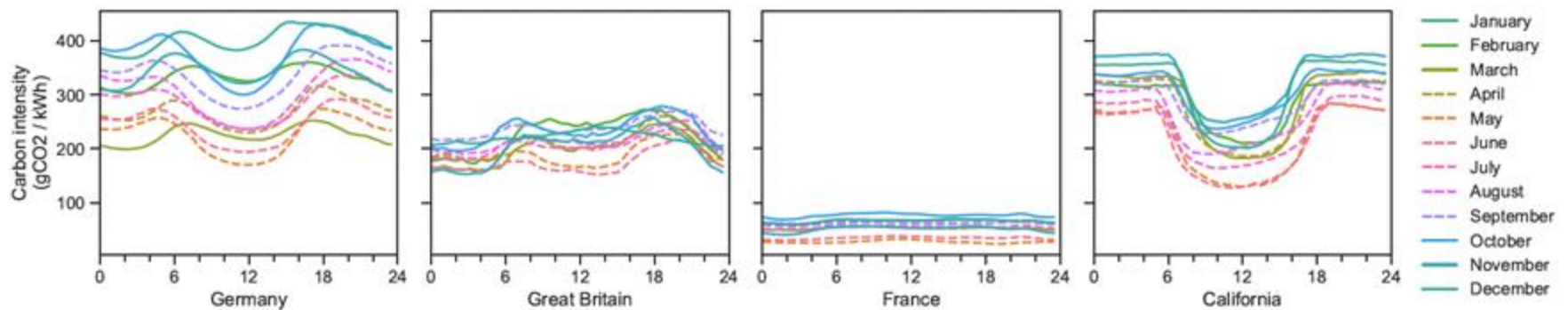
<https://carbonintensity.org.uk/>



<https://app.electricitymaps.com>

# Varying Average Carbon Intensity (1/2) [6]

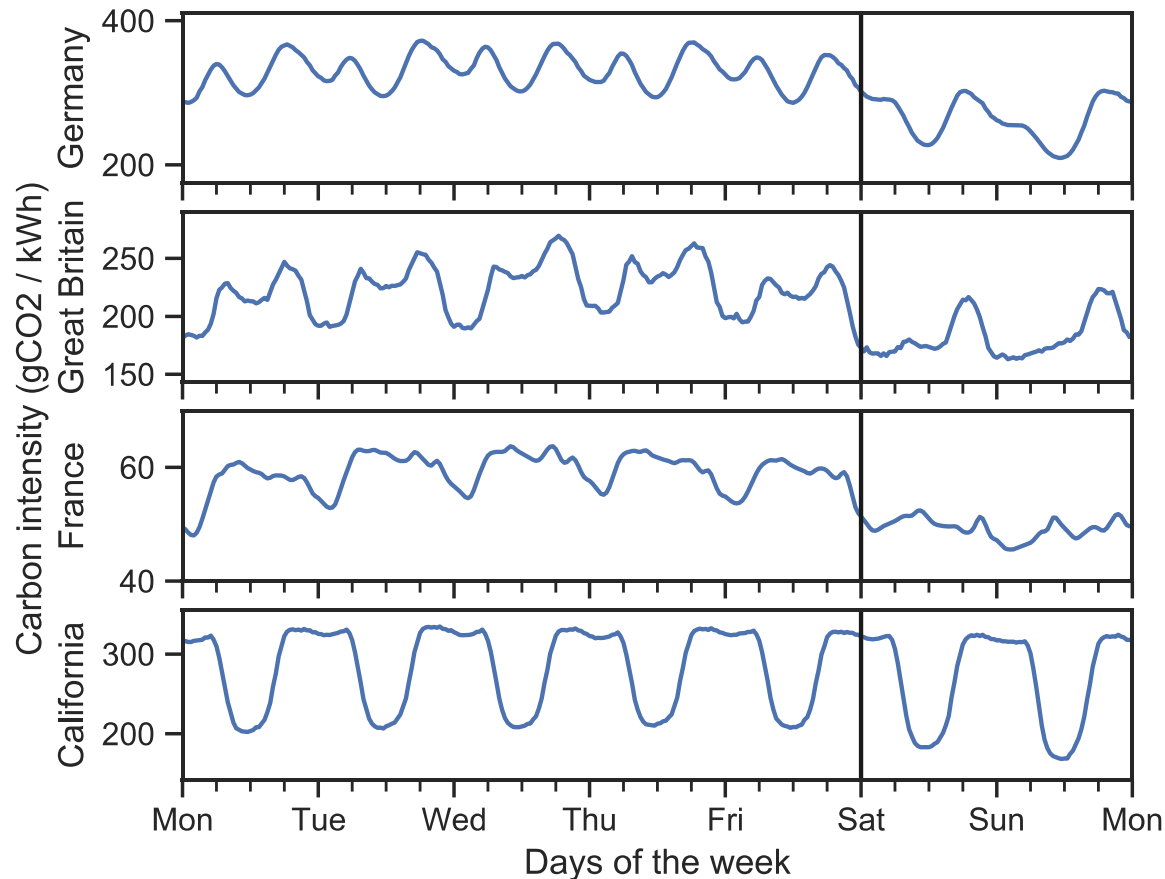
- What are the most promising times to shift work to?



Average carbon intensity (== CO<sub>2</sub>-equiv. greenhouse gas emissions per kilowatt hour of energy) in 2020

# Varying Average Carbon Intensity (2/2) [6]

- What are the most promising days to shift work to?



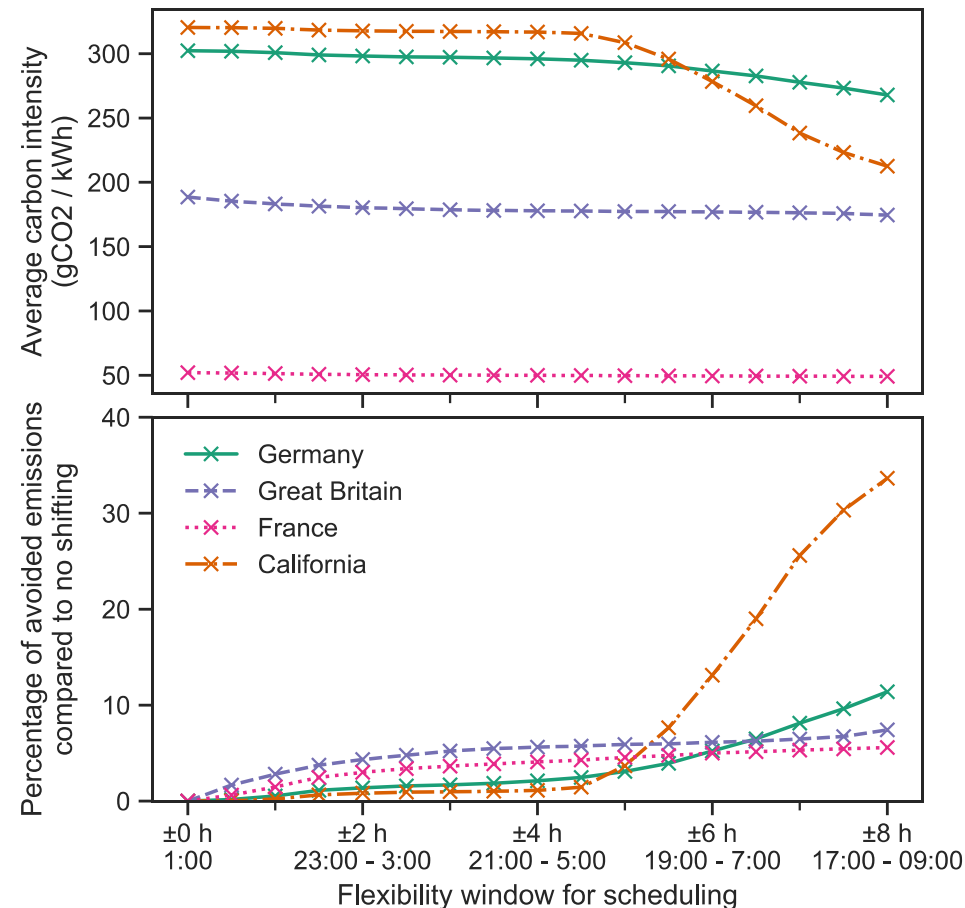
# Simulations of Carbon-Aware Time Shifting [6]

- Evaluation of two scenarios using a simulator (<https://github.com/dos-group/leaf>)
- Scenario 1 – **Periodic Jobs**: Nightly builds, integration tests, recurring generation of business reports, ...
- Scenario 2 – **Ad Hoc Jobs**: ML training jobs, data analysis pipelines, scientific simulations, ...



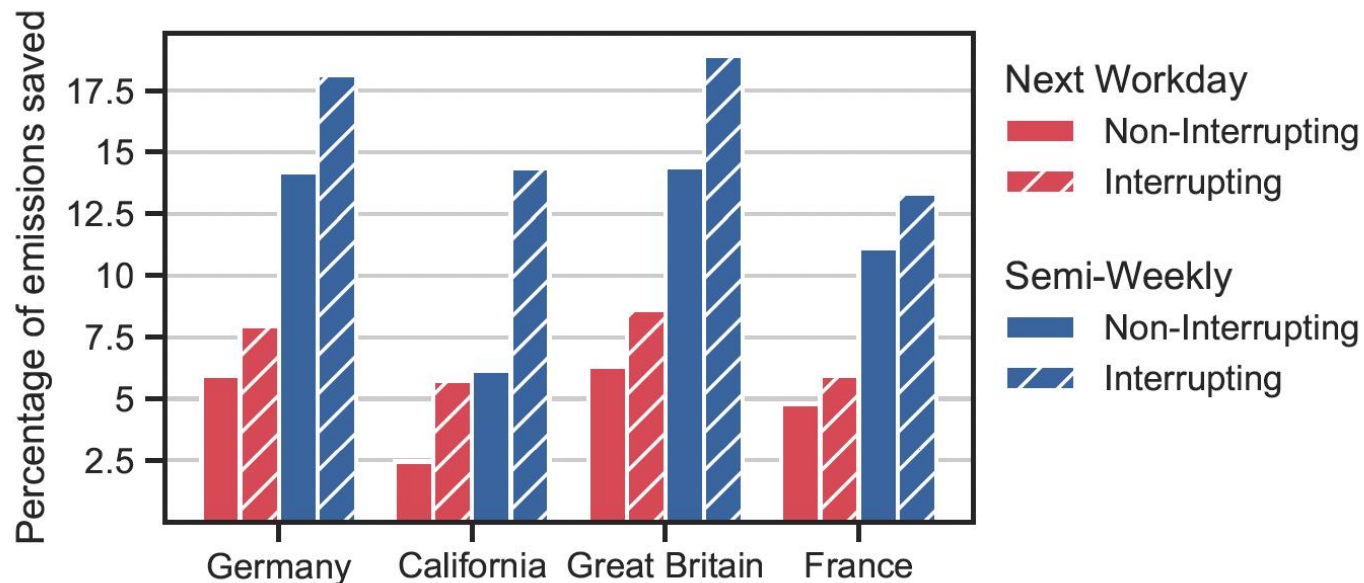
# Scenario 1: Periodic Jobs [6]

- Baseline: All jobs scheduled at 1 am in the night
- Increasing the window by  $\pm 1$ h to allow scheduling between
  - 00:00 to 3:00 ( $\pm 1$ h)
  - 23:00 to 4:00 ( $\pm 2$ h)
  - ...
  - 17:00 to 9:00 ( $\pm 8$ h)



# Scenario 2: Large Ad Hoc Jobs [6]

- Based on an NVIDIA research project, which ran 3387 training jobs using 145.76 GPU years and 325 MWh
- Baseline: Instant scheduling of jobs that arrive randomly during working hours

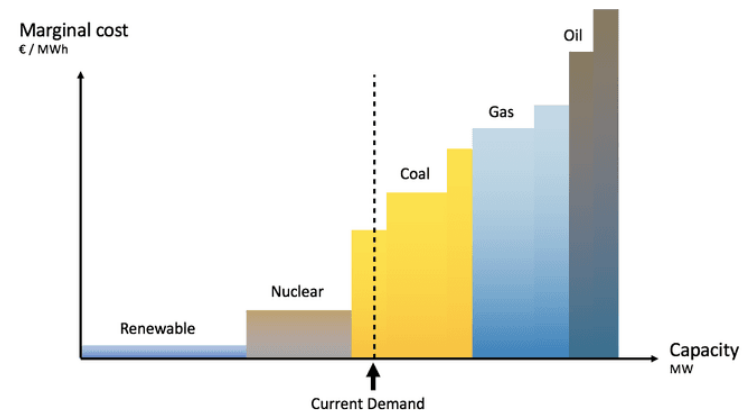


# Limits of Average Carbon Intensity

- Problem: Reporting areas are very large (e.g. entire Germany) so local curtailments are not captured
- Important question not addressed: If we consume an *additional unit* of energy, what is its carbon intensity?

- **Marginal Carbon Intensity**

- Describes the source of any additional unit of energy



<https://www.electricitymaps.com/blog/marginal-emissions-what-they-are-and-when-to-use-them>

# Renewable Excess Energy

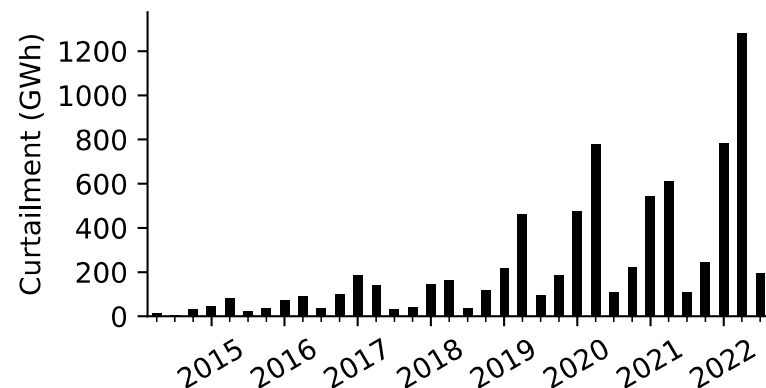
What happens if we have more power supply locally than demand?

1. Energy storage (“move” excess energy through time)
  - Expensive and available in limited capacity
  - Conversion losses
  - Charge cycles increase battery aging
2. Transmit to other area (“move” excess energy through space)
  - Neighboring grids often have excess energy at similar times
  - Feeding energy into the public grid offers very low compensation
  - Limits of transmission infrastructure

If these are no options, we must **curtail** to keep grids balanced

# Local Energy Curtailment

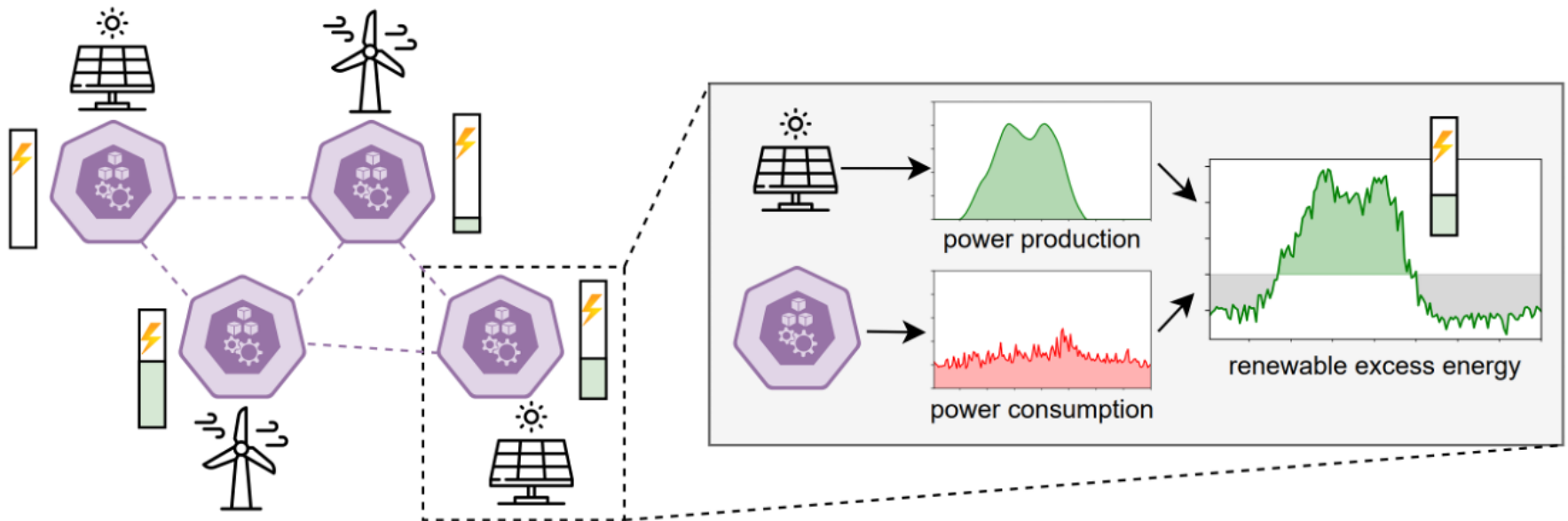
- **Curtailment** describes the deliberate reduction of production to balance power supply and demand
- In 2022
  - California curtailed around 7% of its entire solar production
  - Germany lost 5.4 million MWh of renewable energy due to grid bottlenecks
- Scotland's wind production is increasingly curtailed too



Source: California Independent System Operator (CAISO)

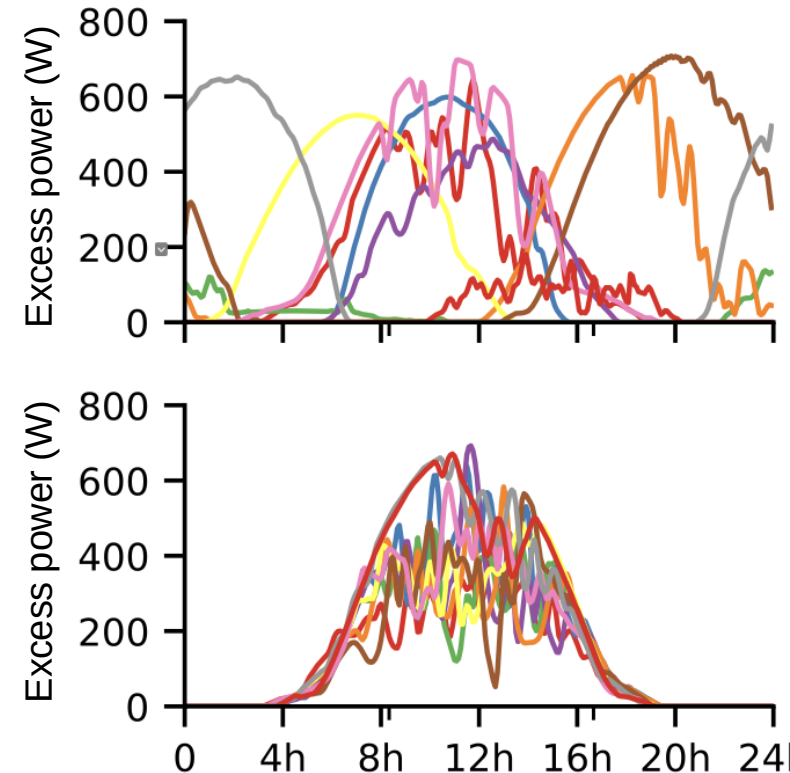
# Renewable Excess Energy [7]

- Can we utilize renewable excess energy in data centers for flexible workloads?



# Some Open Challenges for Carbon-Aware Computing

- Tricky to define metrics
- Reliance on forecast quality
- Energy efficiency (using less energy) vs. carbon efficiency (using clean energy)
- Localized surpluses
- How are future computing and energy systems securely exchanging information?



# Outline of Chapter 5: Cloud Sustainability

---

4.1 The Rising Footprint of ICT and Data Centers

4.2 Energy Efficiency, Energy Proportionality, and Power Usage Effectiveness

— short break (10-15 minutes) —

4.3 Cloud Carbon Footprint Assessment

4.4 Carbon-Aware Cloud Computing



# Summary

- ICT's footprint is substantial and rising
  - Around a third of this is data centers
  - Hyperscale cloud data centers are growing fastest, currently driven by the (Gen)AI hype
  - Footprint goes well beyond emissions
- What can we do?
  - Energy efficiency, resource efficiency, and carbon awareness
  - Rebound effects are to be expected (Jevon's paradox)
  - We will need behavior change too

# Invitation to Get Involved!



- **Glasgow Carbon-Conscious Computing (GC<sub>3</sub>) Lab**
- **Goal: Reducing the carbon footprint** of large-scale systems on edge/cloud computing infrastructure
  1. Compute when and where low-carbon energy is available
  2. Allocate resources for high resource utilization
  3. Save computation and communication through distributed and dynamic system architectures
- **Team:** one academic, one postdoc, two PhD students, L4/L5 project students, visiting PhDs, and summer interns
- **Partners:** AWS, BBC, Barclays, TU Berlin, HU Berlin, ...
- **Website:** <https://lauritzthamsen.org/lab/>

# References

- [1] Freitag, Charlotte, et al. "The Real Climate and Transformative Impact of ICT: A Critique of Estimates, Trends, and Regulations". Patterns 2.9 (2021).
  - [https://www.cell.com/patterns/pdf/S2666-3899\(21\)00188-4.pdf](https://www.cell.com/patterns/pdf/S2666-3899(21)00188-4.pdf)
  - <https://www.lancaster.ac.uk/news/emissions-from-computing-and-ict-could-be-worse-than-previously-thought>
- [2] Gupta, Udit, et al. "Chasing Carbon: The Elusive Environmental Footprint of Computing". 27th IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE. 2021.
  - <https://ieeexplore.ieee.org/document/9407142/>
- [3] Life Cycle Assessment of Dell PowerEdge R740. thinkstep for Dell Technologies. 2019.
  - [https://corporate.delltechnologies.com/content/dam/digitalassets/active/en/unauth/data-sheets/products/servers/lca\\_poweredge\\_r740.pdf](https://corporate.delltechnologies.com/content/dam/digitalassets/active/en/unauth/data-sheets/products/servers/lca_poweredge_r740.pdf)
  - [https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/Full\\_LCA\\_Dell\\_R740.pdf](https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/Full_LCA_Dell_R740.pdf)
- **[4] Barroso, Luiz André, Urs Hölzle, and Parthasarathy Ranganathan. "The Datacenter as a Computer: Designing Warehouse-Scale Machines". Chapter on Energy and Power Efficiency. Springer, 2019.**
  - <https://link.springer.com/book/10.1007/978-3-031-01761-2>
- [5] Green Software Foundation. "Software Carbon Intensity (SCI) Specification".
  - <https://greensoftware.foundation/articles/software-carbon-intensity-sci-specification-project>
  - [https://github.com/Green-Software-Foundation/sci/blob/main/Software\\_Carbon\\_Intensity/Software\\_Carbon\\_Intensity\\_Specification.md](https://github.com/Green-Software-Foundation/sci/blob/main/Software_Carbon_Intensity/Software_Carbon_Intensity_Specification.md)
- [6] Wiesner, Philipp, et al. "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud". 22nd ACM/IFIP International Middleware Conference (Middleware). ACM, 2021.
  - <https://dl.acm.org/doi/10.1145/3464298.3493399>
  - <https://www.bifold.berlin/news-events/news/view/news-detail/scheduling-computing-tasks-can-reduce-emission>
- [7] Wiesner, Philipp, et al. "Cucumber: Renewable-Aware Admission Control for Delay-Tolerant Cloud and Edge Workloads". 28th International Conference on Parallel and Distributed Computing (Euro-Par). Springer, 2022.
  - [https://link.springer.com/chapter/10.1007/978-3-031-12597-3\\_14](https://link.springer.com/chapter/10.1007/978-3-031-12597-3_14)
- [8] International Energy Agency. "Electricity 2024: Analysis and forecast to 2026". Technical Report.
  - <https://www.iea.org/reports/electricity-2024>
- [9] Rynge, Mats, et al. "Producing an infrared multiwave-length galactic plane atlas using montage, pegasus and amazon web services". 23rd Annual Astronomical Data Analysis Software and Systems (ADASS). The Astronomical Society of the Pacific, 2013.
- [10] Jonathan Koomey, et al. "Implications of Historical Trends in the Electrical Efficiency of Computing". IEEE Annals of the History of Computing 33 (3). IEEE 2011.
  - <https://ieeexplore.ieee.org/document/5440129>
- [11] Benjamin Davy / Teads Engineering. "Building an AWS EC2 Carbon Emissions Dataset".
  - <https://medium.com/teads-engineering/building-an-aws-ec2-carbon-emissions-dataset-3f0fd76c98ac>

# Personal Introduction



<https://yelkhatib.github.io/>

Office SAWB 322a

- Reader in Computer Systems
- The GLAsgow Systems Section (GLASS)
- Research: data-driven deployment in complex infrastructures
  - cloud and edge computing
  - system-of-systems composition
  - energy-efficient distributed applications
- Cloud computing expert, publishing research since 2011
- Teaching distributed systems since 2010

# Material

- This is the *first* course in the UK to focus on cloud and sustainability
  - and stress on *both* theoretical underpinnings and practical experience
- Content
  - Interact!
  - Give feedback – both positive and negative
- Pace
  - Material is designed to get you started on the coursework soon
  - If you find me going too slow / fast, do let me know
- Read around the material

# Lectures

- Not the full 2 hours
  - 1 break around the middle
- Some group thinking exercises
  - To encourage active participation
- Source material that is not mine
  - Elkhatib and Wang, “*Distributed Systems*”, Lancaster University
  - Elkhatib and Garraghan, “*Building Big Data Systems*”, Lancaster University
  - Mühl, “*Cloud Computing*”, University of Rostock
  - Thamsen, “*Data-Intensive Systems*”, Humboldt University Berlin

# Labs & Assessment

- Labs (0%)
  - Each labsheet will follow the concepts of the lecture
  - Timetabling jackpot!
  - Estimated time as a very rough guideline
- Assessed Exercise 2 (17%)
  - Released wk6
  - Due wk8 – new School policy
- Quizzes (2x 4%)
  - Released Monday wk9 & 10
  - Due Friday wk9 & 10

# COMPSCI4106/5118 Cloud Systems



Intro to Cloud Computing and Virtual Resources

Virtual Machines

Containers and Container Management

Cloud System Operation and Management

Cloud System Sustainability

## ► Cloud System Design

Modern Cloud Architectures

Flavours of Cloud

A Wider Lens on Sustainability

Emerging Topics and Research Trends

Yehia Elkhatib



# Overview

- Distributed Systems
- Design Goals
  - Quality Attributes
  - Dependability
  - Availability & Reliability
- Failures, Errors, Faults
- Foundations of High Availability
  - Fault Tolerance
  - Error Detection
  - Failover Strategies

Distributed Computing

Software Engineering

Computer Architecture

# What is a computer system?

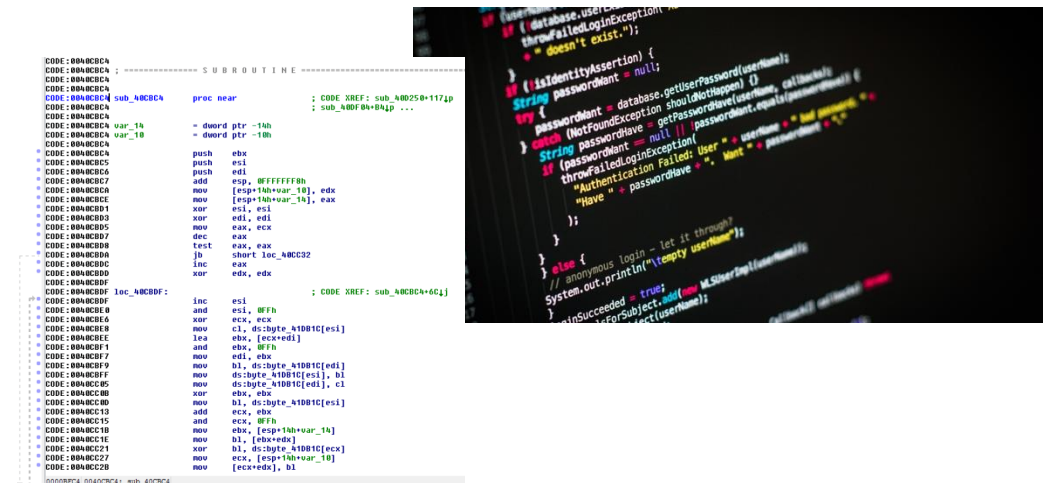
- **Hardware:** CPU, registers, memory, disk, interfaces, peripherals, ...



- **Data:** bits, bytes, ...



- **Software:** assembly, OS, threads, compilers, user-space programs, ...



# What is a distributed system?

“a collection of independent computers that appears to its users as a single coherent system”

[Tanenbaum and van Steen]

“one in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages”

[Coulouris, Dollimore and Kindberg]

“one that stops you getting work done when a machine you’ve never even heard of crashes”

[Lamport]

# Why do we need distributed systems?

- Because the **world** is distributed
  - You want to book a hotel in Prague, but you are in Glasgow
  - You want to retrieve money from *any* ATM, but your bank is in London
  - An airplane has 1 cockpit, 2 wings, 4 engines, 10k sensors, etc.
    - Similarly, railway networks and other distributed transport systems
- Because **problems** rarely hit two different places at the same time
  - As a company having only one database server is a bad idea
  - Having two in the same room is better, but still risky
- Because **joining forces** increases performance, availability, etc.
  - High Performance Computing, replicated web servers, etc.

# Examples of Distributed Systems

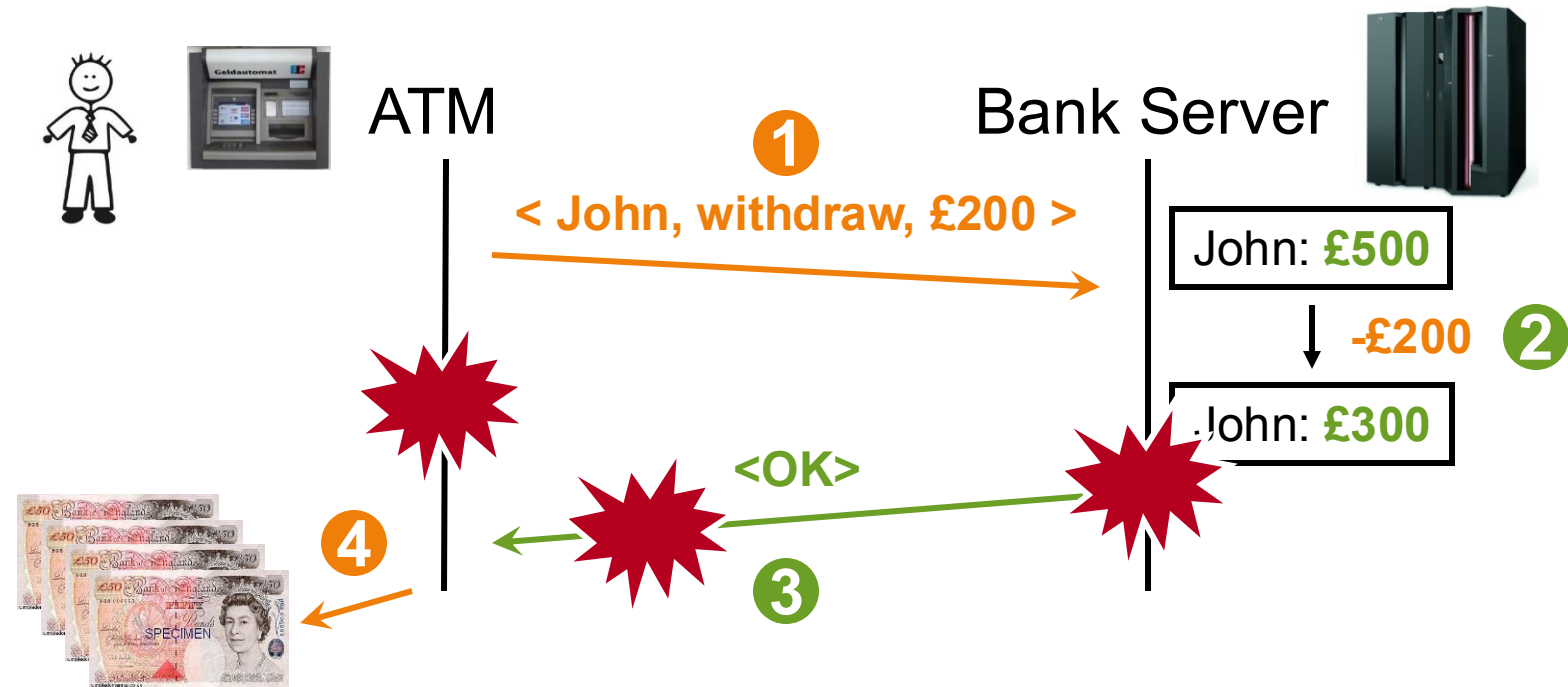
- Financial trading: executing transactions and disseminating events
- Web search: 50+ billion web pages, 10+ billion queries/month
- Social Media Platforms: billions of users worldwide
- Large Language Models (LLMs): trained and served using clusters
- Scientific research: e.g. in 2023 CERN had >1 Exabyte (mil. Terabytes)
- Content Delivery Networks (CDNs): cache closer to users, high traffic
- Online games: players view *and* change a common world in real-time
- ...



# Ok, it's important. But is it easy?

- A bank asks you to program their new **ATM software**
  - Central bank computer (server) stores account information
  - Remote ATMs authenticate customers and deliver money
- **A first version** of the program
  - ATM: (ignoring authentication and security issues)
    1. Ask customer how much money they want
    2. Send message with **<customer ID, withdraw, amount>** to bank server
    3. Wait for bank server answer: **<OK>** or **<refused>**
    4. If **<OK>** give money to customer, else display error message
  - Central Server:
    1. Wait for messages from ATM: **<customer ID, withdraw, amount>**
    2. Check if enough money to withdraw: send **<OK>**, else send **<refused>**

# Easy?



○ But ...

- What if the bank server crashes just after ② and before ③?
- What if the `<OK>` message gets lost? Takes too long to arrive?
- What if the ATM crashes after ①, but before ④?



# Fallacies of Distributed Computing

Asserted at Sun Microsystems (by Deutsch, Joy, Lyon, Gosling)

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. There is one administrator
6. Transport cost is zero
7. The network is homogeneous
8. Topology doesn't change





# Key Aspects of Distributed System Design

- **System Function** – the intended purpose of the system

- Usually documented in a specification of features and capabilities
- *E.g. The system function of a ride-hailing app is to connect riders with drivers, facilitate booking and payment, and provide real-time tracking.*

Functional Requirements define what a system should do

- **System Behaviour** – how the system fulfils its function

- The sequence of actions / states the system goes through to perform its tasks
  - *A rider requests a ride*
  - *The system locates nearby drivers*
  - *Send a request to the driver*
  - *The driver accepts the request*
  - *The app displays the driver's location and ETA*
  - *Process the payment after the ride is complete*

Behaviour is how the system implements the functional requirements

- **Quality Attributes** – core qualities that determine success

- 4: Performance, Cost, Security, and Dependability

Non-functional Requirements define how a system should perform

# Defining Correct Service & Failure

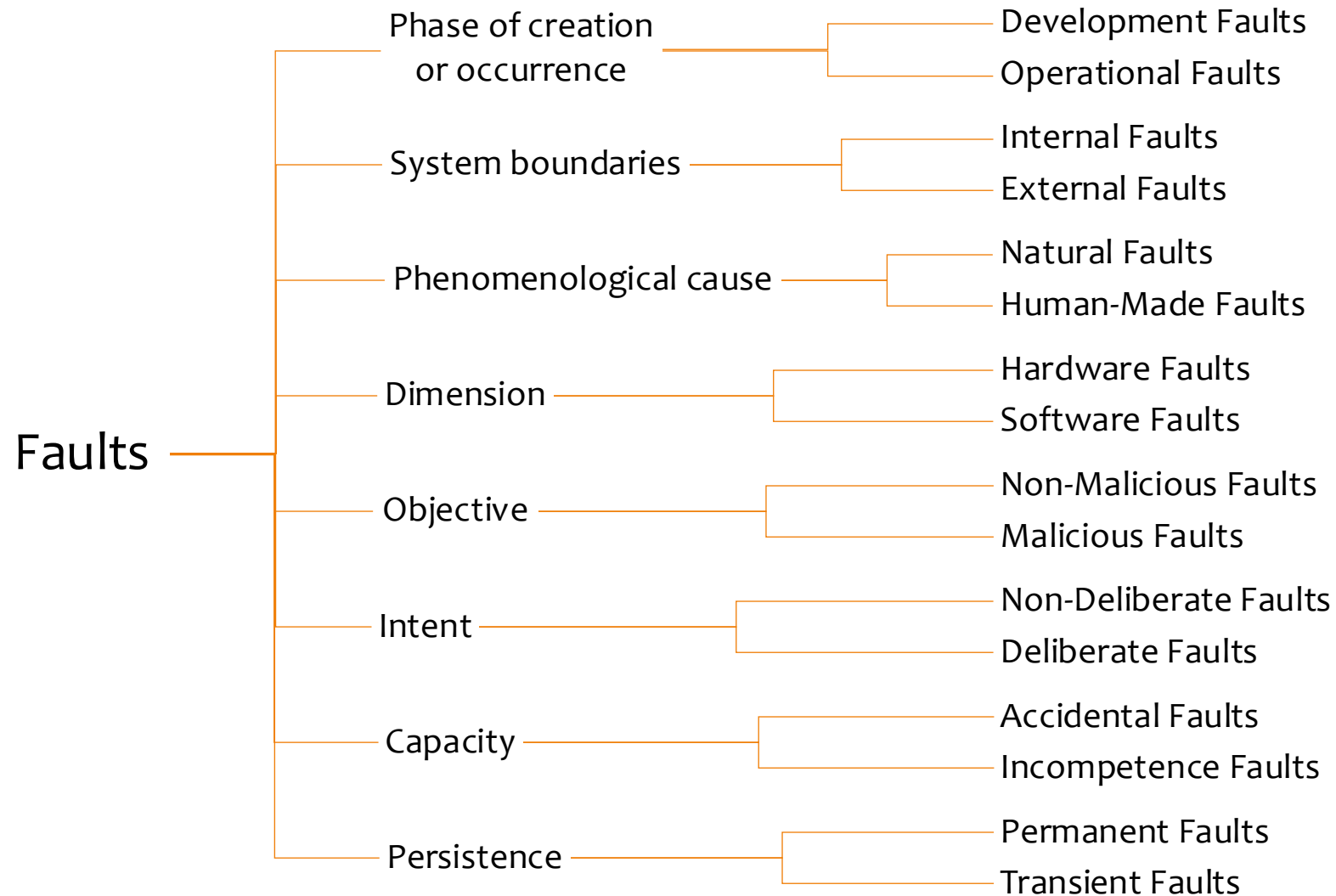
- **Correct service** is delivered when the system implements its function
- **Failure** occurs if service deviates from functional specification
- Failure typically has consequences
  - User dissatisfaction
  - Loss of revenue
  - Damage to reputation
  - Safety risks
  - Further failures

# Failures, Errors & Faults

- Failure: Noncompliance of functional specification
  - Crash, omission, timing, byzantine, etc.
- Error: Transition of internal state into invalid state
  - Erroneous data, inconsistent internal behaviour
- Fault: Hypothesized cause of an error
  - Caused by vulnerability in internal or external state
  - Software/hardware, accidental/incompetence, natural/man-made

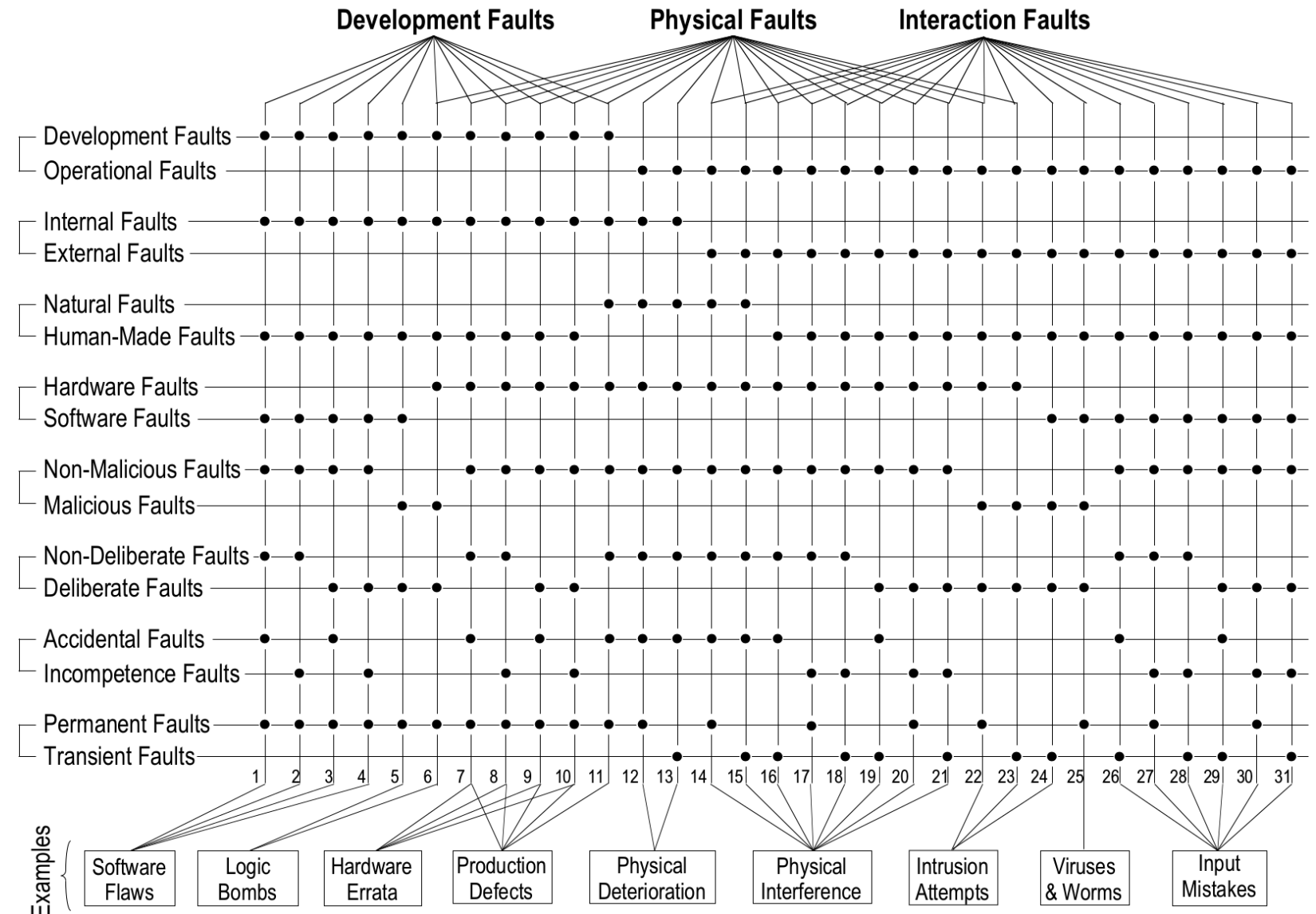
Not all faults and errors result in failures (i.e. reach the service interface)  
However, all failures and errors are caused by faults

# Fault Classification



# Fault Matrix

A. Avizienis, J.C. Laprie, B. Randell, C. Landwehr. “Basic Concepts and Taxonomy of Dependable and Secure Computing” In IEEE Transactions on Dependable and Secure Computing, vol. 1, no. 1, pp. 11-33, Jan-Mar 2004.  
<http://dx.doi.org/10.1109/TDSC.2004.2>



# Failure is Grey

- Example: a web server takes 5 minutes to respond to a request
  - It fulfils its functional specification (i.e. to deliver web pages)
  - It works better than a server that would not reply at all
  - But can it be considered to be functioning correctly?
- The notion of failure is grey; there are varying degrees of failure

optimal service



complete failure

- Optimal Service: meet the functional req's and balance the 4 properties
- Complete Failure: crash, unresponsive, or produces incorrect results
- Partial Failure: parts of the system fail; degraded performance
- Transient Failure: temporary interruption that the system recovers from

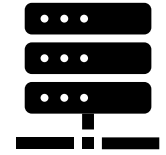
# Quality of Service (QoS)

- A measure of how ‘good’ a system is
- The ability of a system to provide a guaranteed level of performance to specific applications or users
- Multiple facets: latency, bandwidth, security, availability, ...
- Very subjective; system designer defines it given their context
- Goal: highest QoS despite faults (and at the lowest cost)
- Example: In a video streaming service, QoS would prioritise video traffic to ensure smooth playback, even if other network traffic is congested / delayed

# Potential Sources of Failure in Datacentres

- Nodes / servers

- Faulty hardware → crash, timing failure or data corruption
- Power failure → crash and sometimes data corruption
- Any physical accident: fire, flood, earthquake, etc.



- Network (a special type of nodes)

- Routers gateways → whole subnet impacted
- Name servers → whole name domain impacted
- Congestion → dropped packets



- Software, Users / People

- More complex than hardware – more complex state sequences
- Involuntary mistakes (“whoops!”), misconfiguration
- Security attacks, both external and internal (by legitimate users)





# Failure in Data Centres – Examples

- 2008 Amazon S3 major outages for several hours affecting US & EU
- 2009 Microsoft & T-Mobile Sidekick data outage, loss of personal data
- 2011 Gmail failure, 150k customers temporarily lost emails
- 2011 Amazon EBS and RDS outage for 4 days
  - some volumes were not recoverable
- 2015 Apple service disruptions for iTunes, iCloud, Photos, ...
- 2016 Google Cloud Platform significant outage
  - due to a combination of software bugs and unexpected network behaviour
- 2021 OVHcloud fire destroyed DCs in Strasbourg, France
- ...

# Failure in Data Centres – Causes

- Google DCs [GTX14]
  - 40% of servers experience crash / unexpected restart
  - 60,000 production applications affected by server failures
  - Mean Time To Repair (MTTR) of 1.5—10 hours
  - 57% of failures led to VM migrations, hence service disruption
- Global DCs
  - Hard drive cause the most hardware failures (82%) [WZX17]
  - Over 60% of failures result in \$100,000 or more in losses [UTI22]
  - Cyberattacks account for a negligible portion of DC outages (1%) [DCK24]
  - Power & Cooling are the most common cause of outages (71%) [DCK24]

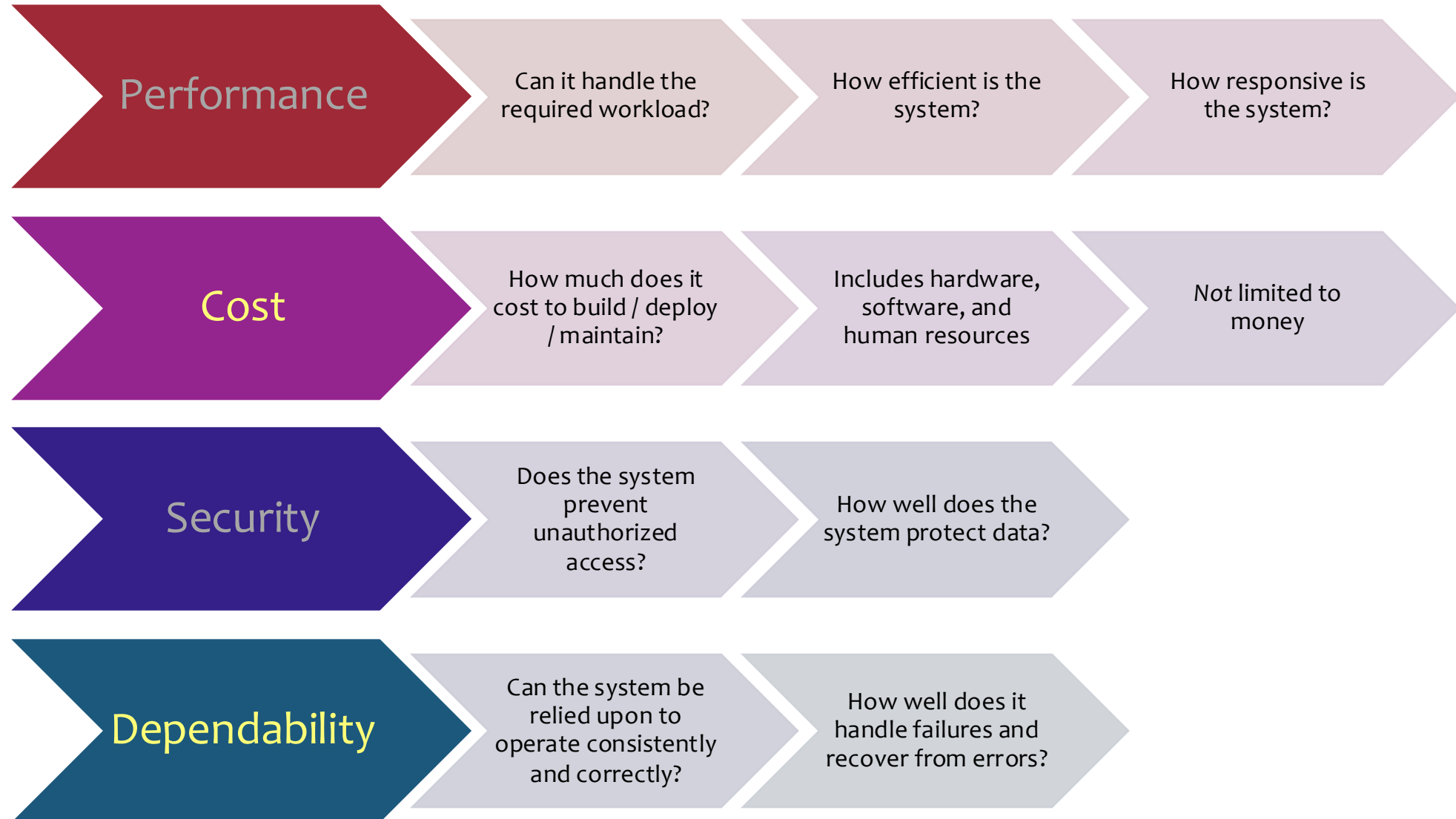
# Further Reading

- Schroeder and Gibson, “*Understanding Failures in Petascale Computers*” J. Phys.: Conf. Ser. 78 2007
- Dean and Barroso, “*The Tail at Scale*”, CACM, 2013.
- Yuan et al., “*Simple testing can prevent most critical failures: An analysis of production failures in distributed Data-Intensive systems*”, 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI) 2014.
- Singh et al., “*Surviving Switch Failures in Cloud Datacenters*”, ACM SIGCOMM Computer Communication Review 51.2: 2-9, 2021.

# Break!



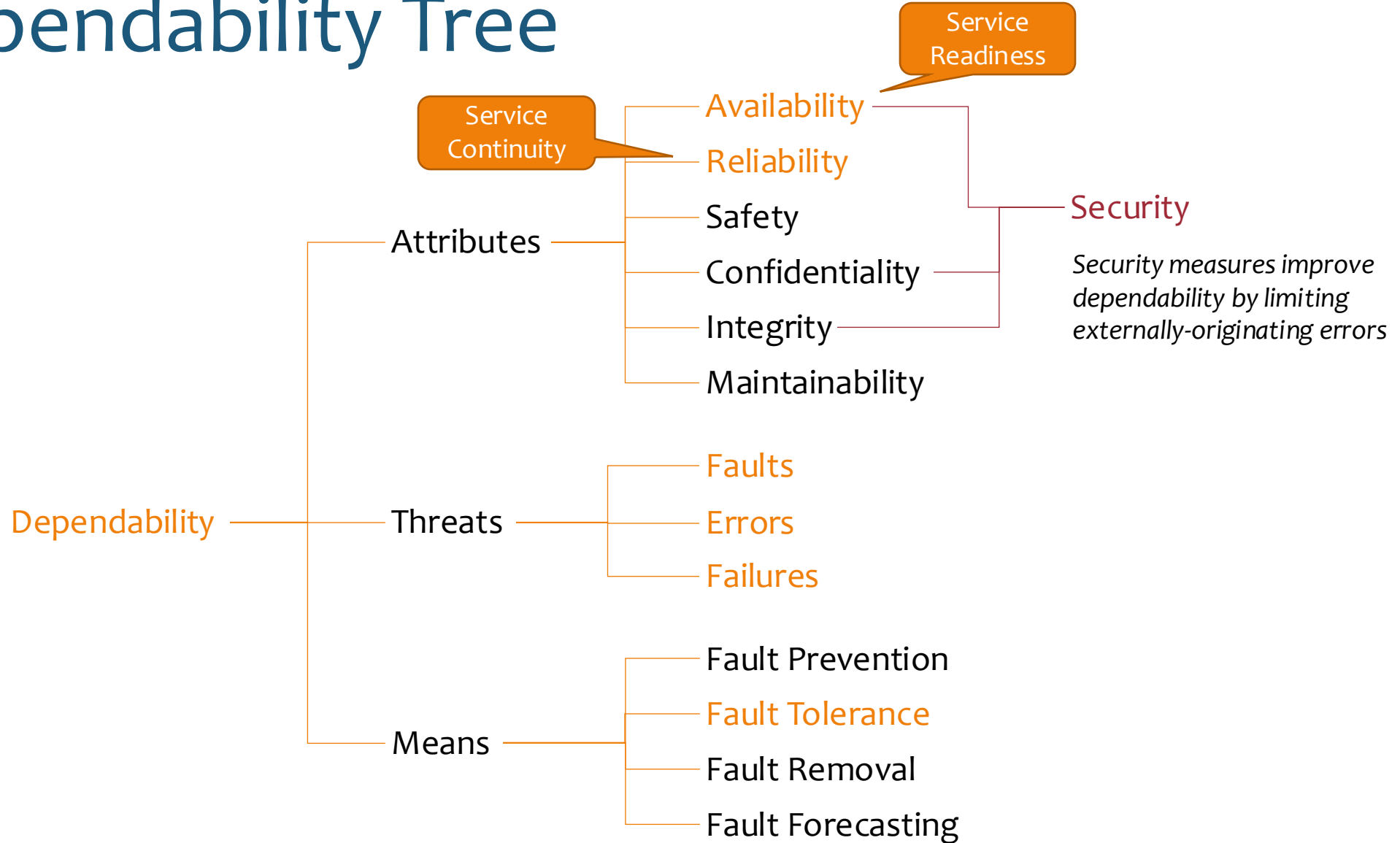
# Quality Attributes



# Dependability Aspects

- **Availability** – readiness for correct service when requested
  - Measured as a percentage of uptime; e.g. 99.99%
- **Reliability** – continuity of correct service, i.e. without failure
  - Expressed as Mean Time Between Failures (MTBF)
- Safety
- Confidentiality
- Integrity
- Maintainability

# Dependability Tree



# Availability & Reliability – Distinction

- Closely related but distinct attributes of dependability
- Example of a car
  - **Availability**: Is the car ready to drive when you need it? Or is it broken down?
  - **Reliability**: How often does it break down? Does it need frequent repairs?
  - Ideally, you want a car that is both available and reliable – ready to go when you need it and unlikely to let you down
- System crashes 1ms/hr OR No crashes but produces incorrect results
  - >99.99% available but is unreliable
- System never crashes but shuts down a week once every year
  - 100% reliable but 98% available



# Availability & Reliability – Examples

- High **availability** systems
  - E-commerce
  - File storage
  - Streaming services
  - Social media
- High **reliability** systems
  - Air traffic control
  - Nuclear power plants
  - Hospital intensive care units

# High Availability – Why?

- Business Impact

- In today's interconnected world, downtime can be extremely costly
- High availability minimises disruptions, ensuring business continuity, customer satisfaction, and revenue generation

- User Expectations

- Users expect services to be available 24/7
- High availability provides a seamless experience

- Critical Systems

- For essential services like healthcare, finance, and emergency response, high availability is crucial for safety and well-being

# Availability 9's

| Availability            | Downtime per Year | Downtime per Month | Downtime per Week |
|-------------------------|-------------------|--------------------|-------------------|
| 90% (one nine)          | 36.5 days         | 72 hours           | 16.8 hours        |
| 99% (two nines)         | 3.65 days         | 7.2 hours          | 1.68 hours        |
| 99.9% (three nines)     | 8.76 hours        | 43.8 min           | 10.1 min          |
| 99.99% (four nines)     | 52.6 min          | 4.38 min           | 1.01 min          |
| 99.999% (five nines)    | 5.26 min          | 25.9 s             | 6.06 s            |
| 99.9999% (six nines)    | 31.56 s           | 2.59 s             | 0.61 s            |
| 99.99999% (seven nines) | 3.16 s            | 259 ms             | 61 ms             |

- Each extra 9 = an order-of-magnitude reduction in downtime
- Need to plan for downtime, even with high availability systems
  - backup systems, disaster recovery plans, clear communication strategies

# Means to Achieve Dependability

- Fault Prevention
  - Prevent occurrence of faults in the first place (proactive)
- Fault Tolerance
  - Design a system that continues operation despite faults
  - i.e. avoid service failures in presence of faults
- Fault Removal
  - Identify and reduce faults already in the system, and reduce their severity
- Fault Forecasting
  - Predict future fault occurrence and their consequences (proactive)

# Means to Improve Dependability

- Fault Prevention
  - Suitable design patterns; Rigorous requirement analysis; Formal verification
- Fault Tolerance
  - Design for redundancy
  - Incorporate error detection and recovery mechanisms
- Fault Removal
  - Early prototyping and testing; Static code analysis
- Fault Forecasting
  - Monitor performance; Analyse incident reports; Conduct vulnerability audits

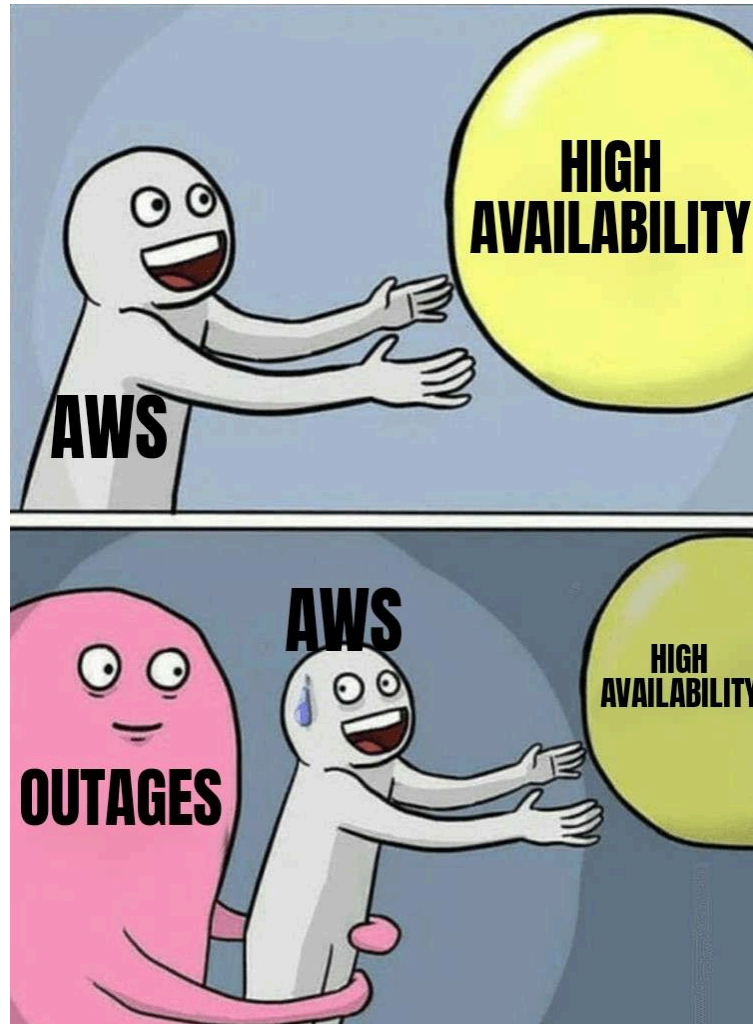
Design time  
Runtime

Runtime

Design time

Runtime and as  
system evolves

# Foundations of High Availability



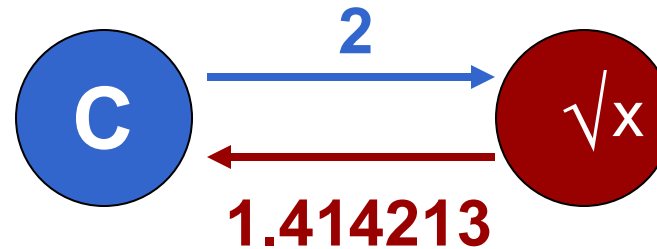
# Fault Tolerance

- Continued system operation even when some components fail
- A key ingredient for both availability and reliability
- Multiple strategies to achieve it
  - Error detection
  - Failover mechanisms (Error recovery)
  - Load balancing
  - Redundancy / Replication
  - Auto-scaling
  - Graceful degradation
  - Fault isolation
  - ...

# Error Detection

- Error detection
  - Failover (Error recovery)
  - Load balancing
  - Redundancy / Replication
  - Auto-scaling

- In the Titanic, this is quite easy: no water inside the hull of the ship
- In distributed systems, this is more difficult
  - What should be the result of a request is usually unknown



- Sometimes, but not always, a sanity check is possible
  - e.g. checksum fields in network packets
- However, this only checks syntax and not semantics



# Error Detection in Data Centres

- Monitoring: collecting key metrics, e.g. CPU, memory, disk I/O
  - Heartbeats (periodic pulse signals) for basic indication of issues
  - Spikes can indicate malfunctioning processes or malware
  - Exceeding thresholds can indicate overloaded systems
- Telemetry: analysing metrics across servers to identify issues
  - Understand usage patterns across time, applications, and users
  - Anomalies can detect potential security threats or hardware failures
  - Insights can help predict future needs and scale infrastructure
- Observability: understand internal state by examining outputs
  - Analysis of system logs and traces of comms through the system
  - Helps understand complex system behaviour

Is it  
working?

What data  
is there?

Why is it  
behaving  
like this?

# Error Detection – Circuit Breaker Pattern

- Inspired by electrical circuit breakers
  - Closed: requests pass through to a service
  - Open: a certain number of failures occur within a specified time window
  - Half-open (after a timeout): allow a limited number of requests through to check if the service has recovered
- Prevents overload of failing services and allowing time to recover
- **Reliability**: prevents cascading failures in distributed systems
- **Availability (long-term)**: fail now rather than degrade under stress
- Performance: quickly reject requests that are likely to fail



# Error Detection in Data Centres – ECC

- Error-Correcting Code (ECC) memory
- Adds extra parity bits to each word to detect and correct errors
  - Single-bit errors
    - detect and automatically correct single-bit errors within a word, transparently without interrupting server operation
  - Double-bit errors
    - detect double-bit errors but typically cannot correct them
    - instead, it triggers an alert or halt system to prevent corruption
- Widely adopted as standard practice across major cloud providers

Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. “DRAM errors in the wild: a large-scale field study”. SIGMETRICS Perform. Eval. Rev. 37, 1 (June 2009), 193–204  
<https://doi.org/10.1145/2492101.1555372>

# Error Detection in Data Centres – Uber

- M3 is a large-scale platform for storing & querying time-series metrics
- Handles massive volume of metrics generated by microservices
  - System-level: cpu, mem, disk, network
  - JVM-level: garbage collection frequency + duration, heap memory, #threads
  - Application-level: service request rates, error frequency, latency
  - Business-specific: #active drivers, trip requests, ETA accuracy, surge pricing
- M3 client periodically pushes metrics to aggregator

<https://m3db.io/>

<https://www.uber.com/en-GB/blog/m3/>

# Error Detection in Data Centres – Netflix



- Stream processing of real-time data (events, metrics, logs) generated by services
- Enables real-time operational monitoring, alerting, and troubleshooting
- Built to be highly scalable and reliable
- Engineers can create custom Mantis Jobs to process streams in specific ways
  - e.g. filtering, aggregation, anomaly detection
- Can ingest from various sources (e.g., Kafka, S3) and output to multiple destinations (e.g., Elasticsearch)
- Provides a query language to interact with and analyse stream data
- Makes it relatively easy to build, deploy, and manage stream processing jobs without needing deep expertise in distributed systems

<https://netflix.github.io/mantis/>

<https://netflixtechblog.com/open-sourcing-mantis-a-platform-for-building-cost-effective-realtime-operations-focused-5b8ff387813a>

- ✓ Error detection
- Failover (Error recovery)
  - Load balancing
  - Redundancy / Replication
  - Auto-scaling

# Failover (Error Recovery)

- Failover: an automatic switch to a redundant system upon failure
- Why is it crucial?
  - Availability – ensures continuous service
  - Reliability – minimises disruption to service
  - Data Integrity – protects against data loss
- How to recover?
  - Heterogeneity / Diversity – reducing chance of common faults
  - Redundancy – accepting that ‘things’ **will** fail (a matter of *when* not *if*)
  - Isolation – preventing faults from propagating

# Failover Patterns

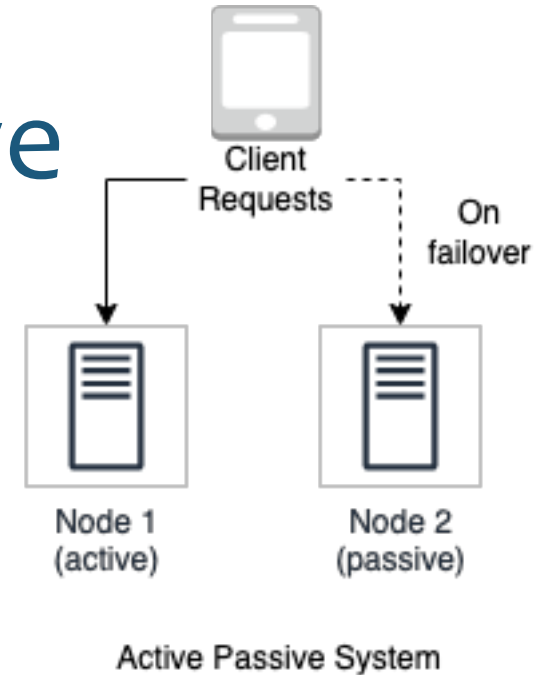
Active-Passive

Variations of  
Active-Passive

Active-Active

# Failover Strategies – Active-Passive

- Active
  - Primary system handling all workload (processing & serving)
- Passive
  - Idle standby system, not handling any workload
  - Kept in sync with the active system (e.g. through database replication)
- Failover
  - When Active fails, Passive becomes the new Active system
- Pros: Simple to implement and cost-effective
- Cons: Can have a longer recovery time
  - including electing new primary if more than one standby



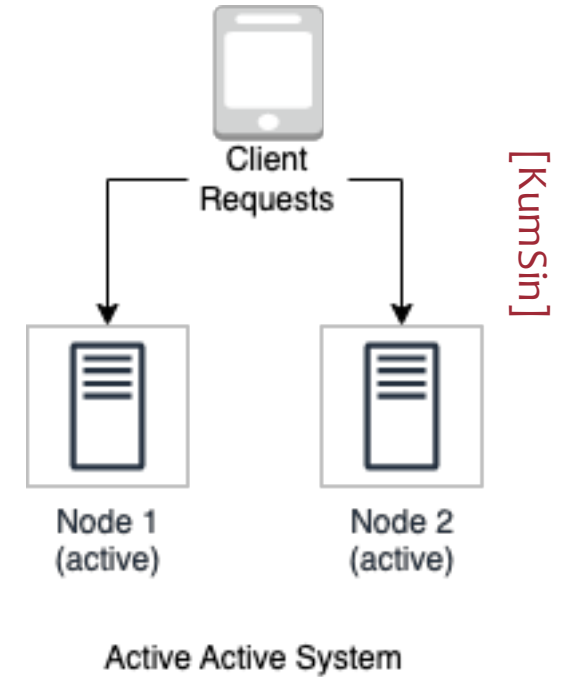


# Failover Strategies – Active-Passive Variants

- Cold Standby
  - Needs to be booted and configured (e.g. to restore data)
- Warm Standby
  - Running but only synchronized periodically (checkpointing)
- Hot Standby
  - Fully synchronized and ready to take over instantly

# Failover Strategies – Active-Active

- Active
  - Two or more systems are simultaneously handling workload (processing & serving)
- Load balancer
  - Distributes incoming traffic across active systems
  - Allocation using various algorithms: round-robin, least load, ...
- Failover
  - When an active system fails, the load balancer automatically redirects its traffic to the remaining active systems
- Pros: Short recovery (immediate)
- Cons: More complex (consistency) and less cost-effective



# Failover Strategies – Key Decision Factors

- State management & consistency requirements
  - Active-Active is not best for stateful systems
- Recovery Time Objective (RTO)
  - Active-Active has the lowest RTO, then Active-Passive and variants
- Cost constraints
  - More synchronisation = more costs (bandwidth, memory)
- Operational complexity
  - Active-Active is complex to implement

# Further Reading

- IBM, “*High Availability Models*”  
<https://www.ibm.com/docs/en/cdfsp/7.6.0?topic=systems-high-availability-models>
- AWS, “*Disaster Recovery Options in the Cloud*”  
<https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html>
- Guo, “*State-of-the-art DCN Topologies*”, In: Data Center Networking, 2022.

# References

- [CNCF22] Cloud Native Computing Foundation, “[CNCF Annual Survey](#)”, 2022
- [DCK24] DataCenterKnowledge, “[Top Data Center Outage Trends and Strategies for Reducing Risk](#)”
- [GTX14] Peter Garraghan, Paul Townend, Jie Xu, “An Empirical Failure-Analysis of a Large-Scale Cloud Computing Environment”, IEEE HASE, 2014.
- [Hum21] Humantiec, “[DevOps Setups – A Benchmarking Study](#)”, 2021
- [KumSin] Jayanth Kumar, Mandeep Singh, “System Design on AWS”, O'Reilly Media, Feb 2025, ISBN 9781098146894.
- [UTI22] UptimeInstitute, “[Uptime Institute’s 2022 Outage Analysis Finds Downtime Costs and Consequences Worsening as Industry Efforts to Curb Outage Frequency Fall Short](#)”, 2022
- [WZX17] Guosai Wang, Lifei Zhang, Wei Xu, “What Can We Learn from Four Years of Data Center Hardware Failures?”, IEEE/IFIP DSN, 2017.

# COMPSCI4106/5118 Cloud Systems



Intro to Cloud Computing and Virtual Resources

Virtual Machines

Containers and Container Management

Cloud System Operation and Management

Cloud System Sustainability

Cloud System Design

## ► **Modern Cloud Architectures**

Flavours of Cloud

A Wider Lens on Sustainability

Emerging Topics and Research Trends

Yehia Elkhatib

# Overview

- Architectural Approaches
  - Layering and Tiering
  - Redundancy by Replication
  - Cloud Scaling
- Advanced Architectures & Technologies
  - Microservices
  - Service Mesh
  - Cloud-native Technologies

# Architectural Approaches

## THE EVOLUTION OF SOFTWARE ARCHITECTURE

### 1990's

SPAGHETTI-ORIENTED  
ARCHITECTURE  
(aka Copy & Paste)



### 2000's

LASAGNA-ORIENTED  
ARCHITECTURE  
(aka Layered Monolith)



### 2010's

RAVIOLI-ORIENTED  
ARCHITECTURE  
(aka Microservices)



### WHAT'S NEXT?

PROBABLY PIZZA-ORIENTED ARCHITECTURE



# Redundancy by Replication

- Why replicate?
  - Performance, e.g. of heavily loaded servers
  - Allows error detection when replicas disagree
  - Allows backward error recovery
- Importance of **fault-model**
  - Effect of replication depends on *how* individual replicas fail
  - Crash faults → availability =  $1 - p^n$   
where  $n$  = no. of replicas,  $p$  = probability of individual failure
  - E.g. 5 servers each with 90% uptime → availability =  $1 - (0.10)^5 = 99.999\%$



# Architecture?

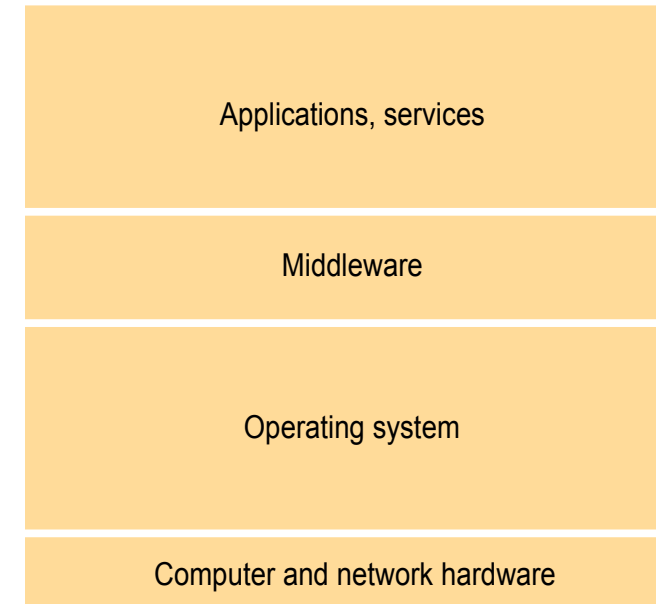
- Foundation Matters: just as buildings need to support their structure, cloud architectures require robust infrastructure layers (compute, storage, networking) to support higher-level services
- Scalability & Modularity: modular construction techniques allow for easier expansion and modification
- Resource Efficiency: through auto-scaling, serverless, and efficient resource allocation
- Evolution: cloud architectures must evolve to accommodate new technologies and requirements while ensuring system stability

# Architectural Concepts – Monolith

- A single, tightly coupled block of code with all application components
- Advantages:
  - Simple to develop and deploy
  - Easy to test and debug in the early stages
- Disadvantages:
  - Complexity – Becomes difficult to maintain application growth / evolution
    - Changes to one part could affect the entire system
  - Scaling challenges – Difficult to scale individual components independently
  - Limited agility – Slow and risky deployments due to the large codebase
  - Technology lock-in – Difficult to adopt new technologies or frameworks

# Architectural Concepts – Layering

- It is common to partition services vertically into layers
  - Lower layers provide services to higher ones
  - Higher layers being unaware of the underlying implementation details
  - Low inter-layer dependency
- Examples:
  - Network protocol stacks, e.g. OSI model
  - Operating systems – kernel, drivers, libraries, GUI, ...
  - Games – engine, logic, AI, UI



# Architectural Concepts – Layering

- Distributed applications commonly feature 3 layers
  - User interface / Presentation: Maintaining interaction between user & system
  - Processing: Often referred to as the business logic of the application
  - Data: Manage persistent state; can comprise Access & Management sublayers
- Advantages:
  - Abstraction
  - Reusability
  - Loose coupling
    - isolated management
    - independent testing
    - software evolution



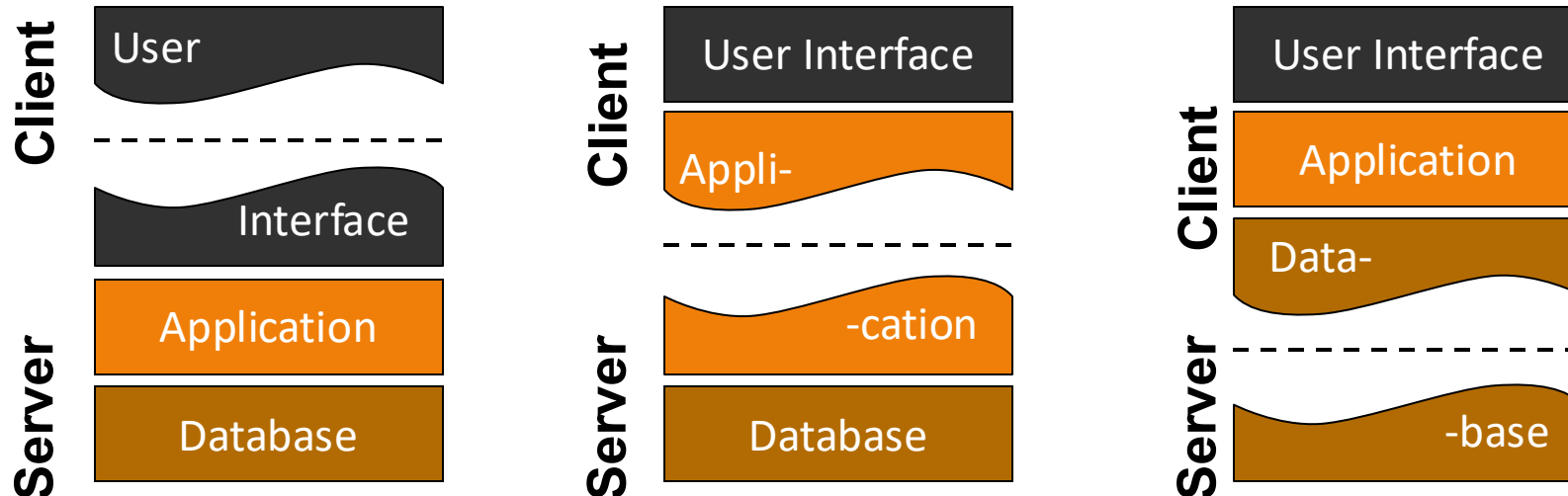
User Interface

Processing

Data

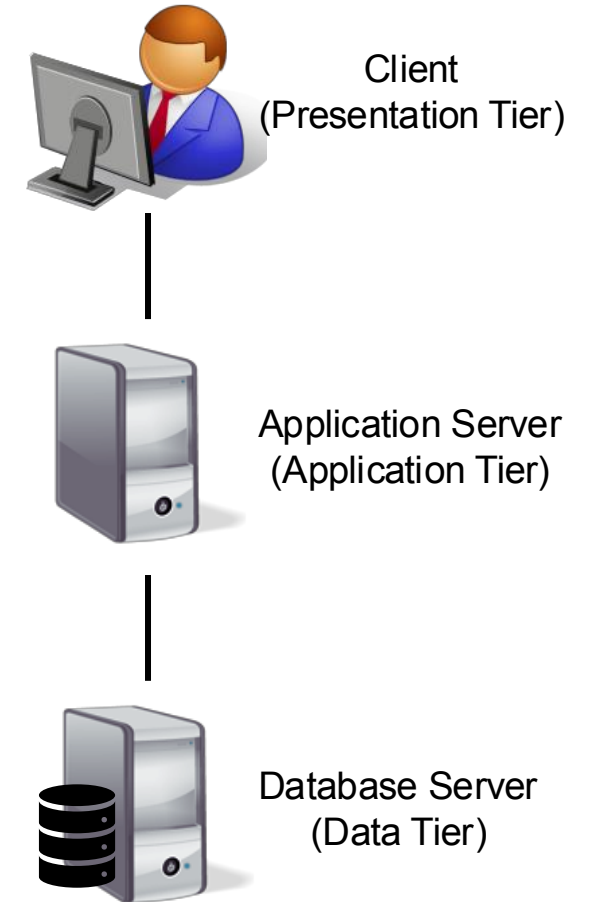
# Architectural Concepts – Tiering

- Tiering complements layering by mapping the organisation *of and within* a layer to **devices**
  - implication about physical location
- Classic 2-tier architecture – *client-server* model
  - split layers between a client and a server in various ways:



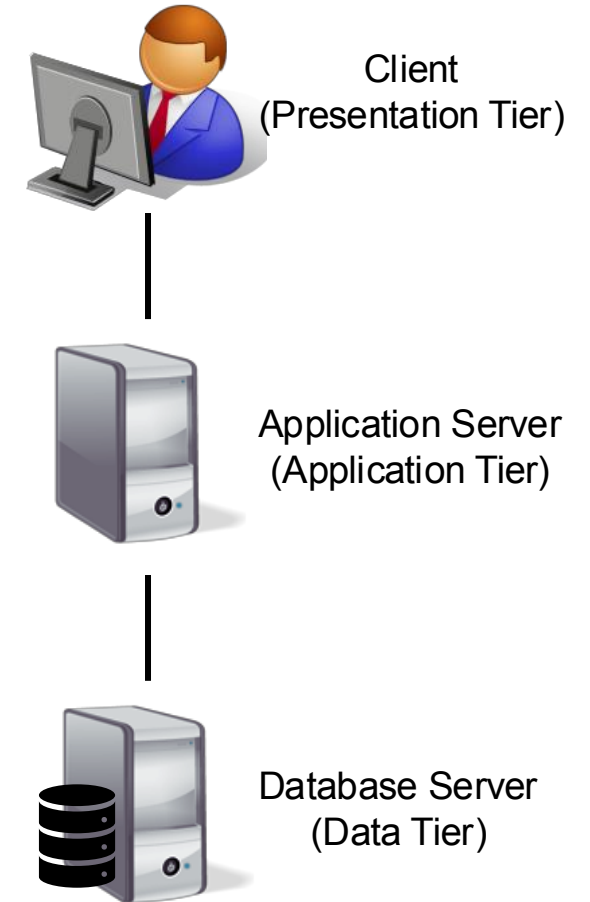
# Architectural Concepts – Tiering

- 3-tiered architecture:
  - Same layers mapped on to three tiers
- 4-tiers ... n-tier (or multi-tier) approaches
  - e.g. microservices
- Advantages:
  - Scalability
  - Availability
  - Flexibility
  - Easy management



# How to increase availability?

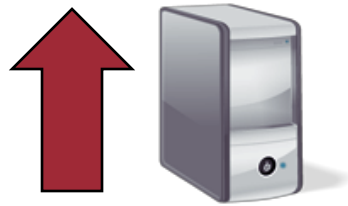
- Classic 3-tier architecture
  - What happens if we increase the number of clients?
  - What happens if one of the components goes down?
- The “cloud approach” – *Scaling*
  - Adjusting the amount of allocated resources
  - When a tier experiences change in load
    - ↑ scale up if the load is increasing
    - ↓ scale down if the load is decreasing





# Cloud Scaling

- ✓ Error detection
- ✓ Failover (Error recovery)
- Load balancing
- Redundancy / Replication
- Auto-scaling



## Vertical Scalability (Scaling Up)

|      |                                                                 |
|------|-----------------------------------------------------------------|
| Idea | Increase performance of a single node (more cores, memory, ...) |
| Pro  | Good speedup up to a particular point; No change to appl. arch. |
| Con  | Beyond that point, speedup becomes very expensive               |



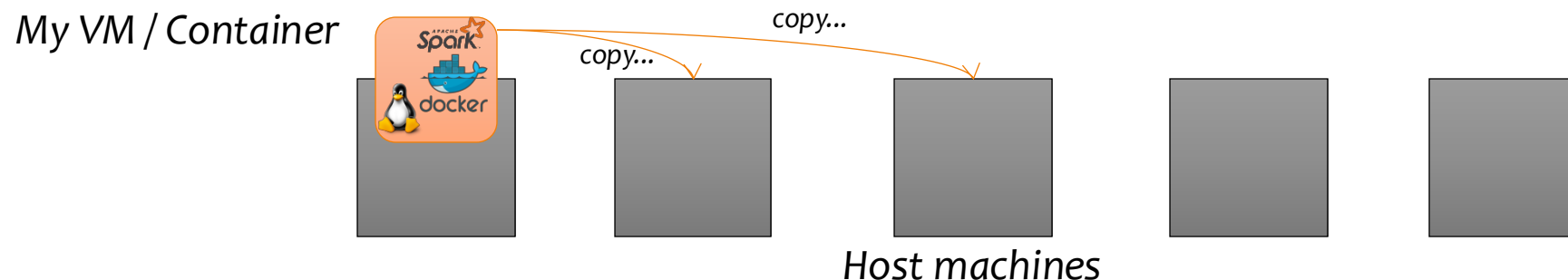
## Horizontal Scalability (Scaling Out)

|      |                                                                       |
|------|-----------------------------------------------------------------------|
| Idea | Increase the number of nodes                                          |
| Pro  | Cheap to grow total amount of resources                               |
| Con  | More difficult to efficiently utilize the resources; Need coord. sys. |

# Cloud Scaling – Out more than Up

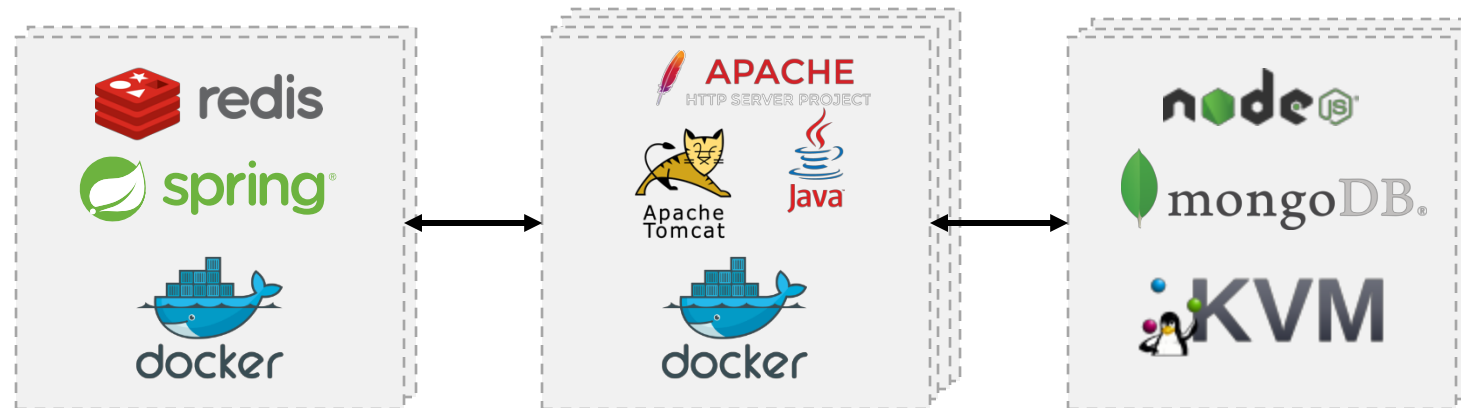
- ✓ Error detection
- ✓ Failover (Error recovery)
- Load balancing
- Redundancy / Replication
- Auto-scaling

- Hardware trend: CPUs are not getting faster as they used to  
→ Datacentres house large sets of inexpensive commodity servers (that fail eventually)
- Scaling applications typically means scaling-out across many nodes
  - Datacentres offer virtually unlimited scale-out services
- VMs and containers make it easy to replicate services across many nodes, especially with IaC and automation / orchestration tools



# Cloud Scaling – Modern Cloud Architecture

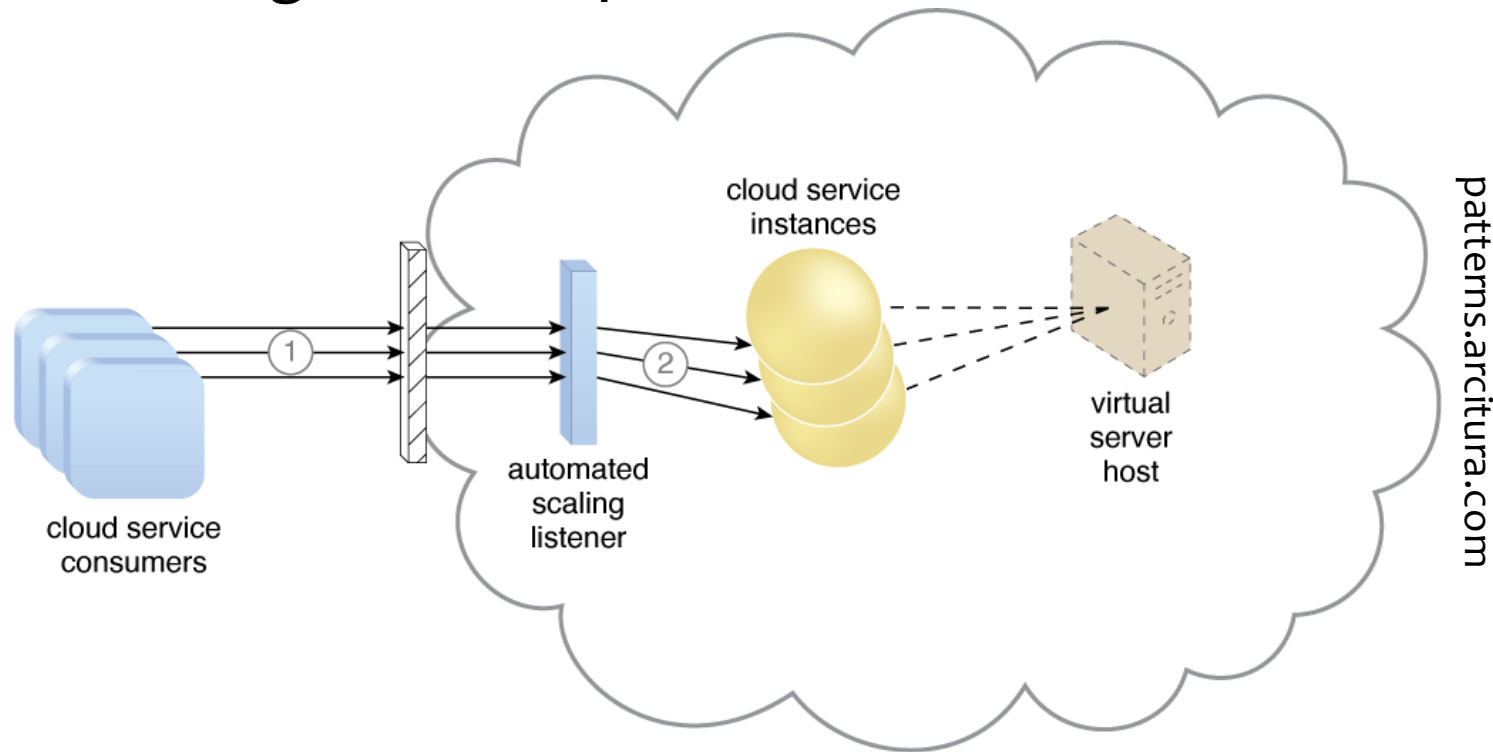
- Today's computer systems are built upon 2 architectural pillars:
  - vertical integration – doing more with each individual tier (service)
  - horizontal scaling – using many commodity computers working together



- RIP monolith applications
- Expanding this created the microservice architecture

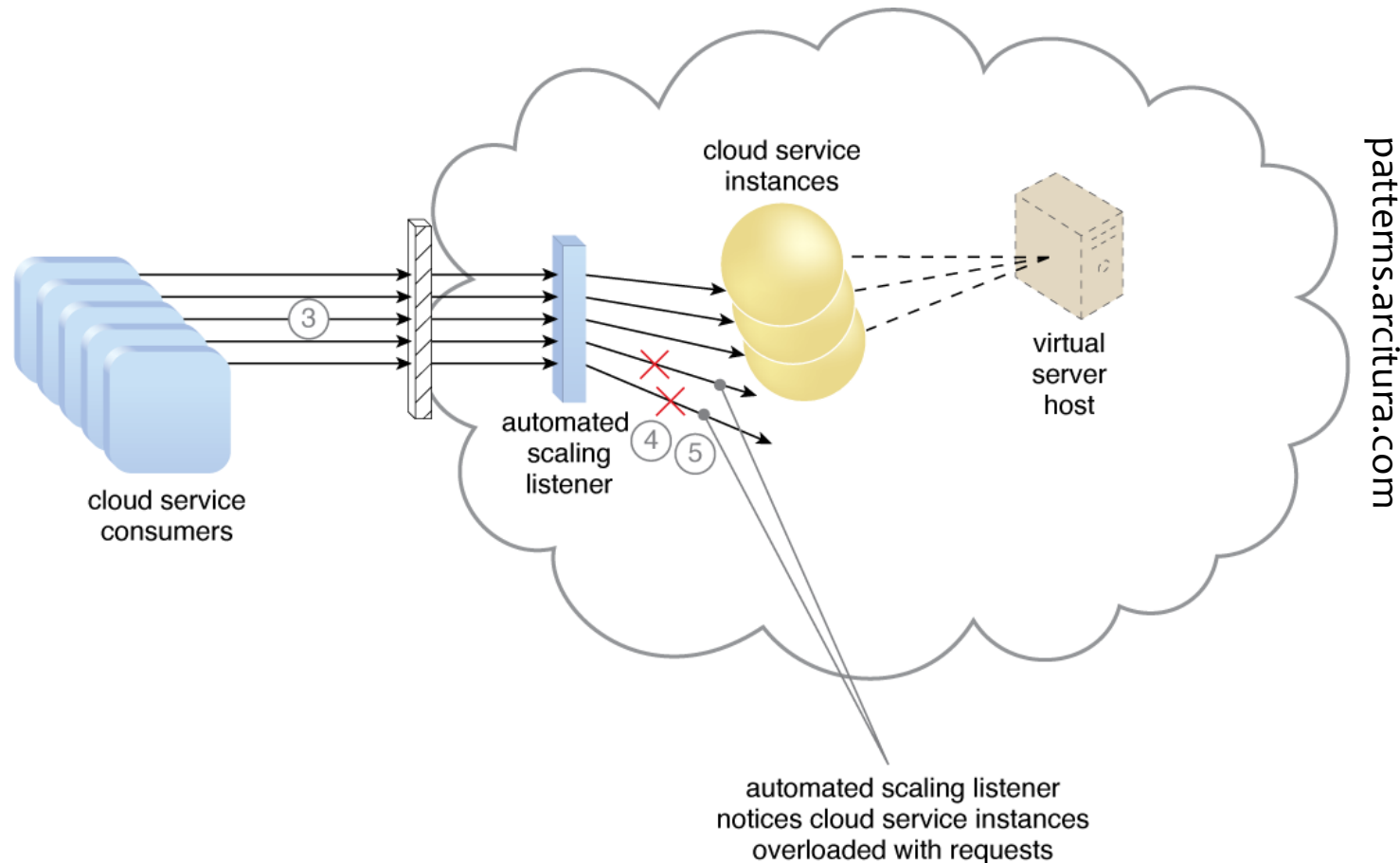
# Simple Dynamic Scaling Architecture

- Consumers sending more requests



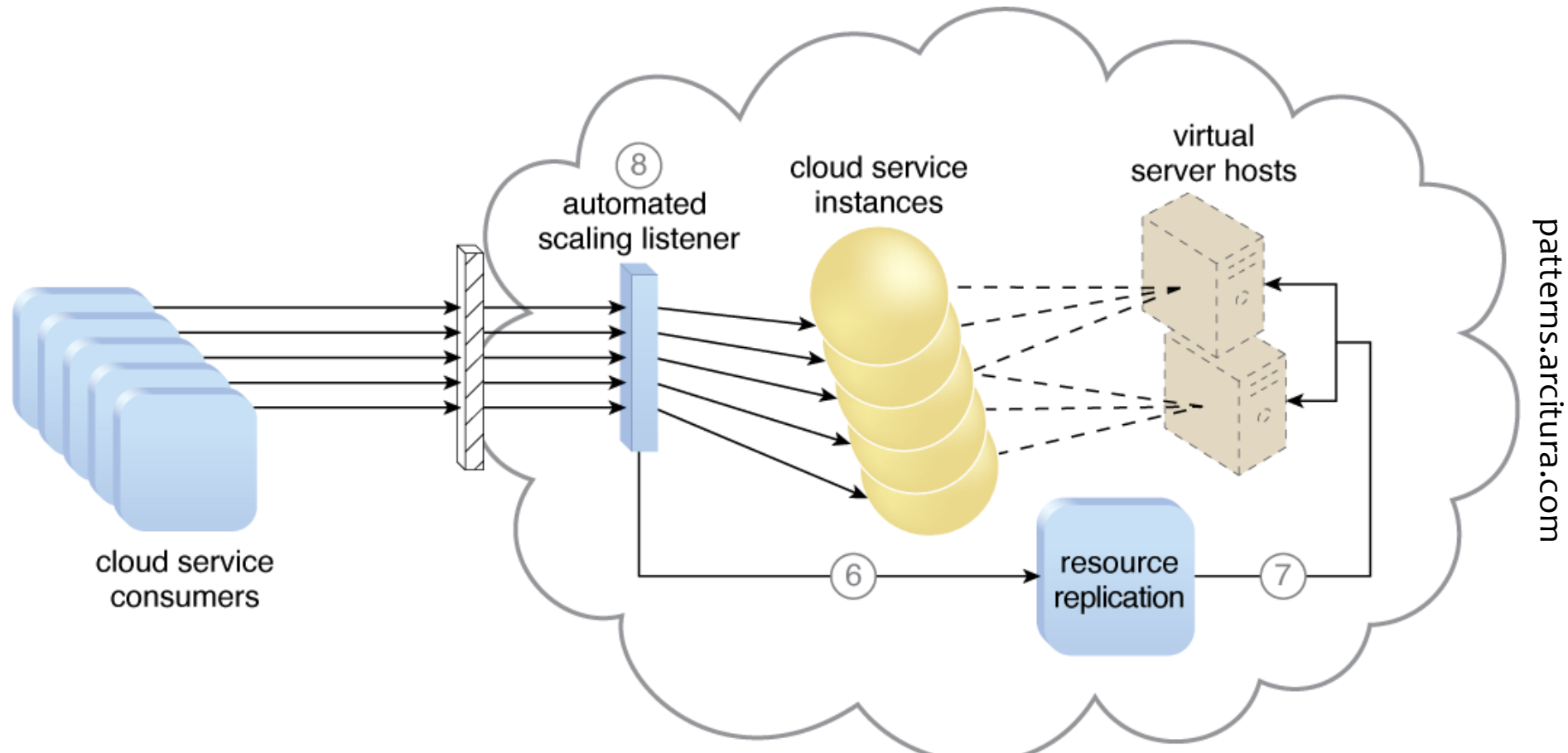
# Simple Dynamic Scaling Architecture

- Requests overload the existing resources, timeouts exceeded



# Simple Dynamic Scaling Architecture

- Scaling up by deploying additional resources



# Scaling and State

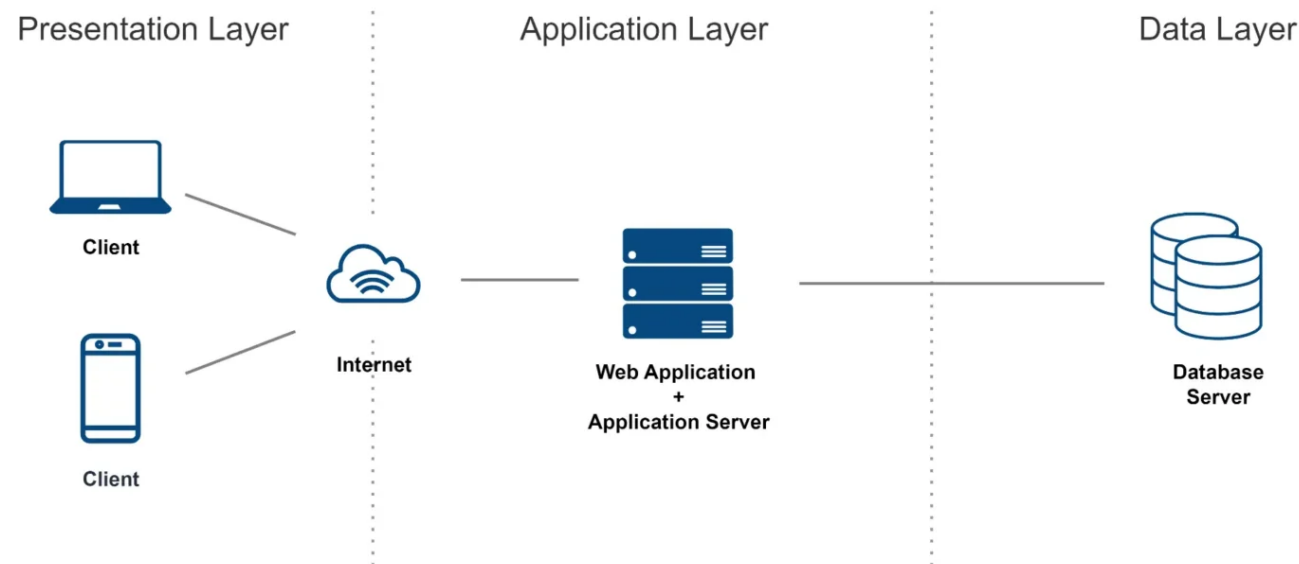
- Multi-tier application scalability entails two questions
  - Where is the bottleneck in the architecture?
  - Is the bottleneck component *stateless* or *stateful*?
- Stateless components
  - Maintain no internal state beyond a request
  - Examples: DNS server, web server with static data, mathematical calc., ...
- Stateful components
  - Maintain state beyond request
    - i.e. prior state is required to process future requests
  - Examples: mail server, firewalls, stateful web server, DBMS, ...



# Scaling Stateless Components

- ✓ Error detection
- ✓ Failover (Error recovery)
- Load balancing
- Redundancy / Replication
- ✓ Auto-scaling

- Assume a bottleneck being a stateless service  
→ Create more instances of said service

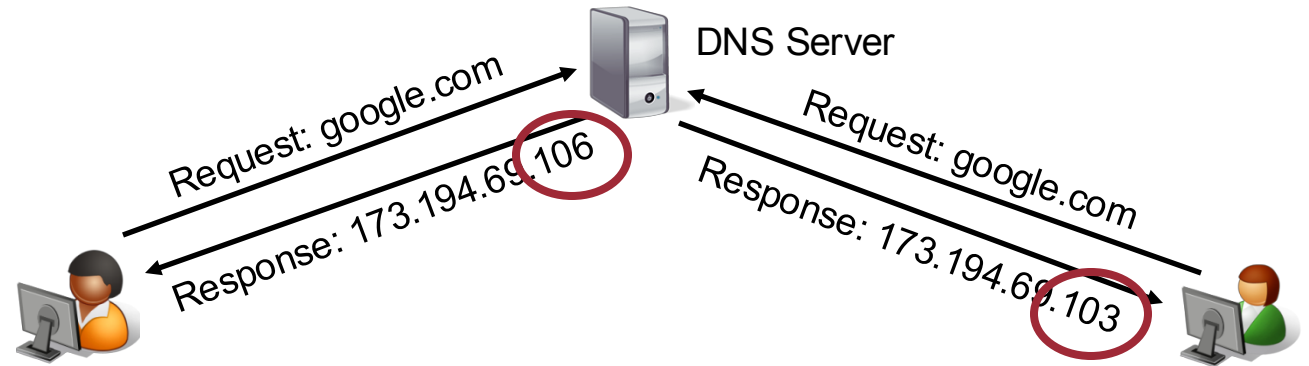


- How do clients figure out which server to contact?



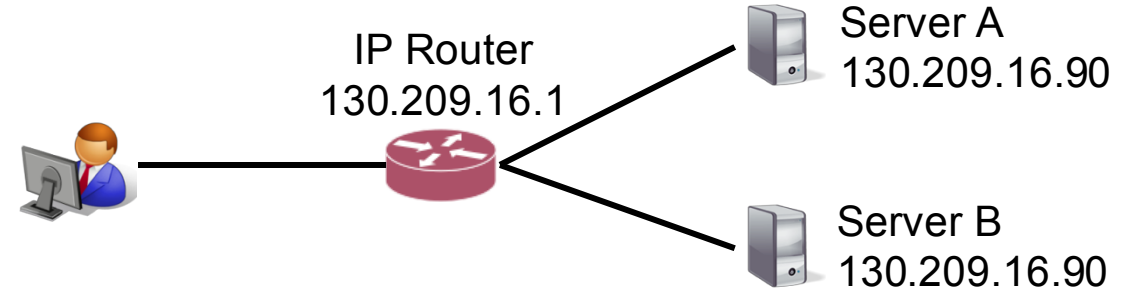
# Stateless Load Balancing – DNS Level

- Balancing is implemented by DNS servers
  - DNS servers resolve domain names to IP addresses
  - Can use geo. location for also complying with sovereignty regs. (GeoDNS)
- Advantages:
  - Simple to implement
  - Cost-effective
  - Can handle high traffic volumes
- Disadvantages:
  - Less control over traffic distribution
  - Can be slow to react to server failures due to DNS caching
  - Limited health check capabilities



# Stateless Load Balancing – IP Level

- Balancing is implemented by routers at the network layer
  - Multiple devices share one IP address (IP anycast)
  - Routers direct clients to different locations (round robin, hashing algorithms)
- Advantages:
  - Relatively simple
  - Fast response to server failures
  - Basic health checks and monitoring
- Disadvantages:
  - Less granular; assumes all requests create equal load
  - More expensive than DNS load balancing (routing configuration)
  - Does not handle application-layer optimisations (e.g. SSL termination)



# Stateless Load Balancing – Application Level

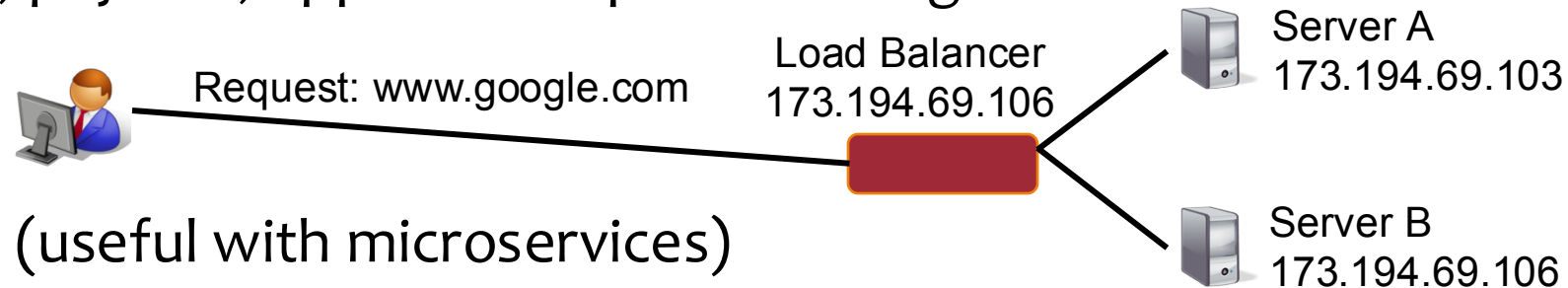
- Bespoke load balancer acting as a temporary front end
  - Distributes requests among machines based on various features
  - HTTP headers, cookies, payload, application-specific configuration

- Advantages:

- Granular control
- Content-based routing (useful with microservices)
- Can reduce load on servers (e.g. SSL offloading)

- Disadvantages:

- Increased complexity; requires deep knowledge of the application
- Performance overhead, higher latency
- Resource intensive, costly



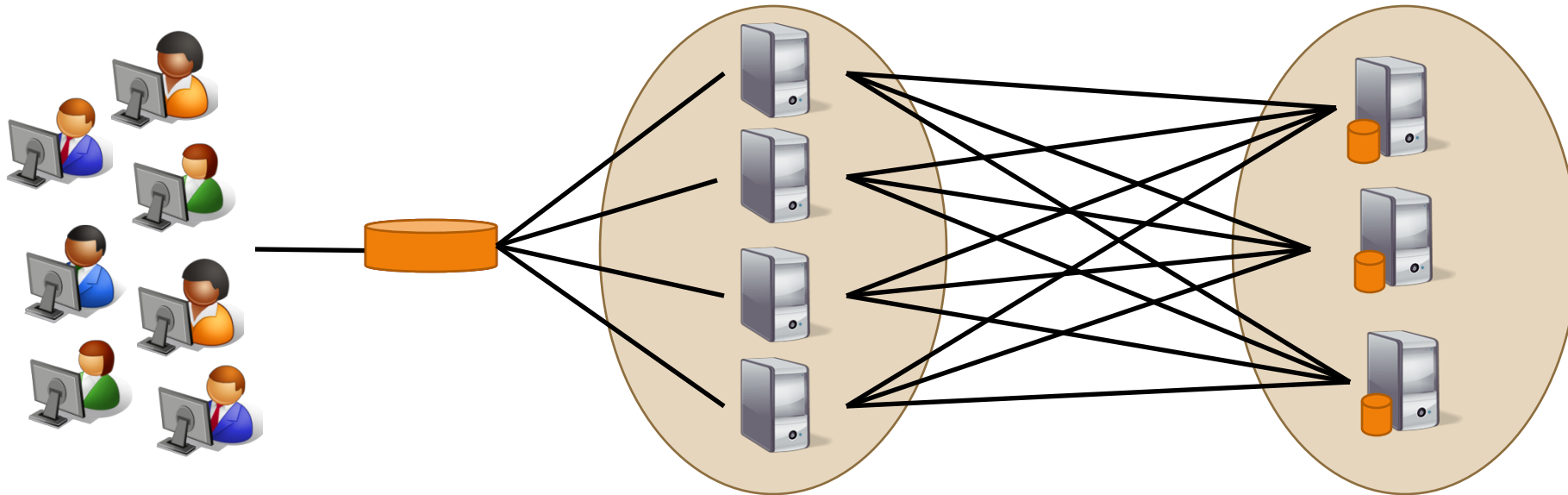
# Stateless Load Balancing – Others

- Load balancing on different levels can be combined
  - e.g. distribute network load among distinct LBs first through DNS
- Other load balancing strategies
  1. Feedback-based – servers report their load levels back to the LB
  2. Client-based – choose the server with the least network latency

# Stateful Scaling

- ✓ Error detection
- ✓ Failover (Error recovery)
- ✓ Load balancing
- Redundancy / Replication
- ✓ Auto-scaling

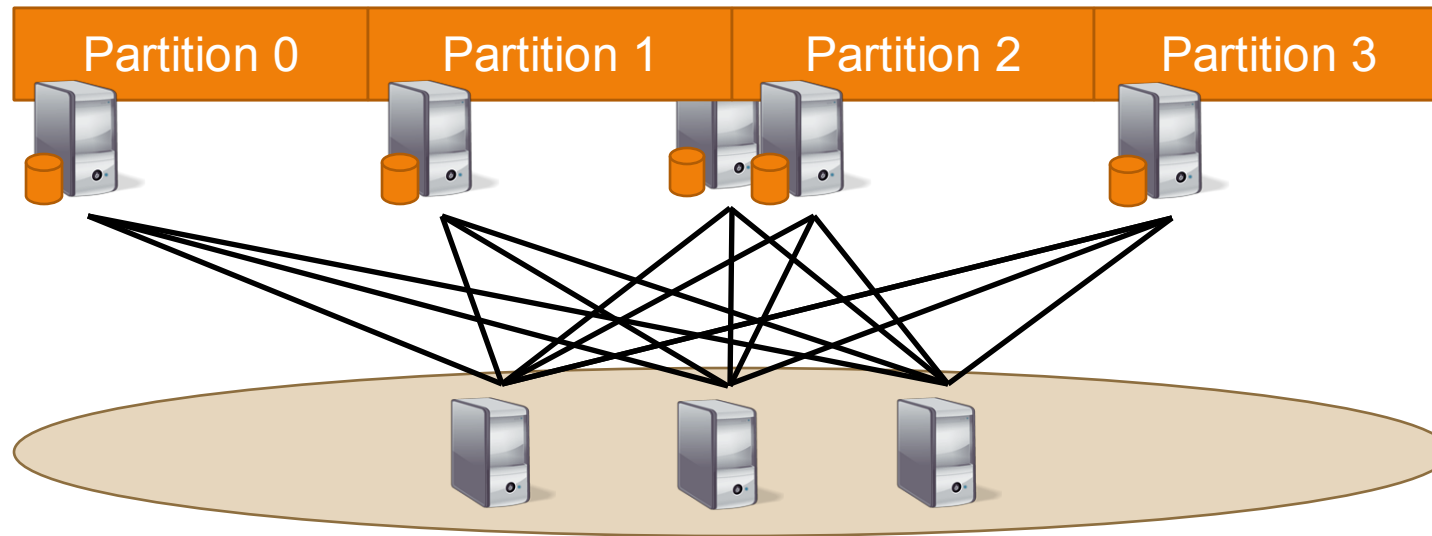
- Now assume that the data tier is the bottleneck



- A database (DB) is stateful: retains information about previous interactions
- Requests from a client must be handled by the same DB instance

# Partitioning

- Divide data into distinct, independent parts
- Each server is responsible for one or more parts



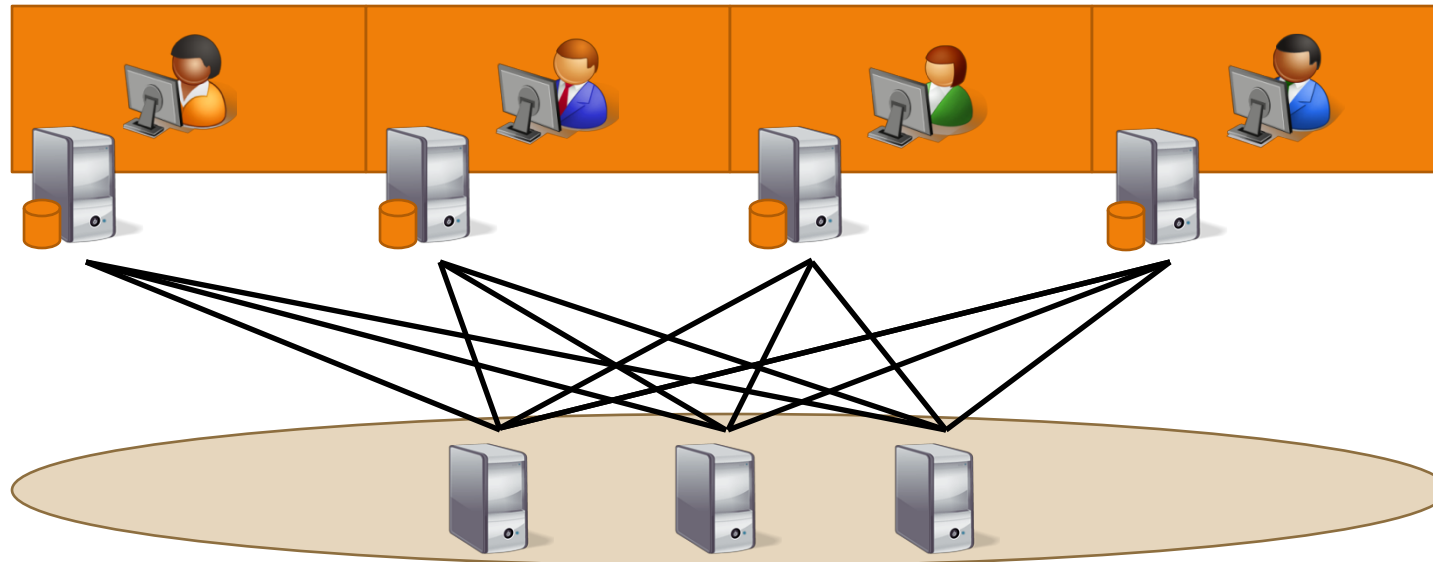
- Partitioning improves just scalability (performance), not availability
  - Each data item is only stored in one partition

# Partitioning – General Considerations

- Data is spread across several machines
- Particular tasks might require reading/writing data from multiple servers
  - causes additional network traffic
- Challenge
  - Network is a scarce resource with limited scalability
  - Becomes scarcer the more machines you add
- For good scalability, the goal of every partitioning scheme is typically to reduce network communication
- However, this is highly application-specific...

# Partitioning Schemes – Per Tenant

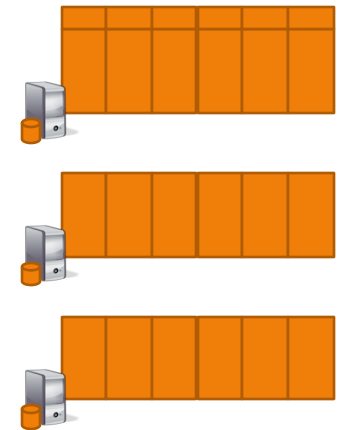
- Put different tenants on different machines
  - Isolation is typically a requirement in cloud environments
  - No network traffic between machines, good scalability
  - Con: Scaling a tenant beyond one machine becomes complex





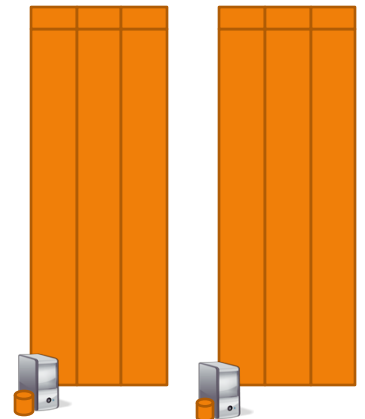
# Partitioning Schemes – Horizontal / Sharding

- Split table by rows across different *shards* distributed across servers
- Each shard has the same schema, but contains only a subset of rows
  - Based on a partitioning key, e.g. user ID, region, etc.
- Reduced number of rows, reduced indices
- Easy to scale out (and up)
- If one shard fails, only that data is affected
- Con: cross-shard queries can be slow; requires balancing
- Examples: Google BigTable and MongoDB



# Partitioning Schemes – Vertical

- Split table by columns, grouping related columns together
- Can improve performance of specific queries (e.g. routine analytics)
- Cons:
  - Does not inherently support scaling across multiple servers
  - Cross-partition queries may require additional joins, increasing latency



# Partitioning – How to Distribute Partitions?

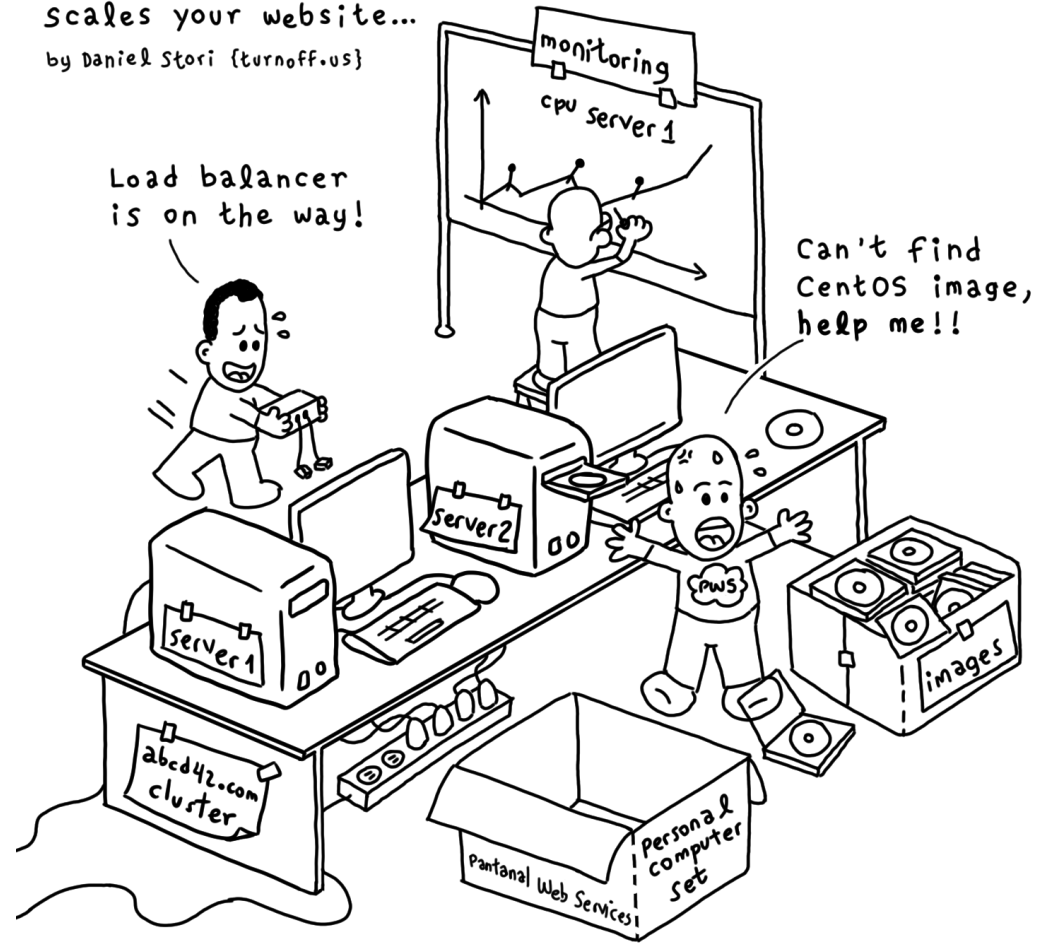
- Range partitioning
  - Goal: Close DB attributes will also be stored close to each other
  - Pro: Easy; efficient for range queries; simple to scale individual partitions
  - Con: Poor load balancing properties; requires manual adjustment
- Hash partitioning
  - Goal: Uniform distribution
  - Pro: Good load balancing characteristics
  - Con: Inefficient for range queries; requires reorganization when number of partitions changes

# Further Reading on Architecture

- Eric Brewer, “[CAP twelve years later: How the ‘rules’ have changed](#)”, IEEE Computer, vol. 45, no. 2, 2012.
- James C. Corbett et al., “[Spanner: Google’s globally distributed database](#)”, ACM TOCS, Volume 31, Issue 3, 2013.
- Giuseppe DeCandia et al., “[Dynamo: Amazon's highly available key-value store](#)”, ACM SIGOPS OS Review, Volume 41, Issue 6, 2007.
- Martin Klepmann, “*Designing Data-Intensive Applications*”, chapter 6, O’Reilly, 2017.

# Break!

This is what happens when  
your cloud provider  
scales your website...  
by Daniel Stori {turnoff.us}



# Advanced Architectures & Technologies

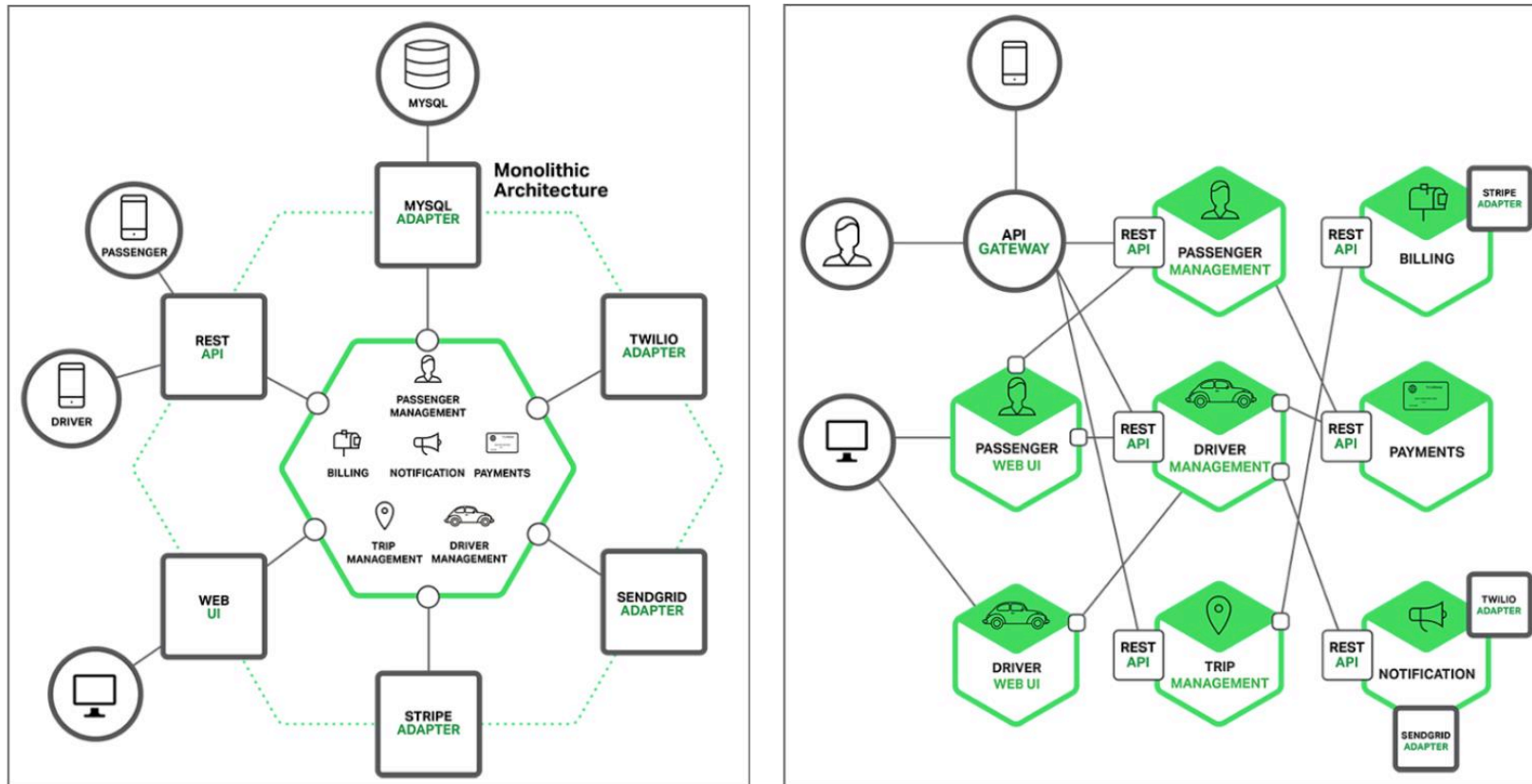


# The Microservice Architecture

- Monoliths are incredibly difficult to maintain and scale
- Even layered architectures can be too complex to evolve
- n-tiered architecture
  - Fragment logic into loosely coupled elements (services)
  - Each service focuses on one job
  - Services can have completely different tech stacks
  - Services communicate through lightweight APIs (e.g. REST)
- The dominant architectural pattern
  - Top performers adopt microservices in 95.5% of applications

Humantiec, “[DevOps Setups – A Benchmarking Study](#)”. Technical Report, 2021

# The Microservice Architecture



Christina Delimitrou, “The Hardware & Software Implications of Microservices and How Big Data Can Help”, Stanford Oct 2018.

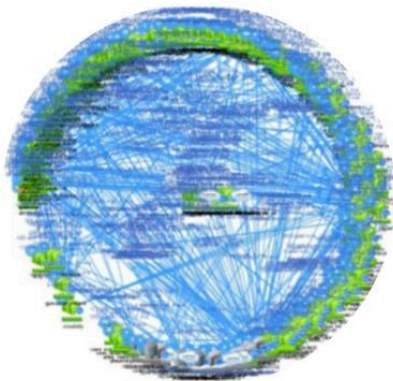


# Microservices – Advantages

- Loose coupling
- Independent deployment, fault isolation (not affecting other services)
- Development flexibility, technology diversity
- Agility – faster development cycles
- Scalability – only scale parts that need it
- Maintainability – smaller codebases are less complex
- Easy reuse in different projects
  - e.g. Netflix [Asgard](#), [Eureka](#)

Any cons?

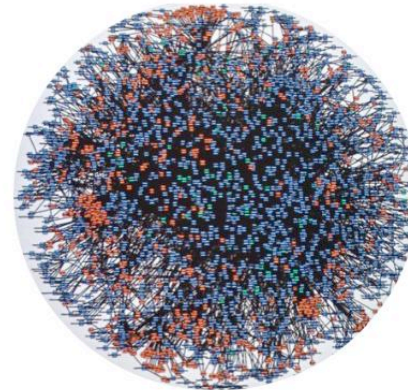
# The Microservice Architecture



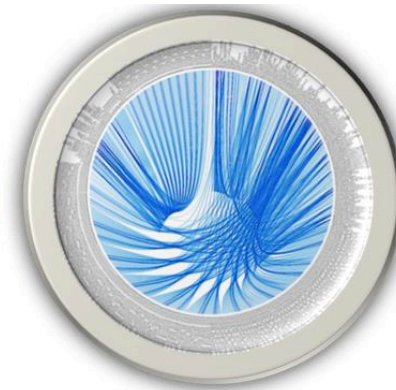
**Netflix**



**Twitter**



**Amazon**



**Social Network**

Y Gan et al. “An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems”, ASPLOS 2019

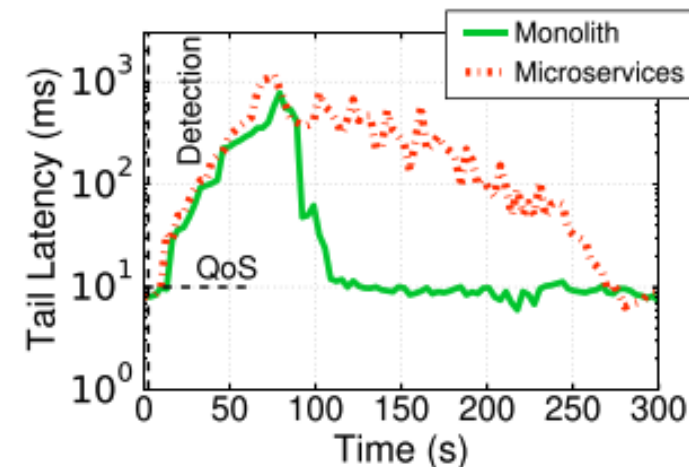
# Microservices – Disadvantages (1/2)

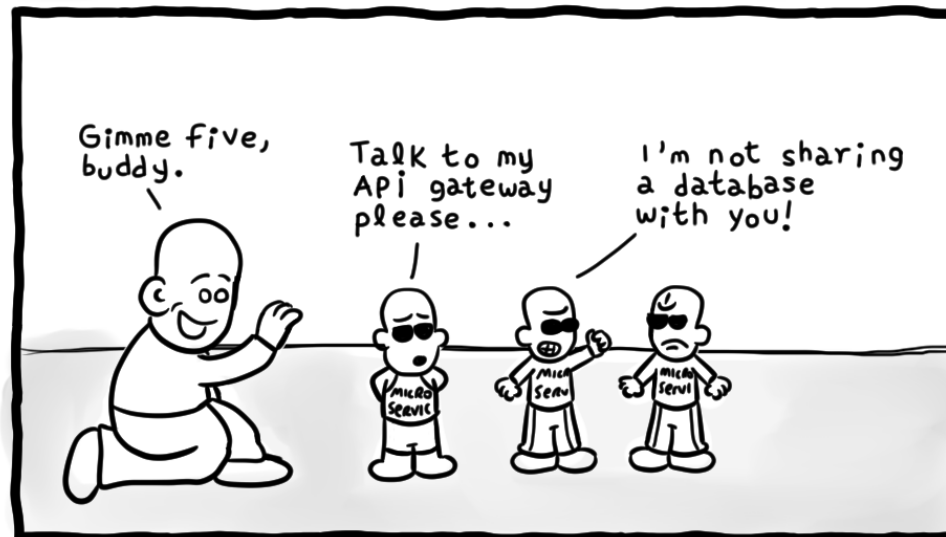
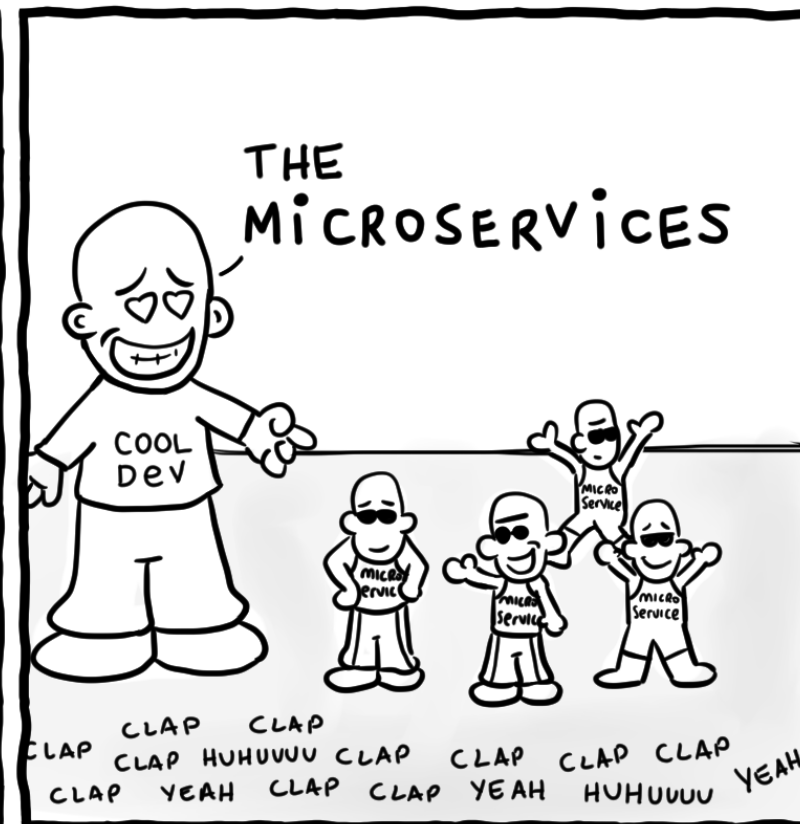
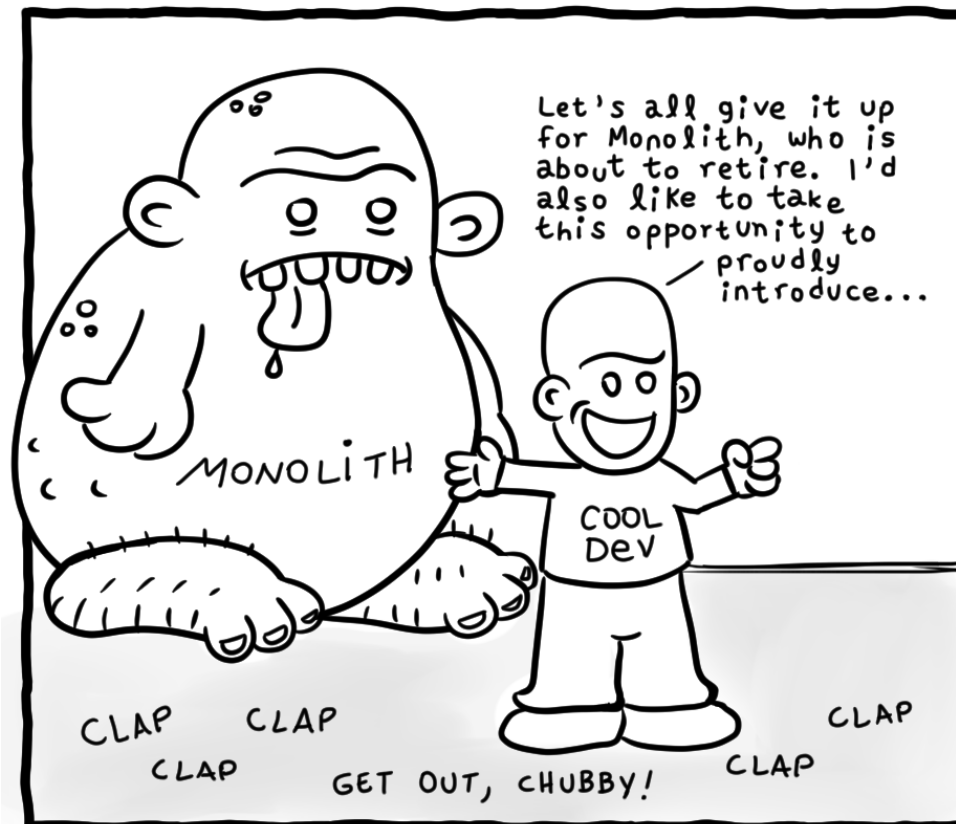
- Complexity
  - Increased operational overhead – more services to manage and monitor
  - Distributed debugging – tracing issues across multiple services
- Latency due to communication between services
- Microservice sprawl – could balloon into 100s or 1,000s of services
- Operational overhead – manage CI/CD pipelines for multiple services
- End-to-end testing can be difficult

# Microservices – Disadvantages (2/2)

- Interdependency chains can cause chaos
- Failure of one service can trigger:
  - failures in dependent services, creating a domino effect (**Cascading failures**)
  - failures in other containers of the same service (**Death spiral**)
  - wasted resources on failed calls, degrading the calling service (**Retry storm**)
- Cascading QoS violations, due to bottleneck services
- Failure recovery could take longer than monoliths

Christina Delimitrou, “The Hardware & Software Implications of Microservices and How Big Data Can Help”, Stanford Oct 2018.





# Microservices – Glueware

- Software to support monitoring, consistency, load balancing, coordination, service discovery, message passing, resilience, etc.

- In 2022, glueware outweighed core microservices

Cloud Native Computing Foundation,  
“[CNCF Annual Survey](#)”, Technical  
Report, 2022.



# Microservice Communication

## Synchronous Communication

- REST APIs (HTTP/HTTPS)
- gRPC (Bidirectional streaming)
- GraphQL (Query-based)

### Pros:

- Immediate response
- Simpler to implement
- Easy to debug

### Cons:

- Tight coupling
- Higher latency
- Lower fault tolerance

## Asynchronous Communication

- Message queues (RabbitMQ, Kafka)
- Event streaming
- Pub/Sub pattern

### Pros:

- Loose coupling
- Better scalability
- Higher fault tolerance

### Cons:

- More complex to implement
- Harder to debug
- Eventually consistent

# Service Mesh Technologies (SMTs)

- A dedicated infrastructure layer for managing and securing comms
- Rely on the pattern of sidecar proxies
  - Microservice = a container with core business logic and a sidecar container
- The sidecar acts as the ingress and egress point, with auxiliary logic
  - Feature-rich data plane (low latency, secure)
  - Control plane (traffic management, service and traffic observability, certs)
- Programmatically define policies for s2s interactions
  - extract this logic out of the core business logic
  - version and apply consistently at different levels
- SMT use increased from 27% (2020) to 47% (2022)

Cloud Native Computing Foundation,  
“[CNCF Annual Survey](#)”, Technical  
Report, 2022.

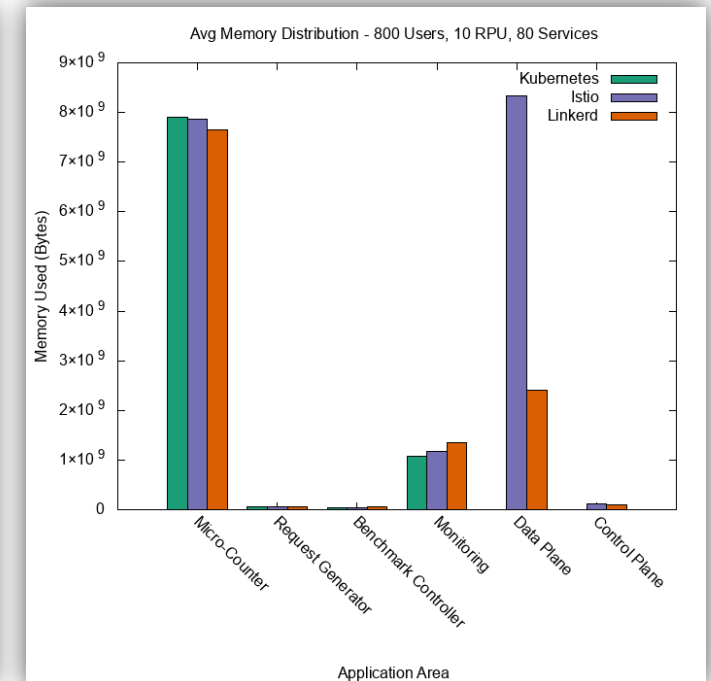
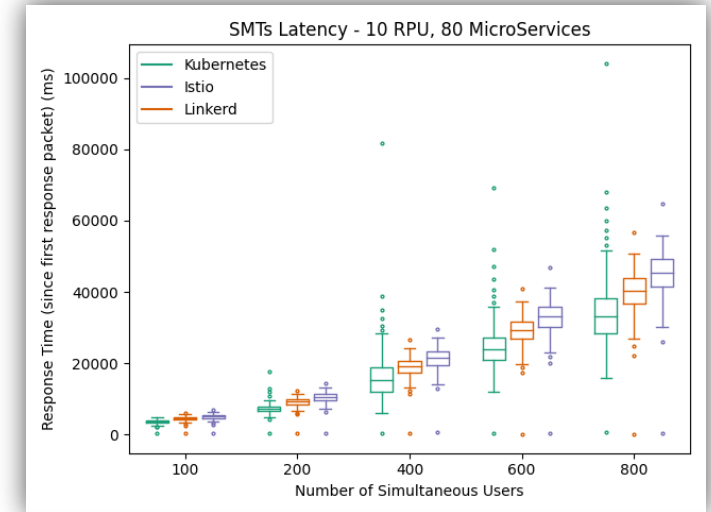
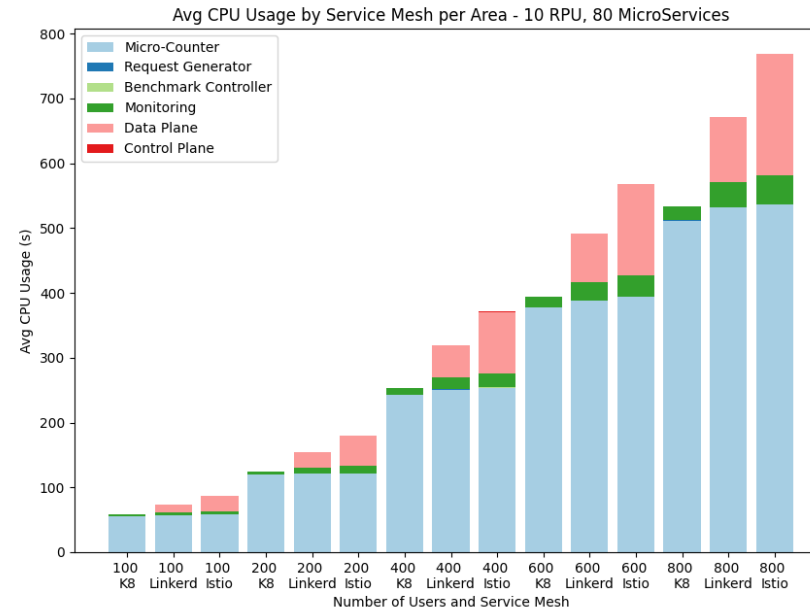


# SMT Comparison



- Linkerd is 6%–12% faster than Istio
- Istio uses 20%–43% memory > Linkerd
- Istio uses 16%–20% cpu > Linkerd

Yehia Elkhatib, and Jose Povedano Poyato, “*An Evaluation of Service Mesh Frameworks for Edge Systems*”, EdgeSys, 2023



# API Gateways



Kong

gravitee.io



krakenD

Tyk



- A reverse proxy that sits between clients and backend services
- Acts as a single-entry point for all API calls
- Key responsibilities
  - Route incoming API requests to backend microservices
    - Based on the request path, method, etc.; Might also do load-balancing
  - Authentication and authorization
    - Verify the identity of clients
    - Ensure they have the necessary permissions to access specific resources
  - Protocol transformation and data format conversion
    - Translate between different protocols (e.g., REST, gRPC, GraphQL) when needed
  - Monitor API usage and rate-limiting
    - Track (using API key or IP address) within a specific time window; otherwise, 429

# Microservices – Avoiding the Sprawl

- Start with a monolith design
  - Gradually break it down into microservices as needed
  - Identify natural boundaries and avoid over-decomposition
- Focus on business capabilities
  - Design around clear purposes rather than technical functions
- Establish clear governance
  - Define guidelines and best practices for microservice development
  - e.g. naming conventions, comm. protocols, deployment procedures
- Follow fault-tolerant microservice design practices
  - e.g. timeouts, bounded retries, circuit breakers

# Further Reading on Microservices

- DORA, “[Loosely Coupled Teams](#)”
- Fowler, “[Microservice Trade-Offs](#)”
- Gan et al. “[Unveiling the Hardware and Software Implications of Microservices in Cloud and Edge Systems](#)”, IEEE Micro, 2020
- Lee et al., “[The Tale of Errors in Microservices](#)”, POMACS, 2024



# Cloud-native Technologies

- A collection of tools and techniques to build, deploy, and manage applications designed for cloud computing environments
- Embrace the cloud's scalability, elasticity, and dynamic nature
- Key Principles: Microservices, Containers, DevOps, Automation
- Benefits:
  - Increased agility, faster development cycles
  - Improved scalability
  - Enhanced resilience, due to distributed architectures and redundancy
  - Greater efficiency, through optimised resource utilization
  - Increased portability

# Cloud-native Technologies – Examples

- Containers: Docker, Containerd, Podman
- Orchestration: Kubernetes, Docker Swarm
- Microservices: Spring Boot, Node.js with Express
- Serverless Computing: AWS Lambda, Google Cloud Functions, Knative
- Message Queues: Azure Queue Storage, Google Cloud Pub/Sub
- Service Mesh: Istio, Linkerd, Consul Connect
- Container Registry: Docker Hub, Amazon ECR, Google GCR, ACR
- API Gateways: Kong, Apigee, Ambassador, Gloo
- Monitoring and Logging: Prometheus, Grafana, Jaeger, Fluentd
- ...

# COMPSCI4106/5118 Cloud Systems



Intro to Cloud Computing and Virtual Resources

Virtual Machines

Containers and Container Management

Cloud System Operation and Management

Cloud System Sustainability

Cloud System Design

Modern Cloud Architectures

## ► Flavours of Cloud

A Wider Lens on Sustainability

Guest Lecture & Revision

Yehia Elkhatib

# Overview

- Provisioning Levels
  - IaaS, PaaS, SaaS, FaaS
- Deployment Models
- Cross-Cloud Computing
- Cloud-to-Edge Continuum



# Quick Recap

- Clouds come in different forms:
  - Provisioning level
  - Deployment model
- Different costs attached to each

# Provisioning Levels

- Software as a Service (SaaS)

- Consumer can access application over the network, managed by provider
- Consumer does not control/manage underlying infrastructure or run environment

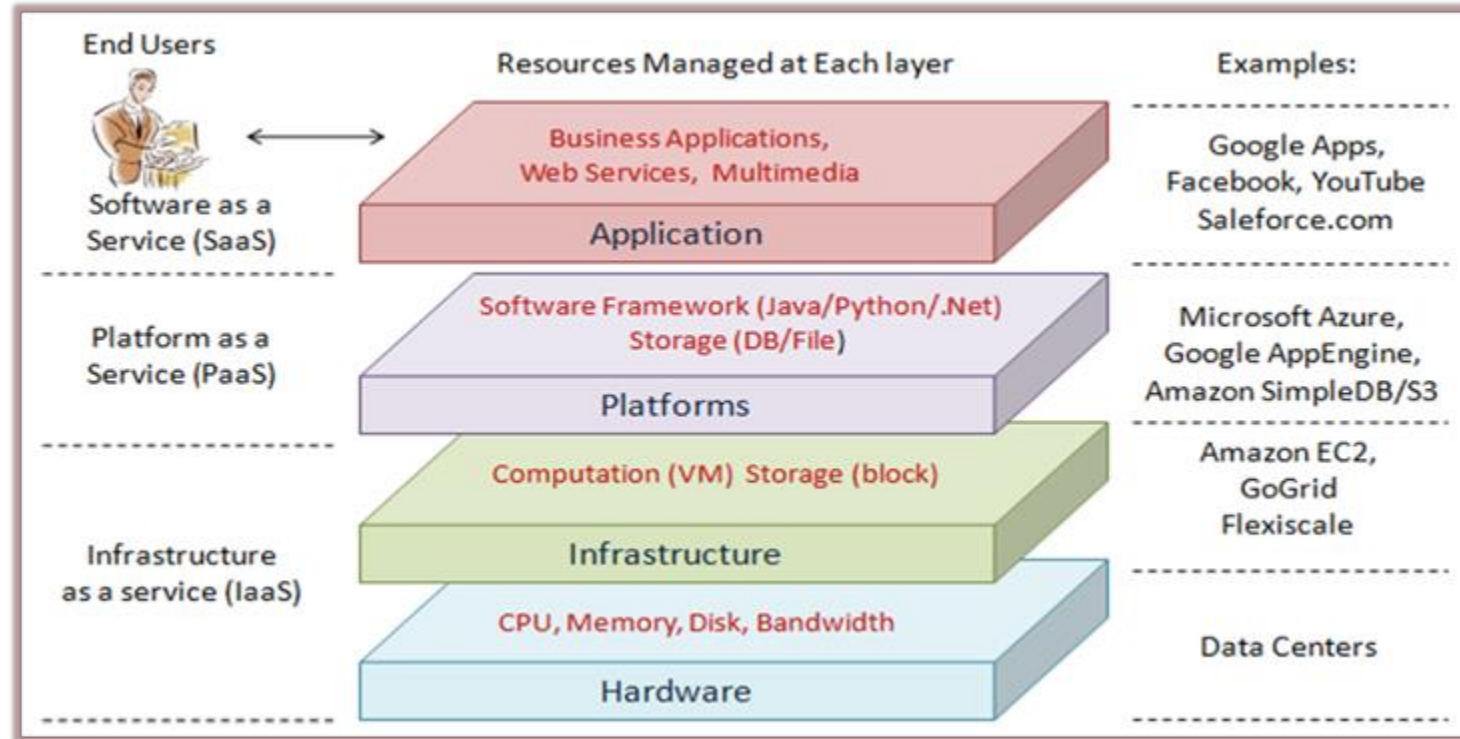
- Platform as a Service (PaaS)

- Consumer deploys custom applications onto cloud infrastructure using programming languages, libraries, services and tools supported by provider
- Consumer does not control/manage underlying infrastructure

- Infrastructure as a Service (IaaS)

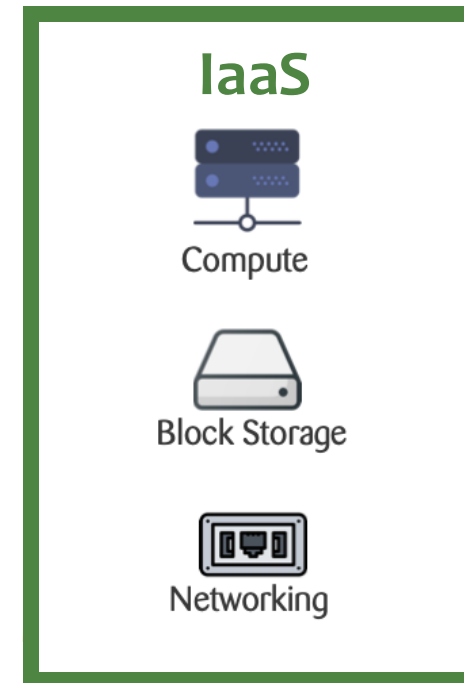
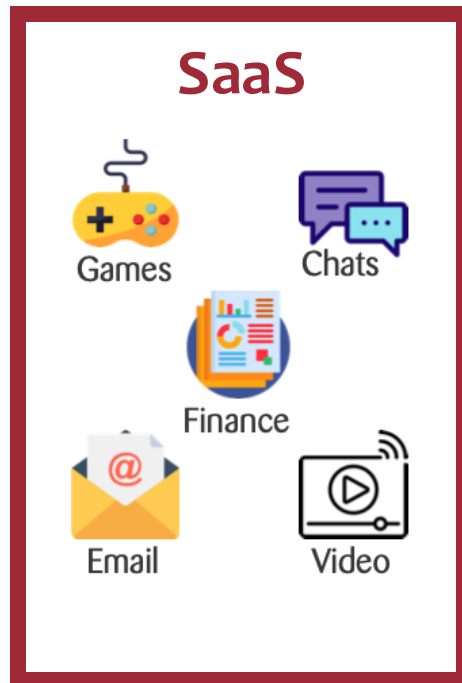
- Provider provisions processing, storage, network, and resources
- Consumer does not control/manage underlying infrastructure, but has control over OS, storage, and deployed applications

# Provisioning Levels

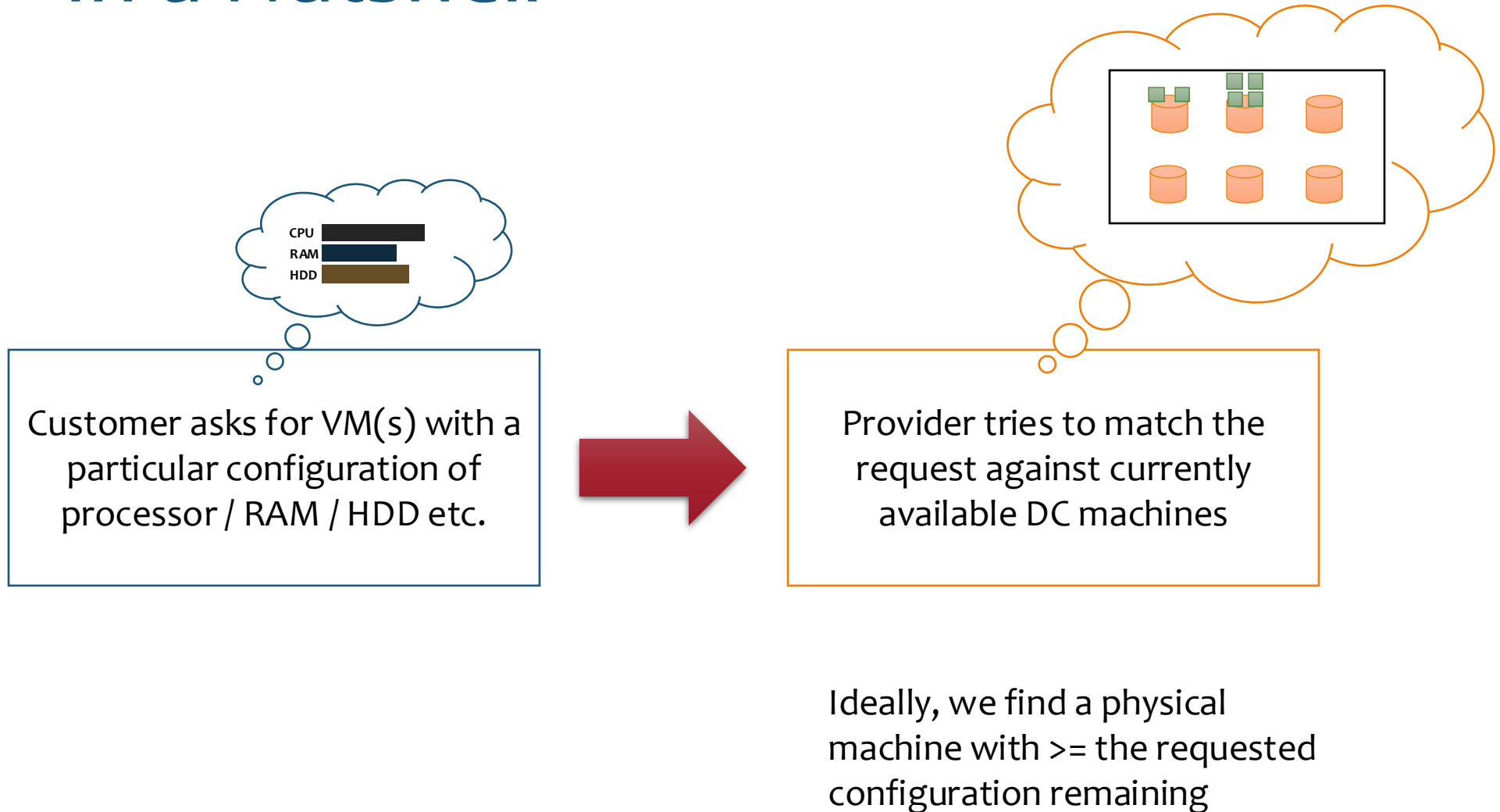


Zhang, Cheng and Boutaba. "Cloud computing: state-of-the-art and research challenges". Journal of Internet Services and Applications 1 (1). May 2010. Springer.

# Provisioning Levels

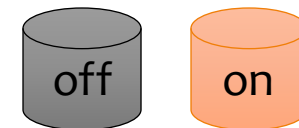
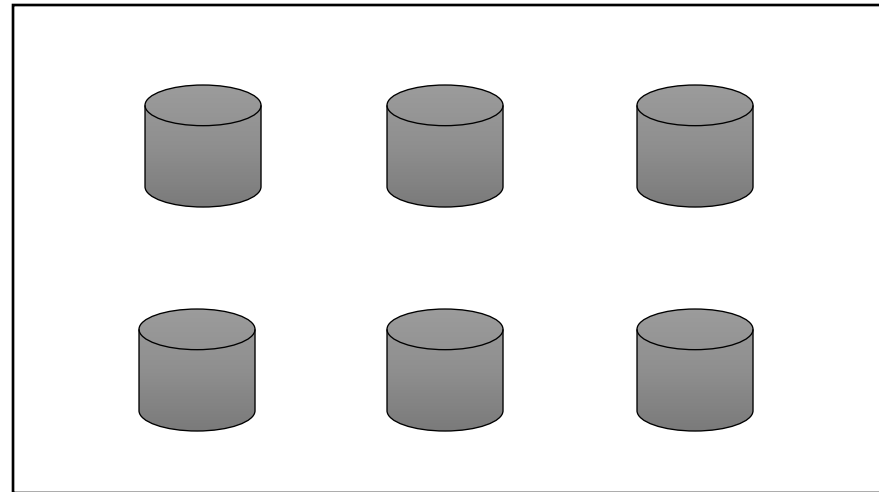


# IaaS – In a Nutshell



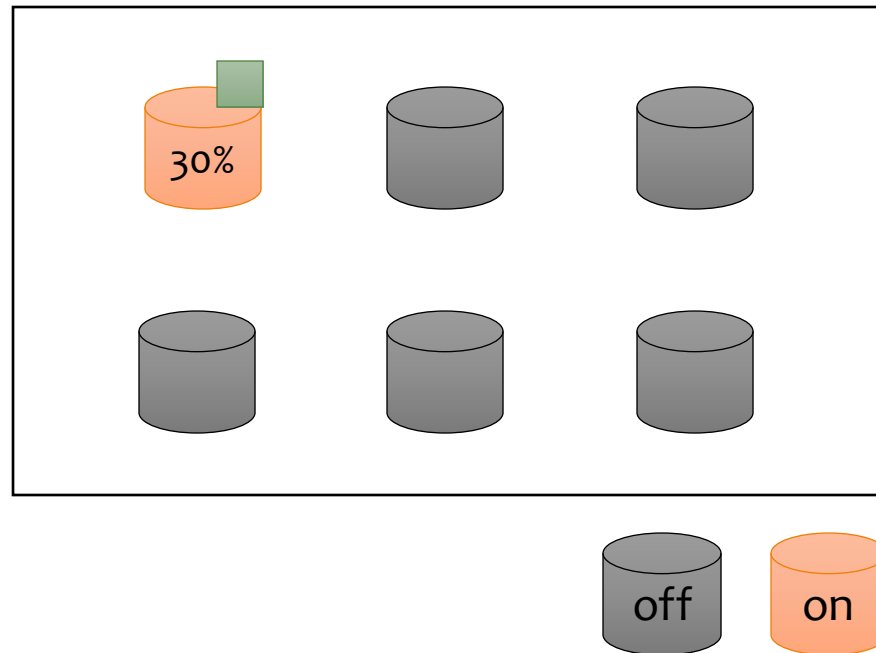
# IaaS – In More Detail

- A DC runs as few machines as possible

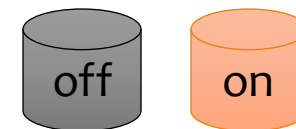
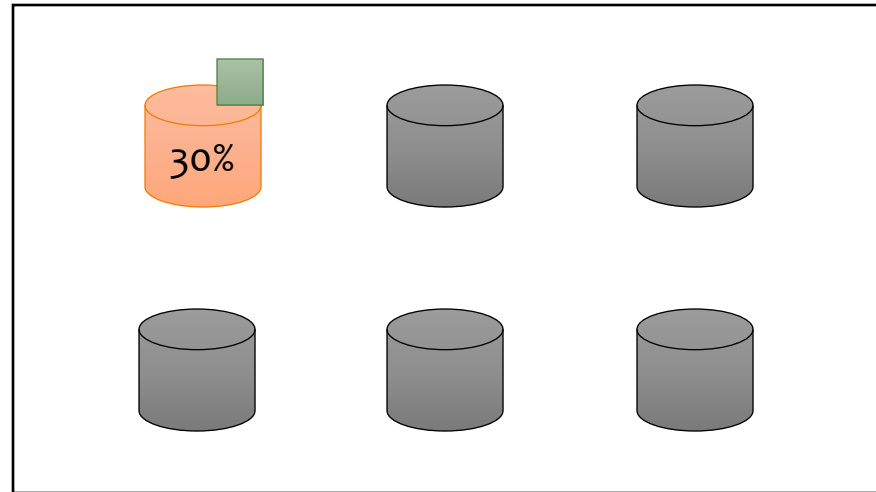


# IaaS – In More Detail

- When there is no more capacity among existing machines, a new one is started

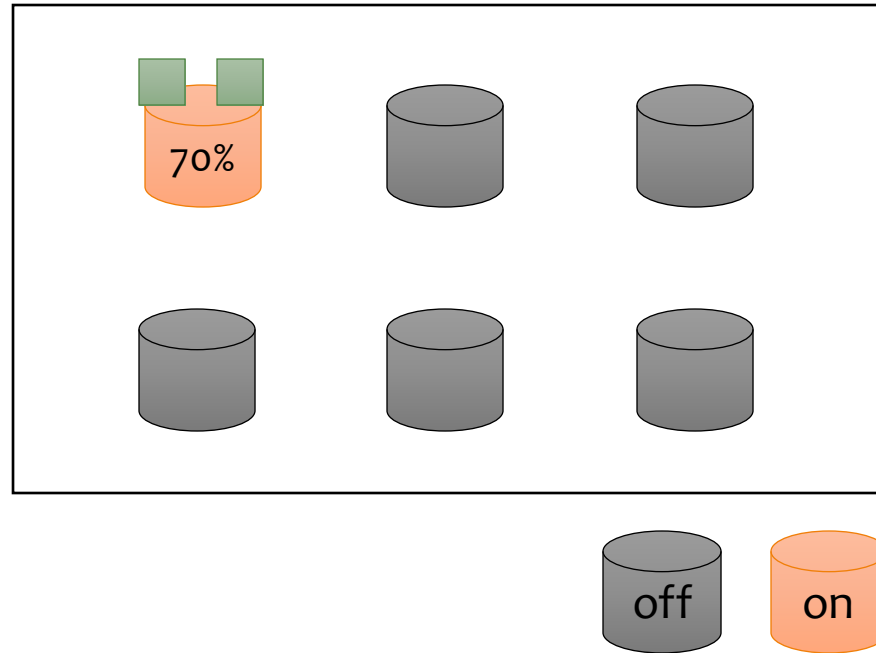


# IaaS – In More Detail

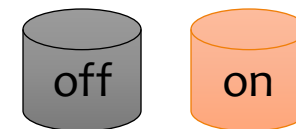
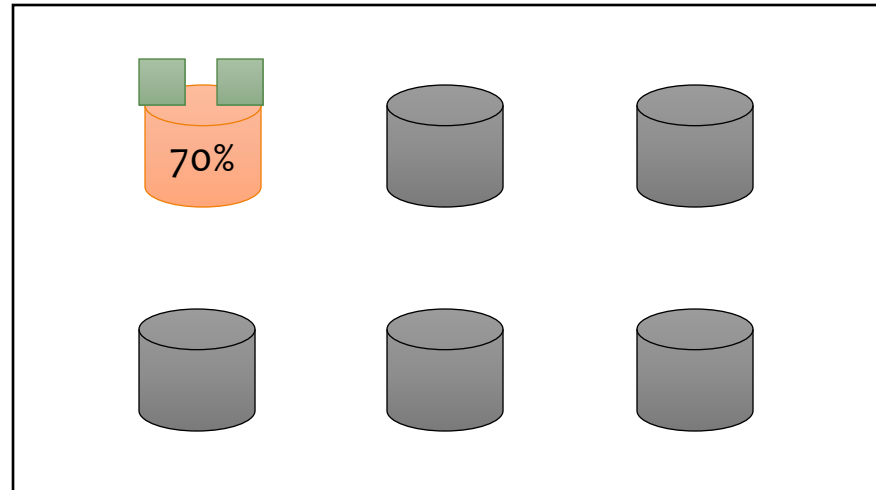




# IaaS – In More Detail

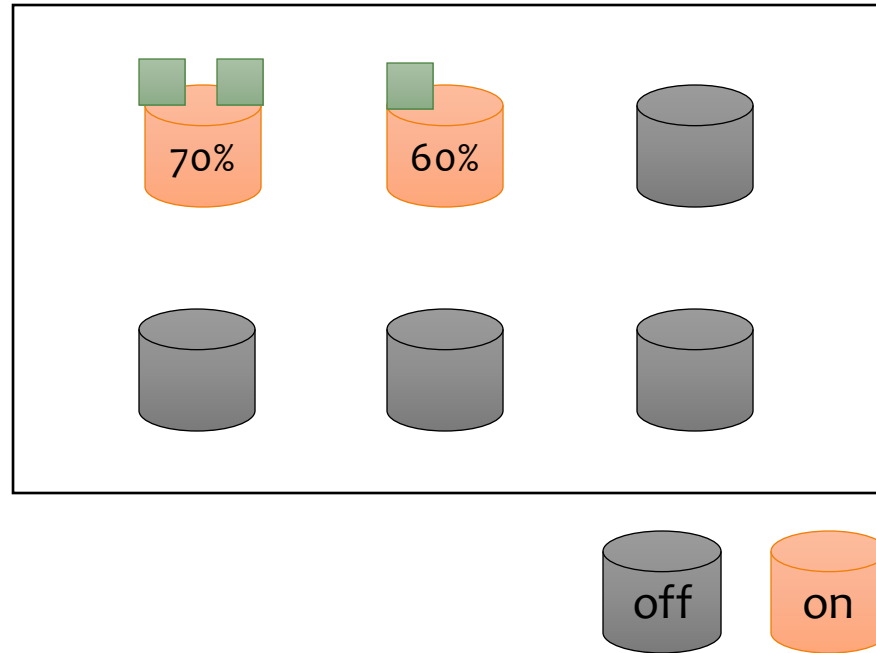


# IaaS – In More Detail



# IaaS – In More Detail

- As our servers fill up, different kinds of VM (CPU/RAM/HDD ratio) may get cheaper or more expensive to fill in the “gaps”



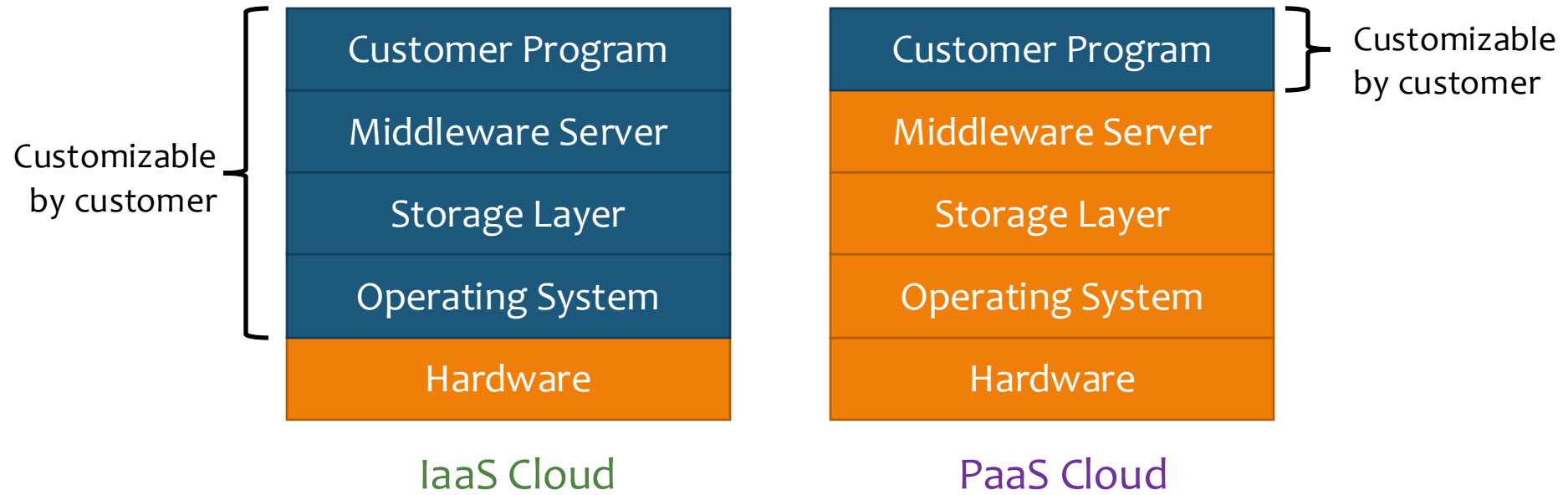
# IaaS – APIs

- On top of the resources, CSPs offer API capabilities
  - interfaces to programmatically control IaaS deployments
- Create, Start, Stop, Clone, ... etc.
- Monitor, get pricing, ... etc.
- Benefits:
  - Flexibility – control infrastructure by code, removing manual effort
  - Automation – fully automated business process logic
  - Integration – with other tools and systems (e.g. data pipeline)

# PaaS

- IaaS assumes that you
  - build all the software within VM images
  - setup the images accordingly; e.g. install libraries and runtimes
  - control how many replicas you need
  - manage said replicas yourself; e.g. scaling
  - setup networking, security groups, middleware
- PaaS hides these levels of detail

# PaaS



# PaaS

- PaaS provides a collection of APIs to program against
- Scaling and load balancing is handled automatically by the platform
- PaaS offers a higher abstraction level compared to IaaS
  - Less development / maintenance effort (e.g. OS patches)
  - Less flexibility, high provider dependence

# PaaS – Pricing

- Higher abstraction level enables different pricing models
- Service usage can be charged
  - by time
  - per query; e.g. for database services
  - per message; e.g. for message queues
  - per CPU usage; e.g. for request-triggered applications
  - ...
- Enables interesting scenarios for customers
  - Deployed applications can have low operational costs (until popular)



# PaaS – Amazon DynamoDB

- Highly-available key-value store used inside Amazon
  - powers parts of AWS, such as S3
- Primary design goals
  - High scalability (100 – 1000 servers)
  - High availability
  - High performance
  - Low latency
- Sacrifices consistency to achieve these goals

# PaaS – Amazon DynamoDB

- Adopts a peer-to-peer approach
  - No server is more important than any other
  - No single point of failure
- Nodes can be added or removed at runtime
  - guarantees **a**vailability and network **p**artition tolerance
  - weak **c**onsistency guarantees (eventual consistency)
- Designed to be a key-value store
  - No support for range queries
  - No need for range partitioning
- Hash partitioning has good load balancing properties

# PaaS – Amazon DynamoDB

Region:

Europe (London) ▼

## DynamoDB Standard table class

| On-Demand Throughput Type | Price                                    |
|---------------------------|------------------------------------------|
| Write Request Units (WRU) | \$0.7423 per million write request units |
| Read Request Units (RRU)  | \$0.1487 per million read request units  |

## DynamoDB Standard-Infrequent Access (DynamoDB Standard-IA) table class

| On-Demand Throughput Type | Price                                    |
|---------------------------|------------------------------------------|
| Write Request Units (WRU) | \$0.9279 per million write request units |
| Read Request Units (RRU)  | \$0.1856 per million read request units  |

## DynamoDB Standard table class

- First 25 GB stored per month is free using the DynamoDB Standard table class
- \$0.29715 per GB-month thereafter

## DynamoDB Standard-Infrequent Access (DynamoDB Standard-IA) table class

- \$0.11886 per GB-month

|                      |                        |
|----------------------|------------------------|
| Warm Backup Storage  | \$0.11886 per GB-month |
| Cold Backup Storage* | \$0.03566 per GB-month |

# SaaS

- Provider offers a ready-made application for you to use
  - possibly customised with a template
- Provider handles
  - writing code
  - managing updates
  - platform integration
  - automated scaling
- Customer (non-expert) just asks for “A service that does X”

# SaaS

- The business model is based on cost
- Provider offers a particular service, like email, cheaper than you could support it yourself
- Major corporations can reduce their IT overheads by outsourcing to SaaS providers
- The downside is that corporations no longer own much of the software that they use
  - they pay monthly fees

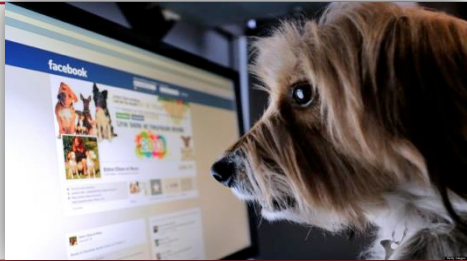
# SaaS – Salesforce

- An IT infrastructure to handle and connect:
  - product development
  - sales opportunities
  - staff roles, progress and development
  - automated analysis
- This replaces things like spreadsheets, to-do lists and email with an integrated purpose-built platform which is backed by an elastic cloud service as your company grows

# SaaS – Salesforce Pricing

|                                                    |                                                                                     | <b>Starter Suite</b><br><b>\$25</b><br><small>user/month</small> | <b>Pro Suite</b><br><b>\$100</b><br><small>user/month</small> | <b>Enterprise</b><br><b>\$165</b><br><small>user/month</small> | <b>Unlimited</b><br><b>\$330</b><br><small>user/month</small> | <b>Einstein 1 Sa</b><br><b>\$500</b><br><small>user/month</small> |
|----------------------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------|
| Account, Contact, Lead, and Opportunity Management |    | ✓                                                                | ✓                                                             | ✓                                                              | ✓                                                             | ✓                                                                 |
| Advanced Forecast & Pipeline Management            |    | —                                                                | —                                                             | ✓                                                              | ✓                                                             | ✓                                                                 |
| Sales Engagement & Conversation Intelligence*      |    | —                                                                | —                                                             | Available for purchase                                         | ✓                                                             | ✓                                                                 |
| Premier Success Plan                               |    | —                                                                | Available for purchase                                        | Available for purchase                                         | ✓                                                             | ✓                                                                 |
| Agentforce*                                        |  | —                                                                | —                                                             | ✓                                                              | ✓                                                             | ✓                                                                 |
| Unified Data                                       |  | —                                                                | —                                                             | Available for purchase                                         | Available for purchase                                        | ✓                                                                 |
| Sales Planning, Programs & Collaboration           |  | —                                                                | —                                                             | Available for purchase                                         | Available for purchase                                        | ✓                                                                 |

# Provisioning Levels – Wrap Up

|             | What is Provided                                                        | Usage                                                           | Example                                                                               |
|-------------|-------------------------------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>SaaS</b> | Turn-key Application                                                    | Application transactions:<br>run jobs, manipulate data,<br>etc. |    |
| <b>PaaS</b> | Runtime environment:<br>software packages and<br>storage support        | Develop applications                                            |    |
| <b>IaaS</b> | Barebones hardware<br>resources (CPU, memory,<br>disk, networking) + OS | Customise runtime<br>environments                               |  |



# Provisioning Levels – Which?

- How to decide?
  - Identify your skillset; real core competency
  - How much want to / can you spend (🕒/\$£) on each layer?
  - How much flexibility can I live with (or without)?
  - Other questions: e.g. legal + privacy concerns
- Applies to both individuals and organisations
- Wait, there's still more!
  - FaaS, NaaS, mBaaS, ...

# Serverless Computing

- aka Function-as-a-Service (FaaS)
- An execution model where provider dynamically manages resources
  - server management is abstracted away from the user
- Ephemeral logic (functions) are created only when needed
  - Event-driven – architecture changes in reaction to specific triggers
  - Scalable – without manual intervention
  - Pay-per-execution – billing based on consumption; “no idle resources”
  - Stateless execution – each function invocation is independent
- Evolution: IaaS -> PaaS -> FaaS

# Serverless Computing

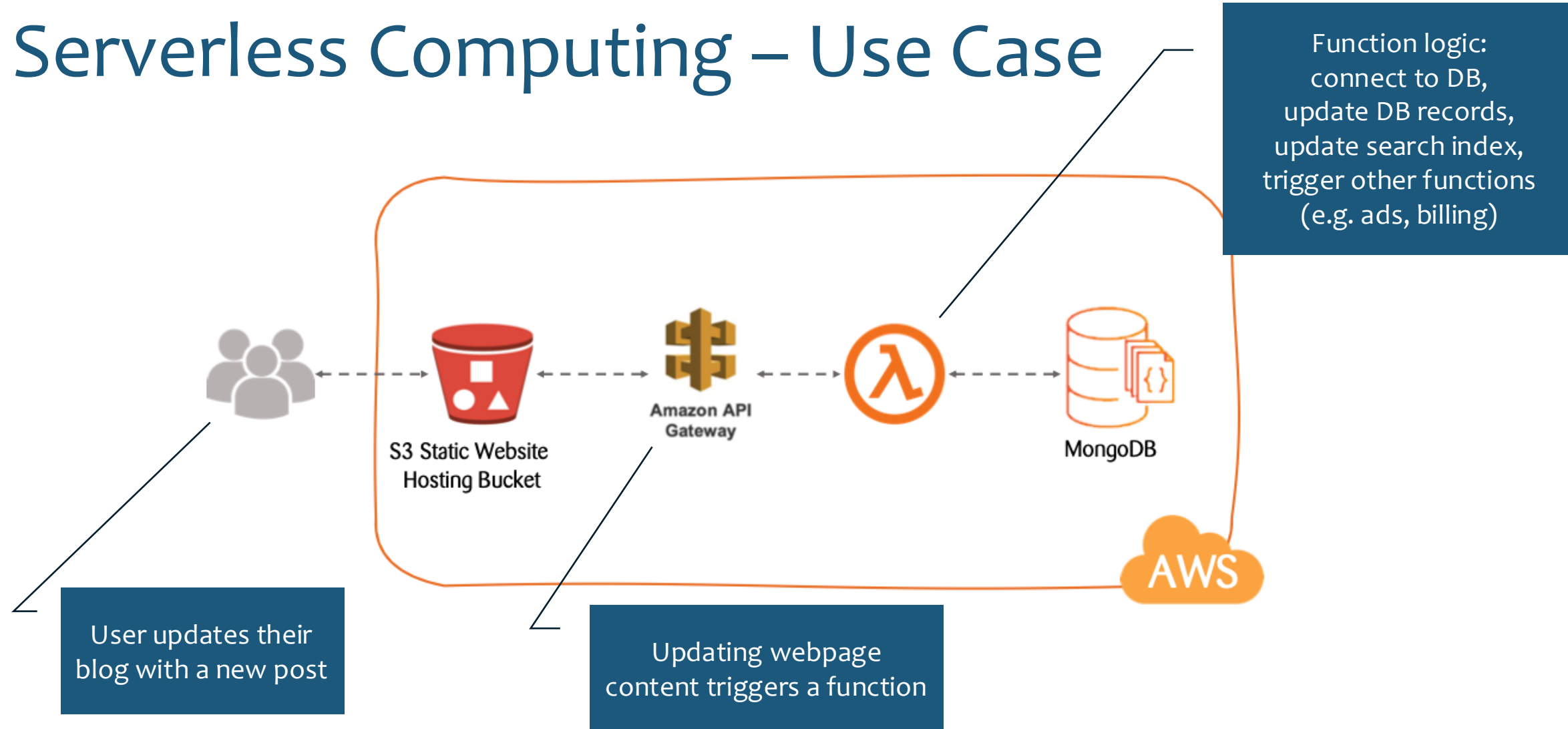
- Function execution environment:
  - Functions are deployed as standalone units of code
  - Cold starts occur when new container instances are initialised
  - Execution is time-limited; typically, 5-15 minutes maximum
  - Environment is ephemeral; no persistent local storage; => stateless
- Supporting services:
  - API Gateway – handles HTTP requests, routing to appropriate functions
  - Event Sources – message queues, storage events, scheduled triggers
  - State Management – external databases, cache services, object storage
  - Identity and Access Management – security and authentication controls

# Serverless Computing

- Examples:

- AWS Lambda
- Azure Functions
- Cloudflare Workers
- DigitalOcean Functions
- Google Cloud Run
- IBM Cloud Functions

# Serverless Computing – Use Case



# Serverless Computing – Benefits

- Lower costs: precise usage-based billing
- No servers to manage: reduced operational complexity
- Enhanced scalability: automatic resource provisioning
- Easy deployment: faster time to market, focus on business logic

# Serverless Computing – Challenges

- Higher latency: cold start impacts response times
- Vendor lock-in: use of platform-specific services and APIs
- Complex state management
- Memory & time constraints: not suitable for all operations
- Complex debugging and monitoring: only local debugging

**Me debugging my serverless  
architecture**



# Further Reading on Serverless

- Shahrad et al., “*Serverless in the Wild: Characterizing and optimizing the serverless workload at a large cloud provider*”, USENIX Annual Technical Conference (ATC), 2020.



# Low/No Code Development

- Visual environments to enable customers to create custom applications requiring minimal/zero understanding of code
  - Drag-and-drop; Pre-built templates; Auto-deployment; Built-in integrations
- Used mainly to build web / mobile apps
- Pros: (no need to wait for developers)
  - Low technical barrier
  - Easy customisation
  - Flexible control of data assets
- Cons:
  - Vendor lock-in; Platform dependencies
  - Limited customisation

# Low/No Code Development – Major CSPs

- Amazon Honeycode, Azure Power Apps, Google AppSheet
  - spreadsheet-like interface to build web/mobile apps
  - data centric; central governance of data (e.g. inventory tracking)
  - good for business applications and process automation
- Azure Logic Apps
  - create custom workflows to make transactions more efficient
- Amazon App Runner
  - manages runtime config, load balancing, network, scaling
- Google Vertex AI
  - use natural language to describe an app that is to be auto-developed

# Deployment Models

- Public cloud
  - Third party service providers offer services to the general public
- Private / Community cloud
  - Organisations (or possibly third parties) set up and maintain cloud services for their own internal use
- Hybrid cloud
  - A mix of both strategies
  - Multi-cloud, via broker, federated cloud

# Deployment Models – Public



# Deployment Models – Public

| Provider     | SaaS                        | PaaS                                 | IaaS                    |
|--------------|-----------------------------|--------------------------------------|-------------------------|
| Google       | Google Docs, Cloud Dataflow | App Engine, Firebase                 | Compute Engine          |
| Microsoft    | Office 365, Outlook.com     | Azure Functions, CosmosDB            | Azure IaaS, Azure Batch |
| Amazon       | KMS, Amplify, QuickSight    | S3, Aurora, EMR, DynamoDB, Beanstalk | EC2, Spot, ELB          |
| Oracle       |                             | GoldenGate                           | OCI                     |
| SalesForce   | SalesCloud                  | VMforce                              | SalesForce              |
| Yahoo        | YQL                         | Hadoop clusters                      |                         |
| IBM          | LotusLive                   | WebSphere CA                         | CloudBurst              |
| RackSpace    |                             | Mosso                                |                         |
| Joyent       |                             | Accelerator                          | SmartDataCenter         |
| DigitalOcean |                             |                                      | DigitalOcean            |





# Deployment Models – Private

- Outsourced private clouds:
  - On-site clouds managed by 3<sup>rd</sup> parties, e.g. IBM, HP, Oracle
- Open-source virtual infrastructure managers:
  - *OpenStack*: A collaboration project between RackSpace, NASA and several commercial companies
  - *OpenNebula*: Advanced virtual environment administration
  - *Eucalyptus*: API compatible with Amazon EC2 & S3
  - *CloudStack Community* : By Cloud.com
  - *Nimbus*: Based on Globus Toolkit 4



# Deployment Models – Comparison

|                                       | 1. Private Cloud                               | 2. Community Cloud                              | 3. Public Cloud                                                  | 4. Hybrid Cloud                                                                                                                                                                                                                 |
|---------------------------------------|------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User of the cloud infrastructure?     | Single organisation                            | Organisations with shared concerns              | Open for the general public                                      | Composition of the three basic cloud models: <ul style="list-style-type: none"> <li>• Remain distinct entities</li> <li>• Bound together by standard mechanisms</li> <li>• Goal: Enable data/application portability</li> </ul> |
| Owner of the cloud infrastructure?    | Organisation, third party, combination thereof | Organisations, third party, combination thereof | Business, academic, government organisation, combination thereof |                                                                                                                                                                                                                                 |
| Location of the cloud infrastructure? | On premise, off premise                        | On premise, off premise                         | On premise of cloud provider                                     |                                                                                                                                                                                                                                 |

# Deployment Models – Which?

|                            | Upfront Cost | Time to Build | Security Risk |
|----------------------------|--------------|---------------|---------------|
| Public                     | Low          | Low           | High          |
| Private Outsourced         | High         | Medium        | Low           |
| Private Internally Managed | High         | High          | Medium        |



# Cross-Cloud Computing – What?

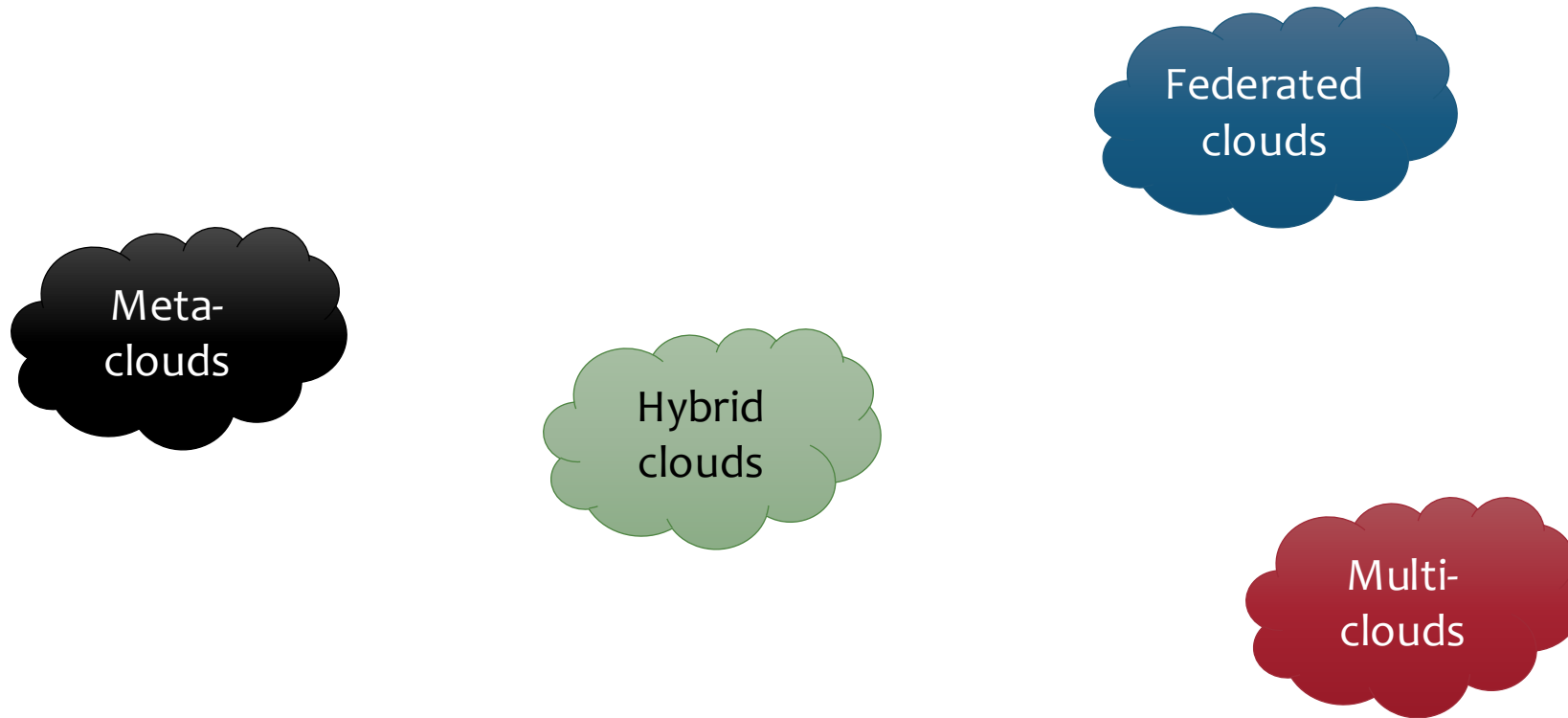
- Operate seamlessly across multiple cloud environments



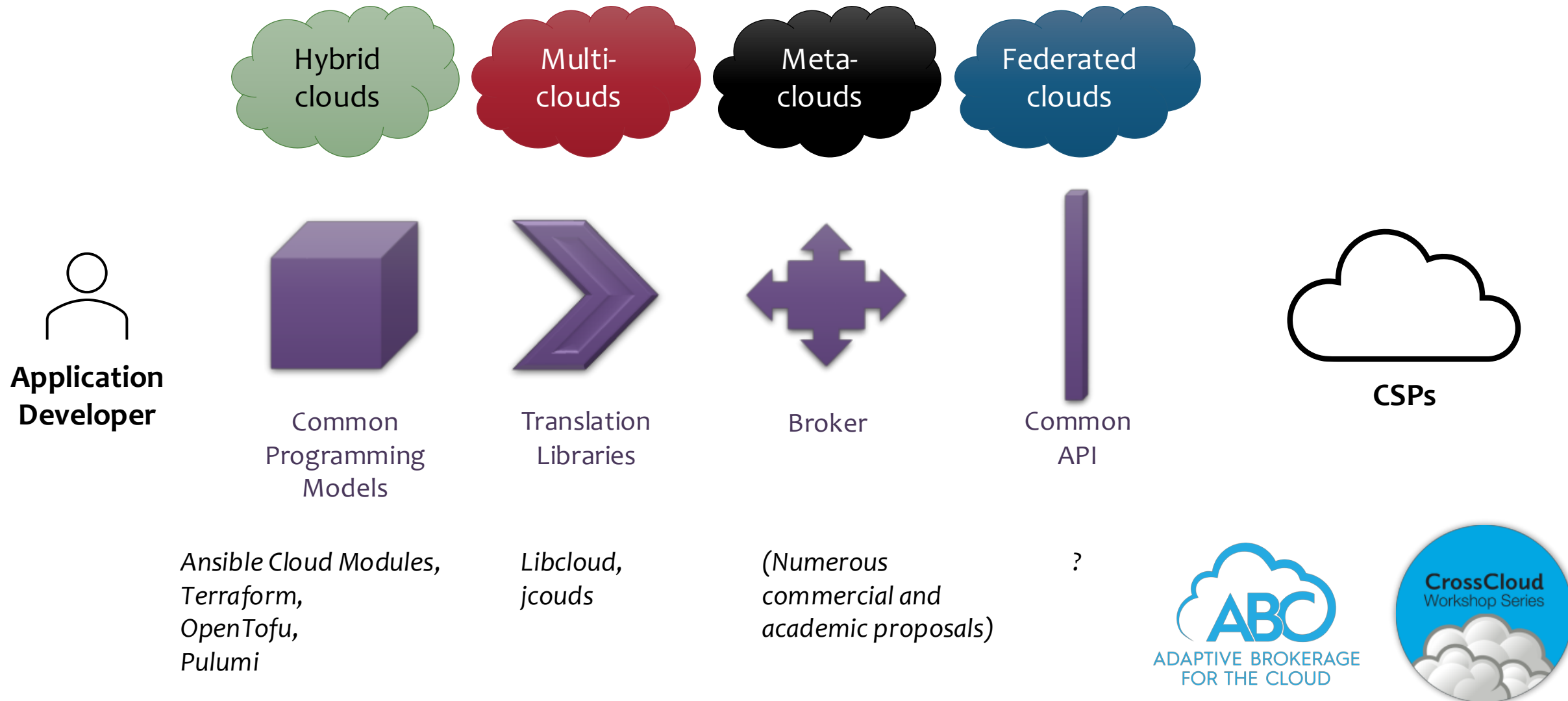
# Cross-Cloud Computing – Why?

- Avoiding vendor lock-in
  - For portability, independence, agility
- Resilience against vendor issues (a higher level of fault tolerance)
  - Outages; QoS fluctuations
- Harness diversity
  - Service provisions
  - Geographical presence
- Compliance with policies
  - e.g. physical location of data replicas

# Cross-Cloud Computing – How?



# Cross-Cloud Computing – How?



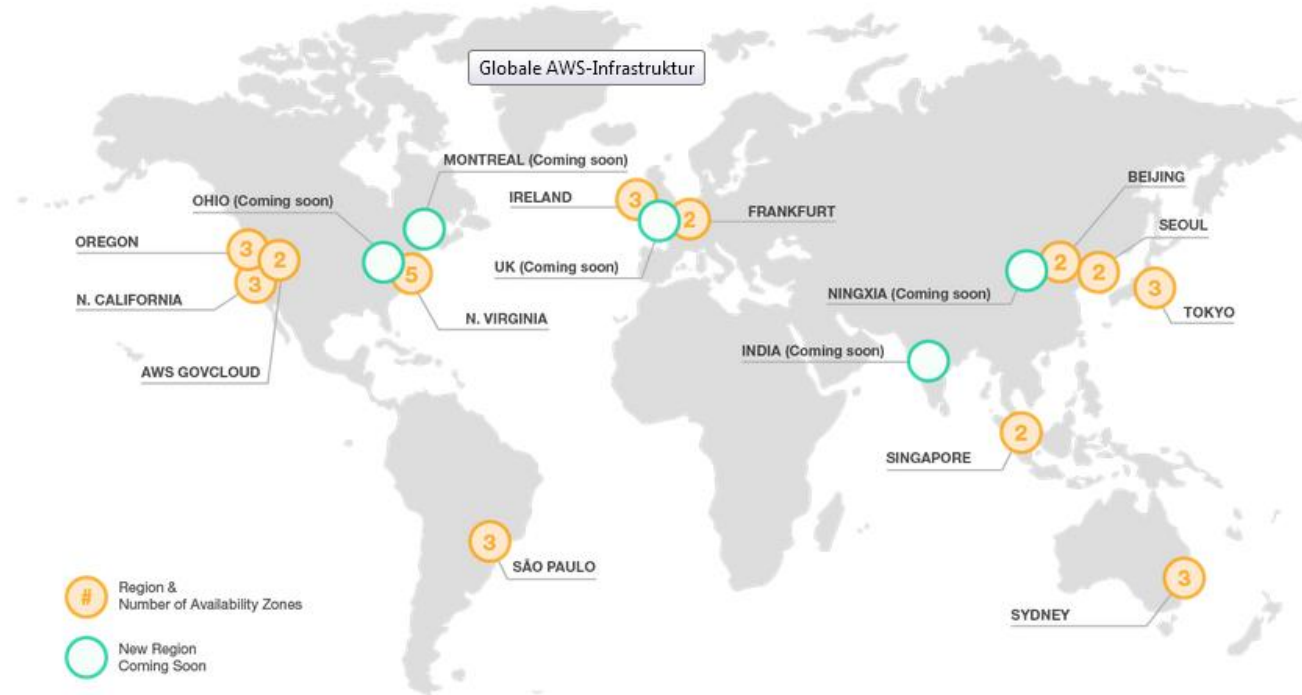
# Cross-Cloud Computing – Trade-offs

- ... all this spawns a whole host of issues:
  - Access control, identity management, performance evaluation, etc.
- Each model comes with nuances of its own
  - Hybrid: hardwiring still involved to some degree
  - Multi-clouds: granularity lost through abstraction
  - Meta-clouds: control lost through delegation
  - Federations: standard interfaces must be established

# Further Reading on Cross-Cloud Computing

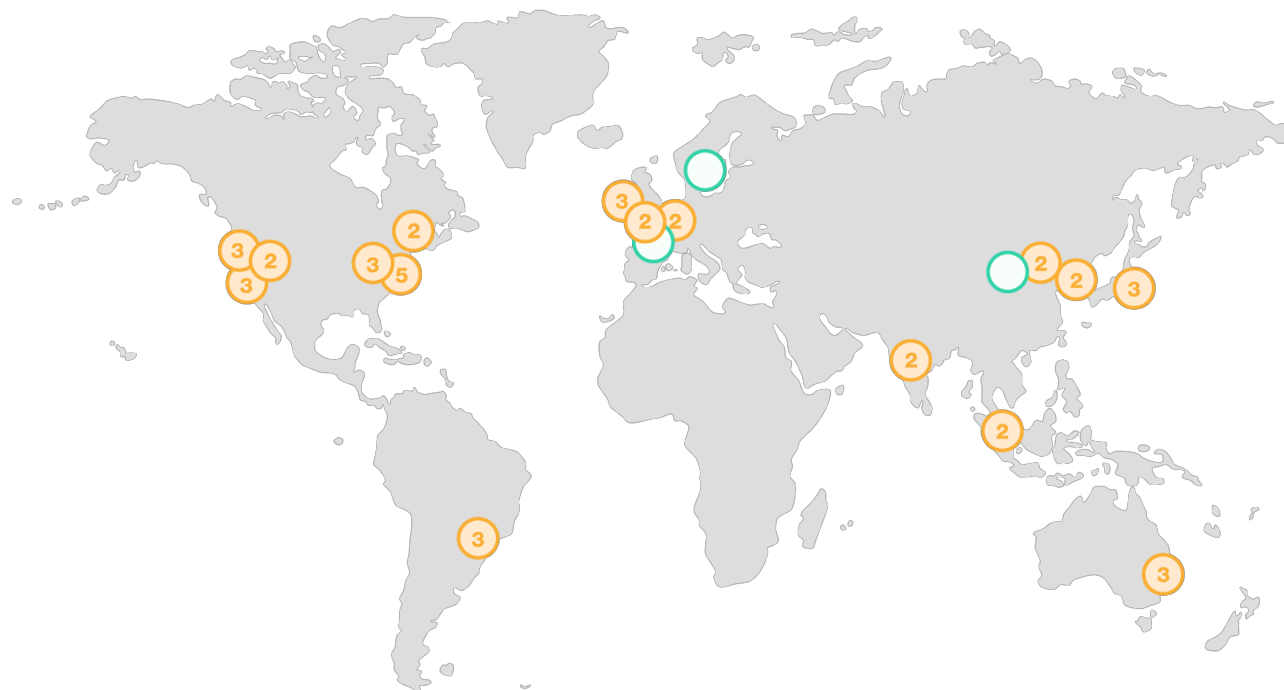
- Elkhatib, “Mapping Cross-Cloud Systems: Challenges and Opportunities”, HotCloud, 2016.
- Elhabbash, Samreen, Hadley, and Elkhatib, “Cloud Brokerage: A Systematic Survey”, Computing Surveys, ACM, 51(6), 2019.
- Elhabbash, Jumagaliyev, Blair, and Elkhatib, “SLO-ML: A Language for Service Level Objective Modelling in Multi-cloud applications”, UCC, 2019.

# AWS Global Infrastructure 2016



- 33 availability zones
- 12 geographic regions

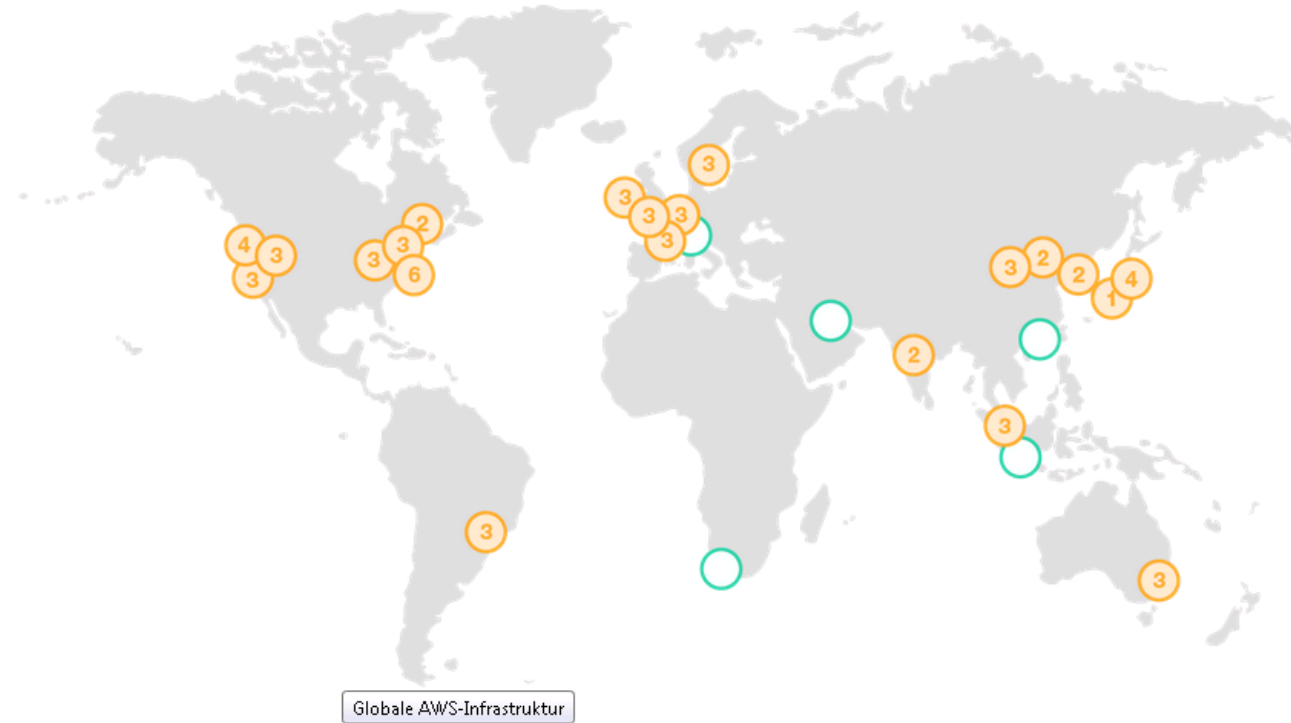
# AWS Global Infrastructure 2017



- 42 availability zones
- 16 geographic regions



# AWS Global Infrastructure 2019

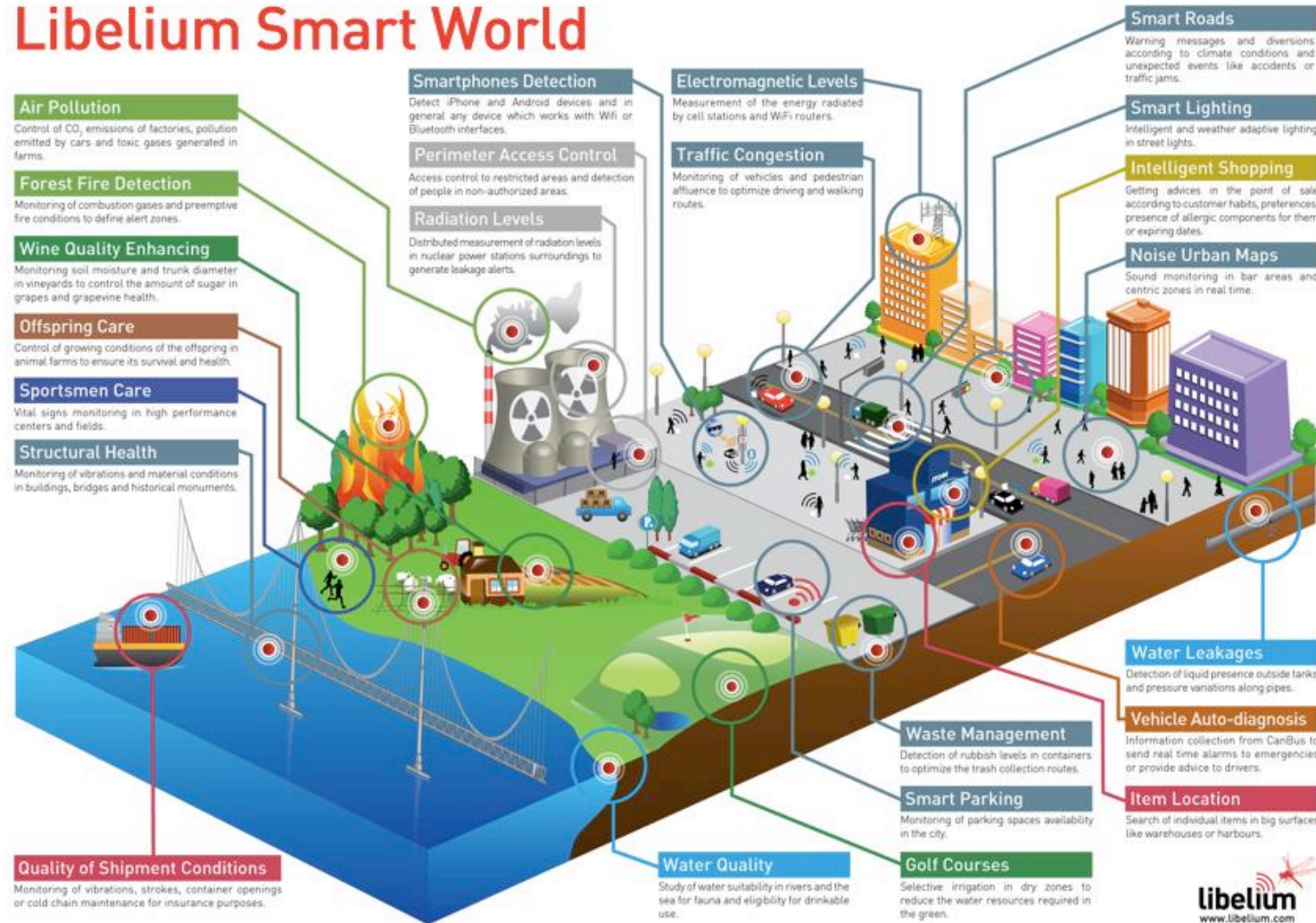


- 61 availability zones
- 20 geographic regions

All good?

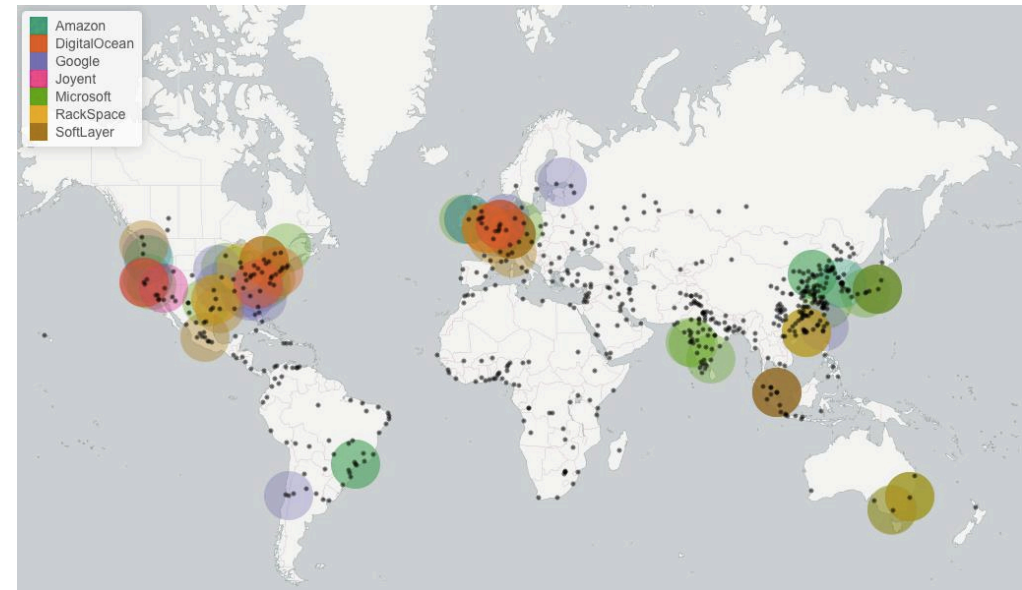
# Computing Continuum – Motivation

## Libelium Smart World



# Computing Continuum – Motivation

- World population : connected devices = 1:4
  - In 2023 – 5.6 billion IoT devices generating >508 zettabytes of data
  - Using 20TB hard drives, we would need about 25.4 billion drives
- Cloud coverage is wide, but not wide enough
- Cloud cannot cater to:
  - Mobility support
  - Centralised model
  - High latency



If you want to update this, get in touch with me

# Computing Continuum – Support Roles

- Lighter execution units to closer to end users
  - Aggregation
  - Pre-processing
  - Fault mitigation
  - Coordination & automation
  - Deployment migration
  - Bridging between boundaries
    - e.g. translation, synchronisation
  - Network middleboxes
    - e.g. caching, traffic scrubbing, traffic shaping

# Computing Continuum – What?

- Extend the cloud paradigm to the edge of the network
  - Compute and data resources on demand
  - Controllable over the network using APIs
- Enhance delivery of current applications and services
  - Location and context awareness; very low latency; ...
- Requires redesigning of applications
  - A hot area of research and business

# Computing Continuum – Fog vs Edge

## ○ Edge

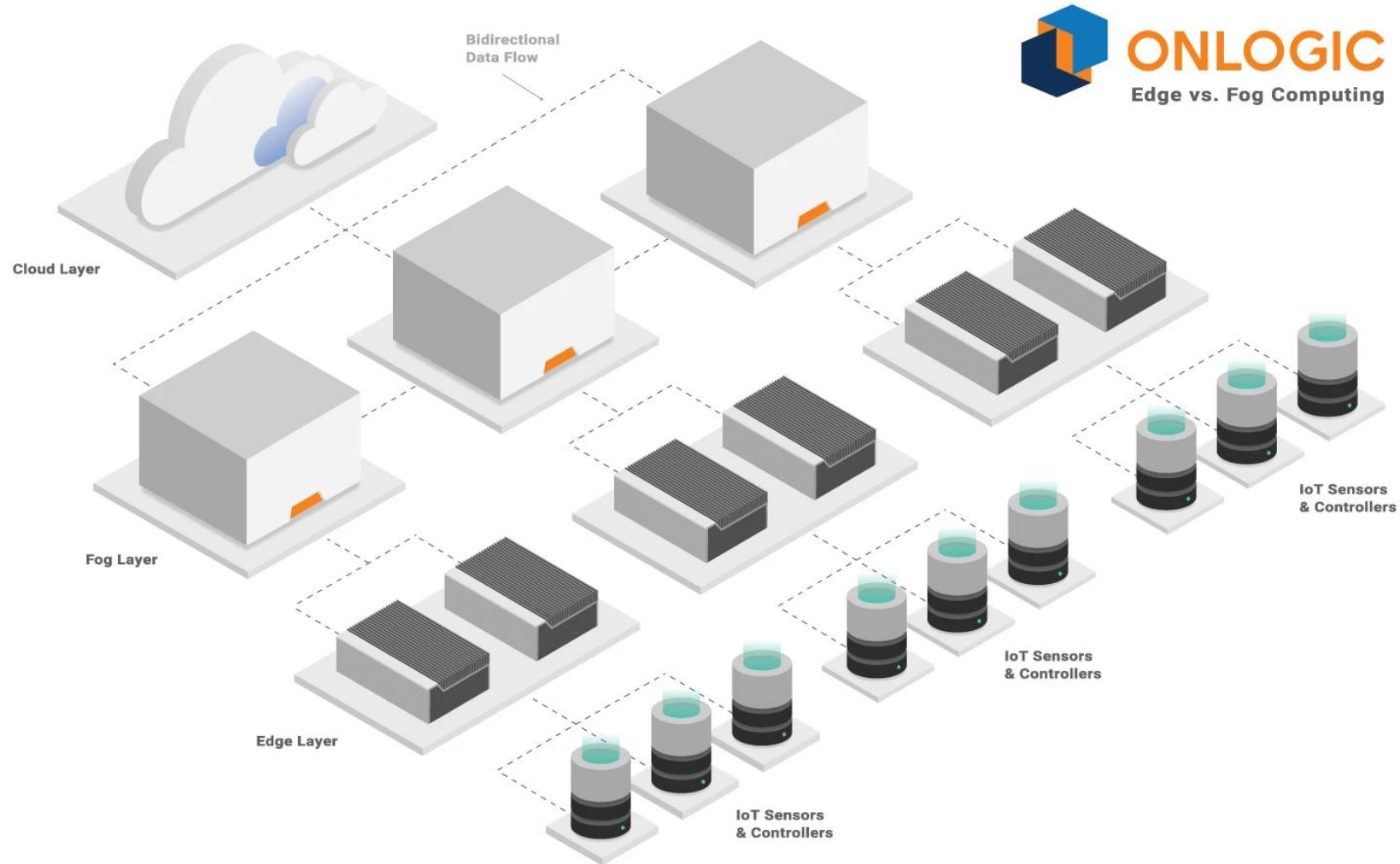
- Focus on processing closest to data source
- Ideal for requirements of ultra-low latency (<10ms), low backhaul bandwidth, privacy sensitivity
- Has limitations in terms of hardware constraints (e.g. storage capacity)
- Can be managed by end user or application provider

## ○ Fog

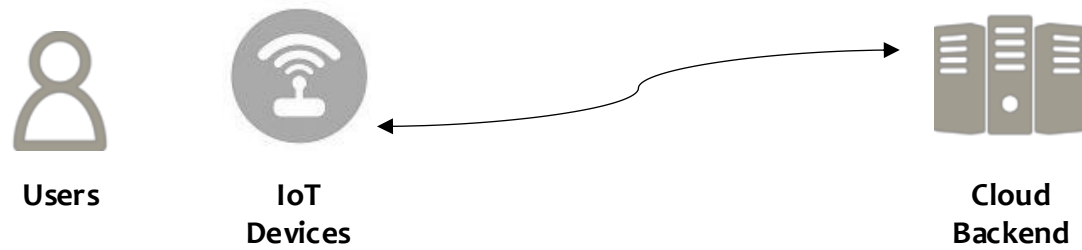
- Extend resources between central DCs and edge devices
- Broader coverage geographically and topologically
- Ideal for greater computational capabilities, but not at cloud scale
- Typically managed by service providers or network operators



# Computing Continuum – Fog vs Edge

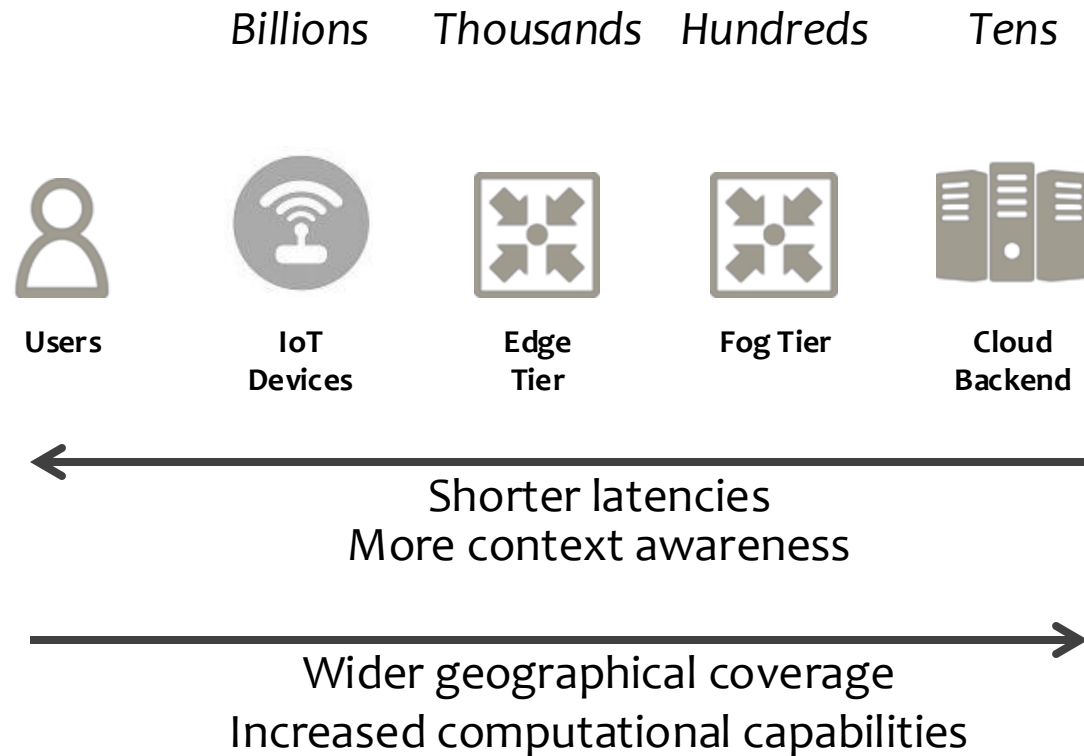


# Computing Continuum – Where?



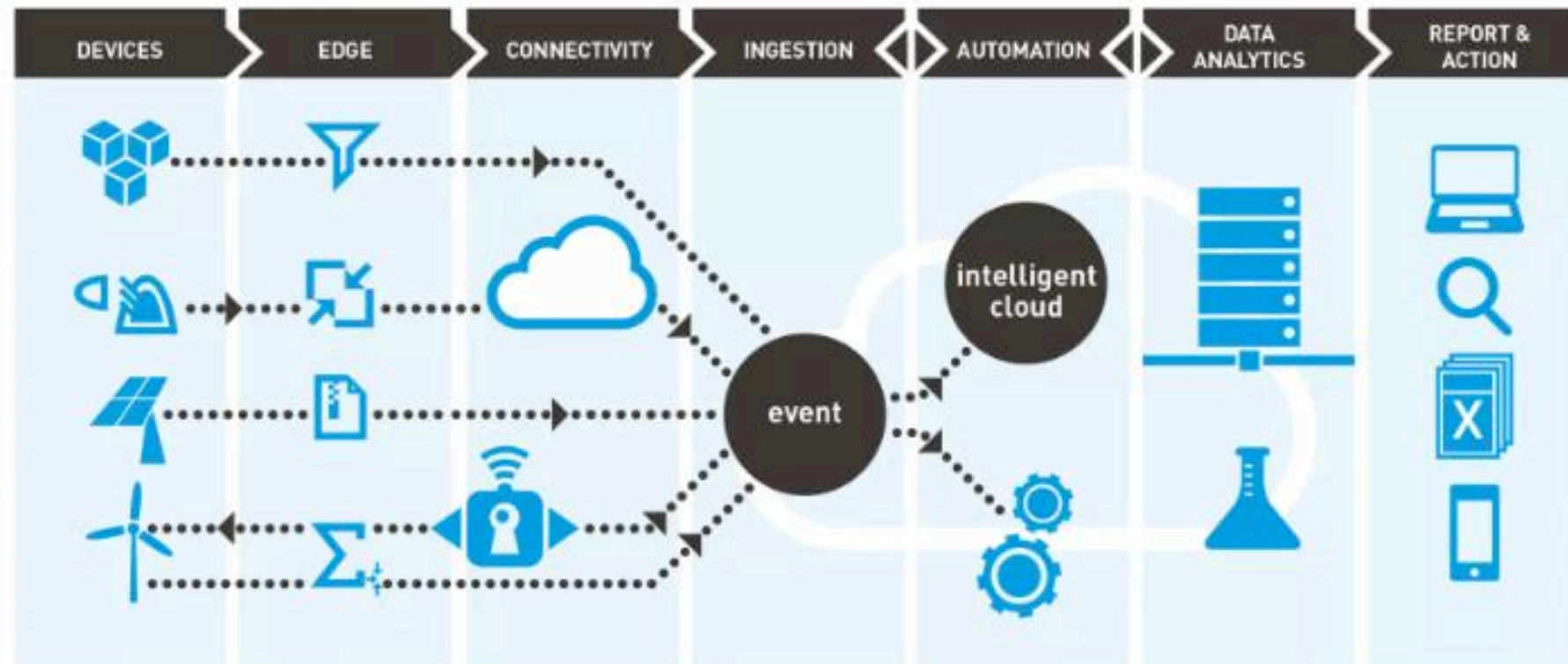


# Computing Continuum – High-level View



# Computing Continuum – High-level View

## IoT End-to-End Value Chain



# Fog / Edge – Where to offer resources?

- Large CSPs
  - AWS (Greengrass), Azure (IoT Edge), Cisco Kinetic
- CDN operators
  - Cloudflare, Akamai, Fastly
- Telecomm operators
  - BT, Vodafone, Verizon, AT&T

# Fog / Edge – Where to offer resources?

- Yourself!
- SBCs are affordable, small, modular
  - Raspberry Pi, Pine64, ODROID-C2, Arduino Uno, BeagleBoard, ...
- Relatively easy to assemble a **micro-cloud** from SBCs



**Iridis-Pi**  
\$4,800



**Pi Beowulf**  
\$2,000



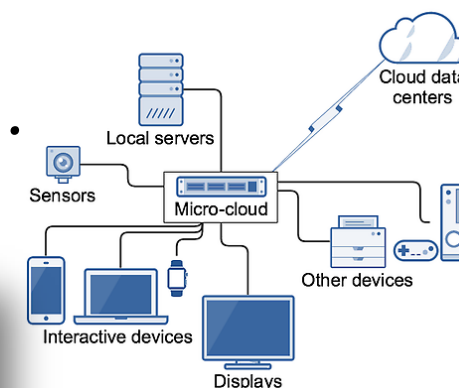
**BitScope**  
\$750

require dedicated infrastructure  
require high expertise to assemble  
not suitable where power is intermittent / scarce



## Grape Cluster

1 leader, 6 worker rPis  
Bespoke PCB (connections, sensors)  
£350  
Open sourced, with Python tools  
<http://se.labri.fr/grape>



# Further Reading on Fog/Edge Computing

- Elkhatib, Porter, Ribeiro, Zhani, Qadir, and Rivière, “[On Using Micro-Clouds to Deliver the Fog](#)”, IEEE Internet Computing, 2017.
- Yu et al. “[A Survey on the Edge Computing for the Internet of Things](#)”, IEEE Access, 2017
- ScaleComputing, “[Exploring Computing Models: Edge Computing vs Fog Computing vs Cloud Computing](#)”

# COMPSCI4106/5118 Cloud Systems



Intro to Cloud Computing and Virtual Resources

Virtual Machines

Containers and Container Management

Cloud System Operation and Management

Cloud System Sustainability

Cloud System Design

Modern Cloud Architectures

Flavours of Cloud

## ► **A Wider Lens on Sustainability**

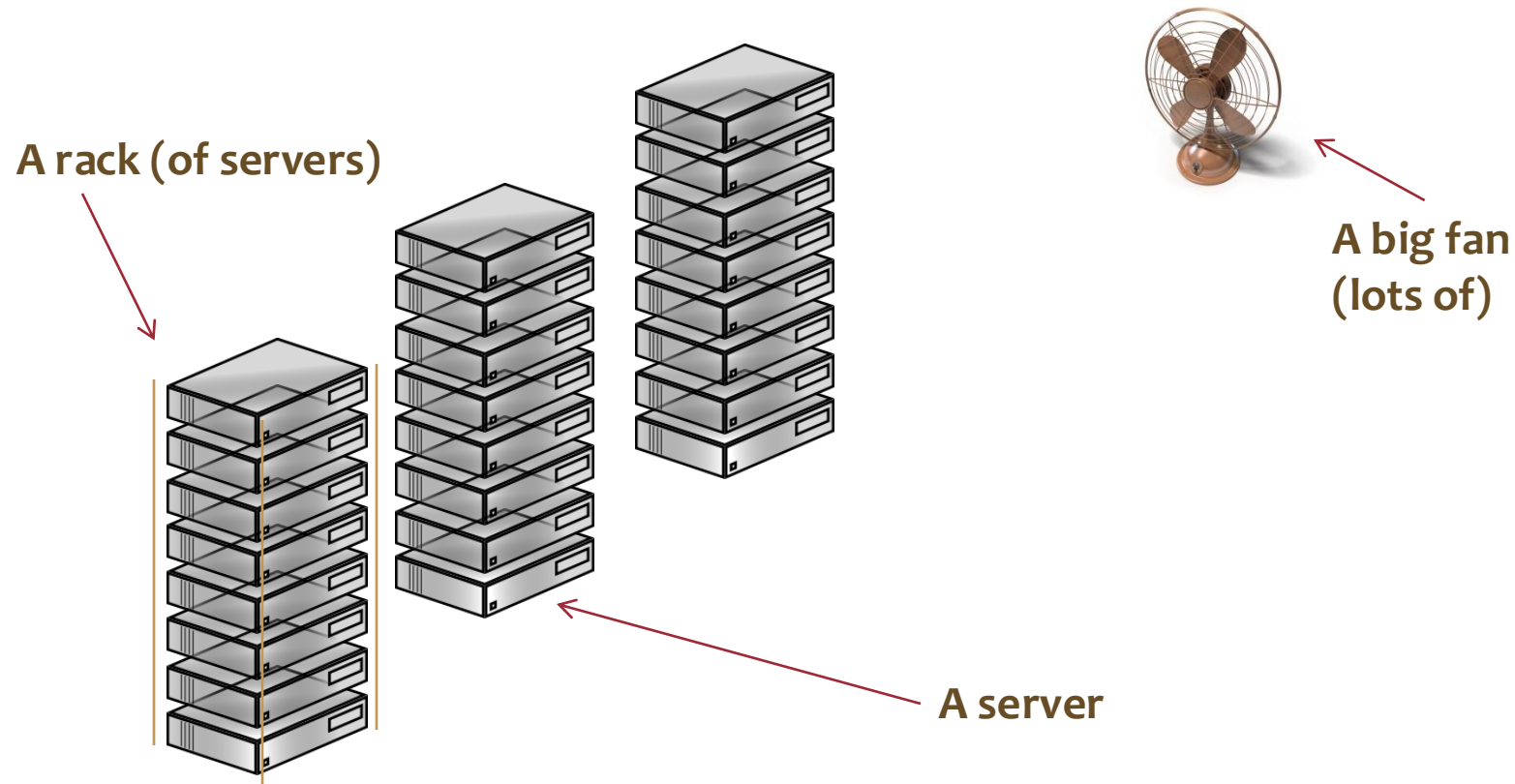
Guest Lecture & Revision

Yehia Elkhatib

# Overview

- Designing Dependable Data Centres
- Carbon Footprint Measurement Frameworks
- Measurement Granularities
- Carbon Intensity
- Carbon-Aware Decision Making

# Data Centre Infrastructure Basics





# Hardware Redundancy

- Geographic redundancy
  - DCs are distributed across multiple geographic regions
  - Mitigate the impact of regional outages, e.g. natural disasters, power grid failures
  - Data is typically replicated across regions to ensuring high availability
- Server redundancy
  - Servers are deployed in clusters with automatic failover mechanisms
  - If one server fails, another takes over seamlessly (managed by virtualisation tech.)
- Storage redundancy
  - Data is replicated across multiple storage devices and technologies
  - Within the same DC and across other DCs
  - RAID configurations to protect against data loss due to disk failures

# Network Redundancy

- Challenge

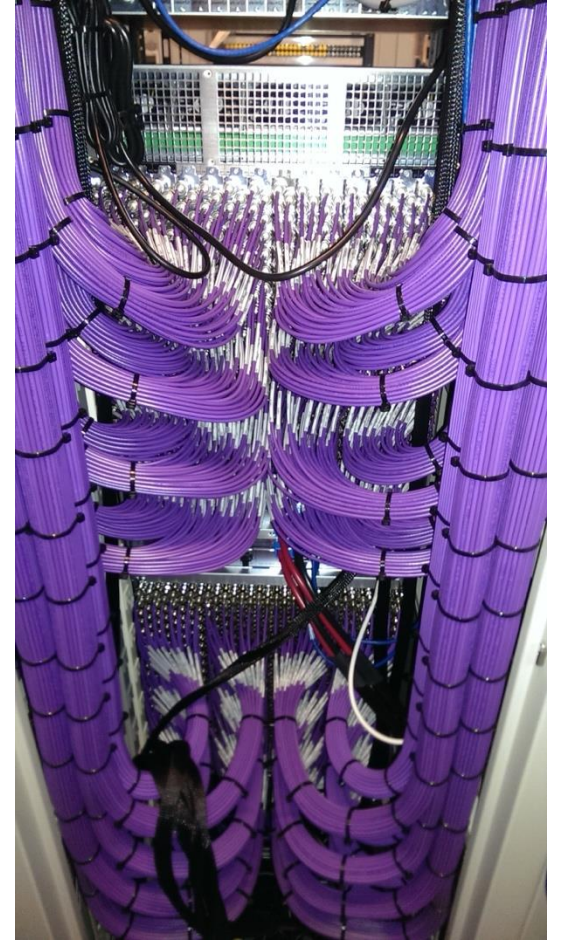
- Reliably connect 100,000s of computers
- Strict requirements of high bandwidth and low latency
- Expect hardware-level failure at any point

- 1. Server-level

- Redundant Network Interface Cards (NICs)
- Dual (or more) power supplies

- 2. Network-level

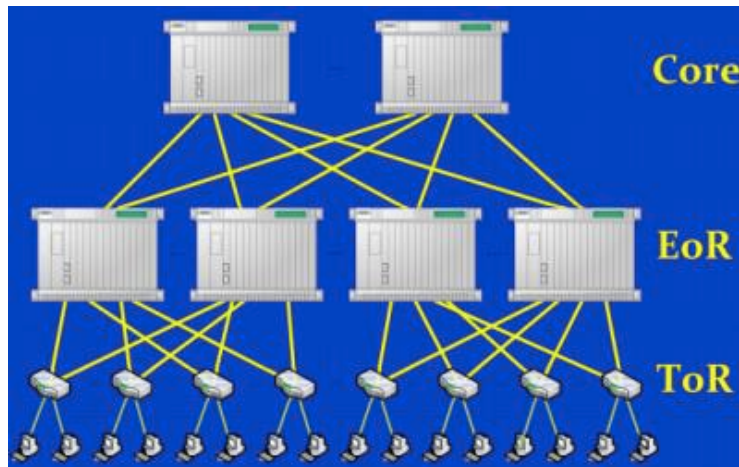
- Redundant switches, routers, firewalls, load balancers



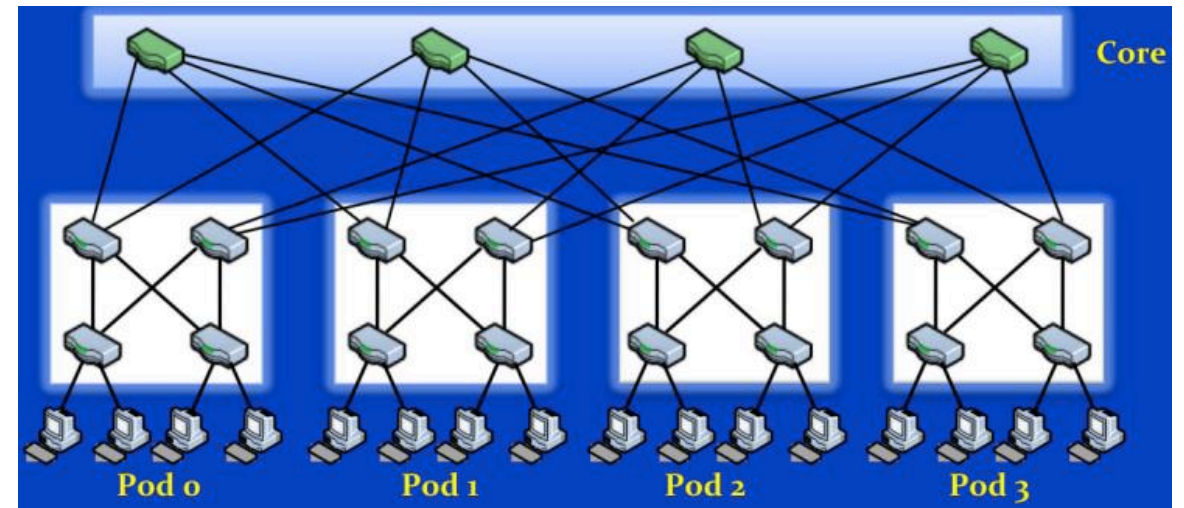
# Network Redundancy

## 3. Link- and path-level

- Link aggregation: multiple links between network devices
- Spanning Tree Protocol (STP) variants to prevent network loops
- Distribute traffic across multiple paths, i.e. load balancing traffic



Hierarchical / 3-tier topology



Fat-tree / clos topology

# Power Redundancy

- DCs require a constant and reliable supply of power
- Power redundancy measures
  - Multiple power feeds from different utility substations
  - Uninterruptible Power Supplies (UPS) as backup for temporary outages
  - Backup diesel / gas generators for medium/long-term outages
  - Power Distribution Units (PDUs) with dual power inputs to ensure continuous rack power

# Cooling Redundancy

- DCs generate a significant amount of heat
- Heating, Ventilation, and Air Conditioning (HVAC) systems
  - to control temperature, humidity, air quality
- Cooling redundancy measures
  - N+1 – having one extra cooling unit than required to handle the load
  - Multiple cooling technologies to mitigate single points of failure
    - e.g., CRAC units, free cooling, in-row cooling
  - Redundant cooling loops – pipes, heat exchangers, pumps
  - Hot/Cold aisle containment – prevent hot and cold air from mixing

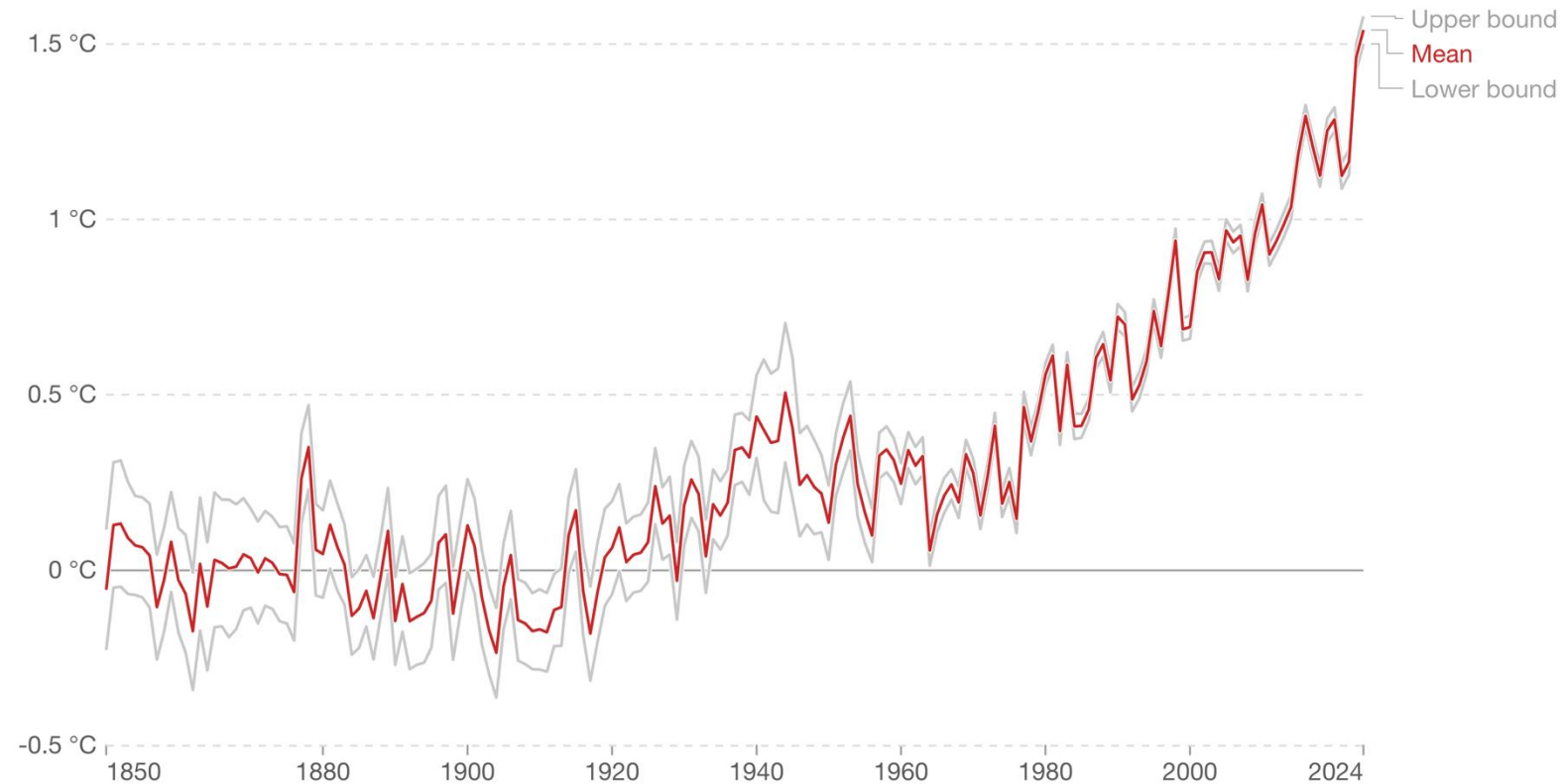
# Further Reading on Data Centre Design

- DgtlInfra, “[Data Center Racks, Cabinets, and Cages: An In-Depth Guide](#)”
- Hatzenbuehler, “[What is Data Center Redundancy? N, N+1, 2N, 2N+1](#)”
- Lovett, “[A Deep Dive into Data Center Redundancy](#)”
- Reboot Monkey, “[Step-by-Step Guide to Designing Your Data Center Layout](#)”

# Sustainability – Why?

## Annual temperature anomalies relative to the pre-industrial period, Global Our World in Data

Average land-sea temperature anomaly relative to the 1861–1890 average temperature baseline.



**Data source:** Met Office Hadley Centre (2025)

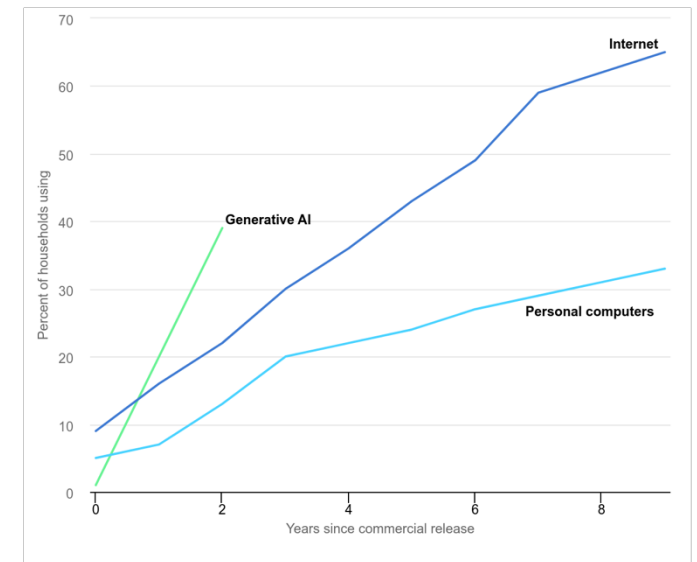
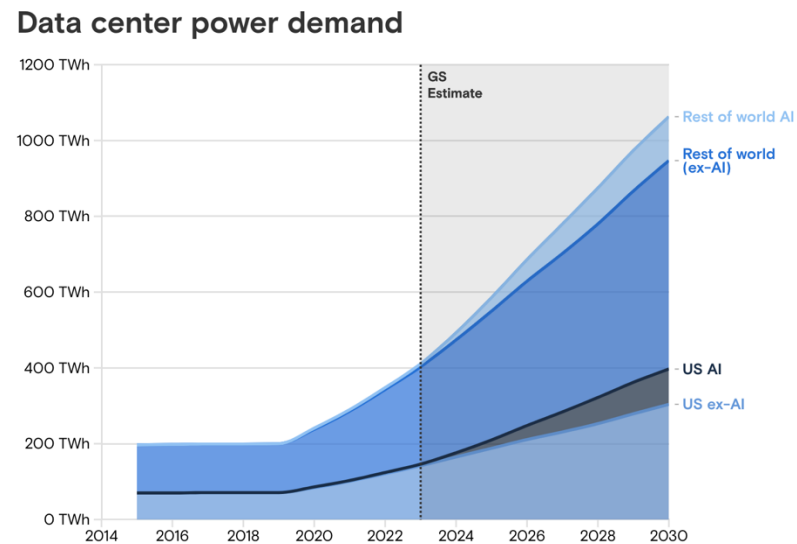
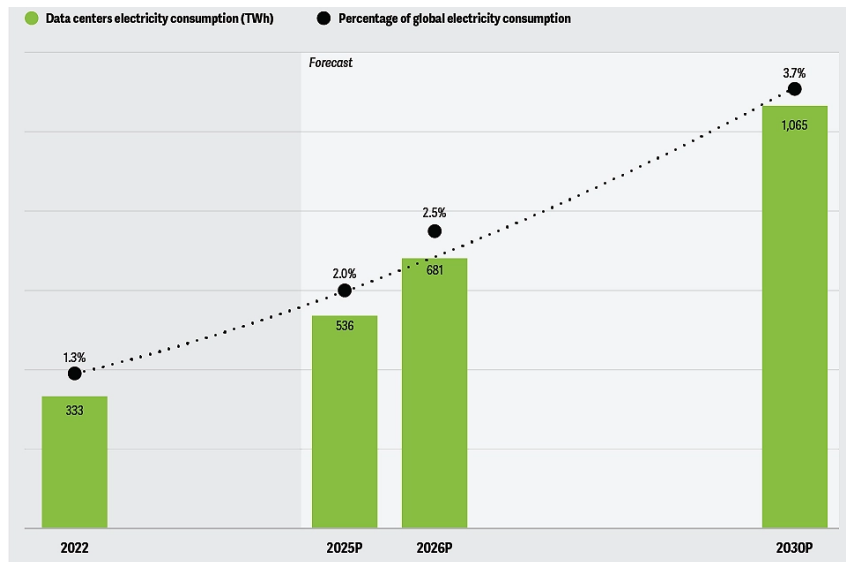
OurWorldinData.org/co2-and-greenhouse-gas-emissions | CC BY

**Note:** The period 1861–1890 is used as the baseline to measure temperature changes relative to pre-industrial times, as recommended by the source.



# Sustainability – The AI Factor

- Global DC electricity consumption to double by 2030 due to GenAI
  - references in picture links

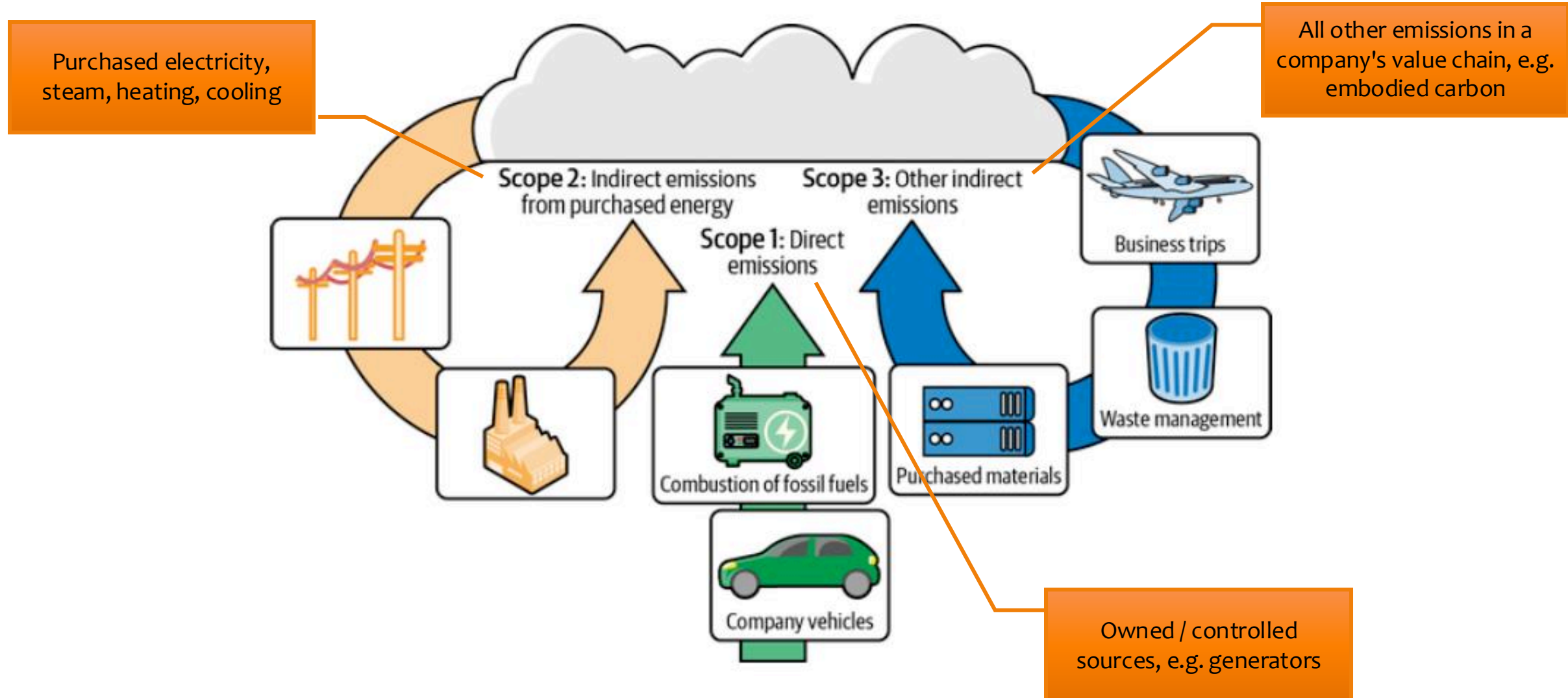




# Carbon Footprint Measurement Frameworks

- Greenhouse Gas Protocol scopes
- Power, Carbon, Water Usage Effectiveness
- Real-time vs. historical measurement approaches
- Life Cycle Assessment methodologies

# Greenhouse Gas Protocol Scopes



# Power/ Carbon/ Water Usage Effectiveness

- PUE = **Total Facility Energy** / IT Equipment Energy
  - Industry average: ~ 1.58 (2022 data)
  - Best practice target: 1.2 or less
- CUE = **Total CO<sub>2</sub> Emissions** / IT Equipment Energy
  - Because not all power is created equal
  - Accounts for carbon intensity of energy sources
- WUE = **DC Water Consumption** / IT Equipment Energy
  - Because power is not everything
  - Note that water source and disposal are also important

# Real-time vs Historical Measurement

- Real-time monitoring
  - Enables dynamic optimisation and immediate response
  - Requires substantial sensor infrastructure and data processing
  - Supports carbon-aware computing decisions
- Historical analysis
  - Provides trend data for strategic planning
  - Less resource-intensive to implement
  - Supports reporting and compliance requirements

# Life Cycle Assessment (LCA) Methodologies

- Cradle-to-grave assessment
  - a holistic and comprehensive view; not just operational consumption
  - from raw material extraction, manufacturing, transport, installation, use, through to end-of-life disposal
- Reveals hidden impacts
  - Embodied carbon in construction materials
  - Resource depletion (rare earth metals, etc.)
  - Water usage in manufacturing and cooling
- Methodology based on standards ISO 14040 and 14044
  - to ensure consistency, transparency, and reliability

# Further Reading on Frameworks

- GHG Protocol, “Corporate Standard”
- ISO, “14040: Environmental management, Life cycle assessment, Principles and framework”, 2006
- ISO, “14044: Environmental management, Life cycle assessment, Requirements and guidelines”, 2006
- Masanet and Lei, “How Much Energy Do Data Centers Really Use?”, 2020

# Measurement Granularities

- Software-level
- Server-level
- Rack-level
- DC-level
- Network-level



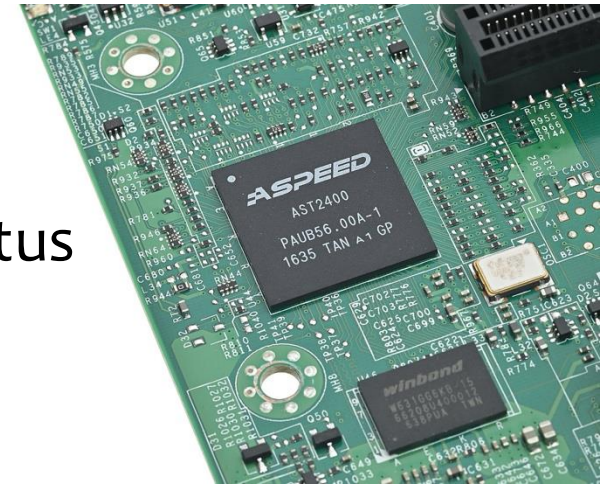
# Measurement Granularities – Software-level

- Intel RAPL (and perf wrapper)
  - previously Power Gadget and PowerLog
  - also compatible with modern AMD CPUs
- NVIDIA SMI (command-line tool) and NVML (C-based library)
- Linux tools
  - PowerTOP
  - powerstat
- Others – estimates based on hardware-specific counters & models
  - CodeCarbon
  - PowerAPI
  - Scaphandre



# Measurement Granularities – Server-level

- Power consumption at component level
  - CPU, memory, storage, network
- Intelligent Platform Management Interface (IPMI)
  - standardised hardware interface for “out-of-band” management
    - control, diagnosis, maintenance even if the server is unresponsive
  - uses a Baseboard Management Controller (BMC)
    - a dedicated microcontroller on the motherboard
  - remote administration regardless of OS or power state
  - monitor: temperature, voltage, fan speed, power supply status
  - control: power cycling, server restart, BIOS configuration
  - log: system events and errors for troubleshooting



# Measurement Granularities – Rack-level

- Intelligent PDUs for per-outlet metering
- Rack inlet/outlet temperature monitoring
- Per-rack cooling efficiency

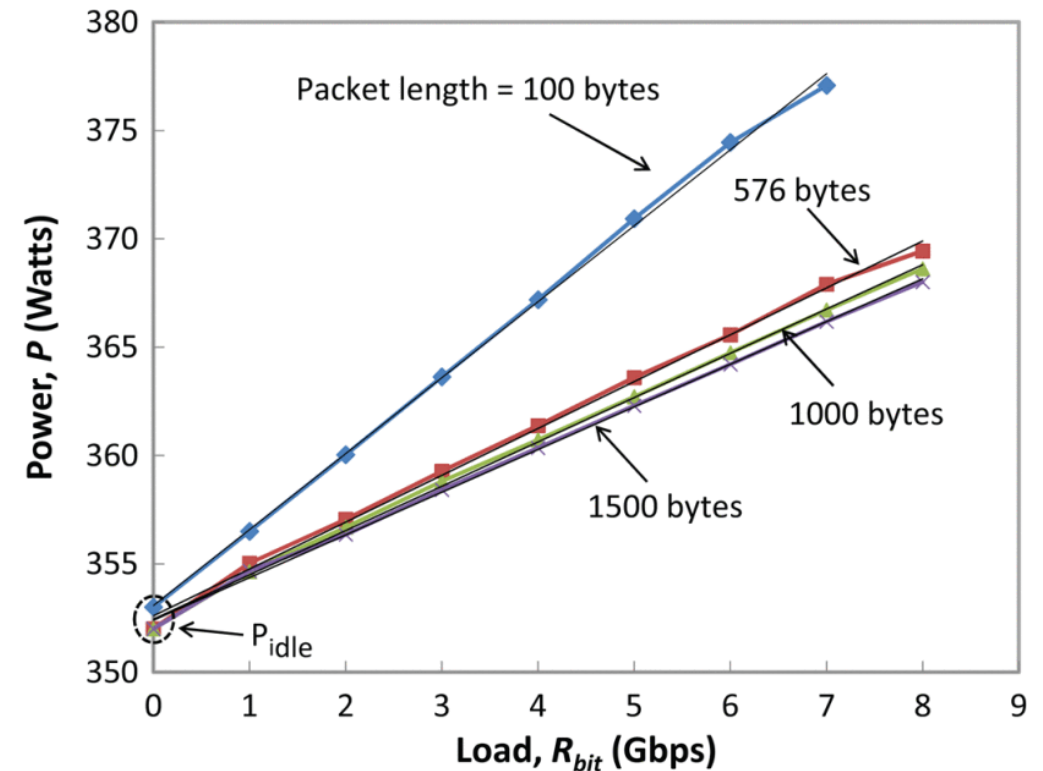


# Measurement Granularities – DC-level

- Total facility power consumption
  - IT equipment
  - HVAC systems
  - Power distribution infrastructure
    - PDUs
    - UPSs and batteries
    - Transformers – voltage regulation, electrical isolation, phase conversion
    - Electrical switchgear – circuit breakers, disconnect switches, fuses
  - Others: lighting, comms, security, building management
- Environmental conditions: temperature, humidity, air flow, smoke
- DC Manageability Interface (DCMI)
  - a standard built upon IPMI to address DC manageability needs
  - power management capabilities include monitoring and capping

# Measurement Granularities – Network-level

- Depends on all the devices and mediums along the network path
- Consumption depends on hw specs, utilisation levels, efficiency
- Estimated to consume ~1% globally
- Tricky as it encompasses various domains, grids, device models, ... and can vary significantly over time



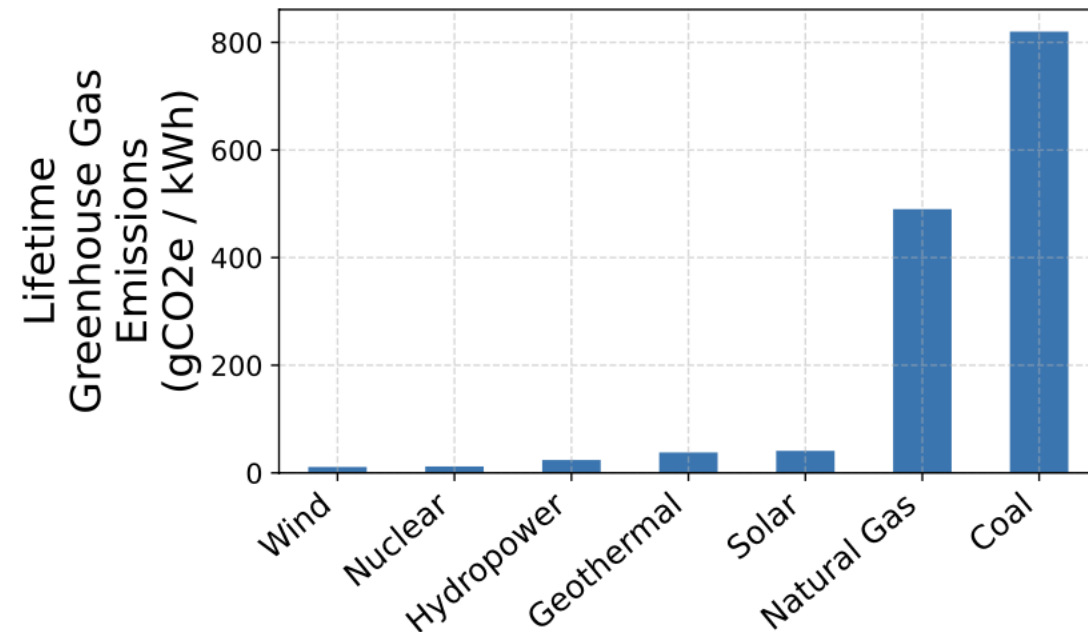
# Further Reading on Measurement

- AFCOM, “[State of the Data Center 2025 Executive Summary](#)”, 2025
- Intel, “[Data Center Manageability Interface Specification v1.5](#)”, 2011



# Carbon Intensity (CI)

- The amount of equivalent CO<sub>2</sub> released per unit of generated power
  - unit: gCO<sub>2</sub>/kWh (effect of greenhouse gases normalised to CO<sub>2</sub>)
- CI varies based on energy source
  - renewables have lower CI, fossil fuels have high CI
  - renewable supply can be intermittent
  - fossil fuels can be used on-demand
- Grid CI =  $\Sigma$ (CI of each energy source)
- Grid CI is very diverse over:
  - geographic regions
  - time of the day / year

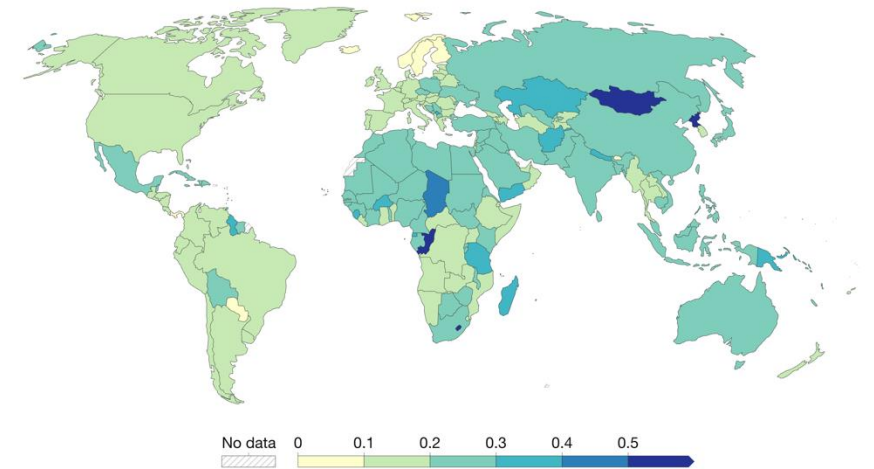


# Carbon Intensity Data

- Historically, has been very coarse and sparse
  - aggregated at country-level (select countries)
  - aggregated over a year -> month
  - e.g. by [The International Energy Agency \(IEA\)](#)
- Becoming more fine-grained
  - in location (region level) and time (sub-hour)
  - captures more variability
  - e.g. [WattTime](#), [ElectricityMaps](#)

## Carbon intensity of energy production, 2023

Amount of carbon dioxide emitted per unit of energy production, measured in kilograms of CO<sub>2</sub> per kilowatt-hour.



Data source: Global Carbon Budget (2024); U.S. Energy Information Administration (2023); Energy Institute - Statistical Review of World Energy (2024)

OurWorldinData.org/co2-and-greenhouse-gas-emissions | CC BY

# Carbin Intensity Signals

## Average CI

- Overall carbon emissions of the electricity mix
- Useful for:
  - Reporting on general emissions; e.g. policy planning, sustainability reporting
  - Long-term (e.g. annual) trend analysis
- Limitations:
  - Does not reflect real-time fluctuations
  - Can obscure the impact of specific energy consumption changes

## Marginal CI

- Emissions from the next unit of electricity (not all created equal)
- Useful for:
  - Optimising real-time decision making, i.e. load-shifting and time-shifting strategies
- Limitations:
  - Data availability can vary across regions and time periods
  - Complex to predict accurately
  - Short term view; not good for long term infrastructure analysis



# Further Reading on CI

- Energy Institute, “[\*Statistical Review of World Energy\*](#)”, 2024

# Sustainability Decisions – Vendor

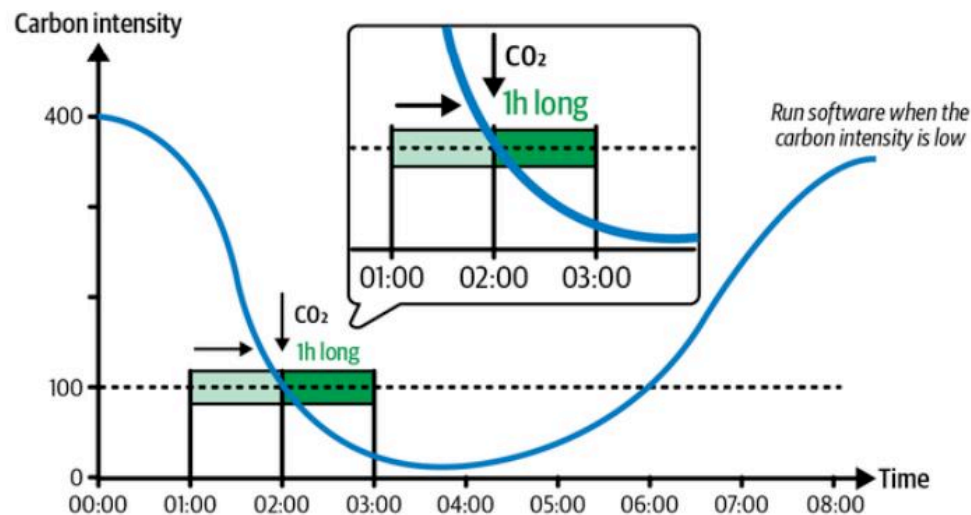
- Energy sources
  - proximity to renewables (hydro, wind, solar, geothermal)
  - invest into renewable backup sources that are reliable across seasons
- Idle power consumption
  - maximise server utilisation; workload consolidation
- Embodied carbon
  - extend hw lifespan strategies (upgrade vs replace); refurbish and reuse
  - account for transportation and installation emissions, e.g. steel, concrete
- Cooling
  - natural cooling, especially in water-scarce locations

# Sustainability Decisions – Vendor

- Lots of activity; watch this space
- Examples:
  - <https://www.microsoft.com/en-us/corporate-responsibility/sustainability>
  - <https://sustainability.aboutamazon.com/products-services/aws-cloud>
  - <https://datacenters.google/operating-sustainably/>
- Transparency and accountability is improving, but more is needed

# Carbon-Aware Decisions – User

- Location-shifting (aka load-shifting)
  - geographical placement where CI is low
- Time-shifting
  - temporal scheduling to times of low CI



# Carbon-Aware Decisions – User

- Curb expectations
  - cannot have high availability, AI, 6G, on-demand everything, etc.
  - be selective about service requirements
- Rethink design goals and practices
  - design applications to use fewer resources (memory, CPU, network)
  - identify and eliminate unnecessary levels of indirection
  - use lazy loading and other techniques to minimise resource consumption
  - test energy consumption as part of your development cycle
  - implement data lifecycle policies; e.g. delete unnecessary data
  - embrace multitenancy in deployment
    - shared VMs, neighbouring containers, managed container platforms, etc.

# Carbon-Aware Decisions – User

- Right-sized commissioning; avoid overprovisioning
  - dynamic assessment of needs and expectations
  - use historical patterns for analysis; monitoring for real-time adjustment
  - dedicated and long running cloud instances are the worst offenders
  - spot instances and serverless are (currently) very efficient
- Service and Provider selection
  - major implications of decisions on cost & perf; same applies for emissions
  - select providers with strong and clear environmental commitments
  - data is not readily available or reliable, but so was performance

# Instance Type Selection

Standard machine types

US

Monthly

Hourly

Machine type

f1-micro Bursting

f1-micro machine types offer bursting capabilities that allow instances to use additional physical CPU for short periods of time. Bursting is not available for all instance types. During the burst, the instance can use up to 3x the vCPU. Note that bursts are not available for all instance types.

n1-standard-1

n1-standard-2

n1-standard-4

n1-standard-8

n1-standard-16

n1-standard-32<sup>1</sup>

Machine type

High-memory machine types

High-memory machine types have 6.50 GB of RAM per virtual core. High-memory instances are ideal for tasks that require more memory relative to virtual CPUs.

US

Monthly

Hourly

Machine type

High-CPU machine types

High-CPU machine types have one virtual core for every 0.90 GB of RAM. High-CPU machine types are ideal for tasks that require more virtual CPUs relative to memory.

n1-highmem-2

n1-highmem-4

n1-highmem-8

n1-highmem-16

n1-highmem-32<sup>1</sup>

Custom machine type

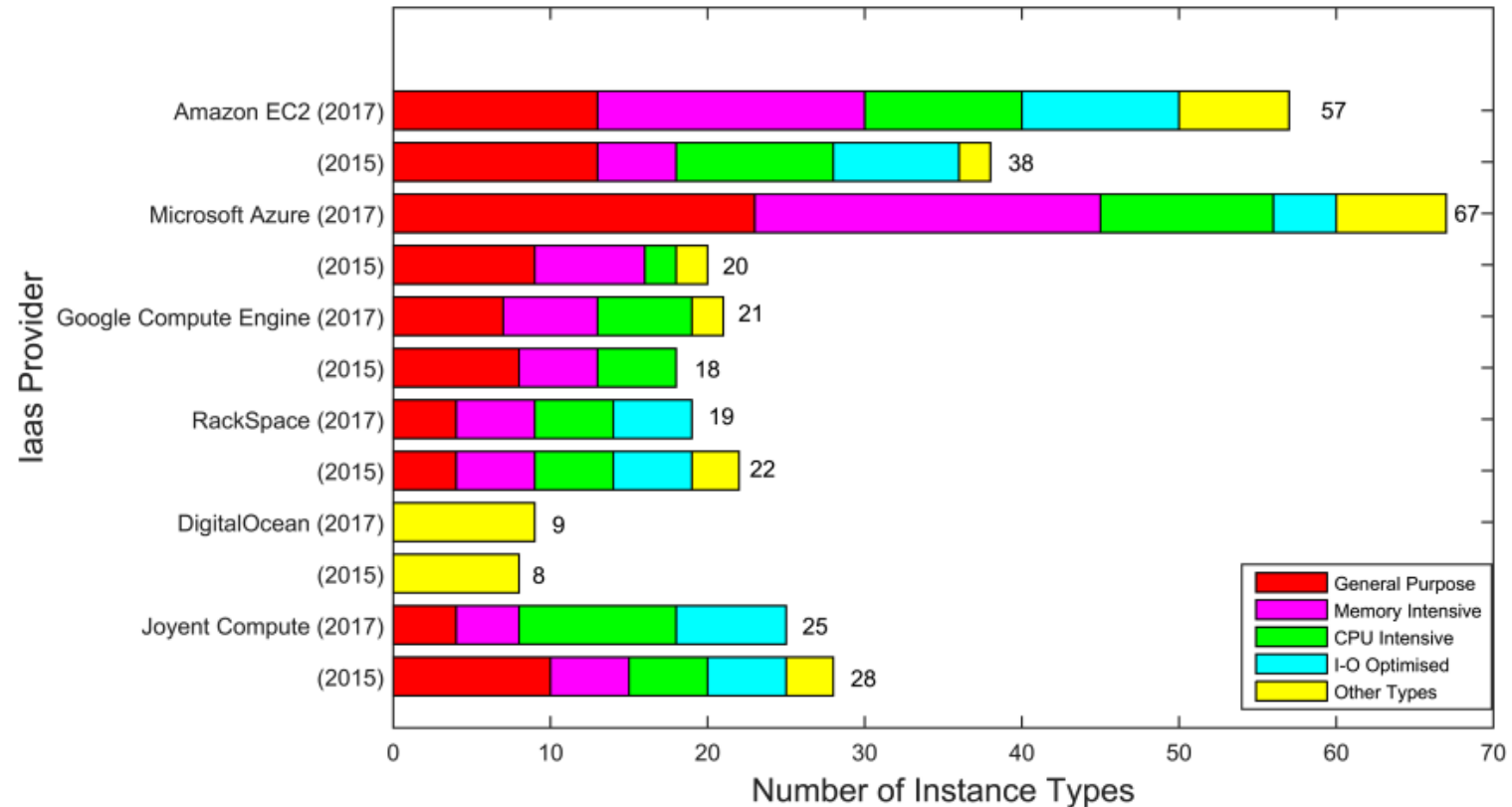
| Machine type               | Virtual CPUs | Memory  | Price (USD) | Preemptible price (USD) |
|----------------------------|--------------|---------|-------------|-------------------------|
| n1-highcpu-2               | 2            | 1.80GB  | \$0.076     | \$0.015                 |
| n1-highcpu-4               | 4            | 3.60GB  | \$0.152     | \$0.030                 |
| n1-highcpu-8               | 8            | 7.20GB  | \$0.304     | \$0.060                 |
| n1-highcpu-16              | 16           | 14.40GB | \$0.608     | \$0.120                 |
| n1-highcpu-32 <sup>1</sup> | 32           | 28.80GB | \$1.216     | \$0.240                 |

Custom machine type

If your ideal machine shape is in between two predefined types, using a custom machine type could save you as much as 40%. Read more about [Custom Machine Types](#).

|                                        | vCPU | ECU      | Memory (GiB) | Instance Storage (GB) | Linux/UNIX Usage  |
|----------------------------------------|------|----------|--------------|-----------------------|-------------------|
| General Purpose - Current Generation   |      |          |              |                       |                   |
| t2.nano                                | 1    | Variable | 0.5          | EBS Only              | \$0.0059 per Hour |
| t2.micro                               | 1    | Variable | 1            | EBS Only              | \$0.012 per Hour  |
| t2.small                               | 1    | Variable | 2            | EBS Only              | \$0.023 per Hour  |
| t2.medium                              | 2    | Variable | 4            | EBS Only              | \$0.047 per Hour  |
| t2.large                               | 2    | Variable | 8            | EBS Only              | \$0.094 per Hour  |
| t2.xlarge                              | 4    | Variable | 16           | EBS Only              | \$0.188 per Hour  |
| t2.2xlarge                             | 8    | Variable | 32           | EBS Only              | \$0.376 per Hour  |
| m4.large                               | 2    | 6.5      | 8            | EBS Only              | \$0.108 per Hour  |
| m4.xlarge                              | 4    | 13       | 16           | EBS Only              | \$0.215 per Hour  |
| m4.2xlarge                             | 8    | 26       | 32           | EBS Only              | \$0.431 per Hour  |
| m4.4xlarge                             | 16   | 53.5     | 64           | EBS Only              | \$0.862 per Hour  |
| m4.10xlarge                            | 40   | 124.5    | 160          | EBS Only              | \$2.155 per Hour  |
| m4.16xlarge                            | 64   | 188      | 256          | EBS Only              | \$3.447 per Hour  |
| Compute Optimized - Current Generation |      |          |              |                       |                   |
| c4.large                               | 2    | 8        | 3.75         | EBS Only              | \$0.1 per Hour    |
| c4.xlarge                              | 4    | 16       | 7.5          | EBS Only              | \$0.199 per Hour  |
| c4.2xlarge                             | 8    | 31       | 15           | EBS Only              | \$0.398 per Hour  |
| c4.4xlarge                             | 16   | 62       | 30           | EBS Only              | \$0.796 per Hour  |

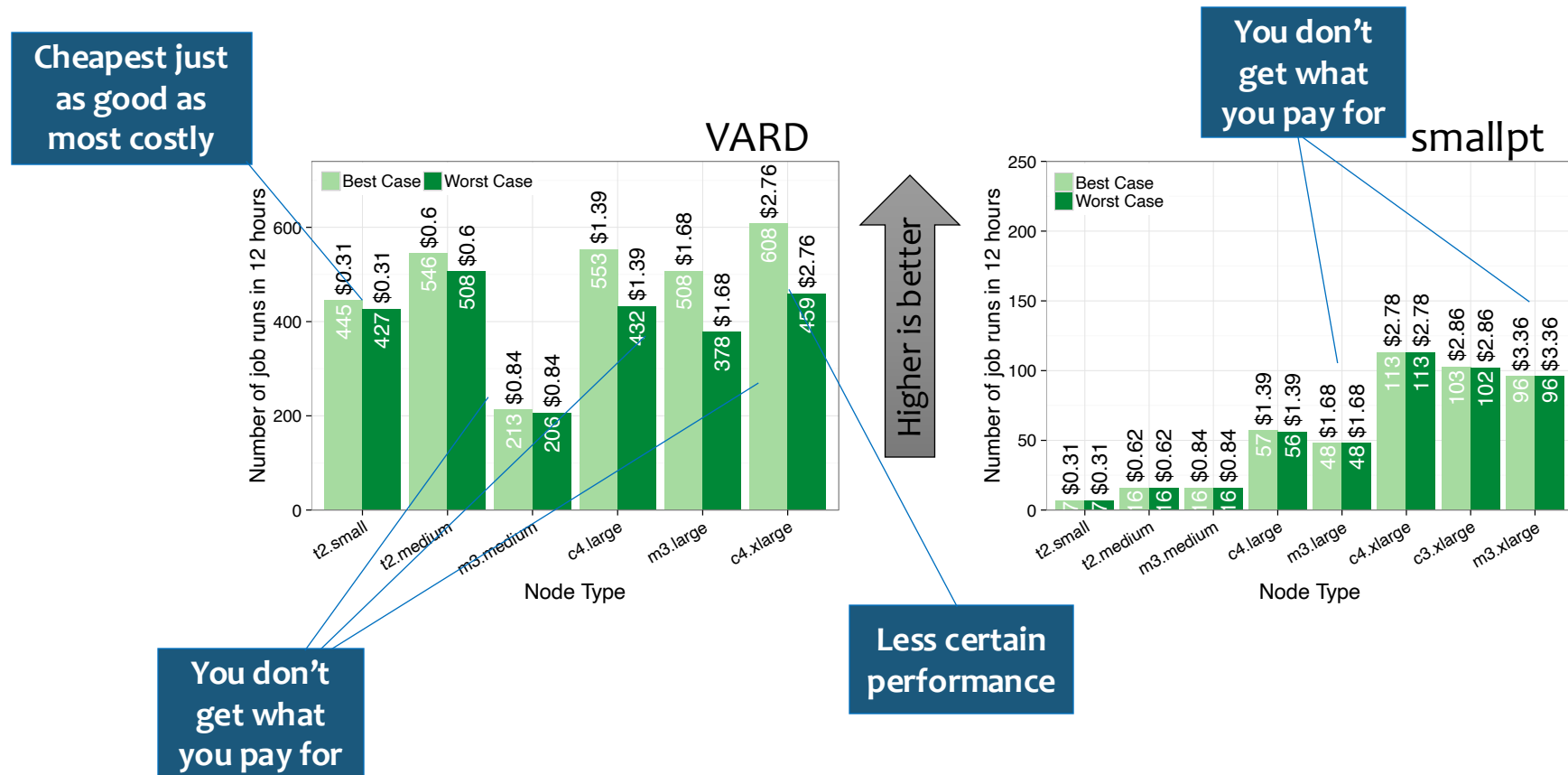
# Instance Type Selection



Abdessalam Elhabbash, Faiza Samreen, James Hadley, and Yehia Elkhatib, “[Cloud Brokerage: A Systematic Survey](#)”, ACM Computing Surveys, January 2019.



# Instance Type Selection – 12h Throughput



Yehia Elkhatib, Faiza Samreen, and Gordon S. Blair, “[Same Same, but Different: A Descriptive Intra-IaaS Differentiation](#)”, CCGrid, May 2019.

# Instance Type Selection – Decisions

- Which provider?
  - Which instance type? How many?
  - Which availability zone?
  - What time?
- 
- The answers are quite subjective
    - What is best for *my* application?

# Instance Type Selection – Perf Prediction

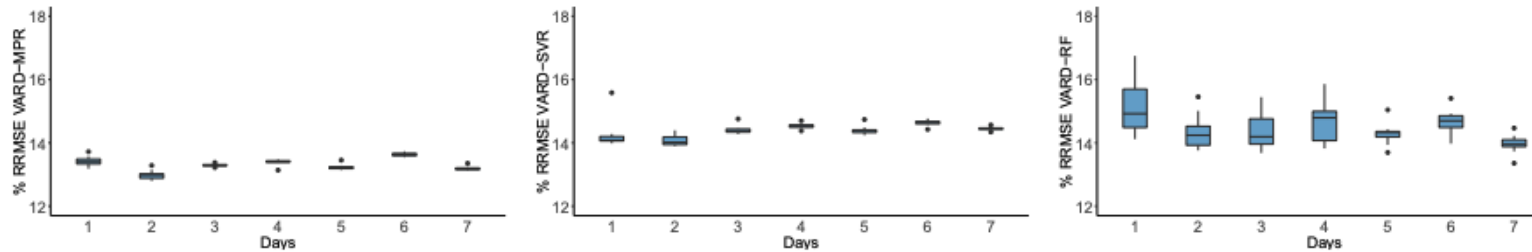


Fig. 8. Model error (%RRMSE) to assess per day prediction accuracy of three base models of VARD application over the span of 7 days. The models displayed from left to right are: MPR, SVR and RF.

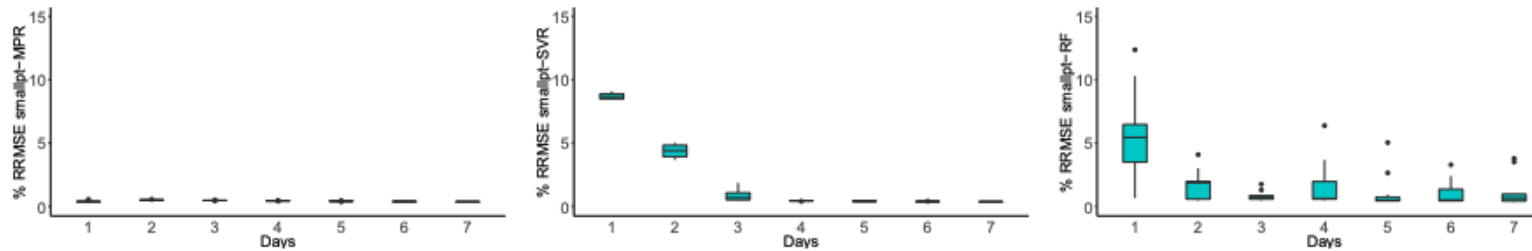


Fig. 9. Model error (%RRMSE) to assess per day prediction accuracy of three base models of smallpt application over the span of 7 days. The models displayed from left to right are: MPR, SVR and RF.

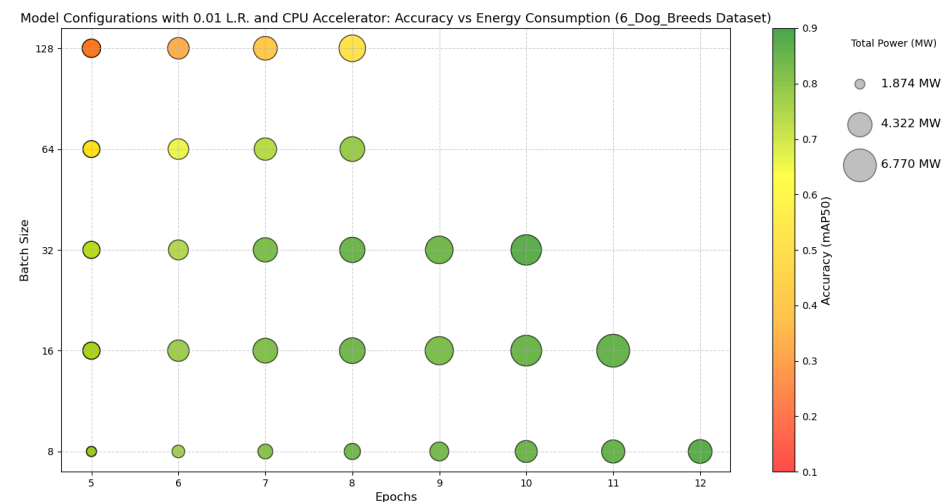
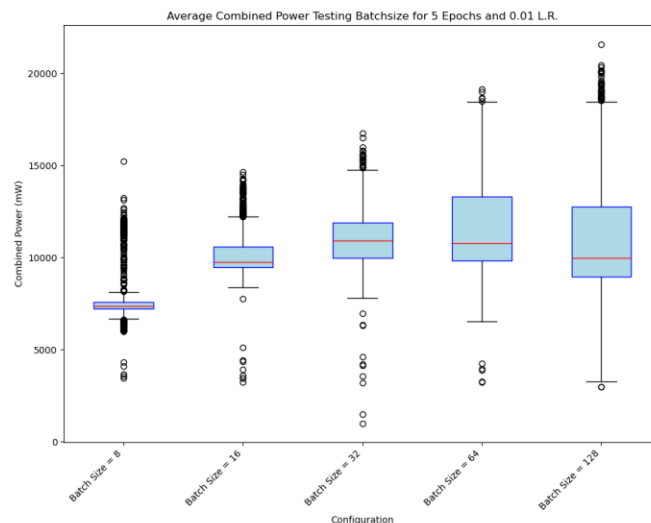
Faiza Samreen, Gordon S Blair, and Yehia Elkhatib, “[Transferable Knowledge for Low-cost Decision Making in Cloud Environments](#)”, Transactions on Cloud Computing, IEEE, July 2022.

# Carbon Footprint Prediction

- Lack of transparency: hardware specs, exact location, tenancy
- Current resources
  - Provider dashboards / APIs – still in their infancy
  - Third-party estimators – e.g. [Cloud Carbon Footprint](#)
  - LCA – comprehensive analysis of the environmental impact of a cloud service throughout its entire life cycle (manufacturing -> disposal)
- Need better data and data acquisition tools
  - important to use credible data sources and methodologies
  - an evolving field; measurement techniques are improving

# Carbon-Aware Decisions - AI

- The choice of DNN, DC, and processor can significantly reduce carbon footprint, up to 100-1000X  
Patterson et al. “[Carbon Emissions and Large Neural Network Training](#)”, arXiv, 2021
- ML training can be tweaked to trade-off some accuracy for significant reduction in emissions  
L4 project by Summer Morton, University of Glasgow, 2025



# Grid-Aware Computing

- Suggests that (current) carbon-aware approaches can be detrimental
- If many time-/location shift, renewable energy could be overused
- This leads to resorting to fossil fuels to cover excess demand
- Important to understand how grids work
- ...and the need for a more responsible and effective implementation
  - termed “grid-aware computing”
- Velasco, “[Carbon Aware Computing: Next Green Breakthrough or New Greenwashing?](#)”, 2024

| Energy source                   | Scalability                       |
|---------------------------------|-----------------------------------|
| Renewables – Solar, Wind, Hydro | Inflexible                        |
| Nuclear                         | Less flexible (safety challenges) |
| Fossil Fuel – Coal              | Flexible                          |
| Fossil Fuel – Gas               | Extremely flexible                |

# Further Reading on Carbon-Awareness

- Acun et al., “[Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters](#)”, ASPLOS, 2023
- Gupta et al., “[Chasing Carbon: The Elusive Environmental Footprint of Computing](#)”, HPCA, 2021
- Radovanovic et al., “[Carbon-Aware Computing for Datacenters](#)”, IEEE TPC, 2023

# Coda – Which GHGP Scope?

- So far, we covered Scope 2 (emissions from purchased energy)
- Very little work on Scope 3 (from other than purchased energy)
- We don't do well at tracking embodied carbon wastage
  - resources are decommissioned early as their performance is insufficient
  - decommissioned resources are not recycled properly because it costs



# Next week – Guest Lectures!

- **We will start at 3.00 sharp**
- **Not** included in assessment (quizzes and exam)
- A great opportunity to learn and ask questions
  - Please do engage

# Topics for Quizzes & Exam Assessment

- ✓ Lecture slides
- ✓ Group exercises during lectures
  - and linked material (Moodle)
- ✗ Further reading pointers
- ✗ References used in lectures
  - except for group exercises
- ✗ Guest lecture(s)

# COMPSCI4106/5118 Cloud Systems



Intro to Cloud Computing and Virtual Resources

Virtual Machines

Containers and Container Management

Cloud System Operation and Management

Cloud System Sustainability

Cloud System Design

Modern Cloud Architectures

Flavours of Cloud

A Wider Lens on Sustainability

► **Guest Lecture & Revision**

Yehia Elkhatib

# Overview

- Guest Lectures
- Exam Prep
  - Format
  - Topics
  - Example Questions
- Things We Missed
- Quiz 2B
- NSS

# Guest Lectures



Dr Kelly Widdicks  
Software Systems Architect, CEH



Dr Faiza Samreen  
Software Systems Architect, CEH

# Break!



# Exam



# Exam Format

- **50%** weight of the overall grade
- Even split between both halves of the course
  - 50% Part 1 – Cloud Resource Management
  - 50% Part 2 – Scalable and Sustainable Architectures
- **Four questions**, 10 marks each
  - Two questions on Part 1, two questions on Part 2
  - Mix of long-form answers (to answer briefly) and MCQ-style questions
- **60 minutes** (for 40 marks)



# Most Important Topics of Part 1 (1)

- **Chapter 1: Introduction**

- NIST definition and its dimensions
- Virtualization terminology and categories

- **Chapter 2: Virtual Machines**

- Categories of instructions, Popek and Goldberg's theorem, and the idea of trap and emulate
- Full virtualization (binary translation, shadow page tables)
- OS-assisted virtualization (CPU and memory virtualization in Xen)
- Hardware-assisted virtualization (CPU and memory features)
- Virtual device drivers

# Most Important Topics of Part 1 (2)

- **Chapter 3: Containers and Container Management**
  - Linux kernel containment features
  - Docker: Images and Dockerfiles
  - Containers vs. VMs
  - Container orchestration with a system like Kubernetes
- **Chapter 4: Cloud Infrastructure Management**
  - Challenges (server sprawl, configuration drift, snowflake servers)
  - Cloud operating systems like OpenStack
  - VM snapshots and migration
  - Infrastructure-as-code and “immutable infrastructures”
  - Continuous deployment strategies (green/blue and canary)

# Most Important Topics of Part 1 (3)

- **Chapter 5: Cloud Sustainability**

- Emissions lifecycle
- Energy efficiency, resource efficiency, carbon-awareness
- Energy proportionality and PUE
- Jevons' paradox and the issue of rebound effects
- High-level procedure for estimating cloud emissions

# Example Questions from Part 1 (1)

- Briefly describe what *critical instructions* are and why they presented a challenge for x86 system virtualization [2]
  - critical instructions are sensitive instructions that are not privileged [1]
  - critical instructions don't trap, so don't pass control to a hypervisor, and would then not have the expected behavior for guest OS code [1]
- Briefly summarize why the physical main memory can simply be partitioned for Xen guests [3]
  - guests are aware of running on a hypervisor, using only parts of the memory [1]
  - large partitions of memory (~ GBs) are typically allocated to each of a few VMs [1]
  - memory addresses for guest processes remain logical/virtual (preserving the well-known benefits of virtual memory, e.g. paging) [1]

# Example Questions from Part 1 (2)

- Briefly explain what the chroot system call on Linux does and how it is useful for containerization [2]
  - makes a specific directory the new root directory of the calling process [1]
  - allows to limit/isolate access to binaries, libraries, configs, etc. [1]
- Briefly explain how Infrastructure-as-Code addresses snowflake servers [2]
  - the configuration of servers is captured in code that can be versioned and executed to reproduce configurations [1]
  - eliminates manual changes to servers that could go undocumented [1]

# Example Questions from Part 1 (3)

- Name and briefly explain an issue with the PUE metric [1]
  - Any valid point of criticism (e.g. inconsistent methodologies, whole system tradeoffs not captured, says nothing about energy sources or IT equipment energy efficiency) [1]
- MCQ-style questions: See the weekly quizzes of Part 1 for examples
  - “select the most appropriate answer and explain your selection in one sentence why you selected it”
  - [1] for selection, [1] for valid justification

# Most Important Topics of Part 2 (1)

- **Chapter 6: Cloud System Design**

- Distributed Systems: Fallacies, Key Aspects, Failures/Errors/Faults/QoS
- Quality Attributes: Dependability, Availability & Reliability
- High Availability: Fault Tolerance, Error Detection, Failover Strategies

- **Chapter 7: Modern Cloud Architectures**

- Architectural Approaches
- Redundancy by Replication
- Stateless Scaling (Load balancing), Stateful Scaling (Partitioning)
- Advanced Architectures: Microservices, Glueware, SMTs, Cloud-native Tech.

# Most Important Topics of Part 2 (2)

- **Chapter 8: Flavours of Cloud**

- Provisioning levels (IaaS, PaaS, SaaS, FaaS)
- Deployment models
- Cross-cloud Computing
- The Computing Continuum

- **Chapter 9: A Wider Lens on Sustainability**

- Designing Dependable Data Centres
- Carbon Footprint Measurement Frameworks
- Measurement Granularities
- Carbon Intensity
- Carbon-Aware Decision Making



# Example Questions from Part 2 (1)

- **Long-form tasks:**

- You are designing a cloud-based ML system with training and inference components. Name one reason why you might deploy the inference service at the edge rather than in the cloud. Provide a brief explanation for your answer. *Length: 20-25 words*
- Match each of the following scenario with the most appropriate failover strategy (Active-Active / Active-Passive). Provide justification. *Length: 20-30 words*

# Example Questions from Part 2 (2)

- **MCQ-style:**

- A financial technology company is developing a payment processing system that handles a very high volume of daily transactions. Which architectural approach would be most appropriate? *Length: 1 sentence*
  - a. Single server with high memory
  - b. Load-balanced servers with database partitioning
  - c. Serverless functions with a single database
  - d. Monolithic application with local caching

# Things We Missed

- Slides from Lecture 6

# Quiz 2B

- It's open!
- ... till Friday

**Improve the student experience  
with a 10 min survey.**

Final year undergraduate students go to  
[glasgow.ac.uk/NSS](https://glasgow.ac.uk/NSS) for a chance to win **£500**.  
Closes 30th April 2025



**Why take part in the NSS?** give feedback on issues that matter to you and **win prizes!** we have 1 x **£500** and 5 x **£100** to win  
(please note that only online responses are eligible for prizes)

**How do I complete the NSS?**

before 30 April 2025 visit [thestudentsurvey.com](https://thestudentsurvey.com) or

